



Camera 中间件 开发参考

文档版本 00B03

发布日期 2023-06-30

版权所有 © 海思技术有限公司2023。保留一切权利。

非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

商标声明



HISILICON、海思和其他海思商标均为海思技术有限公司的商标。

本文档提及的其他所有商标或注册商标，由各自的所有人拥有。

注意

您购买的产品、服务或特性等应受海思公司商业合同和条款的约束，本文档中描述的全部或部分产品、服务或特性可能不在您的购买或使用范围之内。除非合同另有约定，海思公司对本文档内容不做任何明示或默示的声明或保证。

由于产品版本升级或其他原因，本文档内容会不定期进行更新。除非另有约定，本文档仅作为使用指导，本文档中的所有陈述、信息和建议不构成任何明示或暗示的担保。

海思技术有限公司

地址：上海市青浦区虹桥港路2号101室 邮编：201721

网址：<https://www.hisilicon.com/cn/>

客户服务邮箱：support@hisilicon.com



前言

概述

本文为使用中间件进行开发的程序员而写，目的是供您在开发过程中查阅中间件的各种参考信息，包括API、头文件、错误码等。

本文档描述中间件的各个API的使用方法，以及相关的数据结构和错误码。

读者对象

本文档主要适用于以下工程师：

- 技术支持工程师
- 软件开发工程师

符号约定

在本文中可能出现下列标志，它们所代表的含义如下。

符号	说明
 危险	表示如不可避免则将会导致死亡或严重伤害的具有高等级风险的危害。
 警告	表示如不可避免则可能导致死亡或严重伤害的具有中等级风险的危害。
 注意	表示如不可避免则可能导致轻微或中度伤害的具有低等级风险的危害。
 须知	用于传递设备或环境安全警示信息。如不可避免则可能会导致设备损坏、数据丢失、设备性能降低或其它不可预知的结果。 “须知”不涉及人身伤害。
 说明	对正文中重点信息的补充说明。 “说明”不是安全警示信息，不涉及人身、设备及环境伤害信息。



修改记录

文档版本	发布日期	修改说明
00B03	2023-06-30	第3次临时版本发布。 第1章 概述 1.4.5~1.4.9小节和1.4.11小节涉及修改 第2章 录像模块 新增 SS_Track_Source_RequestKeyFrame_FN 第5章 MP4封装解封装模块 “5.4.2 OT_MP4_CONFIG_MUXER_S” 涉及修改 第7章 播放器 “7.4.18 OT_FORMAT_PACKET_S” 涉及修改 第14章 TS封装模块 “14.4.8 OT_TS_STREAM_INFO_S” 涉及修改 第15章 DCF “15.5 错误码” 涉及修改 第16章 DNG封装解封装模块 “16.4.7 OT_DNG_MUXER_EXIF_INFO_S” 涉及修改
00B02	2023-05-20	第2次临时版本发布。 第2章 录像模块 “1.4.29 OT_REC_WRITE_FRAME_FN” 涉及修改 第3章 日期时间文件系统 “1.2.1.1 文件类型”、“1.4.4 OT_DTCF_FILE_TYPE_E” 涉及修改 第5章 MP4封装解封装模块 新增 SS_MP4_GetFileSize接口 第14章 TS封装模块 新增 SS_TS_GetFileSize接口
00B01	2023-03-15	第1次临时版本发布。



目 录

前言.....	i
1 概述.....	1
1.1 中间件概述.....	1
1.1.1 在整个方案中的位置.....	1
1.1.2 模块划分.....	2
1.2 中间件架构.....	2
1.2.1 支持的功能列表.....	2
1.2.2 模块详细说明.....	3
1.2.3 产品层架构图说明.....	3
2 录像模块.....	5
2.1 概述.....	5
2.1.1 基本概念.....	5
2.1.1.1 录像.....	5
2.1.1.2 录制码流.....	6
2.1.1.3 录像切分.....	6
2.1.1.4 录制类型.....	6
2.2 功能描述.....	6
2.2.1 录像.....	6
2.2.2 录制码流.....	6
2.3 API 参考.....	6
2.3.1 SS_REC_Create.....	7
2.3.2 SS_REC_Destroy.....	8
2.3.3 SS_REC_SetParam.....	8
2.3.4 SS_REC_Start.....	9
2.3.5 SS_REC_Stop.....	10
2.3.6 SS_REC_ManualSplit.....	11
2.3.7 SS_REC_WriteData.....	13
2.3.8 SS_REC_RegisterEventCallback.....	14
2.4 数据类型.....	15
2.4.1 OT_REC_TYPE_E.....	16
2.4.2 OT_REC_NORMAL_ATTR_S.....	16
2.4.3 OT_REC_LAPSE_ATTR_S.....	17



2.4.4 OT_REC_SPLIT_TYPE_E.....	17
2.4.5 OT_REC_SPLIT_ATTR_S.....	18
2.4.6 OT_REC_EVENT_E.....	19
2.4.7 OT_REC_STATE_EVENT_INFO_S.....	20
2.4.8 OT_REC_FILE_EVENT_INFO_S.....	21
2.4.9 OT_REC_ERROR_EVENT_INFO_S.....	21
2.4.10 OT_REC_EVENT_INFO_S.....	22
2.4.11 OT_REC_FRAME_DATA_S.....	22
2.4.12 OT_REC_MUXER_OPERATE_FN_S.....	23
2.4.13 OT_REC_BUF_ABSTRACTFUNC_S.....	24
2.4.14 OT_REC_STREAM_ATTR_S.....	24
2.4.15 OT_REC_CALLBACK_S.....	26
2.4.16 OT_REC_ATTR_S.....	26
2.4.17 OT_REC_PARAM.....	27
2.4.18 OT_REC_MANUAL_SPLIT_TYPE_E.....	28
2.4.19 OT_REC_POST_MANUAL_SPLIT_ATTR_S.....	28
2.4.20 OT_REC_PRE_MANUAL_SPLIT_ATTR_S.....	29
2.4.21 OT_REC_NORMAL_MANUAL_SPLIT_ATTR_S.....	29
2.4.22 OT_REC_EXTEND_DURATION_SPLIT_ATTR_S.....	30
2.4.23 OT_REC_MANUAL_SPLIT_ATTR_S.....	30
2.4.24 OT_REC_EVENT_CALLBACK_FN.....	31
2.4.25 OT_REC_REQUEST_FILE_NAMES_FN.....	31
2.4.26 OT_REC_CREATE_MUXER_FN.....	32
2.4.27 OT_REC_DESTROY_MUXER_FN.....	32
2.4.28 OT_REC_CREATE_TRACK_FN.....	33
2.4.29 OT_REC_WRITE_FRAME_FN.....	33
2.4.30 OT_REC_BUF_ALLOC_FN.....	34
2.4.31 OT_REC_BUF_FREE_FN.....	35
2.4.32 OT_REC_STREAM_MAX_CNT.....	35
2.4.33 OT_REC_TRACK_MAX_CNT.....	35
2.4.34 OT_REC_FRAME_MAX_SLICE.....	36
2.4.35 OT_REC_FILE_NAME_LEN.....	36
2.4.36 OT_Track_SourceType_E.....	37
2.4.37 OT_Track_VideoCodec_E.....	37
2.4.38 OT_Track_AudioCodec_E.....	38
2.4.39 OT_Track_VideoSourceInfo_S.....	39
2.4.40 OT_Track_AudioSourceInfo_S.....	39
2.4.41 OT_Track_PrivateSourceInfo_S.....	40
2.4.42 Track_Source_S.....	41
2.4.43 OT_Track_Source_Handle.....	42
2.4.44 SS_Track_Source_Start_FN.....	42
2.4.45 SS_Track_Source_Stop_FN.....	42



2.4.46 SS_Track_Source_RequestKeyFrame_FN.....	43
2.5 错误码.....	44
3 日期时间文件系统.....	46
3.1 概述.....	46
3.2 架构原理.....	46
3.2.1 文件目录组织规则.....	48
3.2.1.1 文件类型.....	48
3.2.1.2 目录类型.....	48
3.2.1.3 文件名称.....	49
3.2.1.4 扫描.....	50
3.2.1.5 检索.....	50
3.2.1.6 删除.....	50
3.3 API 参考.....	50
3.3.1 SS_DTCF_Init.....	51
3.3.2 SS_DTCF_GetDirNames.....	51
3.3.3 SS_DTCF_DelInit.....	52
3.3.4 SS_DTCF_Scan.....	53
3.3.5 SS_DTCF_GetFileByIndex.....	54
3.3.6 SS_DTCF_DelFileByIndex.....	54
3.3.7 SS_DTCF_AddFile.....	55
3.3.8 SS_DTCF_GetOldestFileIndex.....	56
3.3.9 SS_DTCF_GetFileAmount.....	57
3.3.10 SS_DTCF_GetOldestFilePath.....	58
3.3.11 SS_DTCF_CreateFilePath.....	58
3.3.12 SS_DTCF_GetRelatedFilePath.....	59
3.3.13 SS_DTCF_GetEmrFilePath.....	60
3.3.14 SS_DTCF_GetFileDirType.....	61
3.4 数据类型.....	61
3.4.1 OT_FILE_PATH_LEN_MAX.....	62
3.4.2 OT_DIR_LEN_MAX.....	62
3.4.3 OT_DTCF_DIR_E.....	62
3.4.4 OT_DTCF_FILE_TYPE_E.....	63
3.5 错误码.....	64
4 媒体缓存库 libmbuf 使用说明.....	66
4.1 概述.....	66
4.2 功能描述.....	66
4.2.1 基本概念.....	66
4.2.2 功能描述.....	66
4.3 API 参考.....	66
4.3.1 SS_MBUF_GetBuffer.....	67
4.3.2 SS_MBUF_ReleaseBuffer.....	68
4.3.3 SS_MBUF_RegisterPayload.....	68



4.3.4 SS_MBUF_UnregisterPayload.....	69
4.3.5 SS_MBUF_SetRWEnable.....	70
4.3.6 SS_MBUF_GetRWEnable.....	71
4.3.7 SS_MBUF_WriteSlice.....	72
4.3.8 SS_MBUF_ReadSlice.....	73
4.3.9 SS_MBUF_Forward.....	74
4.3.10 SS_MBUF_GetSliceDataOffset.....	74
4.3.11 SS_MBUF_Debug.....	75
4.4 数据结构.....	76
4.4.1 OT_MBUF_FRAME_MAX_BLOCK.....	76
4.4.2 OT_MBUF_ABSTRACTFUNC_S.....	77
4.4.3 OT_MBUF_CFG_S.....	77
4.4.4 OT_MBUF_SLICEINFO_S.....	78
4.4.5 OT_MBUF_FRAMEINFO_S.....	79
4.4.6 Mbuf_Alloc_Func.....	80
4.4.7 Mbuf_Free_Func.....	80
4.4.8 Mbuf_Mcopy_Func.....	81
4.4.9 Mbuf_CreateLock_Func.....	82
4.4.10 Mbuf_DestroyLock_Func.....	82
4.4.11 Mbuf_Lock_Func.....	83
4.4.12 Mbuf_UnLock_Func.....	84
4.5 错误码.....	84
4.6 典型场景使用指南.....	85
4.6.1 视频点播.....	85
4.6.1.1 场景分析.....	85
4.6.1.2 代码示例.....	85
5 MP4 封装解封装模块.....	86
5.1 概述.....	86
5.2 功能描述.....	86
5.2.1 基本概念.....	86
5.2.2 功能描述.....	86
5.2.3 上下文描述.....	86
5.3 API 参考.....	87
5.3.1 SS_MP4_Create.....	87
5.3.2 SS_MP4_Destroy.....	89
5.3.3 SS_MP4_CreateTrack.....	89
5.3.4 SS_MP4_DestroyAllTracks.....	90
5.3.5 SS_MP4_WriteFrame.....	91
5.3.6 SS_MP4_AddAtom.....	92
5.3.7 SS_MP4_RepairFile.....	93
5.3.8 SS_MP4_GetAtom.....	94
5.3.9 SS_MP4_GetFileInfo.....	95



5.3.10 SS_MP4_GetTrackInfoByIndex.....	96
5.3.11 SS_MP4_GetFileSize.....	97
5.4 数据结构.....	98
5.4.1 OT_MP4_CONFIG_S.....	98
5.4.2 OT_MP4_CONFIG_MUXER_S.....	99
5.4.3 OT_MP4_CONFIG_DEMUXER_S.....	100
5.4.4 OT_MP4_CONFIG_APPEND_S.....	101
5.4.5 OT_MP4_ATOM_INFO_S.....	101
5.4.6 OT_MP4_FRAME_DATA_S.....	102
5.4.7 OT_MP4_TRACK_INFO_S.....	102
5.4.8 OT_MP4_DATAINFO_S.....	103
5.4.9 OT_MP4_AUDIOINFO_S.....	104
5.4.10 OT_MP4_VIDEOINFO_S.....	105
5.4.11 OT_MP4_FILE_INFO_S.....	105
5.4.12 OT_MP4_CONFIG_TYPE_E.....	106
5.4.13 OT_MP4_FORMAT_PROFILE_E.....	107
5.4.14 OT_MP4_CODECID_E.....	107
5.4.15 OT_MP4_TRACK_TYPE_E.....	108
5.4.16 OT_MP4_SETDUTATION_FN.....	109
5.4.17 OT_MP4_MAX_FILE_NAME.....	109
5.4.18 OT_MP4_MAX_HDLR_NAME.....	110
5.4.19 OT_MP4_MAX_BOX_PATH.....	110
5.4.20 OT_MP4_BOX_MAX_DATALEN.....	110
5.4.21 OT_MP4_TYPE_SIZE.....	111
5.4.22 OT_MP4_MOOV_LOCATION_E.....	111
5.4.23 OT_MP4_MOOV_AT_TAIL_INFO_S.....	111
5.4.24 OT_MP4_MOOV_AT_FRONT_INFO_S.....	112
5.5 错误码.....	112
5.6 典型场景使用指南.....	114
6 EXIF 模块.....	115
6.1 概述.....	115
6.1.1 EXIF 模块处理流程.....	117
6.2 API 参考.....	118
6.2.1 SS_EXIF_Init.....	119
6.2.2 SS_EXIF_Open.....	119
6.2.3 SS_EXIF_GetInfo.....	120
6.2.4 SS_EXIF_ReadPicData.....	121
6.2.5 SS_EXIF_GetTag.....	122
6.2.6 SS_EXIF_SetBufferSource.....	123
6.2.7 SS_EXIF_SetTag.....	124
6.2.8 SS_EXIF_Close.....	125
6.2.9 SS_EXIF_DeInit.....	126



6.3 数据类型.....	127
6.3.1 OT_EXIF_DATA_S.....	127
6.3.2 OT_EXIF_INFO_S.....	128
6.3.3 OT_EXIF_SUBIMG_S.....	128
6.3.4 OT_EXIF_SUBIMG_MAX_CNT.....	129
6.3.5 OT_EXIF_DATA_SEG_MAX_CNT.....	129
6.3.6 OT_EXIF_DATA_TYPE_E.....	130
6.3.7 OT_EXIF_TAG_LOCATION_E.....	130
6.3.8 OT_EXIF_TAG_ENTRY_S.....	131
6.3.9 OT_EXIF_CONFIG_EDITOR_S.....	132
6.3.10 OT_EXIF_CONFIG_DEMUXER_S.....	132
6.3.11 OT_EXIF_ATTR_S.....	133
6.3.12 OT_EXIF_CONFIG_TYPE_E.....	134
6.3.13 OT_EXIF_BUFFER_SOURCE_S.....	134
6.4 错误码.....	135
6.5 典型场景使用指南.....	136
6.5.1 EXIF 模块图片解封装使用实例.....	136
6.5.2 EXIF 模块图片编辑使用实例.....	137
7 播放器模块.....	140
7.1 概述.....	140
7.1.1 播放器模块层次结构.....	141
7.1.2 播放器模块处理流程.....	141
7.1.3 播放器状态图.....	143
7.2 功能描述.....	143
7.2.1 解封装模块.....	143
7.2.2 播控模块.....	144
7.3 API 参考.....	144
7.3.1 SS_LITEPLAYER_Init.....	145
7.3.2 SS_LITEPLAYER_Delnit.....	145
7.3.3 SS_LITEPLAYER_RegisterDemuxer.....	146
7.3.4 SS_LITEPLAYER_UnRegisterDemuxer.....	147
7.3.5 SS_LITEPLAYER_RegisterAudioDecoder.....	148
7.3.6 SS_LITEPLAYER_UnRegisterAudioDecoder.....	149
7.3.7 SS_LITEPLAYER_Create.....	149
7.3.8 SS_LITEPLAYER_SetVOHandle.....	150
7.3.9 SS_LITEPLAYER_SetAOHandle.....	151
7.3.10 SS_LITEPLAYER_Destroy.....	152
7.3.11 SS_LITEPLAYER_SetDataSource.....	153
7.3.12 SS_LITEPLAYER_Prepare.....	153
7.3.13 SS_LITEPLAYER_Play.....	154
7.3.14 SS_LITEPLAYER_Stop.....	155
7.3.15 SS_LITEPLAYER_Pause.....	156



7.3.16 SS_LITEPLAYER_Seek.....	156
7.3.17 SS_LITEPLAYER_RegCallback.....	157
7.3.18 SS_LITEPLAYER_GetFileInfo.....	158
7.3.19 SS_LITEPLAYER_SetMedia.....	159
7.3.20 SS_LITEPLAYER_GetPlayStatus.....	160
7.3.21 SS_LITEPLAYER_TPlay.....	161
7.4 数据类型.....	162
7.4.1 OT_LITEPLAYER_ERROR_E.....	162
7.4.2 OT_LITEPLAYER_STATE_E.....	163
7.4.3 OT_LITEPLAYER_EVENT_E.....	164
7.4.4 OT_LITEPLAYER_ATTR_S.....	165
7.4.5 OT_LITEPLAYER_EVENT_FN.....	165
7.4.6 OT_LITEPLAYER_PARAM_S.....	166
7.4.7 OT_LITEPLAYER_EXTRA_FILESURFIX_S.....	167
7.4.8 OT_LITEPLAYER_VOUT_OPT_S.....	168
7.4.9 OT_LITEPLAYER_AOUT_OPT_S.....	168
7.4.10 OT_LITEPLAYER_TPLAY_ATTR_S.....	169
7.4.11 OT_LITEPLAYER_VOUT_TYPE_E.....	170
7.4.12 OT_LITEPLAYER_TPLAY_DIRECTION_E.....	170
7.4.13 OT_LITEPLAYER_PLAY_SPEED_E.....	171
7.4.14 OT_LITEPLAYER_FILE_PATH_MAX_LEN.....	171
7.4.15 OT_LITEPLAYER_AUD_DECODER_PATH_LEN.....	172
7.4.16 OT_LITEPLAYER_MAX_FILE_SURFIX_CNT.....	172
7.4.17 OT_LITEPLAYER_MAX_FILE_SURFIX_LEN.....	172
7.4.18 OT_FORMAT_PACKET_S.....	172
7.4.19 OT_FORMAT_STREAM_RESOLUTION_S.....	173
7.4.20 OT_FORMAT_VIDEO_TYPE_E.....	174
7.4.21 OT_FORMAT_AUDIO_TYPE_E.....	175
7.4.22 OT_FORMAT_FILE_INFO_S.....	175
7.4.23 OT_FORMAT_FUN_S.....	177
7.4.24 OT_DEMUX_S.....	177
7.4.25 OT_FORMAT_MEDIA_ATTR_S.....	178
7.4.26 OT_DEMUXER_RESOLUTION_CNT.....	179
7.4.27 OT_DEMUXER_NAME_LEN.....	179
7.4.28 OT_DEMUXER_SUPPORT_NAMES_LEN.....	179
7.4.29 OT_DEMUXER_PROBE_MODE_TYPE_E.....	179
7.5 错误码.....	180
7.6 典型场景使用指南.....	182
7.6.1 播放器使用示例.....	182
7.6.2 播放器使用注意事项.....	185
8 定时器模块.....	186
8.1 概述.....	186



8.2 功能描述.....	186
8.3 API 参考.....	186
8.3.1 SS_Timer_Init.....	186
8.3.2 SS_Timer_DeInit.....	187
8.3.3 SS_Timer_Create.....	188
8.3.4 SS_Timer_Destroy.....	189
8.3.5 SS_Timer_CleanUp.....	190
8.3.6 SS_Timer_Reset.....	190
8.3.7 SS_Timer_SetTickValue.....	191
8.3.8 SS_Timer_SetPeriodicAttr.....	192
8.3.9 SS_Timer_GetPastTime.....	193
8.4 数据类型.....	194
8.4.1 TD_TIMER_HANDLE.....	194
8.4.2 OT_TIMER_S.....	194
8.4.3 SS_TIMER_PROC.....	195
8.5 错误码.....	195
9 层次状态机.....	197
9.1 概述.....	197
9.2 架构原理.....	197
9.2.1 状态机模型.....	198
9.2.2 状态切换规则.....	198
9.3 API 参考.....	199
9.3.1 SS_HFSM_Create.....	199
9.3.2 SS_HFSM_Destroy.....	200
9.3.3 SS_HFSM_AddState.....	200
9.3.4 SS_HFSM_SetInitialState.....	201
9.3.5 SS_HFSM_GetCurrentState.....	202
9.3.6 SS_HFSM_Start.....	203
9.3.7 SS_HFSM_Stop.....	203
9.3.8 SS_HFSM_SendAsyncMessage.....	204
9.4 数据类型.....	205
9.4.1 OT_STATE_NAME_LEN.....	205
9.4.2 OT_STATE_MAX_AMOUNT.....	205
9.4.3 OT_PROCESS_MSG_RESULTE_OK.....	206
9.4.4 OT_PROCESS_MSG_UNHANDLER.....	206
9.4.5 OT_STATE_S.....	206
9.4.6 OT_HFSM_EVENT_E.....	207
9.4.7 OT_HFSM_UNHANDLED_MSG_INFO_S.....	208
9.4.8 OT_HFSM_TRANSTION_INFO_S.....	208
9.4.9 OT_HFSM_EVENT_INFO_S.....	209
9.4.10 OT_HFSM_EVENT_CALLBACK_FN.....	209
9.4.11 OT_HFSM_ATTR_S.....	210



9.4.12 OT_MESSAGE_S.....	211
9.5 错误码.....	211
10 事件路由.....	213
10.1 概述.....	213
10.2 架构原理.....	213
10.3 API 参考.....	214
10.3.1 SS_EVTHUB_Init.....	214
10.3.2 SS_EVTHUB_Deinit.....	215
10.3.3 SS_EVTHUB_Register.....	216
10.3.4 SS_EVTHUB_UnRegister.....	216
10.3.5 SS_EVTHUB_Publish.....	217
10.3.6 SS_EVTHUB_CreateSubscriber.....	218
10.3.7 SS_EVTHUB_DestroySubscriber.....	218
10.3.8 SS_EVTHUB_Subscribe.....	219
10.3.9 SS_EVTHUB_UnSubscribe.....	220
10.3.10 SS_EVTHUB_GetEventHistory.....	221
10.3.11 SS_EVTHUB_SetEnabled.....	221
10.3.12 SS_EVTHUB_GetEnabled.....	222
10.4 数据类型.....	223
10.4.1 EVENT_PAYLOAD_LEN.....	223
10.4.2 OT_EVTHUB_SUBSCRIBE_NAME_LEN.....	223
10.4.3 OT_EVTHUB_MESSAGEQURUR_MAX_SIZE.....	224
10.4.4 OT_EVENT_ID.....	224
10.4.5 OT_EVENT_S.....	224
10.4.6 OT_SUBSCRIBER_S.....	225
10.5 错误码.....	226
11 FSTool.....	227
11.1 概述.....	227
11.2 功能描述.....	228
11.2.1 检查文件系统.....	228
11.2.2 格式化文件系统.....	228
11.3 API 参考.....	228
11.3.1 SS_FSTOOL_Check.....	228
11.3.2 SS_FSTOOL_Format.....	229
11.3.3 SS_FSTOOL_SetLogLevel.....	230
11.3.4 SS_FSTOOL_RegisterBufOps.....	231
11.3.5 SS_FSTOOL_UnregisterBufOps.....	232
11.4 数据结构.....	232
11.4.1 OT_FSTOOL_CHECK_MODE_E.....	233
11.4.2 OT_DISK_LABEL_LEN.....	234
11.4.3 OT_FSTOOL_FORMAT_MODE_E.....	234
11.4.4 OT_FSTOOL_LOG_LEVEL_E.....	234



11.4.5 OT_FSTOOL_BUF_ALLOC_FN.....	235
11.4.6 OT_FSTOOL_BUF_FREE_FN.....	236
11.4.7 OT_FSTOOL_BUF_OPS_S.....	236
11.4.8 OT_FSTOOL_CHECK_CFG_S.....	237
11.4.9 OT_FSTOOL_FORMAT_CFG_S.....	237
11.5 错误码.....	238
11.6 典型场景使用指南.....	239
12 Storage.....	240
12.1 概述.....	240
12.2 功能描述.....	241
12.2.1 监听存储设备.....	241
12.2.2 获取存储设备信息.....	241
12.3 API 参考.....	241
12.3.1 SS_STORAGE_RegisterFSTool.....	241
12.3.2 SS_STORAGE_UnregisterFSTool.....	242
12.3.3 SS_STORAGE_Create.....	243
12.3.4 SS_STORAGE_Destroy.....	244
12.3.5 SS_STORAGE_GetState.....	244
12.3.6 SS_STORAGE_GetFSInfo.....	245
12.3.7 SS_STORAGE_GetDevInfo.....	246
12.3.8 SS_STORAGE_Format.....	247
12.3.9 SS_STORAGE_Pause.....	248
12.3.10 SS_STORAGE_Resume.....	249
12.4 数据结构.....	249
12.4.1 OT_STORAGE_PATH_LEN_MAX.....	250
12.4.2 OT_STORAGE_INST_CNT_MAX.....	250
12.4.3 OT_STORAGE_CFG_S.....	250
12.4.4 OT_STORAGE_STATE_E.....	251
12.4.5 OT_STORAGE_TRANSMISSION_SPEED_E.....	252
12.4.6 OT_STORAGE_DEV_INFO_S.....	253
12.4.7 OT_STORAGE_FS_INFO_S.....	254
12.4.8 OT_STORAGE_FSTOOL_ERR_E.....	254
12.4.9 OT_STORAGE_FS_TOOL_S.....	255
12.4.10 OT_STORAGE_ON_STATE_CHANGE_PFN.....	256
12.5 错误码.....	257
12.6 典型场景使用指南.....	257
12.7 Proc 调试.....	258
13 XMP 封装模块.....	259
13.1 概述.....	259
13.2 功能描述.....	259
13.2.1 基本概念.....	259
13.2.2 功能描述.....	259



13.2.3 上下文描述.....	259
13.3 API 参考.....	260
13.3.1 SS_XMP_Init.....	261
13.3.2 SS_XMP_DeInit.....	261
13.3.3 SS_XMP_Create.....	262
13.3.4 SS_XMP_Destroy.....	263
13.3.5 SS_XMP_GetProperty.....	264
13.3.6 SS_XMP_AddProperty.....	265
13.3.7 SS_XMP_ToString.....	266
13.3.8 SS_XMP_ForeachKeyValue.....	267
13.4 数据结构.....	268
13.4.1 TD_MW_PTR.....	269
13.4.2 OT_XMP_ATTR_S.....	269
13.4.3 OT_XMP_READER_ATTRS_S.....	270
13.4.4 OT_XMP_WRITER_ATTRS_S.....	270
13.4.5 OT_XMP_NS_S.....	271
13.4.6 OT_XMP_HANDLE_KEYVALUE_FN.....	271
13.4.7 OT_XMP_ATTR_TYPE_E.....	272
13.4.8 OT_XMP_MAX_KEY_PATH.....	272
13.4.9 OT_XMP_MAX_NAMESPACE_PATH.....	272
13.4.10 OT_XMP_MAX_NAMESPACE_NUM.....	273
13.5 错误码.....	273
13.6 典型场景使用指南.....	274
13.7 第三方库依赖说明.....	274
14 TS 封装模块.....	275
14.1 概述.....	275
14.2 功能描述.....	275
14.3 API 参考.....	275
14.3.1 SS_TS_Create.....	276
14.3.2 SS_TS_Destroy.....	276
14.3.3 SS_TS_CreateStream.....	277
14.3.4 SS_TS_DestroyAllStreams.....	278
14.3.5 SS_TS_WriteFrame.....	279
14.3.6 SS_TS_GetFileSize.....	279
14.4 数据类型.....	280
14.4.1 OT_TS_MAX_AMOUNT.....	281
14.4.2 OT_TS_MAX_STREAM_AMOUNT.....	281
14.4.3 OT_TS_MAX_FILE_NAME.....	281
14.4.4 OT_TS_VBUF_MAX_SIZE.....	281
14.4.5 OT_TS_PREALLOC_MAX_SIZE.....	282
14.4.6 OT_TS_STREAM_TYPE_E.....	282
14.4.7 OT_TS_CODECID_E.....	283



14.4.8 OT_TS_STREAM_INFO_S.....	283
14.4.9 OT_TS_FRAME_DATA_S.....	284
14.4.10 OT_TS_MUXER_CONFIG_S.....	284
14.4.11 OT_TS_DEMUXER_CONFIG_S.....	285
14.4.12 OT_TS_CONFIG_S.....	286
14.4.13 OT_TS_CONFIG_TYPE_E.....	286
14.5 错误码.....	287
15 DCF.....	288
15.1 概述.....	288
15.2 功能描述.....	288
15.2.1 文件命名.....	289
15.2.1.1 DCF 根目录.....	289
15.2.1.2 DCF 目录.....	289
15.2.1.3 DCF 文件.....	289
15.2.1.4 DCF 对象.....	289
15.2.1.5 DCF 组.....	290
15.2.2 文件检索、添加、删除.....	290
15.2.2.1 检索.....	291
15.2.2.2 添加.....	291
15.2.2.3 删除.....	291
15.3 API 参考.....	292
15.3.1 SS_DCF_WRAPPER_Init.....	293
15.3.2 SS_DCF_WRAPPER_DeInit.....	293
15.3.3 SS_DCF_WRAPPER_Create.....	294
15.3.4 SS_DCF_WRAPPER_Destroy.....	295
15.3.5 SS_DCF_WRAPPER_GetFirstGrp.....	296
15.3.6 SS_DCF_WRAPPER_GetLastGrp.....	297
15.3.7 SS_DCF_WRAPPER_GetNextGrp.....	297
15.3.8 SS_DCF_WRAPPER_GetPrevGrp.....	298
15.3.9 SS_DCF_WRAPPER_DelCurGrp.....	299
15.3.10 SS_DCF_WRAPPER_GetTotalGrpCnt.....	300
15.3.11 SS_DCF_WRAPPER_GetGrpCnt.....	301
15.3.12 SS_DCF_WRAPPER_GetCurGrpFirstObj.....	302
15.3.13 SS_DCF_WRAPPER_GetCurGrpLastObj.....	303
15.3.14 SS_DCF_WRAPPER_GetCurGrpNextObj.....	304
15.3.15 SS_DCF_WRAPPER_GetCurGrpPrevObj.....	304
15.3.16 SS_DCF_WRAPPER_GetNextObjList.....	305
15.3.17 SS_DCF_WRAPPER_GetPrevObjList.....	306
15.3.18 SS_DCF_WRAPPER_AddObj.....	308
15.3.19 SS_DCF_WRAPPER_DelObj.....	308
15.3.20 SS_DCF_WRAPPER_GetTotalObjCnt.....	309
15.3.21 SS_DCF_WRAPPER_GetObjCnt.....	310



15.3.22 SS_DCF_WRAPPER_GetNewFilePaths.....	311
15.4 数据结构.....	312
15.4.1 OT_DCF_WRAPPER_MAX_INSTANCE.....	313
15.4.2 OT_DCF_WRAPPER_MAX_OBJ_TYPE_CNT.....	313
15.4.3 OT_DCF_WRAPPER_MAX_FILE_CNT_IN_OBJ.....	313
15.4.4 OT_DCF_WRAPPER_MAX_FILE_PATH_LENGTH.....	314
15.4.5 OT_DCF_WRAPPER_MAX_ROOT_PATH_LENGTH.....	314
15.4.6 OT_DCF_WRAPPER_DIR_FREE_CHAR_LENGTH.....	314
15.4.7 OT_DCF_WRAPPER_FILE_FREE_CHAR_LENGTH.....	315
15.4.8 OT_DCF_WRAPPER_FILE_EXTENSION_LENGTH.....	315
15.4.9 OT_DCF_WRAPPER_INDEX.....	315
15.4.10 OT_DCF_WRAPPER_OBJ_S.....	316
15.4.11 OT_DCF_WRAPPER_OBJ_LIST_S.....	316
15.4.12 OT_DCF_WRAPPER_GRP_S.....	317
15.4.13 OT_DCF_WRAPPER_SCAN_STATE_E.....	318
15.4.14 OT_DCF_WRAPPER_FILE_TYPE_S.....	318
15.4.15 OT_DCF_WRAPPER_OBJ_TYPE_S.....	319
15.4.16 OT_DCF_WRAPPER_NAMERULE_CFG_S.....	320
15.4.17 OT_DCF_WRAPPER_ON_SCAN_EVENT_PFN.....	321
15.5 错误码.....	321
15.6 典型场景使用指南.....	323
15.7 Proc 调试.....	323
16 DNG 封装解封装模块.....	325
16.1 概述.....	325
16.2 功能描述.....	325
16.2.1 基本概念.....	325
16.2.2 功能描述.....	326
16.2.3 上下文描述.....	326
16.3 API 参考.....	327
16.3.1 SS_DNG_Create.....	327
16.3.2 SS_DNG_Destroy.....	328
16.3.3 SS_DNG_AddIFD.....	329
16.3.4 SS_DNG_GetImageInfo.....	330
16.3.5 SS_DNG_GetImageData.....	331
16.4 数据结构.....	331
16.4.1 OT_DNG_CONFIG_S.....	332
16.4.2 OT_DNG_MUXER_IFD_INFO_S.....	333
16.4.3 OT_DNG_MUXER_IFD0_INFO_S.....	334
16.4.4 OT_DNG_MUXER_RAWIFD_INFO_S.....	336
16.4.5 OT_DNG_MUXER_SCREENIFD_INFO_S.....	337
16.4.6 OT_DNG_MUXER_IMAGE_DATA_S.....	337
16.4.7 OT_DNG_MUXER_EXIF_INFO_S.....	338



16.4.8 OT_DNG_MUXER_GPS_INFO_S.....	340
16.4.9 OT_DNG_MUXER_IMAGE_DYNAMIC_INFO_S.....	340
16.4.10 OT_DNG_MUXER_IMAGE_STATIC_INFO_S.....	341
16.4.11 OT_DNG_MUXER_RAW_FORMAT_S.....	342
16.4.12 OT_DNG_MUXER_CFALAYOUT_TYPE_E.....	343
16.4.13 OT_DNG_MUXER_IMAGE_TYPE_E.....	343
16.4.14 OT_DNG_DEMUXER_IMAGE_INFO_S.....	344
16.4.15 OT_DNG_CONFIG_TYPE_E.....	345
16.4.16 DNG_MAX_FILE_NAME.....	345
16.4.17 DNG_MUXER_DCF_DESCRIPTION_LENGTH.....	346
16.4.18 DNG_MUXER_DATETIME_LENGTH.....	346
16.4.19 DNG_MUXER_PROFILEHUESATMAPDIM_LEN.....	346
16.4.20 DNG_MUXER_LENINFO_LEN.....	346
16.4.21 DNG_MUXER_MATRIX_SIZE.....	347
16.4.22 DNG_MUXER_GPSVER_LEN.....	347
16.4.23 DNG_MUXER_TUDE_LEN.....	347
16.4.24 DNG_MUXER_ISP_BAYER_CHN.....	348
16.4.25 DNG_MUXER_CFACOLORPLANE.....	348
16.4.26 DNG_MUXER_REPEAT_LEN.....	348
16.4.27 DNG_MUXER_SCALE_LEN.....	348
16.4.28 DNG_MUXER_NP_SIZE.....	349
16.4.29 DNG_MUXER_REF_BW.....	349
16.4.30 DNG_MUXER_CROP_LEN.....	349
16.5 错误码.....	350
16.6 典型场景使用指南.....	351
A 缩略语.....	352



插图目录

图 1-1 整体框图.....	1
图 1-2 模块图.....	2
图 1-3 各模块分类.....	3
图 1-4 基于中间件的扩展.....	4
图 2-1 录像模块上下文依赖示意图.....	5
图 2-2 录像、录制码流以及数据源层次关系示意图.....	6
图 2-3 延迟手动切分示意图.....	12
图 2-4 预录手动切分示意图.....	13
图 3-1 日期时间文件系统模块系统框图.....	47
图 3-2 文件命名规则.....	49
图 5-1 文件封装 MP4 组件软件上下文依赖关系.....	87
图 6-1 TAG 的类似信息.....	116
图 6-2 EXIF 模块上下文.....	117
图 6-3 EXIF 模块处理流程.....	118
图 7-1 播放器模块上下文.....	140
图 7-2 播放器模块层次结构图.....	141
图 7-3 播放器模块处理流程.....	142
图 7-4 播放器模块状态图.....	143
图 9-1 状态机模块系统框图.....	197
图 9-2 状态机内部状态组织形式框图.....	198
图 10-1 事件路由模块系统框图.....	213
图 11-1 FSTool 模块上下文.....	227
图 12-1 Storage 模块上下文.....	240
图 12-2 Storage 模块 Proc 信息.....	258
图 13-1 XMP 封装组件软件上下文关系图.....	260
图 14-1 TS 封装组件软件上下文依赖关系.....	275
图 15-1 DCF 模块上下文.....	288
图 15-2 DCF 目录和文件结构.....	290
图 15-3 各状态的切换图.....	292
图 15-4 DCF 模块 Proc 信息.....	324
图 16-1 DNG 文件基本组织结构图.....	326
图 16-2 DNG 组件上下文依赖关系.....	326



表格目录

表 2-1 错误码.....	44
表 3-1 文件类型.....	48
表 3-2 目录类型.....	48
表 3-3 目录类型与文件命名后缀关系.....	49
表 3-4 日期时间文件系统 API 错误码.....	64
表 4-1 媒体缓存模块 API 错误码.....	84
表 5-1 MP4 封装解封装 API 错误码.....	112
表 6-1 EIXF 错误码.....	135
表 7-1 播放器错误码.....	180
表 7-2 解封装错误码.....	182
表 8-1 定时器模块通用错误码.....	195
表 9-1 层次状态机 API 错误码.....	211
表 10-1 事件路由 API 错误码.....	226
表 11-1 FSTool API 错误码.....	238
表 12-1 Storage API 错误码.....	257
表 12-2 Storage Proc 信息.....	258
表 13-1 XMP 封装 API 错误码.....	273
表 14-1 TS 封装的 API 错误码.....	287
表 15-1 DCF 目录命名样例.....	289
表 15-2 DCF 文件命名样例.....	289
表 15-3 DCF API 错误码.....	321
表 15-4 DCF Proc 信息.....	323
表 16-1 DNG 封装 API 错误码.....	350



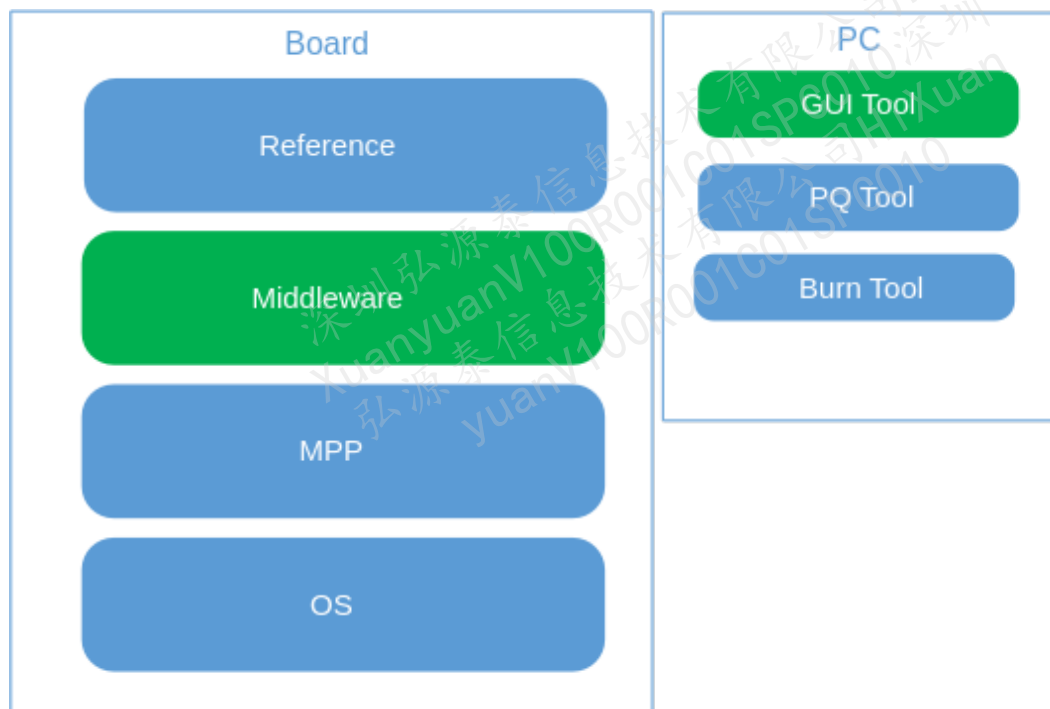
1 概述

1.1 中间件概述

1.1.1 在整个方案中的位置

作为公司监控消费类SDK，包含有下面几个层次，如图1-1所示。

图 1-1 整体框图



板端包含以下几个子系统：

- OS：提供基本的操作系统及基础的外围器件驱动。



- MPP：提供媒体采集、处理、编码功能。
- Middleware：基于业务的接口抽象层。
- Reference：具体的业务实现。

Middleware层在PC端提供UI设计时使用的GUI Builder。

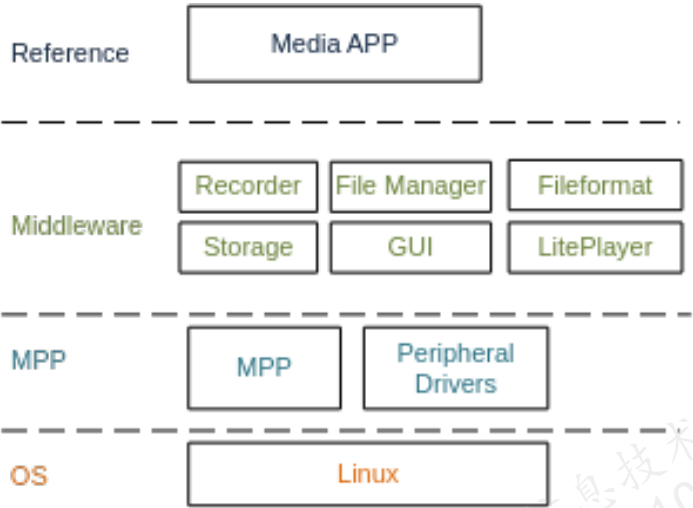
在各个子系统职能划分上，有以下两点约束：

- 为了保持中间件的平台无关性，由MPP层将解决方案差异屏蔽掉。
- 为了保证中间件的通用性，有关特定应用场景的需求放在Reference层处理。

1.1.2 模块划分

模块划分如图1-2所示。

图 1-2 模块图



1.2 中间件架构

1.2.1 支持的功能列表

中间件包括开源和非开源两部分功能，本文档仅针对非开源部分功能进行说明。

- 录像
提供基本的录像功能，将音视频码流录制成特定命名、特定封装格式的视频文件。
- 文件管理
提供常用的日期时间的文件管理，检查和格式化文件系统以及存储设备状态监听的功能。
- 文件封装
提供常用视频图片格式的封装解封装功能。
- GUI



提供基本的UI控件。

- 本地文件回放
提供本地文件回放功能，支持基本的播控操作。

须知

中间件仅支持单进程的功能实现，除个别说明外，不支持中间件库多进程同时使用。

1.2.2 模块详细说明

图 1-3 各模块分类



中间件主要包含有三大模块（如图1-3所示），图形、媒体、存储。

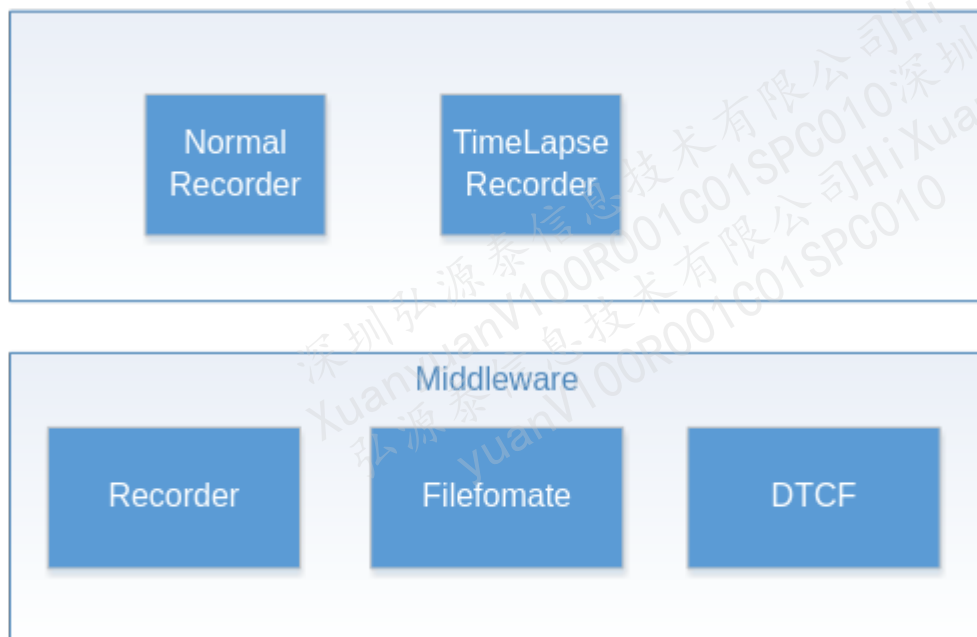
- 图形
提供基本的图形界面控件和开发工具。
- 媒体
提供基础录像功能；提供本地文件回放功能；提供图片视频文件的封装功能。
- 存储
提供文件管理相关接口和功能实现（时间日期文件系统及私有扩展）。

1.2.3 产品层架构图说明

如前所述，中间件是针对摄像头类产品核心功能进行开发，客户只需将各个功能挑出来进行合理的“组装”即可快速完成相关产品类型的软件开发。中间件提供录像、文件回放等基础功能，允许Reference层或者客户对基础功能进行扩展组合，做出差异化的功能。以录像为例，提供给产品层及客户更多的自由度，如图1-4所示。



图 1-4 基于中间件的扩展

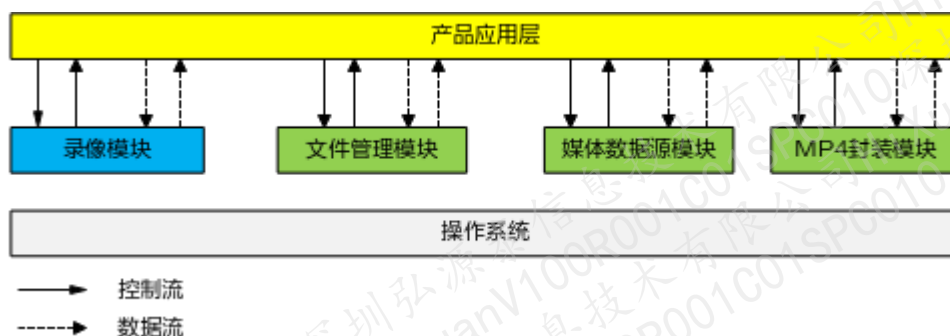


2 录像模块

2.1 概述

录像模块首先负责将外部写入的音视频数据写入录像内部缓存，其次按照用户指定的录制方式以及指定的录像文件命名，使用文件封装器将缓存的音视频数据封装成指定格式的视频文件写入存储介质。录像模块在整个系统中的位置，如图2-1蓝色部分所示。

图 2-1 录像模块上下文依赖示意图



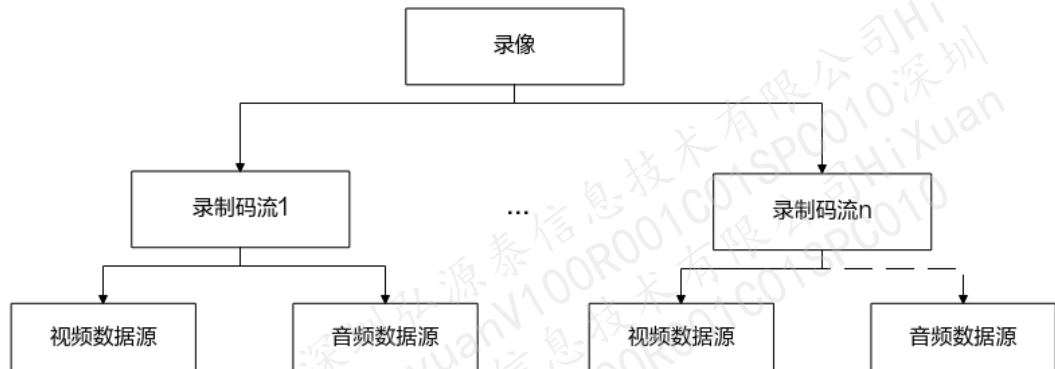
从图2-1可以看出，录像模块与文件管理模块、媒体数据源模块以及MP4封装模块互相解耦合。通过产品层注册回调的方式完成录制的功能。

2.1.1 基本概念

2.1.1.1 录像

一路录像包括若干路录制码流、录制文件名请求操作、录像类型以及切分类型等。录像内部数据层次关系如图2-2所示。

图 2-2 录像、录制码流以及数据源层次关系示意图



2.1.1.2 录制码流

一路录制码流包括一组数据源，以及对应的文件封装操作集合。

2.1.1.3 录像切分

启动录像后，可以按照给定的切分策略对录制码流进行切分，例如按照时间切分，就是指在录制过程中，当文件时长达到指定值后，就进行切分操作，开始下一个文件的录制。

2.1.1.4 录制类型

从数据源的连续性角度，录制类型包括普通录像、缩时录像两种类型。

- 普通录像模式下，录像启动一次数据源，数据源以一定的帧率连续不断的输出编码数据到录像模块，直到收到用户停止录像的命令；
- 缩时录像模式下，录像以一定的时间间隔周期性的请求数据源产生一帧数据，并写入录像模块。

2.2 功能描述

2.2.1 录像

一路录像负责将若干路码流录制成相应的录像文件，包括从外部模块获取各路码流对应的录制文件名，同时控制各路码流的启动，停止以及切分等操作，此外将录像内部的事件上报到应用层。

2.2.2 录制码流

一路录制码流录制首先需要启动码流包含的数据源的启动，将包含的数据源封装到媒体文件中，当到达数据切分点之后完成文件的切分。

2.3 API 参考

录像模块主要提供录像的创建和销毁、录像的启动、停止、手动切分以及数据写入，录像事件回调的注册。该功能模块提供以下API：



- **SS_REC_Create**: 创建录像。
- **SS_REC_Destroy**: 销毁录像。
- **SS_REC_SetParam**: 设置录像参数。
- **SS_REC_Start**: 启动录像。
- **SS_REC_Stop**: 停止录像。
- **SS_REC_ManualSplit**: 手动切分录像。
- **SS_REC_WriteData**: 写入帧数据到录像缓存。
- **SS_REC_RegisterEventCallback**: 注册录像事件处理回调。

2.3.1 SS_REC_Create

【描述】

创建录像。

【语法】

```
TD_S32 SS_REC_Create(const OT_REC_ATTR_S * pstRecAttr, TD_MW_PTR* ppRecorder);
```

【参数】

参数名称	描述	输入/输出
pstRecAttr	录像属性。	输入
ppRecorder	录像句柄指针。	输出

【返回值】

返回值	描述
0	成功。
非0	失败，参见 错误码 。

【需求】

- 头文件: ss_recorder_pro.h、ot_type.h、ot_track_source.h
- 库文件: libreorder_pro、libottimer

【芯片差异】

无

【注意】

- 每调用一次接口，创建一个录像句柄，目前最多支持创建6个录像句柄。
- 每路录像支持管理一路或多路码流的录制。
- 录像统一控制各路录制码流的启停以及切分。



- 各路录制码流采用统一的切分策略以及录像类型进行录制。

【举例】

参看sample_recorder_pro.c。

【相关主题】

无

2.3.2 SS_REC_Destroy

【描述】

销毁录像。

【语法】

```
TD_S32 SS_REC_Destroy(TD_MW_PTR pRecorder);
```

【参数】

参数名称	描述	输入/输出
pRecorder	录像指针。	输入

【返回值】

返回值	描述
0	成功。
非0	失败，参见 错误码 。

【需求】

- 头文件：ss_recorder_pro.h、ot_type.h、ot_track_source.h
- 库文件：librecorder_pro、libottimer

【注意】

对于已启动的录像，必须先停止再销毁。

【举例】

参看sample_recorder_pro.c。

【相关主题】

无。

2.3.3 SS_REC_SetParam

【描述】

设置录像参数。



【语法】

```
TD_S32 SS_REC_SetParam(TD_MW_PTR pRecorder, const OT_REC_PARAM *recParam);
```

【参数】

参数名称	描述	输入/输出
pRecorder	录像指针。	输入
recParam	录像参数。	输入

【返回值】

返回值	描述
0	成功。
非0	失败，参见 错误码 。

【需求】

- 头文件：ss_recorder_pro.h、ot_type.h、ot_track_source.h
- 库文件：librecorder_pro、libottimer

【芯片差异】

无

【注意】

该接口需要在启动录像前调用。

【举例】

参看sample_recorder_pro.c。

【相关主题】

无

2.3.4 SS_REC_Start

【描述】

启动录像。

【语法】

```
TD_S32 SS_REC_Start(TD_MW_PTR pRecorder);
```

【参数】



参数名称	描述	输入/输出
pRecorder	录像指针。	输入

【返回值】

返回值	描述
0	成功。
非0	失败，参见 错误码 。

【需求】

- 头文件：ss_recorder_pro.h、ot_type.h、ot_track_source.h
- 库文件：librecorder_pro、libottimer

【注意】

- 该接口是异步接口，当上报事件OT_REC_EVENT_START后才说明录像已启动完毕。
- 该接口通常在录像未开始或者已停止后调用，如果启动成功，录像状态发生变更会上报OT_REC_EVENT_START事件。
- 在录像已成功启动情况下调用，接口直接返回成功，由于录像状态未发生变更，因此不再上报OT_REC_EVENT_START事件。

【举例】

参看sample_recorder_pro.c。

【相关主题】

无。

2.3.5 SS_REC_Stop

【描述】

停止录像。

【语法】

```
TD_S32 SS_REC_Stop(TD_MW_PTR pRecorder, TD_BOOL bQuickMode);
```

【参数】

参数名称	描述	输入/输出
pRecorder	录像指针。	输入
bQuickMode	录像停止模式。	输入



【返回值】

返回值	描述
0	成功。
非0	失败，参见 错误码 。

【需求】

- 头文件：ss_recorder_pro.h、ot_type.h、ot_track_source.h
- 库文件：librecorder_pro、libottimer

【注意】

- 该接口是异步接口，当上报OT_REC_EVENT_STOP表示录像已停止，此时才可以再次调用[SS_REC_Start](#)。
- 该接口通常在录像已成功启动前提下或者录像发生错误后调用，状态发生改变后上报OT_REC_EVENT_STOP事件。
- 当录像处于已停止状态后，直接返回成功，不再上报OT_REC_EVENT_STOP事件。
- 当bQuickMode为TD_TRUE时，无论录像缓存是否存在未写入文件的数据，都会立即停止录像；当bQuickMode为TD_FALSE时停止录像，会将停止接口调用之前已缓存到录像的数据都写入文件才停止录像。

【举例】

参看sample_recorder_pro.c。

【相关主题】

无。

2.3.6 SS_REC_ManualSplit

【描述】

手动切分录像。

【语法】

```
TD_S32 SS_REC_ManualSplit(TD_MW_PTR pRecorder, const OT\_REC\_MANUAL\_SPLIT\_ATTR\_S\* pstSplitAttr);
```

【参数】

参数名称	描述	输入/输出
pRecorder	录像指针。	输入
pstSplitAttr	手动切分属性。	输入

【返回值】



返回值	描述
0	成功。
非0	失败，参见 错误码 。

【需求】

- 头文件：ss_recorder_pro.h、ot_type.h、ot_track_source.h
- 库文件：librecorder_pro、libottimer

【注意】

- 该接口只对普通录像有效，只在录制过程中可以调用。
- 调用该接口后，录像将按照手动切分类型进行文件切分，待手动切分结束后继续按照录像初始的切分类型进行切分。
- 调用该接口之后，需要等待上报 OT_REC_EVENT_NEW_MANUAL_SPLIT_FILE_END 事件之后，才可以再次调用该接口。
- 当执行手动切分之后，行为如[图2-3](#)和[图2-4](#)所示，其中MFile为手动切分文件，File为普通文件。
 - 当切分类型为OT_REC_POST_MANUAL_SPLIT时，手动切分的行为如[图2-3](#)所示。
 - 当切分类型为OT_REC_PRE_MANUAL_SPLIT时，手动切分行为如[图2-4](#)所示。
- 当切分类型为OT_REC_PRE_MANUAL_SPLIT时，通常情况下的行为是立即切分当前录制的文件，然后开始新文件的录制。在启动录像的同时触发预录的手动切分，由于多路码流存在是否已经录制，因此多路码流行为会有不定性。建议应用层启动录像和启动紧急录像之间需要有一个时间差，建议1个gop的视频时间，例如1s时间。
- 当切分类型为TD_REC_PRE_MANUAL_SPLIT时，原理是从当前视频缓存中最新的I帧向前寻找u32PreRecTimeSec秒数据附近的I帧开始录制，因此预录文件中预录的数据量会有误差，主要有两种情况：
 - 启动录像时间过短，当缓存中数据不够u32PreRecTimeSec的数据时，预录文件中预录的数据会小于u32PreRecTimeSec秒的数据量。
 - 由于起始录制点必须是从I帧开始，手动切分发生在临界时间点或者非预录缓存区过小（ $\text{bufferTimeMSec} / 1000$ 与 u32PreRecTimeSec 差值过小）均会影响预录数据的起始位置，因此预录数据量可能会有1个GOP的时间的误差。

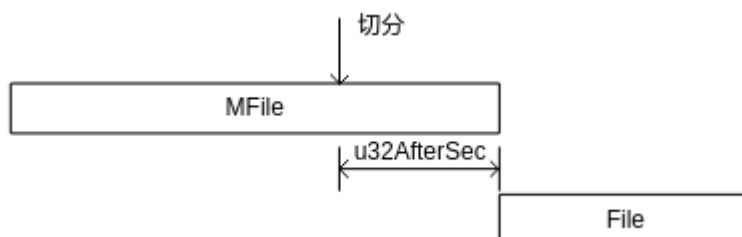
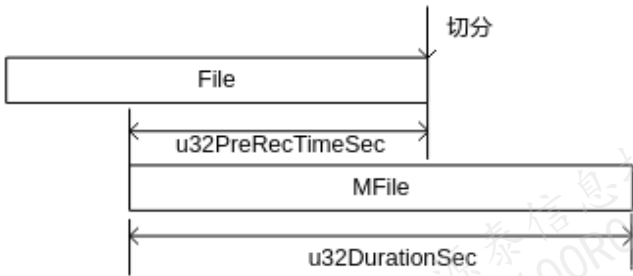
图 2-3 延迟手动切分示意图



图 2-4 预录手动切分示意图



【举例】

参看sample_recorder_pro.c。

【相关主题】

无。

2.3.7 SS_REC_WriteData

【描述】

写帧数据到录像缓存。

【语法】

TD_S32 SS_REC_WriteData(TD_MW_PTR pRecorder, OT_Track_Source_Handle pTrackSrcHandle, const OT_REC_FRAME_DATA_S* pstFrameData);

【参数】

参数名称	描述	输入/输出
pRecorder	录像指针。	输入
pTrackSrcHandle	帧数据对应的数据源句柄。	输入
pstFrameData	录像帧数据指针。	输入

【返回值】

返回值	描述
0	成功。
非0	失败，参见错误码。

【需求】

- 头文件：ss_recorder_pro.h、ot_type.h、ot_track_source.h
- 库文件：librecorder_pro、libottimer



【注意】

录像启动成功后调用，数据的有效性由调用者保证。

【举例】

参看sample_recorder_pro.c。

【相关主题】

无

2.3.8 SS_REC_RegisterEventCallback

【描述】

注册录像事件处理回调。

【语法】

```
TD_S32 SS_REC_RegisterEventCallback(TD_MW_PTR pRecorder, OT_REC_EVENT_CALLBACK_FN  
pfnEventCallback);
```

【参数】

参数名称	描述	输入/输出
pRecorder	录像指针。	输入
pfnEventCallback	录像事件处理回调函数。	输入

【返回值】

返回值	描述
0	成功。
非0	失败，参见 错误码 。

【芯片差异】

无

【需求】

- 头文件：ss_recorder_pro.h、ot_type.h、ot_track_source.h
- 库文件：librecorder_pro、libottimer

【注意】

在录像创建后，录像启动前调用。

【举例】

参看sample_recorder_pro.c。

【相关主题】



无。

2.4 数据类型

录像模块主要提供以下数据类型：

- **OT_REC_TYPE_E**：定义录像类型。
- **OT_REC_NORMAL_ATTR_S**：定义普通录像属性。
- **OT_REC_LAPSE_ATTR_S**：定义缩时录像属性。
- **OT_REC_SPLIT_TYPE_E**：定义文件切分类型。
- **OT_REC_SPLIT_ATTR_S**：定义文件切分属性。
- **OT_REC_EVENT_E**：定义录像事件类型。
- **OT_REC_STATE_EVENT_INFO_S**：定义录像状态事件信息。
- **OT_REC_FILE_EVENT_INFO_S**：定义录像文件事件信息。
- **OT_REC_ERROR_EVENT_INFO_S**：定义录像错误事件信息。
- **OT_REC_EVENT_INFO_S**：定义录像事件信息。
- **OT_REC_FRAME_DATA_S**：定义录像帧数据。
- **OT_REC_MUXER_OPERATE_FN_S**：定义码流封装函数集。
- **OT_REC_BUF_ABSTRACTFUNC_S**：定义码流缓存操作集。
- **OT_REC_STREAM_ATTR_S**：定义录制码流属性。
- **OT_REC_CALLBACK_S**：定义录像回调函数集。
- **OT_REC_ATTR_S**：定义录像属性。
- **OT_REC_PARAM**：定义录像参数。
- **OT_REC_MANUAL_SPLIT_TYPE_E**：定义录像手动切分类型。
- **OT_REC_POST_MANUAL_SPLIT_ATTR_S**：定义延迟手动切分属性。
- **OT_REC_PRE_MANUAL_SPLIT_ATTR_S**：定义预录手动切分属性。
- **OT_REC_NORMAL_MANUAL_SPLIT_ATTR_S**：定义普通预录手动切分属性。
- **OT_REC_EXTEND_DURATION_SPLIT_ATTR_S**：定义延长时长切分属性。
- **OT_REC_MANUAL_SPLIT_ATTR_S**：定义录像手动切分属性。
- **OT_REC_EVENT_CALLBACK_FN**：定义录像事件回调函数。
- **OT_REC_REQUEST_FILE_NAMES_FN**：定义录像文件名请求回调函数。
- **OT_REC_CREATE_MUXER_FN**：定义创建封装器函数。
- **OT_REC_DESTROY_MUXER_FN**：定义封装器销毁回调函数。
- **OT_REC_CREATE_TRACK_FN**：定义封装器音/视轨创建回调函数。
- **OT_REC_WRITE_FRAME_FN**：定义码流封装器写帧回调。
- **OT_REC_BUF_ALLOC_FN**：定义录制码流缓存申请回调函数。
- **OT_REC_BUF_FREE_FN**：定义录制码流缓存释放回调函数。
- **OT_REC_STREAM_MAX_CNT**：定义单路录像支持最大码流数目。
- **OT_REC_TRACK_MAX_CNT**：定义单路码流支持最大媒体轨道数目。
- **OT_REC_FILE_NAME_LEN**：定义录像最大文件名长度。



- **OT_Track_SourceType_E**: 定义数据源类型。
- **OT_Track_VideoCodec_E**: 定义视频编码类型。
- **OT_Track_AudioCodec_E**: 定义音频编码类型。
- **OT_Track_VideoSourceInfo_S**: 定义视频数据源信息。
- **OT_Track_AudioSourceInfo_S**: 定义音频数据源信息。
- **OT_Track_PrivateSourceInfo_S**: 定义私有数据源信息。
- **Track_Source_S**: 定义数据源信息。
- **OT_Track_Source_Handle**: 定义数据源句柄。
- **SS_Track_Source_Start_FN**: 定义启动数据源回调。
- **SS_Track_Source_Stop_FN**: 定义停止数据源回调。

2.4.1 OT_REC_TYPE_E

【说明】

定义录像类型。

【定义】

```
typedef enum {  
    OT_REC_TYPE_NORMAL = 0,  
    OT_REC_TYPE_LAPSE,  
    OT_REC_TYPE_BUTT  
} OT_REC_TYPE_E;
```

【成员】

成员名称	描述
OT_REC_TYPE_NORMAL	普通录像。
OT_REC_TYPE_LAPSE	缩时录像。
OT_REC_TYPE_BUTT	非法类型。

【注意事项】

无。

【相关数据类型及接口】

无。

2.4.2 OT_REC_NORMAL_ATTR_S

【说明】

定义普通录像属性。

【定义】

```
typedef struct {  
    TD_U32 u32Rsv;  
} OT_REC_NORMAL_ATTR_S;
```




【成员】

成员名称	描述
u32Rsv	此值预留

【注意事项】

无。

【相关数据类型及接口】

无。

2.4.3 OT_REC_LAPSE_ATTR_S

【说明】

定义缩时录像属性。

【定义】

```
typedef struct {  
    TD_U32 u32IntervalMs;  
} OT_REC_LAPSE_ATTR_S;
```

【成员】

成员名称	描述
u32IntervalMs	缩时录像帧请求时间间隔，单位：ms。

【注意事项】

帧请求间隔最小值为200 ms，最大值为15*24*60*60*1000 ms（15天）。

【相关数据类型及接口】

无。

2.4.4 OT_REC_SPLIT_TYPE_E

【说明】

定义文件切分类型。

【定义】

```
typedef enum {  
    OT_REC_SPLIT_TYPE_NONE = 0,  
    OT_REC_SPLIT_TYPE_TIME,  
    OT_REC_SPLIT_TYPE_BUTT  
} OT_REC_SPLIT_TYPE_E;
```

【成员】



成员名称	描述
OT_REC_SPLIT_TYPE_NONE	不切分。
OT_REC_SPLIT_TYPE_TIME	按文件时长切分。
OT_REC_SPLIT_TYPE_BUTT	非法切分类型。

【注意事项】

- 不切分类型，录像文件切分不受时长限制，只有当文件达到录像模块支持的最大文件大小后进行文件切分，其中最大文件大小是切分属性 (OT_REC_SPLIT_ATTR_S) 里文件系统所支持的最大文件大小 (maxFileSizeGB)。
- 按时长切分是指录制的文件时长，假如有一路录制码流先达到3G但还未达到设置的文件时长，此时也会强制进行切分操作。

【相关数据类型及接口】

无。

2.4.5 OT_REC_SPLIT_ATTR_S

【说明】

定义文件切分属性。

【定义】

```
typedef struct {  
    OT_REC_SPLIT_TYPE_E enSplitType;  
    TD_U32 u32SplitTimeLenSec;  
    TD_U64 maxFileSizeGB;  
} OT_REC_SPLIT_ATTR_S;
```

【成员】

成员名称	描述
enSplitType	录像文件切分类型。
u32SplitTimeLenSec	文件切分时长，单位：s。
maxFileSizeGB	文件系统所支持的最大文件大小，单位：GB。

【注意事项】

- 切分类型为OT_REC_SPLIT_TYPE_NONE时，文件切分不受切分时长限制，只受 maxFileSizeGB限制。
- 由于每个文件的第一帧必须是视频关键帧，在切分时，当达到切分点后，直到遇到关键帧才进行实际的文件切分。因此u32SplitTimeLenSec的最小取值应该是两个关键帧之间的间隔即就是一个GOP所占的时间。基于切分的原理，u32SplitTimeLenSec必须是录像中各个录制码流关键帧时间间隔的公倍数。该参



数的合理性由调用者负责，设置不合理，会影响各录制码流时长的一致性以及切分的同步。文件内容时长误差 $\pm$ （GOP/帧率），单位秒（s）。

- 对于缩时录像来说，由于视频帧请求间隔一致，所以相同录制时间内各路码流产生的帧数目一致，封装帧率决定视频文件的播放时长，而GOP决定切分点的PTS，为保持切分的同步性以及录制文件时长的一致性，必须要保证各路码流保持相同的帧率，同时保持相同的GOP，否则可能会导致丢帧或者切分文件时长不一致的问题。
- u32SplitTimeLenSec最大值为15*24*60*60秒（15天）。

【相关数据类型及接口】

- OT_REC_TYPE_E
- OT_REC_SPLIT_TYPE_E

2.4.6 OT_REC_EVENT_E

【说明】

定义录像事件类型。

【定义】

```
typedef enum {  
    OT_REC_EVENT_START = 0,  
    OT_REC_EVENT_STOP,  
    OT_REC_EVENT_NEW_FILE_BEGIN,  
    OT_REC_EVENT_NEW_FILE_END,  
    OT_REC_EVENT_NEW_MANUAL_SPLIT_FILE_END,  
    OT_REC_EVENT_ERR_CREATE_FILE_FAIL,  
    OT_REC_EVENT_ERR_WRITE_FILE_FAIL,  
    OT_REC_EVENT_ERR_CLOSE_FILE_FAIL,  
    OT_REC_EVENT_ERR_READ_DATA_ERROR,  
    OT_REC_EVENT_ERR_INTERNAL_OPERATION_FAIL,  
    OT_REC_EVENT_BUTT  
} OT_REC_EVENT_E;
```

【成员】

成员名称	描述
OT_REC_EVENT_START	录像开始状态事件。
OT_REC_EVENT_STOP	录像结束状态事件。
OT_REC_EVENT_NEW_FILE_BEGIN	新文件开始事件。
OT_REC_EVENT_NEW_FILE_END	新文件结束事件。
OT_REC_EVENT_NEW_MANUAL_SPLIT_FILE_END	手动切分文件结束事件。
OT_REC_EVENT_ERR_CREATE_FILE_FAIL	文件创建失败事件。
OT_REC_EVENT_ERR_WRITE_FILE_FAIL	写文件失败事件。
OT_REC_EVENT_ERR_CLOSE_FILE_FAIL	关闭文件失败事件。
OT_REC_EVENT_ERR_READ_DATA_ERROR	读数据错误事件。



成员名称	描述
OT_REC_EVENT_ERR_INTERNAL_OPERATION_FAIL	内部异常错误事件。
OT_REC_EVENT_BUTT	非法类型。

【 注意事项 】

- 录像状态事件OT_REC_EVENT_START、OT_REC_EVENT_STOP相关联的信息为[OT_REC_STATE_EVENT_INFO_S](#)。
- 录像文件状态事件OT_REC_EVENT_NEW_FILE_BEGIN、OT_REC_EVENT_NEW_FILE_END以及OT_REC_EVENT_NEW_MANUAL_SPLIT_FILE_END相关联的事件信息为[OT_REC_FILE_EVENT_INFO_S](#)。
- 录像错误事件OT_REC_EVENT_ERR_CREATE_FILE_FAIL、OT_REC_EVENT_ERR_WRITE_FILE_FAIL、OT_REC_EVENT_ERR_CLOSE_FILE_FAIL、OT_REC_EVENT_ERR_READ_DATA_ERROR、OT_REC_EVENT_ERR_INTERNAL_OPERATION_FAIL相关联的事件信息为[OT_REC_ERROR_EVENT_INFO_S](#)。
- 当收到错误事件之后，需要调用[SS_REC_Stop](#)停止录像。

【 相关数据类型及接口 】

无。

2.4.7 OT_REC_STATE_EVENT_INFO_S

【 说明 】

定义录像状态事件信息。

【 定义 】

```
typedef struct {  
    TD_S32 s32Rsv;  
} OT_REC_STATE_EVENT_INFO_S;
```

【 成员 】

成员名称	描述
s32Rsv	该值预留。

【 注意事项 】

无

【 相关数据类型及接口 】

[OT_REC_EVENT_E](#)



2.4.8 OT_REC_FILE_EVENT_INFO_S

【说明】

定义录像文件事件信息。

【定义】

```
typedef struct {  
    TD_CHAR aszFileName[OT_REC_FILE_NAME_LEN];  
    TD_MW_PTR pMuxer;  
} OT_REC_FILE_EVENT_INFO_S;
```

【成员】

成员名称	描述
aszFileName	事件对应的文件名。
pMuxer	事件对应的封装器句柄。

【注意事项】

- **OT_REC_FILE_NAME_LEN**大小为256。
- hMuxerHdl只在OT_REC_EVENT_NEW_FILE_BEGIN时为有效句柄。

【相关数据类型及接口】

OT_REC_EVENT_E

2.4.9 OT_REC_ERROR_EVENT_INFO_S

【说明】

定义录像错误事件信息。

【定义】

```
typedef struct {  
    TD_CHAR aszFileName[OT_REC_FILE_NAME_LEN];  
    TD_S32 s32ErrorCode;  
} OT_REC_ERROR_EVENT_INFO_S;
```

【成员】

成员名称	描述
aszFileName	错误事件对应录制文件名。
s32ErrorCode	错误事件携带的错误码。

【注意事项】

无

【相关数据类型及接口】



OT_REC_EVENT_E

2.4.10 OT_REC_EVENT_INFO_S

【说明】

定义录像事件信息。

【定义】

```
typedef struct {
    OT_REC_EVENT_E enEventCode;
    union
    {
        OT_REC_STATE_EVENT_INFO_S stStateInfo;
        OT_REC_FILE_EVENT_INFO_S stFileInfo;
        OT_REC_ERROR_EVENT_INFO_S stErrorInfo;
    } unEventInfo;
} OT_REC_EVENT_INFO_S;
```

【成员】

成员名称	描述
enEventCode	事件类型。
unEventInfo	事件附属信息联合体。

【注意事项】

无

【相关数据类型及接口】

- OT_REC_EVENT_E
- OT_REC_STATE_EVENT_INFO_S
- OT_REC_FILE_EVENT_INFO_S
- OT_REC_ERROR_EVENT_INFO_S

2.4.11 OT_REC_FRAME_DATA_S

【说明】

定义帧数据。

【定义】

```
typedef struct {
    TD_U64 u64TimeStamp;
    TD_U8* apu8SliceAdd[OT_REC_FRAME_MAX_SLICE];
    TD_U32 au32SliceLen[OT_REC_FRAME_MAX_SLICE];
    TD_BOOL bKeyFrameFlag;
    TD_U32 u32SliceCnt;
} OT_REC_FRAME_DATA_S;
```

【成员】



成员名称	描述
u64TimeStamp	帧时间戳
apu8SliceAdd	一帧数据分段存储地址数组
au32SliceLen	一帧数据分段存储长度数组
bKeyFrameFlag	是否关键帧标识
u32SliceCnt	一帧数据分为几段地址存储

【注意事项】

- 音频数据没有关键帧概念，因此每一帧bKeyFrameFlag均为TD_FALSE。
- 一帧数据最大支持分OT_REC_FRAME_MAX_SLICE段存储。
- apu8SliceAdd 与au32SliceLen的数据按照顺序一一对应。

【相关数据类型及接口】

- [OT_REC_EVENT_E](#)
- [OT_REC_STATE_EVENT_INFO_S](#)
- [OT_REC_FILE_EVENT_INFO_S](#)
- [OT_REC_ERROR_EVENT_INFO_S](#)

2.4.12 OT_REC_MUXER_OPERATE_FN_S

【说明】

定义码流封装函数集。

【定义】

```
typedef struct {  
    OT_REC_CREATE_MUXER_FN pfnCreateMuxer;  
    OT_REC_DESTROY_MUXER_FN pfnDestroyMuxer;  
    OT_REC_CREATE_TRACK_FN pfnCreateTrack;  
    OT_REC_WRITE_FRAME_FN pfnWriteFrame;  
    TD_VOID* pMuxerCfg;  
} OT_REC_MUXER_OPERATE_FN_S;
```

【成员】

成员名称	描述
pfnCreateMuxer	创建封装器回调函数。
pfnDestroyMuxer	销毁封装器回调函数。
pfnCreateTrack	创建音/视轨回调函数。
pfnWriteFrame	封装器写数据回调函数。
pMuxerCfg	封装器私有配置指针。



【注意事项】

pMuxerCfg 封装私有配置属性指针，指向的内存由调用者管理，录像内部会通过封装器创建回调函数pfnCreateMuxer回传到上层。

【相关数据类型及接口】

- OT_REC_CREATE_MUXER_FN
- OT_REC_DESTROY_MUXER_FN
- OT_REC_CREATE_TRACK_FN
- OT_REC_WRITE_FRAME_FN

2.4.13 OT_REC_BUF_ABSTRACTFUNC_S

【说明】

定义码流缓存操作集。

【定义】

```
typedef struct {  
    OT_REC_BUF_ALLOC_FN pfnBufAlloc;  
    OT_REC_BUF_FREE_FN pfnBufFree;  
    TD_VOID *pBufArg;  
} OT_REC_BUF_ABSTRACTFUNC_S;
```

【成员】

成员名称	描述
pfnBufAlloc	码流缓存申请回调函数。
pfnBufFree	码流缓存释放回调函数。
pBufArg	缓存私有参数指针。

【注意事项】

- pBufArg 缓存私有参数指针，指向的内存由调用者管理，录像内部会通过回调函数pfnBufAlloc和pfnBufFree回传到上层。
- 用户可以根据需要，通过实现pfnBufAlloc和pfnBufFree申请系统内存或者MMZ内存作为录制码流的缓存，需要注意pfnBufAlloc和pfnBufFree的实现要对称，且需要保证实现正确性，否则会引起内存泄露或者其他未知错误。

【相关数据类型及接口】

- OT_REC_BUF_ALLOC_FN
- OT_REC_BUF_FREE_FN

2.4.14 OT_REC_STREAM_ATTR_S

【说明】

定义录制码流属性。

【定义】



```
typedef struct {
    TD_U32 u32TrackCnt;
    TD_U32 bufferTimeMSec;
    OT_Track_Source_Handle aHTrackSrcHandle[OT_REC_TRACK_MAX_CNT];
    OT_REC_MUXER_OPERATE_FN_S stMuxerOps;
    OT_REC_BUF_ABSTRACTFUNC_S stBufOps;
} OT_REC_STREAM_ATTR_S;
```

【成员】

成员名称	描述
u32TrackCnt	码流音/视轨数目。
bufferTimeMSec	码流最大缓存时长（单位：毫秒）。
aHTrackSrcHandle	码流音/视轨数据源句柄数组。
stMuxerOps	码流封装函数操作集。
stBufOps	录制码流缓存操作集。

【注意事项】

- u32TrackCnt码流轨道数目最大为OT_REC_TRACK_MAX_CNT个，即一个音频轨道，一个视频轨道，一个私有数据轨道。当前不支持私有数据轨道。
- aHTrackSrcHandle码流轨道数据源句柄数组，数组中的数据源句柄内存由调用者在录像创建前申请，并确保在录像销毁前一直持有。
- 普通录像支持u32TrackCnt为1或者2，即可以只包含视轨数据源，也可以是视轨数据源加音轨数据源组合方式。
- 缩时录像只支持u32TrackCnt为1，即支持单独视频录制。
- 数据源句柄在数组中存储，需要调用者确保顺序存储，即当u32TrackCnt为1时，默认只有第一个数据源有效，u32TrackCnt为2时，默认前两个数据源有效。
- 无论缩时录像还是普通录像中数据源帧率都是指目的帧率，即就是录制文件的帧率。
- 数据缓存时间bufferTimeMSec为该路码流缓存的数据时长，最小为1000毫秒，最大值为1000*1024*1024*1024/（视频码率/8）毫秒。该值关系到数据缓存大小，受限于系统可用内存，在有效值范围内该值设置过大可能会导致录像创建失败或运行时系统内存不足导致进程被杀掉，而设置过小会导致丢帧，该值设置需要综合系统可用内存和介质写速率。
- 单路码流录制视频所需的缓存大小约为（数据缓存时间*视频码率）/ 8（单位：字节），音频所需缓存大小约为（数据缓存时间*音频采样率*采样位宽*声道数）/ 8（单位：字节）。
- 录像可预录的时间u32PreRecTimeSec小于录像数据缓存时长bufferTimeMSec / 1000秒。
- 数据缓存需要至少缓存1个GOP的帧数据，因此bufferTimeMSec必须要大于等于1000*GOP/帧率 毫秒。
- 如果单个录像多路码流录制，基于多路同步的考虑，多个码流的bufferTimeMSec尽可能保持一致，最大差异保持在2000ms之内。

【相关数据类型及接口】



- [OT_Track_Source_Handle](#)
- [OT_REC_ATTR_S](#)

2.4.15 OT_REC_CALLBACK_S

【说明】

定义录像回调函数集。

【定义】

```
typedef struct {  
    OT\_REC\_REQUEST\_FILE\_NAMES\_FN pfnRequestFileNames;  
} OT_REC_CALLBACK_S;
```

【成员】

成员名称	描述
pfnRequestFileNames	请求一组文件名回调函数。

【注意事项】

无。

【相关数据类型及接口】

无。

2.4.16 OT_REC_ATTR_S

【说明】

定义录像属性。

【定义】

```
typedef struct {  
    OT\_REC\_TYPE\_E enRecType;  
    union  
    {  
        OT\_REC\_NORMAL\_ATTR\_S stNormalRecAttr;  
        OT\_REC\_LAPSE\_ATTR\_S stLapseRecAttr;  
    } unRecAttr;  
    OT\_REC\_SPLIT\_ATTR\_S stSplitAttr;  
    TD_U32 u32StreamCnt;  
    OT\_REC\_STREAM\_ATTR\_S astStreamAttr[OT\_REC\_STREAM\_MAX\_CNT];  
    OT\_REC\_CALLBACK\_S stRecCallbacks;  
    TD_U32 u32PreRecTimeSec;  
} OT_REC_ATTR_S;
```

【成员】

成员名称	描述
enRecType	文件切分属性。



成员名称	描述
unRecAttr	录像类型属性联合体。
stSplitAttr	录像切分类型属性
u32StreamCnt	录制码流数目。
astStreamAttr	录制码流属性数组。
stRecCallbacks	录像回调。
u32PreRecTimeSec	录像预录时间（单位：秒）。

【注意事项】

- 录制类型enRecType需要与unRecAttr联合体设置相对应。
- stSplitAttr按照时长切分的切分时长和合理性需要综合考虑各录制码流的视频参数设定。
- astStreamAttr录制码流属性数组与u32StreamCnt数目对应，默认只有数组中的前u32StreamCnt个码流有效，有效性由参数传递者保证。
- 录像回调中请求一组文件名回调pfnRequestFileNames，文件名数组中文件名的顺序与astStreamAttr码流的顺序一致。

【相关数据类型及接口】

- [OT_REC_TYPE_E](#)
- [OT_REC_SPLIT_ATTR_S](#)
- [OT_REC_LAPSE_ATTR_S](#)
- [OT_REC_STREAM_ATTR_S](#)
- [OT_REC_CALLBACK_S](#)

2.4.17 OT_REC_PARAM

【说明】

定义录像参数。

【定义】

```
typedef enum {  
    TD_U32 streamPriority;  
} OT_REC_PARAM;
```

【成员】

成员名称	描述
streamPriority	录制码流线程优先级

【注意事项】



Huawei LiteOS上支持设置的录制码流线程优先级范围是0~31，Linux上优先级范围是0~99。

2.4.18 OT_REC_MANUAL_SPLIT_TYPE_E

【说明】

定义录像手动切分类型。

【定义】

```
typedef enum {  
    OT_REC_POST_MANUAL_SPLIT = 0,  
    OT_REC_PRE_MANUAL_SPLIT,  
    OT_REC_NORMAL_MANUAL_SPLIT,  
    OT_REC_EXTEND_DURATION_SPLIT,  
    OT_REC_MANUAL_SPLIT_BUTT,  
} OT_REC_MANUAL_SPLIT_TYPE_E;
```

【成员】

成员名称	描述
OT_REC_POST_MANUAL_SPLIT	延迟手动切分类型。
OT_REC_PRE_MANUAL_SPLIT	预录手动切分类型。
OT_REC_NORMAL_MANUAL_SPLIT	普通手动切分类型。
OT_REC_EXTEND_DURATION_SPLIT	延长时长切分类型。
OT_REC_MANUAL_SPLIT_BUTT	非法手动切分类型。

【注意事项】

无

【相关数据类型及接口】

[OT_REC_MANUAL_SPLIT_ATTR_S](#)

2.4.19 OT_REC_POST_MANUAL_SPLIT_ATTR_S

【说明】

定义延迟手动切分属性。

【定义】

```
typedef struct {  
    TD_U32 u32AfterSec;  
} OT_REC_POST_MANUAL_SPLIT_ATTR_S;
```

【成员】

成员名称	描述
u32AfterSec	延迟切分时间，单位秒。



【注意事项】

u32AfterSec最小值1秒，最大值15*24*60*60秒（15天）。

【相关数据类型及接口】

无。

2.4.20 OT_REC_PRE_MANUAL_SPLIT_ATTR_S

【说明】

定义预录手动切分属性。

【定义】

```
typedef struct {
    TD_U32 u32DurationSec;
} OT_REC_PRE_MANUAL_SPLIT_ATTR_S;
```

【成员】

成员名称	描述
u32DurationSec	预录文件时长，单位秒。

【注意事项】

u32DurationSec最小值为1秒，最大值为15*24*60*60秒（15天）。

【相关数据类型及接口】

无。

2.4.21 OT_REC_NORMAL_MANUAL_SPLIT_ATTR_S

【说明】

定义普通预录手动切分属性。

【定义】

```
typedef struct {
    TD_U32 u32Rsv;
} OT_REC_NORMAL_MANUAL_SPLIT_ATTR_S;
```

【成员】

成员名称	描述
u32Rsv	预留值。

【注意事项】



无。

【相关数据类型及接口】

无。

2.4.22 OT_REC_EXTEND_DURATION_SPLIT_ATTR_S

【说明】

定义延长时长切分属性。

【定义】

```
typedef struct {  
    TD_U32 extendDurSec;  
} OT_REC_EXTEND_DURATION_SPLIT_ATTR_S;
```

【成员】

成员名称	描述
extendDurSec	延长切分的时长，单位：s。

【注意事项】

extendDurSec最小值为1秒，最大值为15*24*60*60秒（15天）。

【相关数据类型及接口】

无。

2.4.23 OT_REC_MANUAL_SPLIT_ATTR_S

【说明】

定义录像手动切分属性。

【定义】

```
typedef struct {  
    OT_REC_MANUAL_SPLIT_TYPE_E enManualType  
    union  
    {  
        OT_REC_POST_MANUAL_SPLIT_ATTR_S stPostSplitAttr;  
        OT_REC_PRE_MANUAL_SPLIT_ATTR_S stPreSplitAttr;  
        OT_REC_NORMAL_MANUAL_SPLIT_ATTR_S stNormalSplitAttr;  
        OT_REC_EXTEND_DURATION_SPLIT_ATTR_S stExtendSplitDurAttr;  
    };  
} OT_REC_MANUAL_SPLIT_ATTR_S;
```

【成员】

成员名称	描述
enManualType	手动切分类型。
stPostSplitAttr	延迟手动切分属性。



成员名称	描述
stPreSplitAttr	预录手动切分属性。
stNormalSplitAttr	普通手动切分属性。
stExtendSplitDurAttr	延长时长切分属性。

【 注意事项 】

无

【 相关数据类型及接口 】

- [OT_REC_MANUAL_SPLIT_TYPE_E](#)
- [OT_REC_POST_MANUAL_SPLIT_ATTR_S](#)
- [OT_REC_MANUAL_SPLIT_ATTR_S](#)
- [OT_REC_NORMAL_MANUAL_SPLIT_ATTR_S](#)
- [SS_REC_ManualSplit](#)

2.4.24 OT_REC_EVENT_CALLBACK_FN

【 说明 】

定义录像事件回调函数。

【 定义 】

```
typedef TD_S32 (*OT_REC_EVENT_CALLBACK_FN)(TD_MW_PTR pRecorder, const
OT\_REC\_EVENT\_INFO\_S* pstEventInfo);
```

【 成员 】

成员名称	描述
pRecorder	录像指针，回调函数输入参数。
pstEventInfo	录像事件信息，回调函数输入参数。

【 注意事项 】

无。

【 相关数据类型及接口 】

[SS_REC_RegisterEventCallback](#)

2.4.25 OT_REC_REQUEST_FILE_NAMES_FN

【 说明 】

定义录像文件名请求回调函数。

【 定义 】



```
typedef TD_S32 (*OT_REC_REQUEST_FILE_NAMES_FN)(TD_MW_PTR pRecorder, TD_U32  
u32FileCnt, TD_CHAR (*pszFilename)[OT_REC_FILE_NAME_LEN]);
```

【成员】

成员名称	描述
pRecorder	录像指针，回调函数输入参数。
u32FileCnt	请求文件名个数，回调函数输入参数。
pszFilename	文件名数组，回调函数输出参数。

【注意事项】

文件名在数组中的存放需要按照顺序存放，文件名的顺序需要与录制码流的顺序一致，pszFilename 内部数据的有效性由注册用户确保。

【相关数据类型及接口】

OT_REC_CALLBACK_S

2.4.26 OT_REC_CREATE_MUXER_FN

【说明】

定义创建封装器函数。

【定义】

```
typedef TD_S32 (*OT_REC_CREATE_MUXER_FN) (const TD_CHAR* pszFileName, const  
TD_VOID* pMuxerCfg, TD_MW_PTR* ppMuxer);
```

【成员】

成员名称	描述
pszFileName	文件名，回调函数输入参数。
pMuxerCfg	封装器私有属性，回调函数输入参数。
ppMuxer	封装器句柄，回调输出参数。

【注意事项】

- pMuxerCfg 与OT_REC_STREAM_ATTR_S中的pMuxerCfg一致。
- 回调输出参数的有效性由注册用户确保。

【相关数据类型及接口】

OT_REC_MUXER_OPERATE_FN_S

2.4.27 OT_REC_DESTROY_MUXER_FN

【说明】



定义封装器销毁回调函数。

【定义】

```
typedef TD_S32 (*OT_REC_DESTROY_MUXER_FN)(TD_MW_PTR pMuxer);
```

【成员】

成员名称	描述
pMuxer	封装器句柄，回调函数输入参数。

【注意事项】

无。

【相关数据类型及接口】

[OT_REC_MUXER_OPERATE_FN_S](#)

2.4.28 OT_REC_CREATE_TRACK_FN

【说明】

定义封装器音/视轨创建回调函数。

【定义】

```
typedef TD_S32 (*OT_REC_CREATE_TRACK_FN)(TD_MW_PTR pMuxer,  
OT\_Track\_Source\_Handle pstTrackSrcHandle, TD_MW_PTR* pTrack);
```

【成员】

成员名称	描述
pMuxer	封装器句柄，回调输入参数。
pstTrackSrcHandle	数据源句柄，回调输入参数。
pTrack	封装轨道句柄，回调输出参数。

【注意事项】

回调输出参数有效性由注册用户保证。

【相关数据类型及接口】

[OT_REC_MUXER_OPERATE_FN_S](#)

2.4.29 OT_REC_WRITE_FRAME_FN

【说明】

定义码流封装器写帧回调。

【定义】



```
typedef TD_S32 (*OT_REC_WRITE_FRAME_FN)(TD_MW_PTR pMuxer, TD_MW_PTR  
hTrackHandle, const OT_REC_FRAME_DATA_S *pstFrameData, TD_U64 *fileSize);
```

【成员】

成员名称	描述
pMuxer	封装器句柄，回调输入参数。
pTrack	封装轨道句柄，回调输入参数。
pstFrameData	封装帧数据，回调输入参数。
fileSize	文件大小句柄，回调输入参数。

【注意事项】

在回调函数中调用SS_MP4_GetFileSize或者SS_TS_GetFileSize获取文件大小，并赋值给fileSize。

【相关数据类型及接口】

[OT_REC_MUXER_OPERATE_FN_S](#)

2.4.30 OT_REC_BUF_ALLOC_FN

【说明】

定义录制码流缓存申请回调函数。

【定义】

```
typedef TD_S32 (*OT_REC_BUF_ALLOC_FN)(TD_VOID* pBufArg, TD_U32 u32BufSize, TD_VOID**  
ppVmAddr);
```

【成员】

成员名称	描述
pBufArg	缓存私有数据指针，回调输入参数。
u32BufSize	需要申请的缓存大小，回调输入参数。
ppVmAddr	缓存地址指针，回调输出参数。

【注意事项】

- 回调函数输出参数的有效性由注册用户保证，需要确保申请的缓存地址有效，缓存空间连续且缓存大小正确，否则会引发未知错误。
- 回调函数返回值必须真实有效，否则会引起缓存使用和缓存释放等方面的一些问题。

【相关数据类型及接口】

[OT_REC_MUXER_OPERATE_FN_S](#)



2.4.31 OT_REC_BUF_FREE_FN

【说明】

定义录制码流缓存释放回调函数。

【定义】

```
typedef TD_S32 (*OT_REC_BUF_FREE_FN)(TD_VOID* pBufArg, TD_U32 u32BufSize, TD_VOID* pVmAddr);
```

【成员】

成员名称	描述
pBufArg	缓存私有数据指针，回调输入参数。
u32BufSize	需要申请的缓存大小，回调输入参数。
pVmAddr	缓存地址指针，回调输入参数。

【注意事项】

回调实现的正确性由调用者保证，需要确保pVmAddr指向的缓存能够正确释放，否则可能会引起内存泄露等问题。

【相关数据类型及接口】

[OT_REC_MUXER_OPERATE_FN_S](#)

2.4.32 OT_REC_STREAM_MAX_CNT

【说明】

单路录像支持的最大录制码流数目。

【定义】

```
#define OT_REC_STREAM_MAX_CNT(4)
```

【成员】

无。

【注意事项】

无。

【相关数据类型及接口】

[OT_REC_ATTR_S](#)

2.4.33 OT_REC_TRACK_MAX_CNT

【说明】

单个录制码流包含的最大轨道数据源个数。

【定义】



```
#define OT_REC_TRACK_MAX_CNT (5)
```

【成员】

无。

【注意事项】

无。

【相关数据类型及接口】

[OT_REC_STREAM_ATTR_S](#)

2.4.34 OT_REC_FRAME_MAX_SLICE

【说明】

单帧数组最大分段数目。

【定义】

```
#define OT_REC_FRAME_MAX_SLICE (8)
```

【成员】

无。

【注意事项】

无。

【相关数据类型及接口】

[OT_REC_FRAME_DATA_S](#)

2.4.35 OT_REC_FILE_NAME_LEN

【说明】

录制文件名最大长度。

【定义】

```
#define OT_REC_FILE_NAME_LEN (256)
```

【成员】

无。

【注意事项】

支持的录制文件名最大长度为256，其中包含一个字符串结束符。

【相关数据类型及接口】

- [OT_REC_FILE_EVENT_INFO_S](#)
- [OT_REC_ERROR_EVENT_INFO_S](#)
- [OT_REC_REQUEST_FILE_NAMES_FN](#)



2.4.36 OT_Track_SourceType_E

【说明】

定义数据源类型。

【定义】

```
typedef enum Track_SourceType_E
{
    OT_TRACK_SOURCE_TYPE_PRIV,
    OT_TRACK_SOURCE_TYPE_VIDEO,
    OT_TRACK_SOURCE_TYPE_AUDIO,
    OT_TRACK_SOURCE_TYPE_BUTT
} OT_Track_SourceType_E;
```

【成员】

成员名称	描述
OT_TRACK_SOURCE_TYPE_PRIV	私有数据类型。
OT_TRACK_SOURCE_TYPE_VIDEO	视频数据类型。
OT_TRACK_SOURCE_TYPE_AUDIO	音频数据源类型。

【注意事项】

目前只支持视频数据与音频数据。

【相关数据类型及接口】

无。

2.4.37 OT_Track_VideoCodec_E

【说明】

定义视频数据编码类型。

【定义】

```
typedef enum Track_VideoCodec_E
{
    OT_TRACK_VIDEO_CODEC_H264 = 96,
    OT_TRACK_VIDEO_CODEC_H265 = 98,
    OT_TRACK_VIDEO_CODEC_MJPEG = 102,
    OT_TRACK_VIDEO_CODEC_BUTT
} OT_Track_VideoCodec_E;
```

【成员】

成员名称	描述
OT_TRACK_VIDEO_CODEC_H264	H.264编码
OT_TRACK_VIDEO_CODEC_H265	H.265编码
OT_TRACK_VIDEO_CODEC_MJPEG	MJPEG编码



【注意事项】

目前只支持H.264、H.265数据。

【相关数据类型及接口】

无。

2.4.38 OT_Track_AudioCodec_E

【说明】

定义音频数据编码类型。

【定义】

```
typedef enum Track_AudioCodec_E
{
    OT_TRACK_AUDIO_CODEC_G711Mu = 0,
    OT_TRACK_AUDIO_CODEC_G711A = 8,
    OT_TRACK_AUDIO_CODEC_LPCM = 23,
    OT_TRACK_AUDIO_CODEC_G726 = 97,
    OT_TRACK_AUDIO_CODEC_AMR = 101,
    OT_TRACK_AUDIO_CODEC_ADPCM = 104,
    OT_TRACK_AUDIO_CODEC_AAC = 105,
    OT_TRACK_AUDIO_CODEC_WAV = 108,
    OT_TRACK_AUDIO_CODEC_MP3 = 109,
    OT_TRACK_AUDIO_CODEC_BUTT
} OT_Track_AudioCodec_E;
```

【成员】

成员名称	描述
OT_TRACK_AUDIO_CODEC_G711Mu	G.711 μ -law编码
OT_TRACK_AUDIO_CODEC_G711A	G.711 A-law编码
OT_TRACK_AUDIO_CODEC_G726	G726编码。
OT_TRACK_AUDIO_CODEC_LPCM	LPCM 编码
OT_TRACK_AUDIO_CODEC_AMR	AMR编码
OT_TRACK_AUDIO_CODEC_ADPCM	APDPCM编码
OT_TRACK_AUDIO_CODEC_AAC	AAC编码
OT_TRACK_AUDIO_CODEC_WAV	WAV编码
OT_REC_CODEC_TYPE_MP3	MP3编码

【注意事项】

目前支持AAC、G.711 μ -law和G.711 A-law编码格式。

【相关数据类型及接口】

无。



2.4.39 OT_Track_VideoSourceInfo_S

【说明】

定义视频数据源信息。

【定义】

```
typedef struct Track_VideoSourceInfo_S
{
    OT_Track_VideoCodec_E enCodecType;
    TD_U32 u32Width;
    TD_U32 u32Height;
    TD_U32 u32BitRate;
    TD_FLOAT frameRate;
    TD_U32 u32Gop;
    TD_FLOAT fSpeed;
} OT_Track_VideoSourceInfo_S;
```

【成员】

成员名称	描述
enCodecType	数据编码类型。
u32Width	视频宽度。
u32Height	视频高度。
u32BitRate	视频码率。
frameRate	视频目的帧率。
u32Gop	视频GOP值。
fSpeed	视频帧播放速率。

【注意事项】

fSpeed决定了录制文件以几倍速播放，对于普通录像来说，fSpeed等于目的帧率/原始帧率；对于缩时录像来说fSpeed等于目的帧率* u32IntervalMs（缩时录像帧间隔）/ 1000。

【相关数据类型及接口】

[OT_Track_VideoCodec_E](#)

2.4.40 OT_Track_AudioSourceInfo_S

【说明】

定义音频数据源信息。

【定义】

```
typedef struct Track_AudioSourceInfo_S
{
    OT_Track_AudioCodec_E enCodecType;
    TD_U32 u32ChnCnt;
    TD_U32 u32SampleRate;
```



```
TD_U32 u32AvgBytesPerSec;  
TD_U32 u32SamplesPerFrame;  
TD_U16 u16SampleBitWidth;  
} OT_Track_AudioSourceInfo_S;
```

【成员】

成员名称	描述
enCodecType	音频编码类型。
u32ChnCnt	音频声道数目
u32SampleRate	音频采样率。
u32AvgBytesPerSec	平均每秒BYTE数。
u32SamplesPerFrame	平均每帧采样点。
u16SampleBitWidth	采样位宽。

【注意事项】

无。

【相关数据类型及接口】

[OT_Track_AudioCodec_E](#)

2.4.41 OT_Track_PrivateSourceInfo_S

【说明】

定义私有数据源信息。

【定义】

```
typedef struct Track_PrivateSourceInfo_S  
{  
    TD_U32 u32PrivateData;  
    TD_FLOAT frameRate;  
    TD_U32 u32BytesPerSec;  
    TD_BOOL bStrictSync;  
} OT_Track_PrivateSourceInfo_S;
```

【成员】

成员名称	描述
u32PrivateData	私有数据，预留。
frameRate	私有数据帧率。
u32BytesPerSec	私有数据每秒字节数。
bStrictSync	私有数据帧是否严格同步视频帧标识。



【注意事项】

当私有数据的帧率大于或等于视频帧率，且私有数据帧的时间戳与视频帧严格同步时，建议bStrictSync设置为TD_TRUE，除此之外建议设置为TD_FALSE。

【相关数据类型及接口】

无

2.4.42 Track_Source_S

【说明】

定义数据源信息。

【定义】

```
struct Track_Source_S
{
    TD_S32 s32PrivateHandle;
    SS_Track_Source_Start_FN pfnSourceStart;
    SS_Track_Source_Stop_FN pfnSourceStop;
    SS_Track_Source_RequestKeyFrame_FN pfnRequestKeyFrame
    OT_Track_SourceType_E enTrackType;
    union
    {
        OT_Track_VideoSourceInfo_S stVideoInfo;
        OT_Track_AudioSourceInfo_S stAudioInfo;
        OT_Track_PrivateSourceInfo_S stPrivInfo;
    } unTrackSourceAttr;
};
```

【成员】

成员名称	描述
s32PrivateHandle	数据源私有句柄。
pfnSourceStart	数据源启动回调。
pfnSourceStop	数据源停止回调。
pfnRequestKeyFrame	数据源关键帧回调。
enTrackType	数据源类型。
unTrackSourceAttr	数据源属性枚举。

【注意事项】

无。

【相关数据类型及接口】

- [SS_Track_Source_Start_FN](#)
- [SS_Track_Source_Stop_FN](#)
- [OT_Track_SourceType_E](#)



- [OT_Track_VideoSourceInfo_S](#)
- [OT_Track_AudioSourceInfo_S](#)
- [OT_Track_PrivateSourceInfo_S](#)

2.4.43 OT_Track_Source_Handle

【说明】

定义数据源句柄。

【定义】

```
typedef Track\_Source\_S* OT_Track_Source_Handle;
```

【成员】

无。

【注意事项】

句柄指向的内存由调用者负责申请和释放。

【相关数据类型及接口】

[Track_Source_S](#)

2.4.44 SS_Track_Source_Start_FN

【说明】

定义启动数据源回调函数。

【定义】

```
typedef TD_S32 (*SS_Track_Source_Start_FN)(OT\_Track\_Source\_Handle  
pTrackSource,TD_MW_PTR pCaller);
```

【成员】

成员名称	描述
pTrackSource	数据源句柄。
pCaller	调用模块句柄。

【注意事项】

回调输出参数有效性由注册用户保证。

【相关数据类型及接口】

- [Track_Source_S](#)
- [OT_Track_Source_Handle](#)

2.4.45 SS_Track_Source_Stop_FN

【说明】



定义停止数据源回调函数。

【定义】

```
typedef TD_S32 (*SS_Track_Source_Stop_FN)(OT_Track_Source_Handle
pTrackSource,TD_MW_PTR pCaller);
```

【成员】

成员名称	描述
pTrackSource	数据源句柄。
pCaller	调用模块句柄。

【注意事项】

无。

【相关数据类型及接口】

- [Track_Source_S](#)
- [OT_Track_Source_Handle](#)

2.4.46 SS_Track_Source_RequestKeyFrame_FN

【说明】

定义数据源关键帧回调。

【定义】

```
typedef TD_S32 (*SS_Track_Source_RequestKeyFrame_FN)(OT_Track_Source_Handle
pTrackSource, TD_MW_PTR pCaller);
```

【成员】

成员名称	描述
pTrackSource	数据源句柄。
pCaller	调用模块句柄。

【注意事项】

无。

【相关数据类型及接口】

- [Track_Source_S](#)
- [OT_Track_Source_Handle](#)



2.5 错误码

录像API错误码如表2-1所示。

表 2-1 错误码

错误代码	宏定义	描述
0xA10B8001	OT_ERR_RECORD_ILLEGAL_PARAM	参数非法
0xA10B8002	OT_ERR_RECORD_NULL_PTR	空指针
0xA10B8040	OT_ERR_RECORD_HANDLE_INVALID	非法句柄
0xA10B8041	OT_ERR_RECORD_NO_AVAILABLE_HANDLE	没有可用录像句柄
0xA10B8042	OT_ERR_RECORD_MALLOC_FAIL	内存分配失败
0xA10B8043	OT_ERR_RECORD_THREAD_CREATE_FAIL	线程创建失败
0xA10B8044	OT_ERR_RECORD_MUTEX_INIT_FAIL	互斥锁初始化失败
0xA10B8045	OT_ERR_RECORD_COND_VAR_INIT_FAIL	条件锁初始化失败
0xA10B8046	OT_ERR_RECORD_INIT_TIMER_FAIL	定时器初始化失败
0xA10B8060	OT_ERR_RECORD_STREAM_START_FAIL	码流数据源启动失败
0xA10B8061	OT_ERR_RECORD_STREAM_STOP_FAIL	码流数据源停止失败
0xA10B8070	OT_ERR_RECORD_DO_STOPPING	录像未停止
0xA10B8071	OT_ERR_RECORD_OPT_NOT_ALLOWED	操作不允许
0xA10B8072	OT_ERR_RECORD_DO_MANUAL_SPLIT	正在手动切分中
0xA10B8080	OT_ERR_RECORD_ALLOC_FILENAME_FAIL	请求文件名失败
0xA10B8090	OT_ERR_RECORD_BUFFER_CREATE_FAIL	缓存创建失败
0xA10B8091	OT_ERR_RECORD_BUFFER_DESTROY_FAIL	缓存销毁失败
0xA10B8092	OT_ERR_RECORD_BUFFER_SET_MODE_FAIL	设置缓存模式失败
0xA10B8093	OT_ERR_RECORD_BUFFER_WRITE_FRAME_FAIL	缓存写帧失败



错误代码	宏定义	描述
0xA10B8094	OT_ERR_RECORD_BUFFER_READ_FRA ME_FAIL	缓存读帧失败



3 日期时间文件系统

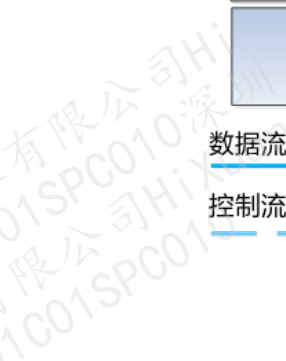
3.1 概述

日期时间文件系统模块是录像和拍照模块的支撑模块，同时也向预览、回放等业务层应用提供媒体文件的命名添加、检索、删除和关联功能。

文件的命名及目录组织方式，是决定文件管理方式的核心要素之一；不同的产品形态根据业务需求可以选择不同的文件管理方式；目前本模块主要提供了按日期时间的文件管理方式。

3.2 架构原理

日期时间文件系统模块，主要提供文件的检索、命名添加、删除和关联四个功能。主要是日期时间文件管理规则的集合。具体系统框图组成及对外接口关系如[图3-1](#)所示。



录下的所有文

- 文档版本 00



3.2.1 文件目录组织规则

日期时间文件系统主要是按日期时间文件命名和一系列规则的集合。下面详细阐述其中的规则。

3.2.1.1 文件类型

当前支持的文件类型如表3-1所示。

表 3-1 文件类型

文件类型	文件后缀	说明
OT_DTCF_FILE_TYPE_MP4	MP4	录像文件，缩略图集成在内部。文件扫描时识别该后缀。仅匹配大写。
OT_DTCF_FILE_TYPE_JPG	JPG	用于抓拍文件。文件扫描时识别该后缀。仅匹配大写。
OT_DTCF_FILE_TYPE_TS	TS	TS文件。文件扫描时识别该后缀。仅匹配大写。
OT_DTCF_FILE_TYPE_DNG	DNG	DNG文件。文件扫描时识别该后缀。仅匹配大写。

3.2.1.2 目录类型

如表3-2所示，当前支持14种目录类型，每种目录类型都有默认的名称。支持用户修改目录名称。

表 3-2 目录类型

目录类型	默认名称	说明
DTCF_DIR_EMR_FRONT	"EMR"	前摄像头紧急录像大码流目录
DTCF_DIR_EMR_FRONT_SUB	"EMR_s"	前摄像头紧急录像小码流目录
DTCF_DIR_NORM_FRONT	"Movie"	前摄像头普通录像大码流目录
DTCF_DIR_NORM_FRONT_SUB	"Movie_s"	前摄像头普通录像小码流目录
DTCF_DIR_PARK_FRONT	Park	前摄像头停车录像大码流目录
DTCF_DIR_PARK_FRONT_SUB	Park_s	前摄像头停车录像小码流目录
DTCF_DIR_EMR_REAR	"EMR_b"	后摄像头紧急录像大码流目录
DTCF_DIR_EMR_REAR_SUB	"EMR_b_s"	后摄像头紧急录像小码流目录
DTCF_DIR_NORM_REAR	"Movie_b"	后摄像头普通录像大码流目录
DTCF_DIR_NORM_REAR_SUB	"Movie_b_s"	后摄像头普通录像小码流目录
DTCF_DIR_PARK_REAR	Park_b	后摄像头停车录像大码流目录



目录类型	默认名称	说明
DTCF_DIR_PARK_REAR_SUB	Park_b_s	后摄像头停车录像小码流目录
DTCF_DIR_PHOTO_FRONT	"Photo"	前摄像头抓拍目录
DTCF_DIR_PHOTO_REAR	"Photo_b"	后摄像头抓拍目录

3.2.1.3 文件名称

文件的名称是由日期时间字符串、文件序号、文件名后缀、文件类型后缀四部分组成。文件的全路径是由根路径、二级目录名、文件名三部分组成。规则示意图如图3-2所示。

图 3-2 文件命名规则

规则: `/ROOT_PATH/DIR_NAME/DATETIME_STR_NO_SUFFIX.TYPE`

示例: `/Tmp/SD/EMR_s/2017_0220_143930_00_S.MP4`

目录类型与文件命名后缀的关系如表3-3所示。

表 3-3 目录类型与文件命名后缀关系

目录类型	后缀
DTCF_DIR_EMR_FRONT	无
DTCF_DIR_EMR_FRONT_SUB	_s
DTCF_DIR_NORM_FRONT	无
DTCF_DIR_NORM_FRONT_SUB	_s
DTCF_DIR_PARK_FRONT	无
DTCF_DIR_PARK_FRONT_SUB	_s
DTCF_DIR_EMR_REAR	_b
DTCF_DIR_EMR_REAR_SUB	_b_s
DTCF_DIR_NORM_REAR	_b
DTCF_DIR_NORM_REAR_SUB	_b_s



目录类型	后缀
DTCF_DIR_PARK_REAR	无
DTCF_DIR_PARK_REAR_SUB	_b_s
DTCF_DIR_PHOTO_FRONT	无
DTCF_DIR_PHOTO_REAR	_b

3.2.1.4 扫描

文件扫描是同步接口、接口调用后，先清空上次扫描的结果，再根据当前参数重新扫描，然后按时间从新到老排序。

3.2.1.5 检索

支持通过索引获取文件的信息，依赖扫描接口。扫描某个目录或者多个目录后，才能通过索引获取文件信息。

3.2.1.6 删除

文件管理模块提供了通过索引删除指定文件的接口，供用户使用。索引的范围是根据扫描的结果来确定的，删除接口也只是删除扫描结果中的一个节点，不会删除真实的文件。

3.3 API 参考

该功能模块为用户提供以下API：

- **SS_DTCF_Init**：初始化DTCF。
- **SS_DTCF_GetDirNames**：获取所有二级目录名称。
- **SS_DTCF_Delinit**：去初始化DTCF。
- **SS_DTCF_Scan**：文件目录扫描。
- **SS_DTCF_GetFileByIndex**：通过索引获取文件。
- **SS_DTCF_DelFileByIndex**：通过索引删除文件。
- **SS_DTCF_AddFile**：添加文件。
- **SS_DTCF_GetOldestFileIndex**：获取最老文件索引。
- **SS_DTCF_GetFileAmount**：获取文件数量。
- **SS_DTCF_GetOldestFilePath**：获取最老文件路径。
- **SS_DTCF_CreateFilePath**：创建文件路径。
- **SS_DTCF_GetRelatedFilePath**：获取关联文件路径。
- **SS_DTCF_GetEmrFilePath**：获取紧急录像文件路径。
- **SS_DTCF_GetFileDirType**：获取文件所在目录类型。



3.3.1 SS_DTCF_Init

【描述】

初始化DTCF。

【定义】

```
TD_S32 SS_DTCF_Init(const TD_CHAR *pazRootDir, const TD_CHAR azDirNames
[DTCF_DIR_BUTT][OT_DIR_LEN_MAX]);
```

【参数】

参数名称	描述	输入/输出
pazRootDir	文件系统根目录指针。	输入
azDirNames	二级目录名称。	输入

【返回值】

返回值	描述
0	成功。
非0	失败，参见 错误码 。

【需求】

- 头文件：ss_dtcf.h
- 库文件：libdtcf.a

【注意】

不支持多实例。

【举例】

无。

3.3.2 SS_DTCF_GetDirNames

【描述】

获取所有二级目录名称。

【定义】

```
TD_S32 SS_DTCF_GetDirNames(TD_CHAR (*pazDirNames)[OT_DIR_LEN_MAX], TD_U32
u32DirAmount);
```

【参数】



参数名称	描述	输入/输出
pazDirNames	二级目录名称指针。	输出
u32DirAmount	二级目录个数。 取值范围：(0, DTCF_DIR_BUTT]	输入

【返回值】

返回值	描述
0	成功。
非0	失败，参见 错误码 。

【需求】

- 头文件：ss_dtcf.h
- 库文件：libdtcf.a

【注意】

DTCF创建后才能调用该接口。

【举例】

无。

3.3.3 SS_DTCF_DeInit

【描述】

去初始化DTCF。

【定义】

```
TD_S32 SS_DTCF_DeInit(void);
```

【参数】

无。

【返回值】

返回值	描述
0	成功。
非0	失败，参见 错误码 。

【需求】



- 头文件：ss_dtcf.h
- 库文件：libdtcf.a、libdtcf.so

【注意】

无。

【举例】

无。

3.3.4 SS_DTCF_Scan

【描述】

文件目录扫描。

【定义】

```
TD_S32 SS_DTCF_Scan(const OT\_DTCF\_DIR\_E enDirs[DTCF_DIR_BUTT], TD_U32 u32DirCount,  
TD_U32 *pu32FileAmount);
```

【参数】

参数名称	描述	输入/输出
enDirs	要扫描的文件目录类型的数组。	输入
u32DirCount	要扫描的文件目录类型的数量。	输入
pu32FileAmount	扫描到的文件数量的指针。	输出

【返回值】

返回值	描述
0	成功。
非0	失败，参见 错误码 。

【需求】

- 头文件：ss_dtcf.h
- 库文件：libdtcf.a、libdtcf.so

【注意】

- DTCF创建后才能调用该接口。
- 扫描后的文件默认按文件创建时间从新到旧排序。
- 最大支持10240个文件。

【举例】

无。



3.3.5 SS_DTCF_GetFileByIndex

【描述】

通过索引获取文件信息。

【定义】

TD_S32 SS_DTCF_GetFileByIndex(TD_U32 u32Index, TD_CHAR *pazFileName, TD_U32 u32Length, [OT_DTCF_DIR_E](#) *penDir);

【参数】

参数名称	描述	输入/输出
u32Index	文件索引。	输入
pazFileName	文件名称的指针。	输出
u32Length	文件名称的长度。	输入
penDir	文件目录类型的指针。	输出

【返回值】

返回值	描述
0	成功。
非0	失败，参见 错误码 。

【需求】

- 头文件：ss_dtcf.h
- 库文件：libdtcf.a、libdtcf.so

【注意】

- 扫描后才能调用该接口。
- 索引不能大于或者等于扫描到的文件数量。

【举例】

无。

3.3.6 SS_DTCF_DelFileByIndex

【描述】

通过索引删除文件。

【定义】

TD_S32 SS_DTCF_DelFileByIndex(TD_U32 u32Index, TD_U32 *pu32FileAmount);



【参数】

参数名称	描述	输入/输出
u32Index	文件索引。	输入
pu32FileAmount	文件删除后，剩余文件的总数量。	输出

【返回值】

返回值	描述
0	成功。
非0	失败，参见 错误码 。

【需求】

- 头文件：ss_dtcf.h
- 库文件：libdtcf.a、libdtcf.so

【注意】

只删除内部扫描的文件信息，不删除真实文件。

【举例】

无。

3.3.7 SS_DTCF_AddFile

【描述】

添加文件。

【定义】

```
TD_S32 SS_DTCF_AddFile(const TD_CHAR *pazSrcFilePath, OT\_DTCF\_DIR\_E enDir);
```

【参数】

参数名称	描述	输入/输出
pazSrcFilePath	文件路径。	输入
enDir	文件目录类型。	输入

【返回值】



返回值	描述
0	成功。
非0	失败，参见 错误码 。

【需求】

- 头文件：ss_dtcf.h
- 库文件：libdtcf.a、libdtcf.so

【注意】

- 扫描后才能使用该接口。
- 最大支持10240个文件。

【举例】

无。

3.3.8 SS_DTCF_GetOldestFileIndex

【描述】

获取最老文件索引。

【定义】

TD_S32 SS_DTCF_GetOldestFileIndex(const [OT_DTCF_DIR_E](#) enDirs[DTCF_DIR_BUTT], TD_U32 u32DirCount,TD_U32 *pu32Index);

【参数】

参数名称	描述	输入/输出
enDirs	目录类型数组。	输入
u32DirCount	目录个数	输入
pu32Index	文件索引指针。	输出

【返回值】

返回值	描述
0	成功。
非0	失败，参见 错误码 。

【需求】



- 头文件：ss_dtcf.h
- 库文件：libdtcf.a、libdtcf.so

【注意】

- 扫描后才能使用该接口。
- 文件目录类型仅支持扫描的文件类型。
- 实现删除最老文件功能时，需要与SS_DTCF_DelFileByIndex配合使用。
- 请注意两个接口调用的中间不能有SS_DTCF_AddFile接口的调用。

【举例】

无。

3.3.9 SS_DTCF_GetFileAmount

【描述】

获取文件数量。

【定义】

```
TD_S32 SS_DTCF_GetFileAmount(TD_U32 *pu32FileAmount);
```

【参数】

参数名称	描述	输入/输出
pu32FileAmount	文件数量指针。	输出

【返回值】

返回值	描述
0	成功。
非0	失败，参见 错误码 。

【需求】

- 头文件：ss_dtcf.h
- 库文件：libdtcf.a、libdtcf.so

【注意】

扫描后才能使用该接口。

【举例】

无。



3.3.10 SS_DTCF_GetOldestFilePath

【描述】

获取目录下最老文件路径。

【定义】

TD_S32 SS_DTCF_GetOldestFilePath(**OT_DTCF_DIR_E** enDir, TD_CHAR *pazFilePath, TD_U32 u32Length);

【参数】

参数名称	描述	输入/输出
enDir	文件目录类型。	输入
pazFilePath	最老文件路径指针。	输出
u32Length	最老文件路径长度。	输入

【返回值】

返回值	描述
0	成功。
非0	失败，参见 错误码 。

【需求】

- 头文件：ss_dtcf.h
- 库文件：libdtcf.a、libdtcf.so

【注意】

每次调用该接口都会扫描目录下所有文件，排序后获取最老文件，占用CPU，所以不推荐频繁调用该接口。

【举例】

无。

3.3.11 SS_DTCF_CreateFilePath

【描述】

创建文件路径。

【定义】

TD_S32 SS_DTCF_CreateFilePath(**OT_DTCF_FILE_TYPE_E** enFileType, **OT_DTCF_DIR_E** enDir, TD_CHAR *pazFilePath, TD_U32 u32Length);

【参数】



参数名称	描述	输入/输出
enFileType	文件类型。	输入
enDir	文件目录类型。	输入
pazFilePath	文件路径指针。	输出
u32Length	文件路径长度。	输入

【返回值】

返回值	描述
0	成功。
非0	失败，参见 错误码 。

【需求】

- 头文件：ss_dtcf.h
- 库文件：libdtcf.a、libdtcf.so

【注意】

仅按规则创建路径，不真实创建文件。

【举例】

无。

3.3.12 SS_DTCF_GetRelatedFilePath

【描述】

获取关联文件路径。

【定义】

```
TD_S32 SS_DTCF_GetRelatedFilePath(const TD_CHAR *pazSrcFilePath, OT\_DTCF\_DIR\_E enDir,  
TD_CHAR *pazDstFilePath, TD_U32 u32Length);
```

【参数】

参数名称	描述	输入/输出
pazSrcFilePath	原文件路径。	输入
enDir	关联文件目录类型。	输入
pazDstFilePath	关联文件路径指针。	输出
u32Length	关联文件路径长度。	输入



【返回值】

返回值	描述
0	成功。
非0	失败，参见 错误码 。

【需求】

- 头文件：ss_dtcf.h
- 库文件：libdtcf.a、libdtcf.so

【注意】

无。

【举例】

无。

3.3.13 SS_DTCF_GetEmrFilePath

【描述】

获取紧急文件路径。

【定义】

```
TD_S32 SS_DTCF_GetEmrFilePath(const TD_CHAR *pazSrcFilePath, TD_CHAR *pazDstFilePath,  
TD_U32 u32Length);
```

【参数】

参数名称	描述	输入/输出
pazSrcFilePath	源文件路径。	输入
pazDstFilePath	紧急录像路径指针。	输出
u32Length	紧急录像路径长度。	输入

【返回值】

返回值	描述
0	成功。
非0	失败，参见 错误码 。

【需求】



- 头文件：ss_dtcf.h
- 库文件：libdtcf.a、libdtcf.so

【注意】

停车没有紧急录像。

【举例】

无。

3.3.14 SS_DTCF_GetFileDirType

【描述】

获取文件的目录类型。

【定义】

TD_S32 SS_DTCF_GetFileDirType(const TD_CHAR *pazSrcFilePath, OT_DTCF_DIR_E *penDir);

【参数】

参数名称	描述	输入/输出
pazSrcFilePath	源文件路径。	输入
penDir	文件目录类型指针。	输出

【返回值】

返回值	描述
0	成功。
非0	失败，参见 错误码 。

【需求】

- 头文件：ss_dtcf.h
- 库文件：libdtcf.a、libdtcf.so

【注意】

无。

【举例】

无。

3.4 数据类型

日期时间文件系统模块相关数据类型定义如下：



- **OT_FILE_PATH_LEN_MAX**: 定义文件路径的最大长度。
- **OT_DIR_LEN_MAX**: 定义目录名称的最大长度。
- **OT_DTCF_DIR_E**: 定义文件目录类型。
- **OT_DTCF_FILE_TYPE_E**: 定义文件类型集。

3.4.1 OT_FILE_PATH_LEN_MAX

【说明】

定义文件路径的最大长度。

【定义】

```
#define OT_FILE_PATH_LEN_MAX (256)
```

【注意事项】

无。

【相关数据结构及接口】

无。

3.4.2 OT_DIR_LEN_MAX

【说明】

定义目录名称的最大长度。

【定义】

```
#define OT_DIR_LEN_MAX (64)
```

【注意事项】

无。

【相关数据类型及接口】

无。

3.4.3 OT_DTCF_DIR_E

【说明】

定义文件目录类型。

【定义】

```
typedef enum {  
    DTCF_DIR_EMR_FRONT = 0,  
    DTCF_DIR_EMR_FRONT_SUB,  
    DTCF_DIR_NORM_FRONT,  
    DTCF_DIR_NORM_FRONT_SUB,  
    DTCF_DIR_PARK_FRONT,  
    DTCF_DIR_PARK_FRONT_SUB,  
    DTCF_DIR_EMR_REAR,  
    DTCF_DIR_EMR_REAR_SUB,  
    DTCF_DIR_NORM_REAR,  
    DTCF_DIR_NORM_REAR_SUB,  
}
```



```
DTCF_DIR_PARK_REAR,  
DTCF_DIR_PARK_REAR_SUB,  
DTCF_DIR_PHOTO_FRONT,  
DTCF_DIR_PHOTO_REAR,  
DTCF_DIR_BUTT  
} OT_DTCF_DIR_E;
```

【成员】

成员名称	描述
DTCF_DIR_EMR_FRONT	前摄像头紧急录像大码流目录
DTCF_DIR_EMR_FRONT_SUB	前摄像头紧急录像小码流目录
DTCF_DIR_NORM_FRONT	前摄像头普通录像大码流目录
DTCF_DIR_NORM_FRONT_SUB	前摄像头普通录像小码流目录
DTCF_DIR_PARK_FRONT	前摄像头停车录像大码流目录
DTCF_DIR_PARK_FRONT_SUB	前摄像头停车录像小码流目录
DTCF_DIR_EMR_REAR	后摄像头紧急录像大码流目录
DTCF_DIR_EMR_REAR_SUB	后摄像头紧急录像小码流目录
DTCF_DIR_NORM_REAR	后摄像头普通录像大码流目录
DTCF_DIR_NORM_REAR_SUB	后摄像头普通录像小码流目录
DTCF_DIR_PARK_REAR	后摄像头停车录像大码流目录
DTCF_DIR_PARK_REAR_SUB	后摄像头停车录像小码流目录
DTCF_DIR_PHOTO_FRONT	前摄像头拍照目录
DTCF_DIR_PHOTO_REAR	后摄像头拍照目录
DTCF_DIR_BUTT	无效目录类型

【注意事项】

停车没有紧急录像。

【相关数据类型及接口】

无。

3.4.4 OT_DTCF_FILE_TYPE_E

【说明】

定义文件类型集。

【定义】

```
typedef enum {  
    OT_DTCF_FILE_TYPE_MP4,  
    OT_DTCF_FILE_TYPE_JPG,  
}
```



```
OT_DTCF_FILE_TYPE_TS,  
OT_DTCF_FILE_TYPE_DNG,  
OT_DTCF_FILE_TYPE_BUTT  
} OT_DTCF_FILE_TYPE_E;
```

【成员】

成员名称	描述
OT_DTCF_FILE_TYPE_MP4	MP4文件类型。
OT_DTCF_FILE_TYPE_JPG	JPG文件类型。
OT_DTCF_FILE_TYPE_TS	TS文件类型。
OT_DTCF_FILE_TYPE_DNG	DNG文件类型。
OT_DTCF_FILE_TYPE_BUTT	无效文件类型。

【注意事项】

无。

【相关数据类型及接口】

无。

3.5 错误码

日期时间文件系统API错误码如表3-4所示。

表 3-4 日期时间文件系统 API 错误码

错误代码	宏定义	描述
0x2A01	OT_DTCF_ERR_EINVAL_PAEAMETER	非法参数
0x2A02	OT_DTCF_ERR_NULL_PTR	指针为空
0x2A03	OT_DTCF_ERR_NOMEM	内存不够，或者申请内存失败。
0x2A04	OT_DTCF_UNSUPPORT_PATH	不支持的文件路径
0x2A05	OT_DTCF_SAME_DIR_PATH	目标文件目录与源文件目录相同
0x2A06	OT_DTCF_ERR_STATUS_ERROR	状态错误，或者操作不支持
0x2A07	OT_DTCF_ERR_EMPTY_DIR	目录为空
0x2A08	OT_DTCF_ERR_UNDEFINE_DIR	未定义的目录类型
0x2A09	OT_DTCF_ERR_SYSTEM_ERROR	系统操作错误



错误代码	宏定义	描述
0x2A0A	OT_DTCF_SAME_FILENAME_PATH	文件名相同，不能添加
0x2A0B	OT_DTCF_PATH_IS_NOT_DIR_ERROR	非目录路径



4 媒体缓存库 libmbuf 使用说明

4.1 概述

媒体缓存库libmbuf提供媒体数据的缓存空间，通过函数挂接可用于媒体传输，实现对媒体缓存数据的写入、读取、简化传输流程的复杂度。

4.2 功能描述

4.2.1 基本概念

整块数据分片规则：

对于一些大块数据，时常会有分块存储的需求，并可选择分块读取或整块读取；以H.264视频为例，编码器会有Slice模式，这是进行Mbuf缓存时，需要以Slice片模式进行写入，并根据需要进行Slice读取，或者Frame读取；1个Frame可以包括1到多个Slice。

4.2.2 功能描述

媒体缓存模块用于对媒体数据的缓存，提供缓存空间的初始化，外部调用者在提供一些必备信息申请一块缓存，并注册使能媒体数据类型，就可以进行媒体数据写入或读取了，其数据的组织过程不用调用者理会。

4.3 API 参考

媒体缓存模块提供模块的初始化去初始化、缓存空间的申请释放、缓存数据的读写、以及缓存状态的获取。

该功能模块提供以下API：

- **SS_MBUF_GetBuffer**：申请一块Mbuf缓存。
- **SS_MBUF_ReleaseBuffer**：释放一块Mbuf缓存。
- **SS_MBUF_RegisterPayload**：注册一种缓存可写入数据类型。



- **SS_MBUF_UnregisterPayload**: 取消注册一种数据类型。
- **SS_MBUF_SetRWEnable**: 设置Mbuf缓存中对应数据类型读写使能权限。
- **SS_MBUF_GetRWEnable**: 获取Mbuf缓存中对应数据类型的读写使能权限。
- **SS_MBUF_WriteSlice**: 向Mbuf缓存写入对应payload类型的分片数据。
- **SS_MBUF_ReadSlice**: 从Mbuf缓存中读取对应payload类型的分片数据。
- **SS_MBUF_Forward**: 将读指针后移，以读取到新的数据片/帧。
- **SS_MBUF_Debug**: 打印Mbuf缓存模块调试信息。

4.3.1 SS_MBUF_GetBuffer

【描述】

申请一块Mbuf缓存，或者获取某块缓存的相关属性。

【语法】

```
TD_S32 SS_MBUF_GetBuffer(const TD_VOID* argv, OT_MBUF_CFG_S* pstMbufCfg, TD_HANDLE* pHandle);
```

【参数】

参数名称	描述	输入/输出
argv	用户透传数据。	输入
pstMbufCfg	对应缓存Mbuf的相关属性。	输入
phandle	缓存Mbuf句柄。	输入/输出

【返回值】

返回值	描述
0	成功。
非0	失败，返回 错误码 。

【需求】

- 头文件：ss_mbuf_ext.h，ot_mbuf_define_ext.h
- 库文件：libmbuf.a

【举例】

无。

【相关主题】

无。



4.3.2 SS_MBUF_ReleaseBuffer

【描述】

释放一块Mbuf缓存。

【语法】

```
TD_S32 SS_MBUF_ReleaseBuffer(const TD_VOID *argv, TD_MW_PTR hHandle) ;
```

【参数】

参数名称	描述	输入/输出
argv	用户透传数据。	输入
hHandle	Mbuf缓存句柄。	输入

【返回值】

返回值	描述
0	成功。
非0	失败，返回 错误码 。

【需求】

- 头文件：ss_mbuf_ext.h，ot_mbuf_define_ext.h
- 库文件：libmbuf.a

【注意】

与[SS_MBUF_GetBuffer](#)接口使用。

【举例】

无。

【相关主题】

无。

4.3.3 SS_MBUF_RegisterPayload

【描述】

注册payloadType数据类型，可注册多种数据类型，只有已注册数据类型才支持Mbuf读写。

【语法】

```
TD_S32 SS_MBUF_RegisterPayload(TD_HANDLE hHandle, TDTD_U8u8PayloadType);
```

【参数】



参数名称	描述	输入/输出
hHandle	缓存句柄。	输入
u8PayloadType	负载数据类型，范围：[0, 254]	输入

【返回值】

返回值	描述
0	成功。
非0	失败，返回 错误码 。

【需求】

- 头文件：ss_mbuf_ext.h, ot_mbuf_define_ext.h
- 库文件：libmbuf.a

【注意】

如果相应数据类型没有注册，则[SS_MBUF_WriteSlice](#)会返回失败。

【举例】

无。

【相关主题】

无。

4.3.4 SS_MBUF_UnregisterPayload

【描述】

取消注册一种数据类型。

【语法】

```
TD_S32 SS_MBUF_UnregisterPayload(TD_HANDLE hHandle, TD_U8 u8PayloadType);
```

【参数】

参数名称	描述	输入/输出
hHandle	缓存句柄。	输入
u8PayloadType	负载数据类型，范围：[0, 254]	输入

【返回值】



返回值	描述
0	成功。
非0	失败，返回 错误码 。

【需求】

- 头文件：ss_mbuf_ext.h, ot_mbuf_define_ext.h
- 库文件：libmbuf.a

【注意】

如果该数据类型未注册，返回失败。

【举例】

无。

【相关主题】

无。

4.3.5 SS_MBUF_SetRWEnable

【描述】

设置Mbuf缓存中对应数据类型读写使能权限。

【语法】

```
TD_S32SS_MBUF_SetRWEnable(TD_HANDLE hHandle, TD_U8 u8PayloadType, TD_BOOL bEnableRead, TD_BOOL bEnableWrite);
```

【参数】

参数名称	描述	输入/输出
hHandle	缓存句柄。	输入
u8PayloadType	负载数据类型，范围：[0, 254]	输入
bEnableRead	使能读取。	输入
bEnableWrite	使能写入。	输入

【返回值】

返回值	描述
0	成功。
非0	失败，返回 错误码 。



【需求】

- 头文件：ss_mbuf_ext.h, ot_mbuf_define_ext.h
- 库文件：libmbuf_v2.a

【注意】

如果传入的pHandle未申请，或者u8PayloadType未注册，返回失败。

【举例】

无。

【相关主题】

无。

4.3.6 SS_MBUF_GetRWEnable

【描述】

获取Mbuf缓存中对应数据类型的读写使能权限。

【语法】

```
TD_S32 SS_MBUF_GetRWEnable(TD_HANDLE hHandle, TD_U8 u8PayloadType, TD_BOOL* pbEnableRead, TD_BOOL* pbEnableWrite);
```

【参数】

参数名称	描述	输入/输出
hHandle	缓存句柄。	输入
u8PayloadType	负载数据类型，范围：[0, 254]	输入
pbEnableRead	使能读取。	输出
pbEnableWrite	使能写入。	输出

【返回值】

返回值	描述
0	成功。
非0	失败，返回错误码。

【需求】

- 头文件：ss_mbuf_ext.h, ot_mbuf_define_ext.h
- 库文件：libmbuf.a

【注意】

如果传入的pHandle未申请，或者u8PayloadType未注册，返回失败。



【举例】

无。

【相关主题】

无。

4.3.7 SS_MBUF_WriteSlice

【描述】

向Mbuf缓存写入对应payload类型的分片数据。

【语法】

```
TD_S32 SS_MBUF_WriteSlice(TD_HANDLE hHandle, const OT_MBUF_SLICEINFO_S*  
pstSliceInfo);
```

【参数】

参数名称	描述	输入/输出
hHandle	缓存句柄。	输入
pstSliceInfo	写入分片数据相关信息，包括数据长度，数据类型等。	输入

【返回值】

返回值	描述
0	成功。
非0	失败，返回 错误码 。

【需求】

- 头文件：ss_mbuf_ext.h, ot_mbuf_define_ext.h
- 库文件：libmbuf.a

【注意】

- 如果传入的pHandle未申请，或者传入的数据类型未注册，或者未使能写，则会返回失败。
- 这里一整块数据可以分为多片数据进行写入，典型场景为视频Slice模式，多个Slice组成一帧，可以分片写入Mbuf，支持单片读取和组帧后读取Mbuf。同时一帧数据也可以只有一片。
- 这里的**OT_MBUF_SLICEINFO_S**也可是一帧数据分为多个block概念，地址数组依次为每个数据块的起始地址，blocklen长度数组为每个数据块的长度，一个frame或者一个slice可以由多个block组成，一般情况下一帧或者一片数据由一块block组成。



- 首次写入的数据必须为关键帧数据，否则报错。
- 写数据失败后，下次写入数据必须为关键帧数据，否则报错。

【举例】

无。

【相关主题】

无。

4.3.8 SS_MBUF_ReadSlice

【描述】

从Mbuf缓存中读取对应payload类型的分片数据。

【语法】

```
TD_S32 SS_MBUF_ReadSlice(TD_HANDLE hHandle, TD_U8 u8PayloadType,  
OT_MBUF_SLICEINFO_S* pstSliceInfo);
```

【参数】

参数名称	描述	输入/输出
hHandle	缓存句柄。	输入
u8PayloadType	负载数据类型，范围：[0, 254]	输入
pstSliceInfo	输出的分片数据信息，包括数据。	输出

【返回值】

返回值	描述
0	成功。
非0	失败，返回错误码。

【需求】

- 头文件：ss_mbuf_ext.h，ot_mbuf_define_ext.h
- 库文件：libmbuf.a

【注意】

仅读取数据，但不更新读指针，需要调用SS_MBUF_Forward才能读到新一片数据，否则读到的会一直是同一片数据。

【举例】

无。

【相关主题】



无。

4.3.9 SS_MBUF_Forward

【描述】

将读指针后移，以读取到新的数据片/帧。

【语法】

```
TD_S32 SS_MBUF_Forward(TD_HANDLE hHandle, TD_U8 u8PayloadType, TD_U32 u32Step);
```

【参数】

参数名称	描述	输入/输出
hHandle	缓存句柄。	输入
u8PayloadType	负载数据类型，范围：[0, 254]	输入
u32Step	移动步长，以Slice片为单位，1表示读指针后移一片。	输入

【返回值】

返回值	描述
0	成功。
非0	失败，返回错误码。

【需求】

- 头文件：ss_mbuf_ext.h，ot_mbuf_define_ext.h
- 库文件：libmbuf.a

【注意】

调用SS_MBUF_ReadSlice不会更新读指针，只有调用该接口后，才能读取到后续数据。

【举例】

无。

【相关主题】

无。

4.3.10 SS_MBUF_GetSliceDataOffset

【描述】

获取帧数据偏移地址。



【语法】

```
TD_S32 SS_MBUF_GetSliceDataOffset(TD_HANDLE hHandle, const OT_MBUF_SLICEINFO_S*  
pstSliceInfo, TD_U32* pu32Offset);
```

【参数】

参数名称	描述	输入/输出
hHandle	缓存句柄。	输入
pstSliceInfo	数据信息。	输入
pu32Offset	偏移地址。	输出

【返回值】

无。

【需求】

- 头文件：ss_mbuf_ext.h, ot_mbuf_define_ext.h
- 库文件：libmbuf.a

【注意】

无。

【举例】

无。

【相关主题】

无。

4.3.11 SS_MBUF_Debug

【描述】

打印Mbuf缓存模块调试信息。

【语法】

```
TD_VOID SS_MBUF_Debug(TD_HANDLE hHandle, TD_U8 u8PayloadType);
```

【参数】

参数名称	描述	输入/输出
hHandle	缓存句柄。	输入
u8PayloadType	负载数据类型。	输入

【返回值】

无。

**【需求】**

- 头文件：ss_mbuf_ext.h, ot_mbuf_define_ext.h
- 库文件：libmbuf.a

【注意】

无。

【举例】

无。

【相关主题】

无。

4.4 数据结构

相关数据类型、数据结构定义如下：

- **OT_MBUF_FRAME_MAX_BLOCK**：定义Mbuf缓存帧最大块数。
- **OT_MBUF_ABSTRACTFUNC_S**：定义模块内部使用缓存空间相关操作的抽象接口集合。
- **OT_MBUF_CFG_S**：定义Mbuf缓存属性结构体。
- **OT_MBUF_SLICEINFO_S**：定义写入分片数据信息结构体。
- **OT_MBUF_FRAMEINFO_S**：定义从Mbuf缓存读取帧数据的相关属性。
- **Mbuf_Alloc_Func**：定义用户申请内存空间接口。
- **Mbuf_Free_Func**：定义用户释放内存空间接口。
- **Mbuf_Mcopy_Func**：定义用户拷贝内存空间接口。
- **Mbuf_CreateLock_Func**：定义用户创建跨进程互斥锁接口。
- **Mbuf_DestroyLock_Func**：定义用户销毁跨进程互斥锁接口。
- **Mbuf_Lock_Func**：定义用户互斥锁加锁接口。
- **Mbuf_UnLock_Func**：定义用户互斥锁解锁接口。

4.4.1 OT_MBUF_FRAME_MAX_BLOCK

【说明】

定义Mbuf缓存帧最大块数。

【定义】

```
#define OT_MBUF_FRAME_MAX_BLOCK (8)
```

【注意事项】

无

【相关数据类型及接口】

无



4.4.2 OT_MBUF_ABSTRACTFUNC_S

【说明】

定义模块内部使用缓存空间相关操作的抽象接口集合，用户通过实现自有的缓存空间申请，释放，拷贝，加解锁操作，从而自由选择缓存所需内存空间的来源，而不局限于系统malloc接口，比如从媒体MMZ内存分配空间。

【定义】

```
typedef struct ot_MBUF_ABSTRACTFUNC_S
{
    Mbuf_Alloc_Func      pfnAlloc;
    Mbuf_Free_Func       pfnFree;
    Mbuf_Mcopy_Func      pfnMemcpy;
    Mbuf_CreateLock_Func pfnCreateLock;
    Mbuf_DestroyLock_Func pfnDestroyLock;
    Mbuf_Lock_Func        pfnLock;
    Mbuf_UnLock_Func      pfnUnLock;
} OT_MBUF_ABSTRACTFUNC_S;
```

【成员】

成员名称	描述
pfnAlloc	内存申请抽象接口。
pfnFree	内存释放抽象接口，对应于内申请。
pfnMemcpy	内存拷贝抽象接口。
pfnCreateLock	多进程访问共享内存的互斥锁，创建接口。
pfnDestroyLock	多进程访问共享内存的互斥锁，销毁接口。
pfnLock	多进程对共享内存访问的加锁接口。
pfnUnLock	多进程对共享内存访问的解锁接口。

【注意事项】

pfnAlloc新申请的空间，请务必在接口实现中进行清零操作。

【相关数据类型及接口】

无。

4.4.3 OT_MBUF_CFG_S

【说明】

定义Mbuf缓存句柄相关属性。

【定义】

```
typedef struct ot_MBUF_CFG_S
{
    TD_U32 u32BufSize;
    TD_S32 s32MaxPayload;
}
```



```
OT_MBUF_ABSTRACTFUNC_S stMBufFunc;  
}OT_MBUF_CFG_S;
```

【成员】

成员名称	描述
u32BufSize	缓存区域大小。
s32MaxPayload	缓存中支持的可存储最大数据类型，范围：[0, 254]
stMBufFunc	注册缓存函数。

【注意事项】

无。

【相关数据类型及接口】

无。

4.4.4 OT_MBUF_SLICEINFO_S

【说明】

定义向Mbuf缓存读写片数据的相关属性。

【定义】

```
typedef struct ot_MBUF_SLICEINFO_S  
{  
    TD_U64 u64Pts;  
    TD_U32 u32Seq;  
    TD_U32 u32FrameLen;  
    TD_U32 u32SliceLen;  
    TD_U32 u32HeadLen;  
    TD_U32 u32TailLen;  
    TD_S32 s32SliceNum;  
    TD_S32 s32SliceCnt;  
    TD_BOOL blsKeyFrame;  
    TD_U8 u8PayloadType;  
    TD_U8 u8SliceType;  
    TD_VOID *pAddr[OT_MBUF_FRAME_MAX_BLOCK];  
    TD_U32 au32BlockLen[OT_MBUF_FRAME_MAX_BLOCK];TD_U32 u32BlockCount;  
}OT_MBUF_SLICEINFO_S;
```

【成员】

成员名称	描述
u64Pts	数据时间戳
u32Seq	数据序列号
u32FrameLen	帧数据长度，一帧由1到多个片组成
u32SliceLen	分片数据长度



成员名称	描述
u32HeadLen	分片数据头部预留空间长度，一般取0
u32TailLen	分片数据尾部预留空间长度，一般取0
s32SliceNum	分片序列号，从1开始
s32SliceCnt	一帧数据由几个分片组成
blsKeyFrame	当前帧是否是关键帧
u8PayloadType	当前分片的数据类型
u8SliceType	当前分片的类型。 0x01：起Slice； 0x02：中间Slice； 0x04：结束Slice； 0x05：单帧Slice。
pAddr	分片数据地址
au32BlockLen	分片数据块长度
u32BlockCount	分片数据块数

【注意事项】

对于一块数据由多个片数据组成的数据类型，比如视频帧Frame，可以是多个分片Slice组成，支持Slice写入，以frame方式读取。

【相关数据类型及接口】

无。

4.4.5 OT_MBUF_FRAMEINFO_S

【说明】

定义从Mbuf缓存读取帧数据的相关属性。

【定义】

```
typedef struct ot_MBUF_FRAMEINFO_S
{
    TD_S32 s32SliceCnt;
    OT_MBUF_SLICEINFO_S *pstSliceInfo;
}OT_MBUF_FRAMEINFO_S;
```

【成员】

成员名称	描述
s32SliceCnt	帧内包含片个数。
pstSliceInfo	每个片相关数据信息。



【注意事项】

无。

【相关数据类型及接口】

无。

4.4.6 Mbuf_Alloc_Func

【说明】

定义Mbuf模块缓存所需内存空间申请的抽象接口类型。

【定义】

```
typedef TD_S32 (*Mbuf_Alloc_Func)(const TD_VOID *argv, TD_U32 u32BufSize, TD_VOID  
**ppVmAddr);
```

【参数】

参数名称	描述	输入/输出
argv	用户透传数据。	输入
u32BufSize	申请空间大小	输入
ppVmAddr	返回的内存地址	输出

【返回值】

返回值	描述
0	成功。
非0	失败。

【注意事项】

此处返回的地址，是用户定义的地址，可以为栈内存，也可以是MMZ内存地址。

【相关数据类型及接口】

无。

4.4.7 Mbuf_Free_Func

【说明】

定义Mbuf模块缓存所需内存空间释放的抽象接口类型。

【定义】



```
typedef TD_S32 (*Mbuf_Free_Func)(const TD_VOID *argv,TD_VOID *pVmAddr);
```

【参数】

参数名称	描述	输入/输出
argv	用户透传数据。	输入
pVmAddr	释放的内存地址。	输入

【返回值】

返回值	描述
0	成功。
非0	失败。

【注意事项】

无。

【相关数据类型及接口】

无。

4.4.8 Mbuf_Mcopy_Func

【说明】

定义Mbuf模块缓存所需内存空间数据拷贝的抽象接口类型。

【定义】

```
typedef TD_S32 (*Mbuf_Mcopy_Func)(const TD_VOID *argv,TD_VOID *pDest,  
const TD_VOID *pSrc,TD_U32 u32Size);
```

【参数】

参数名称	描述	输入/输出
argv	用户透传数据。	输入
pDest	目的地址。	输入
pSrc	源地址。	输入
u32Size	拷贝数据大小。	输入

【返回值】



返回值	描述
0	成功。
非0	失败。

【注意事项】

无。

【相关数据类型及接口】

无。

4.4.9 Mbuf_CreateLock_Func

【说明】

定义Mbuf模块缓存进行读写所需互斥锁创建抽象接口类型。

【定义】

```
typedef TD_S32 (*Mbuf_CreateLock_Func)(const TD_VOID *argv);
```

【参数】

参数名称	描述	输入/输出
argv	用户透传数据。	输入

【返回值】

返回值	描述
0	成功。
非0	失败。

【注意事项】

当不需要跨进程访问时，该接口实现可为空实现。

【相关数据类型及接口】

无。

4.4.10 Mbuf_DestroyLock_Func

【说明】

定义Mbuf模块缓存进行读写所需互斥锁销毁抽象接口类型。

【定义】



```
typedef TD_S32 (*Mbuf_DestroyLock_Func)(const TD_VOID *argv);
```

【参数】

参数名称	描述	输入/输出
argv	用户透传数据。	输入

【返回值】

返回值	描述
0	成功。
非0	失败。

【注意事项】

当不需要跨进程访问时，该接口实现可为空实现。

【相关数据类型及接口】

无。

4.4.11 Mbuf_Lock_Func

【说明】

定义Mbuf模块缓存进行读写所需互斥锁加锁抽象接口类型。

【定义】

```
typedef TD_S32 (*Mbuf_Lock_Func)(const TD_VOID *argv);
```

【参数】

参数名称	描述	输入/输出
argv	用户透传数据。	输入

【返回值】

返回值	描述
0	成功。
非0	失败。

【注意事项】



当不需要跨进程访问时，该接口实现可为空实现。

【相关数据类型及接口】

无。

4.4.12 Mbuf_UnLock_Func

【说明】

定义Mbuf模块缓存进行读写所需互斥锁解锁抽象接口类型。

【定义】

```
typedef TD_S32 (*Mbuf_UnLock_Func)(const TD_VOID *argv);
```

【参数】

参数名称	描述	输入/输出
argv	用户透传数据。	输入

【返回值】

返回值	描述
0	成功。
非0	失败。

【注意事项】

当不需要跨进程访问时，该接口实现可为空实现。

【相关数据类型及接口】

无

4.5 错误码

媒体缓存API错误码如表4-1所示。

表 4-1 媒体缓存模块 API 错误码

错误代码	宏定义	描述
0	OT_SUCCESS	成功
-1	OT_FAILURE	失败
-2	OT_ERR_MBUF_ILLEGALPARAM	接口参数非法
-3	OT_ERR_MBUF_REINTIALIZE	重复初始化



错误代码	宏定义	描述
-4	OT_ERR_MBUF_UNINITIALIZE	未初始化
-5	OT_ERR_MBUF_FULL	Buffer满，无法写入
-6	OT_ERR_MBUF_NODATA	无指定PayloadType类型的数据
-7	OT_ERR_MBUF_REACHLIMIT	PayloadType达到注册上限
-8	OT_ERR_MBUF_DISABLED	读/写未使能，导致无法读/写
-9	OT_ERR_MBUF_DISLOCATED	当前可读取片不是起始片，也就是读指针错位，该情况下只能按Slice读取,无法按Frame读取
-10	OT_ERR_MBUF_UNCOMPLETED	当前帧数据未写完全，只写入了部分分片，导致无法读取到完整帧
-11	OT_ERR_MBUF_NOTKEYFRAME	当前帧为非关键帧。

4.6 典型场景使用指南

4.6.1 视频点播

4.6.1.1 场景分析

视频点播时，媒体传输模块使用媒体缓存模块进行数据的缓存。

4.6.1.2 代码示例

参见版本sample。



5 MP4 封装解封装模块

5.1 概述

MP4封装解封装模块用于录像时的标准MP4格式文件封装，本地回放时的MP4文件解封装，支持MP4标准协议。

5.2 功能描述

5.2.1 基本概念

MP4：是一套用于音频、视频信息的压缩编码标准，由国际标准化组织（ISO）和国际电工委员会（IEC）下属的“动态图像专家组”（Moving Picture Experts Group，即MPEG）制定。

- 2001年，MP4文件格式第一版本MPEG-4 Part1发布。
- 2003年，引入Apple MOV文件格式，创建MPEG-4 Part12规范“ISO Base Media File Format”。2004年，基于Part12，加入一些新特性，形成Part14“MP4 File Format”，同年，发布Part15“AVC File Format”。

5.2.2 功能描述

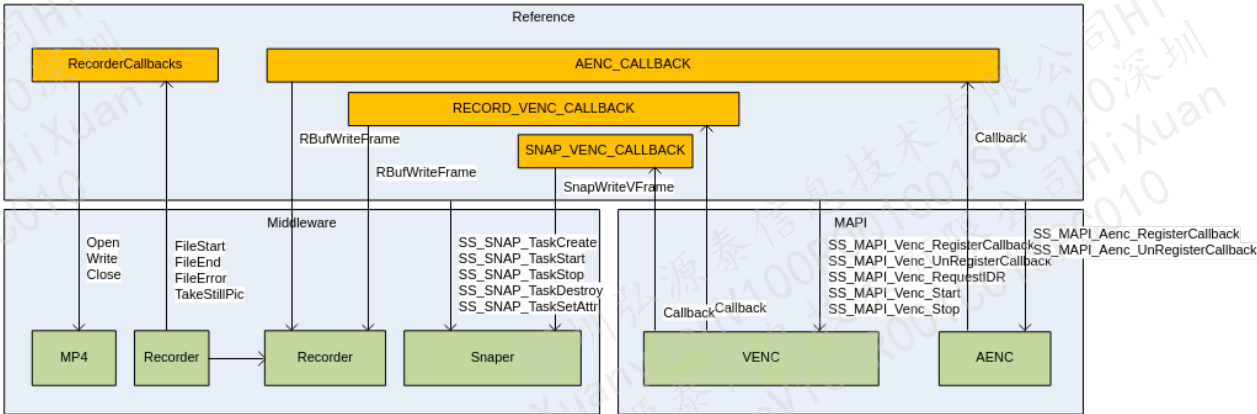
当前文件封装MP4支持标准H.264、H.265视频编码格式，AAC、MP4、G.711 μ -law和G.711 A-law音频编码格式MP4文件封装，支持GPS、陀螺仪、mjpeg等私有数据封装，支持不依赖备份文件的修复功能，支持用户选择moov在MP4文件里相对于mdat的位置。

5.2.3 上下文描述

文件封装MP4组件软件上下文依赖关系如图5-1所示。对下依赖具体封装实现，MP4根据上层sample app对录像（RECORD）功能调用，依赖文件管理(DCF)模块提供的文件路径文件名，对接收到的媒体流数据进行标准格式MP4文件封装。上层用户开机或插卡遇到损坏文件依赖MP4模块实现文件修复功能。用户调用MP4组件的解封装接口实现对mp4文件的解析，包括文件时长、厂商信息、缩略图等内容的解析。



图 5-1 文件封装 MP4 组件软件上下文依赖关系



5.3 API 参考

该模块支持：

- 文件封装解封装mp4实例创建、销毁；
- track创建、销毁、track信息获取；
- 私有atom添加及读取；
- 文件信息获取；
- 文件修复等。

该功能模块提供以下API：

- **SS_MP4_Create**：MP4实例创建。
- **SS_MP4_Destroy**：MP4实例销毁。
- **SS_MP4_CreateTrack**：为MP4实例创建track。
- **SS_MP4_DestroyAllTracks**：为MP4实例销毁track。
- **SS_MP4_WriteFrame**：MP4实例中写帧数据。
- **SS_MP4_AddAtom**：MP4实例添加私有atom。
- **SS_MP4_RepairFile**：进行文件修复。
- **SS_MP4_GetAtom**：获取atom信息。
- **SS_MP4_GetFileInfo**：获取文件信息。
- **SS_MP4_GetTrackInfoByIndex**：根据下标获取track信息。
- **SS_MP4_GetFileSize**：获取文件大小。

5.3.1 SS_MP4_Create

【描述】

MP4实例创建。

【语法】

TD_S32 SS_MP4_Create(TD_MW_PTR* ppMp4, const OT_MP4_CONFIG_S * pstMP4Cfg);



【参数】

参数名称	描述	输入/输出
ppMp4	MP4实例句柄。	输出
pstMP4Cfg	参数配置信息。	输入

【返回值】

返回值	描述
0	成功。
非0	失败，参见 错误码 。

【需求】

- 头文件：ss_mp4_format.h
- 库文件：libmp4.a、libmp4.so

【注意】

支持重复创建实例，但会返回错误，该模块支持多实例，最多支持32个MP4实例。

【举例】

本例中演示了MP4模块的初始化过程，包括必要的参数设置，具体可以参见本模块的sample代码。

```
/*init the g_pstMP4Cfg and create MP4*/
OT_MP4_CONFIG_S stMp4Cfg;
memset(&stMp4Cfg, 0x00, sizeof(OT_MP4_CONFIG_S));
snprintf(stMp4Cfg.aszFileName, OT_MP4_MAX_FILE_NAME, "/tmp/sample/test_avc.mp4");
stMp4Cfg.enConfigType = OT_MP4_CONFIG_MUXER;
stMp4Cfg.stMuxerConfig.enFormatProfile = OT_MP4_FORMAT_MP42;
stMp4Cfg.stMuxerConfig.u32PreAllocUnit = 0;
stMp4Cfg.stMuxerConfig.u32VBufSize = 1024 * 1024;
stMp4Cfg.stMuxerConfig.bConstantFps = TD_TRUE;
stMp4Cfg.stMuxerConfig.bCo64Flag = TD_FALSE;
stMp4Cfg.stMuxerConfig.u32BackupUnit = 5 * 1024 * 1024;
/*create muxer*/
printf("create muxer\n");
s32Ret = SS_MP4_Create(&pMP4Muxer, &stMp4Cfg);
if (TD_SUCCESS != s32Ret)
{
    printf("create mp4 fail %d \n", s32Ret);
    return TD_FAILURE;
}
```

【相关主题】

无。



5.3.2 SS_MP4_Destroy

【描述】

MP4实例销毁。

【语法】

```
TD_S32 SS_MP4_Destroy(TD_MW_PTR pMp4, TD_U64* pu64Duration);
```

【参数】

参数名称	描述	输入/输出
pMp4	MP4实例句柄。	输入
pu64Duration	MP4文件录制时长。	输出

【返回值】

返回值	描述
0	成功。
非0	失败，返回 错误码 。

【需求】

- 头文件：ss_mp4_format.h
- 库文件：libmp4.a、libmp4.so

【注意】

支持重复销毁实例，但会返回错误。

【相关主题】

无。

5.3.3 SS_MP4_CreateTrack

【描述】

为MP4创建track。

【语法】

```
TD_S32 SS_MP4_CreateTrack(TD_MW_PTR pMp4, TD_MW_PTR* ppTrack, const  
OT\_MP4\_TRACK\_INFO\_S\* pstTrackInfo);
```

【参数】



参数名称	描述	输入/输出
pMp4	MP4实例句柄。	输入
ppTrack	Track句柄。	输出
pstTrackInfo	Track配置数据。	输入

【返回值】

返回值	描述
0	成功。
非0	失败，返回 错误码 。

【需求】

- 头文件：ss_mp4_format.h
- 库文件：libmp4.a、libmp4.so

【注意】

- 请在创建MP4实例后调用此接口。
- 目前pstTrackInfo下的enTrackType参数为Video时只支持H264、H265；enTrackType为Audio时支持AAC、MP4、G.711 μ -law和G.711 A-law。当封装的视频track为H.264、H.265码流时，请设置muxer config中的enFormatProfile为OT_MP4_FORMAT_MP42，即为base media/version 2格式MP4文件。

【举例】

无。

【相关主题】

无。

5.3.4 SS_MP4_DestroyAllTracks

【描述】

为MP4销毁track。

【语法】

TD_S32 SS_MP4_DestroyAllTracks(TD_MW_PTR pMp4, [OT_MP4_SETDUTAION_FN](#) pfnSetDuration);

【参数】

参数名称	描述	输入/输出
pMp4	MP4实例track句柄。	输入



参数名称	描述	输入/输出
pfnSetDuration	设置duration回调。	输入

【返回值】

返回值	描述
0	成功。
非0	失败，返回错误码。

【需求】

- 头文件：ss_mp4_format.h
- 库文件：libmp4.a、libmp4.so

【注意】

- 请在创建MP4实例后调用此接口，该接口支持在写moov数据前设置封装duration属性，该回调参数为空时表示不设置duration属性，采用封装中实际的时长信息。
- 在调用SS_MP4_DestroyAllTracks接口前请确保已经调用SS_MP4_WriteFrame接口写入帧数据。

【举例】

无。

【相关主题】

无。

5.3.5 SS_MP4_WriteFrame

【描述】

MP4中写入帧数据。

【语法】

```
TD_S32 SS_MP4_WriteFrame(TD_MW_PTR pMp4, TD_MW_PTR pTrack, const OT_MP4_FRAME_DATA_S* pstFrameData);
```

【参数】

参数名称	描述	输入/输出
pMp4	MP4实例句柄。	输入
pTrack	Track句柄。	输入
pstFrameData	帧数据信息。	输入



【返回值】

返回值	描述
0	成功。
非0	失败，返回 错误码 。

【需求】

- 头文件：ss_mp4_format.h
- 库文件：libmp4.a、libmp4.so

【注意】

- 请在创建MP4实例及track之后调用该接口。
- MP4封装组件内存占用和录制的帧数强相关，封装时各个box的信息存在内存中，除每个trak中包含的一个stbl大小不确定，trak个数不确定，其余box占用内存为固定值。假设长时间录制一共生成了n帧数据，每个stbl占用内存计算公式如下：
 $Memory = 2n * 4 (stts \text{ 最小为 } 8) + n * 4 / GOP(stss) + 4n(stsz) + 4n(stco)$ 。
本模块采用C++语言实现，由于C++存在一些隐藏的内存分配，实际占用内存会略大于Memory。
- 用户请确保系统内存支持录制的业务场景。
- 同理，解析mp4文件的时候为了获取文件的各项信息，会将文件中各trak信息读取到内存中，请确保系统内存能够支持用户解析mp4文件的场景。
- 如果当前文件大小达到最大限制，则不再写入数据并且返回错误码。当前文件大小受限于FAT32限制，为3.99GB。

【举例】

无。

【相关主题】

无。

5.3.6 SS_MP4_AddAtom

【描述】

MP4添加私有atom。

【语法】

```
TD_S32 SS_MP4_AddAtom(TD_MW_PTR pMp4, const TD_CHAR* pszAtomPath, const  
OT\_MP4\_ATOM\_INFO\_S\* pstAtomInfo);
```

【参数】

参数名称	描述	输入/输出
pMp4	MP4实例句柄。	输入



参数名称	描述	输入/输出
pszAtomPath	加入MP4的路径。	输入
pstAtomInfo	私有atom的信息。	输入

【返回值】

返回值	描述
0	成功。
非0	失败，返回 错误码 。

【需求】

- 头文件：ss_mp4_format.h
- 库文件：libmp4.a、libmp4.so

【注意】

添加私有atom时，可以添加在最外层和moov同级，此时pszAtomPath为“/”；可以添加在moov上和mvhd同级，此时pszAtomPath为“/moov”；也可以添加在track上，此时pszAtomPath为“/moov/trak[index]”（index表示第几个添加的trak），以此类推路径可自己选择，请注意不支持在顶级目录添加mp4封装中的特有类型ftyp、mdat、moov等私有atom，不支持在固定格式的atom下添加私有atom（如ftyp、mdat、stbl、stts下等）。

【举例】

无。

【相关主题】

无。

5.3.7 SS_MP4_RepairFile

【描述】

进行文件修复。

【语法】

```
TD_S32 SS_MP4_RepairFile(const TD_CHAR* aszFilename, TD_BOOL bParseExtraMdat);
```

【参数】

参数名称	描述	输入/输出
aszFilename	待修复MP4主文件名。	输入



参数名称	描述	输入/输出
bParseExtraMdat	是否修复至最后一帧。	输入

【返回值】

返回值	描述
0	修复成功。
非0	修复失败，返回 错误码 。

【需求】

- 头文件：ss_mp4_format.h
- 库文件：libmp4.a、libmp4.so

【注意】

- 使用时该接口耗时涉及读文件操作及文件seek操作，以及分段表的解析操作，如果用户需要降低修复大文件的耗时，需要适当降低修复精度：在文件封装时增大备份间隔u32BackupUnit，或者将本接口的bParseExtraMdat标志置为0。
- 为了能够顺利解析并修复文件内容，请用户遵循接口调用规则：创建mp4 muxer实例(开启文件修复)，创建所需各个mp4 track，向各track写数据或者添加私有box，销毁各个mp4 track，销毁mp4 muxer实例。

【举例】

无。

【相关主题】

无。

5.3.8 SS_MP4_GetAtom

【描述】

MP4获取atom信息。

【语法】

```
TD_S32 SS_MP4_GetAtom(TD_MW_PTR pMp4, TD_CHAR* pszAtomPath,  
OT_MP4_ATOM_INFO_S* pstAtomInfo);
```

【参数】

参数名称	描述	输入/输出
pMp4	MP4实例句柄。	输入
pszAtomPath	atom的路径。	输入



参数名称	描述	输入/输出
pstAtomInfo	atom的信息。	输出

【返回值】

返回值	描述
0	成功。
非0	失败，返回 错误码 。

【需求】

- 头文件：ss_mp4_format.h
- 库文件：libmp4.a、libmp4.so

【注意】

根据路径获取atom信息，如获取缩略图“thm”atom的长度，类型，数据buffer等信息，内存由外部申请释放，当入参atom长度小于实际长度时，返回错误并返回实际长度，当buffer长度够时，返回atom数据。

【举例】

无。

【相关主题】

无。

5.3.9 SS_MP4_GetFileInfo

【描述】

MP4获取文件信息。

【语法】

```
TD_S32 SS_MP4_GetFileInfo(TD_MW_PTR pMp4, OT_MP4_FILE_INFO_S* pstFileInfo);
```

【参数】

参数名称	描述	输入/输出
pMp4	MP4实例句柄。	输入
pstFileInfo	文件信息。	输出

【返回值】



返回值	描述
0	成功。
非0	失败，返回 错误码 。

【需求】

- 头文件：ss_mp4_format.h
- 库文件：libmp4.a、libmp4.so

【注意】

打开已存在MP4文件后，调用该接口获取该文件信息。

【举例】

无。

【相关主题】

无。

5.3.10 SS_MP4_GetTrackInfoByIndex

【描述】

MP4根据下标获取track信息。

【语法】

```
TD_S32 SS_MP4_GetTrackInfoByIndex(TD_MW_PTR pMp4, TD_U32 u32Index,  
OT\_MP4\_TRACK\_INFO\_S\* pTrackInfo,TD_MW_PTR *ppTrack);
```

【参数】

参数名称	描述	输入/输出
pMp4	MP4实例句柄。	输入
u32Index	下标。	输入
pTrackInfo	track信息。	输出
ppTrack	Track句柄。	输出

【返回值】

返回值	描述
0	成功。
非0	失败，返回 错误码 。



【需求】

- 头文件：ss_mp4_format.h
- 库文件：libmp4.a、libmp4.so

【注意】

- 根据下标获取track信息以及对应的句柄，第一个加入mp4文件的track下标为0，当下标超过MP4文件中track总数目时返回错误。
- 当前支持解析的track类型：pTrackInfo下的enTrackType参数为Video时只支持H264、H265；enTrackType为Audio时支持AAC、MP3、G.711 μ -law和G.711 A-law。
- MP4文件具有备份信息时，从track里获取出的fSpeed有效，反之则无效。
- MP4文件具有备份信息时，从音频track里获取出的u32SamplePerFrame有效，反之则无效。

【举例】

无。

【相关主题】

无。

5.3.11 SS_MP4_GetFileSize

【描述】

获取文件大小。

【语法】

TD_U64 SS_MP4_GetFileSize(TD_MW_PTR pMp4Handle)

【参数】

参数名称	描述	输入/输出
pMp4	MP4实例句柄。	输入

【返回值】

返回值	描述
非0	文件大小。

【需求】

- 头文件：ss_ts_format.h
- 库文件：libts.a、libts.so

【注意】



无。

【举例】

无。

5.4 数据结构

相关数据类型、数据结构定义如下：

- **OT_MP4_CONFIG_S**：MP4配置参数结构体。
- **OT_MP4_CONFIG_MUXER_S**：MP4封装配置参数结构体。
- **OT_MP4_CONFIG_DEMUXER_S**：MP4解封装配置参数结构体。
- **OT_MP4_CONFIG_APPEND_S**：MP4追加封装配置参数结构体。
- **OT_MP4_ATOM_INFO_S**：私有atom信息结构体。
- **OT_MP4_FRAME_DATA_S**：帧数据信息结构体。
- **OT_MP4_TRACK_INFO_S**：track信息结构体。
- **OT_MP4_DATAINFO_S**：metadata信息结构体。
- **OT_MP4_AUDIOINFO_S**：audio音频信息结构体。
- **OT_MP4_VIDEOINFO_S**：视频信息结构体。
- **OT_MP4_FILE_INFO_S**：文件信息结构体。
- **OT_MP4_CONFIG_TYPE_E**：mp4配置类型。
- **OT_MP4_FORMAT_PROFILE_E**：文件格式。
- **OT_MP4_CODECID_E**：编码类型枚举。
- **OT_MP4_TRACK_TYPE_E**：track类型枚举。
- **OT_MP4_SETDURATION_FN**：设置封装文件中duration时长信息的回调。
- **OT_MP4_MAX_FILE_NAME**：文件名最大长度（包含文件路径）。
- **OT_MP4_MAX_BOX_PATH**：私有atom路径名最大长度。
- **OT_MP4_MAX_HDLR_NAME**：hdlrname最大长度。
- **OT_MP4_BOX_MAX_DATALEN**：私有atom最大数据长度。
- **OT_MP4_TYPE_SIZE**：私有atom类型名的长度。
- **OT_MP4_MOOV_LOCATION_E**：moov相对于mdat位置的枚举。
- **OT_MP4_MOOV_AT_TAIL_INFO_S**：moov放置在mdat后面时的参数结构体。
- **OT_MP4_MOOV_AT_FRONT_INFO_S**：moov放置在mdat前面时的参数结构体。

5.4.1 OT_MP4_CONFIG_S

【说明】

定义MP4配置参数结构体。

【定义】

```
typedef struct MP4_CONFIG_S
{
```



```
TD_CHAR aszFileName[OT_MP4_MAX_FILE_NAME];/*file path and file name*/
OT_MP4_CONFIG_TYPE_E enConfigType;
union
{
    OT_MP4_CONFIG_MUXER_S stMuxerConfig;
    OT_MP4_CONFIG_DEMUXER_S stDemuxerConfig;
    OT_MP4_CONFIG_APPEND_S stAppendConfig;
};
} OT_MP4_CONFIG_S;
```

【成员】

成员名称	描述
aszFileName	文件名，包含文件路径。
enConfigType	配置类型枚举：封装实例、解封装实例、追加封装实例配置信息。
stMuxerConfig	封装实例配置信息。
stDemuxerConfig	解封装实例配置信息
stAppendConfig	追加封装实例配置信息。

【注意事项】

aszFileName保证路径存在，路径不存在接口返回错误。

【相关数据类型及接口】

无。

5.4.2 OT_MP4_CONFIG_MUXER_S

【说明】

定义MP4封装配置参数结构体。

【定义】

```
typedef struct MP4_CONFIG_MUXER_S {
    /* pre allocate size in bytes, [0,100M], 0 for not use pre allocate function,suggest 20M,
    unit :byte */
    TD_U32 u32PreAllocUnit;
    TD_U32 u32VBufSize;          /* set the vbuf size for fwrite (0,5M] unit :byte */
    TD_BOOL bCo64Flag;          /* if true;use co64,or use stco,used for muxer */
    TD_BOOL bConstantFps;       /* if true, use constant framerate to calculate time,or use
    pst delta */
    TD_U32 u32BackupUnit;        /* stbl group unit [512k,500M], if 0 not use backup data
    and repair */
    TD_CHAR fopenMode[OT_MODE_SIZE]; /* muxer fopen mode */
    OT_MP4_FORMAT_PROFILE_E enFormatProfile; // mpeg-4 file profile,base medie or
    quicktime
    enum Mp4MoovLocation moovLocation;
    union {
        OT_MP4_MOOV_AT_TAIL_INFO_S moovAtTail;
        Mp4MoovAtFront moovAtFront;
    };
} OT_MP4_CONFIG_MUXER_S;
```



【成员】

成员名称	描述
u32PreAllocUnit	预分配单元大小，单位：字节，当前采用多次预分配方案，为0时表示不开启预分配功能，取值范围：[0, 100M]。
u32VBufSize	文件IO写数据缓冲大小，单位：字节。
bCo64Flag	是否开启64位chunk offset封装标志。
bConstantFps	是否采用固定帧率模式计算文件时长。
u32BackupUnit	备份数据单元，每u32BackupUnit字节数据在MP4文件中存入一份分段索引表（stbl group），用于修复时重组moov信息。
fopenMode	封装器fopen的参数配置
enFormatProfile	文件格式配置。
moovLocation	选择moov与mdat位置关系。
moovAtTail	mdat在moov前的参数。
moovAtFront	mdat在moov后的参数。

【注意事项】

无。

【相关数据类型及接口】

无。

5.4.3 OT_MP4_CONFIG_DEMUXER_S

【说明】

定义MP4解封装配置参数结构体。

【定义】

```
typedef struct MP4_CONFIG_DEMUXER_S
{
    TD_U32 u32VBufSize; /*set the vbuf size for fwrite (0,5M] unit :byte*/
} OT_MP4_CONFIG_DEMUXER_S;
```

【成员】

成员名称	描述
u32VBufSize	文件IO写数据缓冲大小，单位：字节。

【注意事项】

无。



【相关数据类型及接口】

无。

5.4.4 OT_MP4_CONFIG_APPEND_S

【说明】

定义MP4追加封装配置参数结构体。

【定义】

```
typedef struct MP4_CONFIG_APPENDS_S
{
    TD_U32 u32VBufSize; /*set the vbuf size for fwrite (0,5M] unit :byte*/
} OT_MP4_CONFIG_APPEND_S;
```

【成员】

成员名称	描述
u32VBufSize	文件IO写数据缓冲大小，单位：字节。

【注意事项】

无。

【相关数据类型及接口】

无。

5.4.5 OT_MP4_ATOM_INFO_S

【说明】

定义私有atom信息结构体。

【定义】

```
typedef struct MP4_ATOM_INFO_S
{
    TD_U32 u32DataLen; /*user define atom data len */
    TD_CHAR aszType[OT_MP4_TYPE_SIZE + 1]; /*user define atom type*/
    TD_U8* pu8DataBuf; /*user define atom data buffer*/
} OT_MP4_ATOM_INFO_S;
```

【成员】

成员名称	描述
u32DataLen	私有atom数据长度。
aszType	私有atom类型。
pu8DataBuf	私有atom数据指针。



【 注意事项 】

无。

【 相关数据类型及接口 】

无。

5.4.6 OT_MP4_FRAME_DATA_S

【 说明 】

帧数据信息结构体。

【 定义 】

```
typedef struct MP4_FRAME_DATA_S
{
    TD_U64 u64TimeStamp ;/*frame timestamp*/
    TD_U8* pu8DataBuffer; /*frame data buffer*/
    TD_U32 u32DataLength; /*frame data len*/
    TD_BOOL bKeyFrameFlag; /*key frame flag*/
} OT_MP4_FRAME_DATA_S;
```

【 成员 】

成员名称	描述
u64TimeStamp	媒体流帧数据时间戳。
pu8DataBuffer	媒体流帧数据buffer。
u32DataLength	数据流帧长度。
bKeyFrameFlag	媒体流数据是否为I帧。

【 注意事项 】

无。

【 相关数据类型及接口 】

无。

5.4.7 OT_MP4_TRACK_INFO_S

【 说明 】

定义track信息。

【 定义 】

```
typedef struct MP4_TRACK_INFO_S
{
    OT_MP4_TRACK_TYPE_E enTrackType; /*stream type*/
    TD_U32 u32TimeScale; /*time scale for each trak*/
    TD_FLOAT fSpeed; /*play speed, (0,1]for slow,(1,~) for fast*/
    TD_CHAR aszHdlrName[OT_MP4_MAX_HDLR_NAME]; /*manufacturer name */
    union
```



```
{
    OT_MP4_VIDEOINFO_S stVideoInfo; /*video info*/
    OT_MP4_AUDIOINFO_S stAudioInfo; /*audio info*/
    OT_MP4_DATAINFO_S stDataInfo; /*metadata info*/
};
} OT_MP4_TRACK_INFO_S;
```

【成员】

成员名称	描述
enTrackType	Track类型。
u32TimeScale	Track的TimeScale，时间精度，即一秒被分为TimeScale份，一般video和data的TimeScale为120000，audio的TimeScale和音频采样率一致（例如音频采样率sampleRate为48000，则音频TimeScale为48000）。
fSpeed	播放速率。
aszHdlrName	制造商信息hdlrname。
stVideoInfo	视频信息。
stAudioInfo	音频信息。
stDataInfo	metadata数据信息。

【注意事项】

无。

【相关数据类型及接口】

- OT_MP4_VIDEOINFO_S
- OT_MP4_AUDIOINFO_S
- OT_MP4_DATAINFO_S

5.4.8 OT_MP4_DATAINFO_S

【说明】

媒体流metadata数据信息结构体。

【定义】

```
typedef struct MP4_DATAINFO_S
{
    TD_U32 u32Width; /*meta data width*/
    TD_U32 u32Height; /*meta data height*/
    TD_FLOAT frameRate; /*frame rate fps*/
    OT_MP4_CODECID_E enCodecID; /*codecId*/
} OT_MP4_DATAINFO_S;
```

【成员】



成员名称	描述
u32Width	meta data数据宽信息（当meta data存视频数据时需要设置该属性）
u32Height	meta data数据高信息（当meta data存视频数据时需要设置该属性）
frameRate	meta data数据帧率信息（当meta data为视频数据时设置该属性）
enCodecID	编码类型。

【注意事项】

无。

【相关数据类型及接口】

无。

5.4.9 OT_MP4_AUDIOINFO_S

【说明】

媒体流音频信息结构体。

【定义】

```
typedef struct MP4_AUDIOINFO_S
{
    TD_U32      u32Channels;/*audio channel num 2 */
    TD_U32      u32SampleRate;/*audio sample rate 48k*/
    TD_U32      u32SamplePerFrame;/*audio sample per frame*/
    TD_U16      u16SampleSize;/*bit per sample , 16*/
    OT_MP4_CODECID_E enCodecID;/*codec type*/
} OT_MP4_AUDIOINFO_S;
```

【成员】

成员名称	描述
u32Channels	音频通道。
u32SampleRate	音频采样率。
u32SamplePerFrame	音频每帧采样点数。
u16SampleSize	每个采样点大小。
enCodecID	音频编码类型。

【注意事项】

无。



【相关数据类型及接口】

无。

5.4.10 OT_MP4_VIDEOINFO_S

【说明】

媒体流视频信息结构体。

【定义】

```
typedef struct MP4_VIDEOINFO_S
{
    TD_U32 u32Width; /*video width*/
    TD_U32 u32Height; /*video height*/
    TD_U32      u32BitRate; /*bitrate bps*/
    TD_FLOAT      frameRate; /*frame rate fps*/
    OT_MP4_CODECID_E enCodecID; /*codec type*/
} OT_MP4_VIDEOINFO_S;
```

【成员】

成员名称	描述
u32Width	视频宽分辨率。
u32Height	视频高分辨率。
u32BitRate	码率。
frameRate	目的帧率。
enCodecID	编码类型。

【注意事项】

无。

【相关数据类型及接口】

无。

5.4.11 OT_MP4_FILE_INFO_S

【说明】

MP4文件信息结构体。

【定义】

```
typedef struct MP4_FILE_INFO_S
{
    TD_U32 u32Duration; //ms
    TD_CHAR aszHandlrName[OT_MP4_MAX_HDLR_NAME];
    TD_BOOL bCo64Flag;
    TD_U32 u32TrackNum;
    TD_U32 u32BackupUnit;
} OT_MP4_FILE_INFO_S;
```



【成员】

成员名称	描述
u32Duration	文件长度，使用单位：ms
aszHandlrName	制造商名称。
bCo64Flag	是否是64bit文件系统大文件。
u32TrackNum	Track数目。
u32BackupUnit	备份单元大小。

【注意事项】

无。

【相关数据类型及接口】

无。

5.4.12 OT_MP4_CONFIG_TYPE_E

【说明】

Mp4配置类型枚举。

【定义】

```
typedef enum MP4_CONFIG_TYPE_E{
    OT_MP4_CONFIG_MUXER = 1,
    OT_MP4_CONFIG_DEMUXER,
    OT_MP4_CONFIG_BUTT
} OT_MP4_CONFIG_TYPE_E;
```

【成员】

成员名称	描述
OT_MP4_CONFIG_MUXER	文件封装配置信息。
OT_MP4_CONFIG_DEMUXER	文件解封装配置信息。
OT_MP4_CONFIG_BUTT	无效值。

【注意事项】

无。

【相关数据类型及接口】

无。



5.4.13 OT_MP4_FORMAT_PROFILE_E

【说明】

文件格式。

【定义】

```
typedef enum MP4_FORMAT_PROFILE_E {  
    OT_MP4_FORMAT_MP42 = 0, // base media/version 2  
    OT_MP4_FORMAT_QT,       // quicktime movie,  
    OT_MP4_FORMAT_BUTT  
} OT_MP4_FORMAT_PROFILE_E;
```

【成员】

成员名称	描述
OT_MP4_FORMAT_MP42	MP42格式文件。
OT_MP4_FORMAT_QT	Quicktime格式文件。
OT_MP4_FORMAT_BUTT	无效值。

【注意事项】

无。

【相关数据类型及接口】

无。

5.4.14 OT_MP4_CODECID_E

【说明】

媒体流编码类型枚举。

【定义】

```
typedef enum MP4_CODECID_E {  
    OT_MP4_CODEC_ID_H264 = 0,  
    OT_MP4_CODEC_ID_H265,  
    OT_MP4_CODEC_ID_MJPEG,  
    OT_MP4_CODEC_ID_PRORES_422_PROXY,//apco  
    OT_MP4_CODEC_ID_PRORES_422_LT,//apcs  
    OT_MP4_CODEC_ID_PRORES_422,//apcn  
    OT_MP4_CODEC_ID_PRORES_422_HQ,//apch  
    OT_MP4_CODEC_ID_AACLC,  
    OT_MP4_CODEC_ID_MP3,  
    OT_MP4_CODEC_ID_G726,  
    OT_MP4_CODEC_ID_G711_A,  
    OT_MP4_CODEC_ID_G711_U,  
    OT_MP4_CODEC_ID_PCM,  
    OT_MP4_CODEC_ID_BUTT  
} OT_MP4_CODECID_E;
```

【成员】



成员名称	描述
OT_MP4_CODEC_ID_H264	视频H.264编码格式。
OT_MP4_CODEC_ID_H265	视频H.265编码格式。
OT_MP4_CODEC_ID_MJPEG	MJPEG编码格式。
OT_MP4_CODEC_ID_PRORES_422_PROXY	不支持。
OT_MP4_CODEC_ID_PRORES_422_LT	不支持。
OT_MP4_CODEC_ID_PRORES_422	不支持。
OT_MP4_CODEC_ID_PRORES_422_HQ	不支持。
OT_MP4_CODEC_ID_AACLC	AAC编码格式。
OT_MP4_CODEC_ID_MP3	MP3编码格式。
OT_MP4_CODEC_ID_G726	G726编码格式。
OT_MP4_CODEC_ID_G711_A	G.711 A-law编码格式。
OT_MP4_CODEC_ID_G711_U	G.711 μ -law编码格式。
OT_MP4_CODEC_ID_PCM	PCM编码格式。
OT_MP4_CODEC_ID_BUTT	无效值。

【注意事项】

无。

【相关数据类型及接口】

无。

5.4.15 OT_MP4_TRACK_TYPE_E

【说明】

track类型枚举。

【定义】

```
typedef enum MP4_TRACK_TYPE_E {  
    OT_MP4_STREAM_VIDEO = 1,  
    OT_MP4_STREAM_AUDIO,  
    OT_MP4_STREAM_DATA,  
    OT_MP4_STREAM_BUTT  
} OT_MP4_TRACK_TYPE_E;
```

【成员】

成员名称	描述
OT_MP4_STREAM_VIDEO	视频类型。



成员名称	描述
OT_MP4_STREAM_AUDIO	音频类型。
OT_MP4_STREAM_DATA	DATA类型。
OT_MP4_STREAM_BUTT	无效值。

【 注意事项 】

无。

【 相关数据类型及接口 】

无。

5.4.16 OT_MP4_SETDUTAION_FN

【 说明 】

设置封装文件中duration时长信息的回调。

【 定义 】

```
typedef TD_S32 (*OT_MP4_SETDUTAION_FN)(TD_MW_PTR pMp4, TD_U32 u32Timescale, TD_U64* pu64Duration);
```

【 成员 】

参数名称	描述
pMp4	MP4实例的句柄。
u32Timescale	封装文件的时间精度信息。
pu64Duration	封装文件中实际时长信息（以u32Timescale为精度）。

【 注意事项 】

该回调由上层用户实现，用户根据模块内上抛的MP4句柄、时间精度和实际时长确定是否根据丢帧情况及切分时长修改当前实际的时长信息。

【 相关数据类型及接口 】

无。

5.4.17 OT_MP4_MAX_FILE_NAME

【 说明 】

定义文件名最大长度。

【 定义 】

```
#define OT_MP4_MAX_FILE_NAME(256)
```

**【注意事项】**

包含文件路径及字符串结束符。

【相关数据类型及接口】

无。

5.4.18 OT_MP4_MAX_HDLR_NAME

【说明】

定义hdlrname最大长度。

【定义】

```
#define OT_MP4_MAX_HDLR_NAME (256)
```

【注意事项】

无。

【相关数据类型及接口】

无。

5.4.19 OT_MP4_MAX_BOX_PATH

【说明】

定义私有atom路径名最大长度。

【定义】

```
#define OT_MP4_MAX_BOX_PATH (64)
```

【注意事项】

无。

【相关数据类型及接口】

无。

5.4.20 OT_MP4_BOX_MAX_DATALEN

【说明】

定义私有atom最大数据长度。

【定义】

```
#define OT_MP4_BOX_MAX_DATALEN (64)
```

【注意事项】

无。

【相关数据类型及接口】

无。



5.4.21 OT_MP4_TYPE_SIZE

【说明】

定义私有atom类型名的长度。

【定义】

```
#define OT_MP4_TYPE_SIZE (4)
```

【注意事项】

无。

【相关数据类型及接口】

无。

5.4.22 OT_MP4_MOOV_LOCATION_E

【说明】

moov相对于mdat位置的枚举。

【定义】

```
typedef enum MP4_MOOV_LOCATION_E{  
    OT_MP4_MOOV_AT_TAIL = 0, /* write moov after mdat */  
    OT_MP4_MOOV_AT_FRONT, /* write moov before mdat, and user need to set nonzero  
fileDuration */  
    OT_MP4_MOOV_BUTT  
} OT_MP4_MOOV_LOCATION_E;
```

【成员】

成员名称	描述
OT_MP4_MOOV_AT_TAIL	moov放置在mdat后面。
OT_MP4_MOOV_AT_FRONT	moov放置在mdat的前面。
OT_MP4_MOOV_BUTT	无效值。

【注意事项】

无。

【相关数据类型及接口】

- [OT_MP4_MOOV_AT_FRONT_INFO_S](#)
- [OT_MP4_MOOV_AT_TAIL_INFO_S](#)

5.4.23 OT_MP4_MOOV_AT_TAIL_INFO_S

【说明】

moov放置在mdat后面时的参数结构体。

【定义】



```
typedef struct MP4_MOOV_AT_TAIL_INFO_S {
    TD_U32 reserved;
} OT_MP4_MOOV_AT_TAIL_INFO_S;
```

【成员】

成员名称	描述
reserved	保留。

【注意事项】

无。

【相关数据类型及接口】

OT_MP4_MOOV_LOCATION_E

5.4.24 OT_MP4_MOOV_AT_FRONT_INFO_S

【说明】

moov放置在mdat前面时的参数结构体。

【定义】

```
typedef struct MP4_MOOV_AT_FRONT_INFO_S {
    TD_U32 fileDuration; /* file duration for calculating the size of moov atom, in second */
} OT_MP4_MOOV_AT_FRONT_INFO_S;
```

【成员】

成员名称	描述
fileDuration	计算moov atom大小的文件时长，单位：秒。

【注意事项】

无。

【相关数据类型及接口】

OT_MP4_MOOV_LOCATION_E

5.5 错误码

MP4封装解封装API错误码如表5-1所示。

表 5-1 MP4 封装解封装 API 错误码

错误代码	宏定义	描述
0	TD_SUCCESS	成功



错误代码	宏定义	描述
-1	TD_FAILURE	失败
0xA1298002	OT_ERR_MP4_NULL_PTR	空指针
0xA1298040	OT_ERR_MP4_HANDLE_INVALID	无效句柄
0xA1298041	OT_ERR_MP4_INVALIDARG	非法参数
0xA1298042	OT_ERR_MP4_MALLOC	内存分配失败
0xA1298043	OT_ERR_MP4_CREATE	MP4创建失败
0xA1298044	OT_ERR_MP4_DESTROY	MP4销毁失败
0xA1298045	OT_ERR_MP4_WRITE_HEAD	MP4写文件头失败
0xA1298046	OT_ERR_MP4_WRITE_TAIL	MP4写文件尾失败
0xA1298047	OT_ERR_MP4_CREATE_AGAIN	MP4重复创建
0xA1298048	OT_ERR_MP4_NOT_CREATE	MP4未创建
0xA1298049	OT_ERR_MP4_READ_FRAME	MP4读帧失败
0xA1298050	OT_ERR_MP4_WRITE_FRAME	MP4写帧失败
0xA1298051	OT_ERR_MP4_REPAIR_UPDATE	MP4备份数据更新失败
0xA1298052	OT_ERR_MP4_CREATE_STREAM	MP4创建track失败
0xA1298053	OT_ERR_MP4_DESTROY_STREAM	MP4销毁track失败
0xA1298054	OT_ERR_MP4_GET_ATOM	获取atom失败
0xA1298055	OT_ERR_MP4_ADD_ATOM	添加atom失败
0xA1298056	OT_ERR_MP4_ATOM_LEN	获取atom长度失败
0xA1298061	OT_ERR_MP4_OPERATION_FAILED	操作失败
0xA1298062	OT_ERR_MP4_UNSUPPORTED_CODE C	不支持的编码格式
0xA1298063	OT_ERR_MP4_OPEN_FILE	打开文件失败
0xA1298064	OT_ERR_MP4_END_FILE	关闭文件失败
0xA1298065	OT_ERR_MP4_READ_FILE	读文件失败
0xA1298066	OT_ERR_MP4_SEEK_FILE	Seek文件失败
0xA1298067	OT_ERR_MP4_WRITE_FILE	写文件失败
0xA1298068	OT_ERR_MP4_REMOVE_FILE	删除文件失败
0xA1298069	OT_ERR_MP4_FILE_SIZE_OVERFLOW	文件大小超出限制



5.6 典型场景使用指南

参见版本sample。



6 EXIF 模块

6.1 概述

EXIF模块为用户提供图片解封装和图片编辑的功能。它可以获取MPF图片各子图的分辨率信息，MPF图片的缩略图数据以及EXIF图片中TAG信息。它也可以在EXIF图片中添加或更新用户相关的TAG，如XMP信息、厂商注释等。其它跟用户不相关的如光圈、快门速度、色彩空间等等由SDK内部封装。

TAG信息类似图6-1所示。

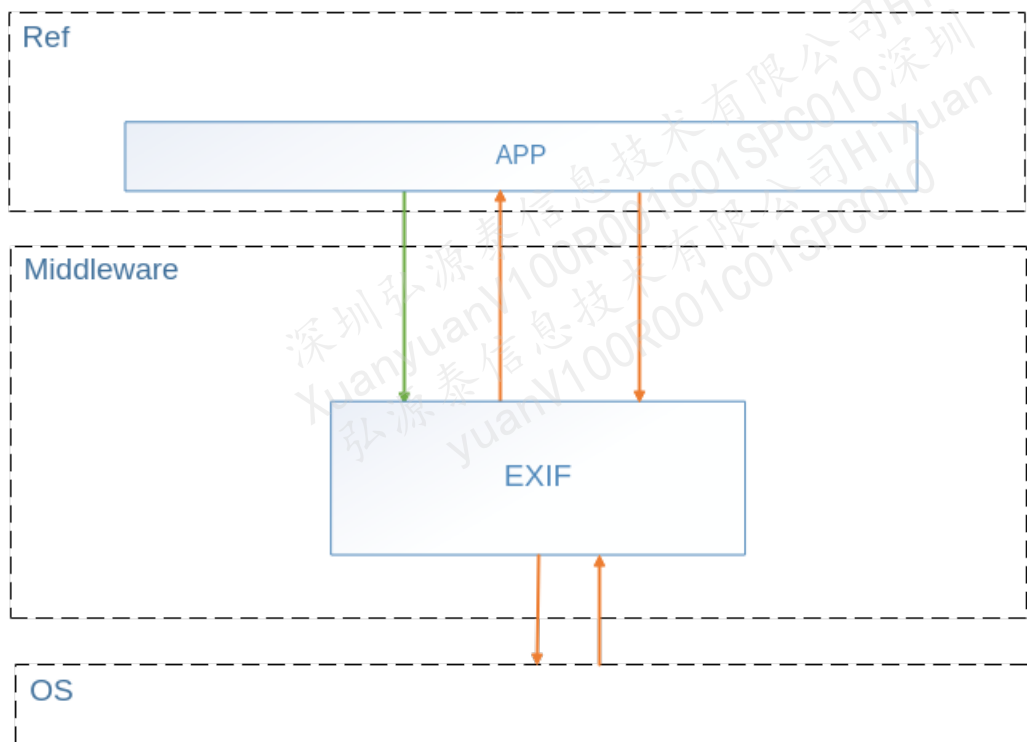


图 6-1 TAG 的类似信息

Metadata GoogleMap	
Exif Xmp Iptc Maker ALL Custom	
Workspace	
Tag name	Value
---- IFD0 ----	
ImageDescription	DCIM
Make	Hsilicon
Model	himc-dv-3559-imx117-linux
Orientation	Horizontal (normal)
XResolution	72
YResolution	72
ResolutionUnit	inches
Software	HiMobileCam v2.1.1.1.20170915
ModifyDate	2017:01:21 21:47:40
---- ExifIFD ----	
ExposureTime	1/100
FNumber	32.0
ExposureProgram	Program AE
ISO	400
ExifVersion	0221
DateTimeOriginal	2017:01:21 21:47:40
CreateDate	2017:01:21 21:47:40
ComponentsConfiguration	Y, Cb, Cr, -
CompressedBitsPerPixel	0.8974609375
ShutterSpeedValue	1/100
ApertureValue	32.0
ExposureCompensation	0
MaxApertureValue	65536.0
MeteringMode	Average
LightSource	Unknown
Flash	No Flash
FocalLength	0.0 mm
ColorSpace	sRGB
ExifImageWidth	4000
ExifImageHeight	3000
SensingMethod	One-chip color area
FileSource	Digital Camera
SceneType	Directly photographed
CustomRendered	Normal
ExposureMode	Auto
WhiteBalance	Auto
DigitalZoomRatio	0.9913044949
FocalLengthIn35mmFormat	0 mm
SceneCaptureType	Standard
GainControl	None

EXIF模块在整个中间件解决方案中的位置，如图6-2所示。

图 6-2 EXIF 模块上下文



—————> 数据流(数据拷贝)
—————> 调用

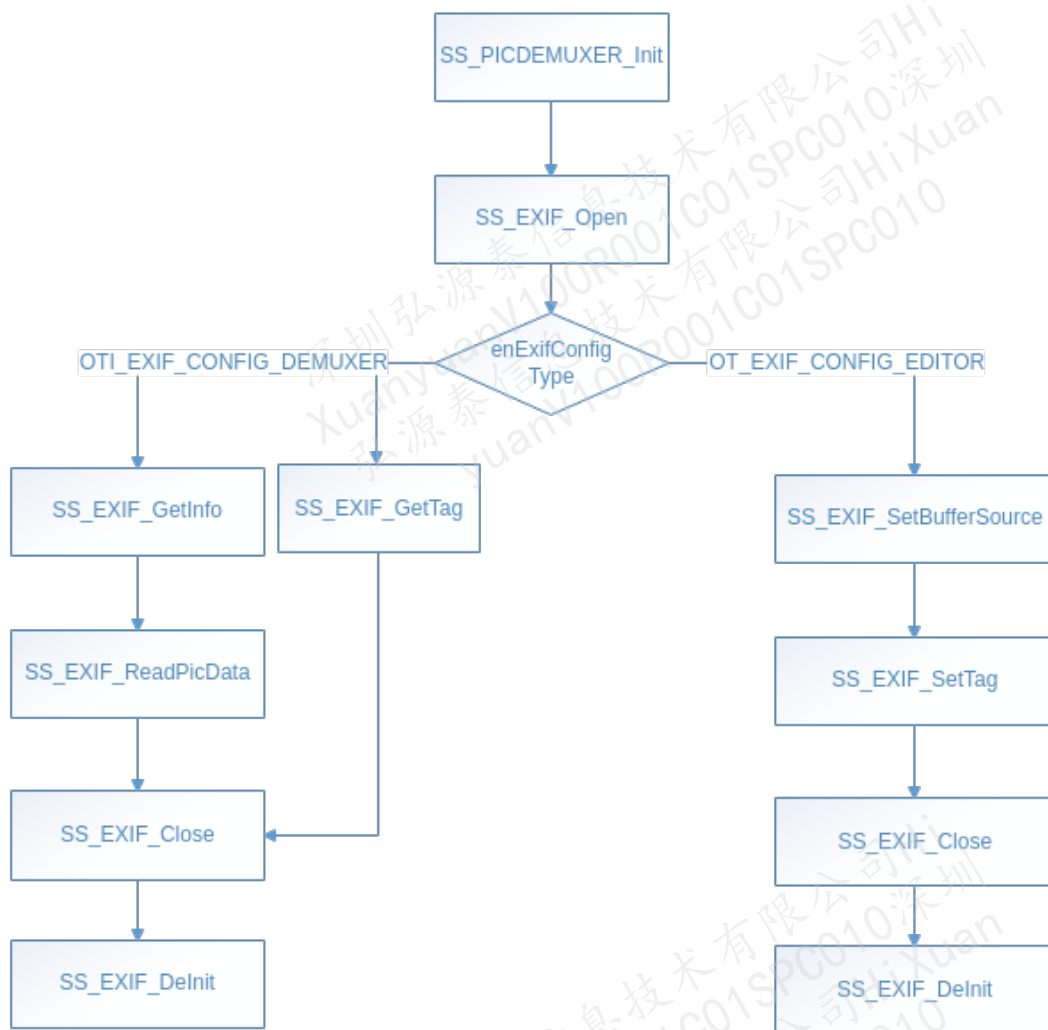
须知

本EXIF模块仅支持本平台抓拍的图片文件，且图片文件封装格式为MPF或者EXIF。

6.1.1 EXIF 模块处理流程

EXIF模块的处理流程如图6-3所示。

图 6-3 EXIF 模块处理流程



从图6-3可以看出整个EXIF模块的处理流程是：

- 步骤1** 首先初始化EXIF模块，对于解封装功能，设置enExifConfigType为OT_EXIF_CONFIG_DEMUXER。
- 步骤2** 打开一张图片获取一个EXIF模块实例，获取图片子图信息，根据子图信息选择将要获取的子图索引，也可获取图片标签内容。
- 步骤3** 读取指定索引的子图数据或者获取图片标签内容，关闭实例后再去初始化。对于标签编辑功能，设置enExifConfigType为OT_EXIF_CONFIG_EDITOR，然后获取一个标签编辑模块实例，设置图片数据buffer，设置tag标签内容，销毁实例，标签编辑模块去初始化。

----结束

6.2 API 参考

该功能模块提供以下API。

- **SS_EXIF_Init**: EXIF模块初始化。



- **SS_EXIF_Open**: 创建EXIF模块实例。
- **SS_EXIF_GetInfo**: 获取对应EXIF实例的子图信息。
- **SS_EXIF_ReadPicData**: 获取指定的子图数据。
- **SS_EXIF_GetTag**: 获取图片tag标签数据。
- **SS_EXIF_SetBufferSource**: 设置图片buffer数据。
- **SS_EXIF_SetTag**: 设置要增加或者修改的tag标签数据。
- **SS_EXIF_Close**: EXIF模块实例销毁。
- **SS_EXIF_DeInit**: EXIF模块去初始化。

6.2.1 SS_EXIF_Init

【描述】

EXIF模块初始化。

【语法】

```
TD_S32 SS_EXIF_Init(TD_VOID);
```

【参数】

无

【返回值】

返回值	描述
0	成功。
非0	失败，参见 错误码 。

【需求】

- 头文件: ss_exif.h
- 库文件: libexif.a、libexif.so

【注意】

- 创建EXIF实例前必须先调用初始化接口。
- 初始化接口中内有引用计数。第一次初始化成功，内部引用计数为1。重复初始化，内部引用计数加1。

【举例】

参考["EXIF模块图片解封装使用实例"](#)。

【相关主题】

无。

6.2.2 SS_EXIF_Open

【描述】



创建EXIF模块实例。

【语法】

```
TD_S32 SS_EXIF_Open(TD_MW_PTR* exif, OT_EXIF_ATTR_S* pstExifAttr);
```

【参数】

参数名称	描述	输入/输出
exif	EXIF模块实例的地址。	输出
pstExifAttr	EXIF模块实例属性。	输入

【返回值】

返回值	描述
0	成功。
非0	失败，参见 错误码 。

【需求】

- 头文件：ss_exif.h
- 库文件：libexif.a、libexif.so

【注意】

- 需要跟初始化接口配套使用。
- EXIF模块最大支持同时存在6个实例。
- 若enExifConfigType为OT_EXIF_CONFIG_DEMUXER_S则创建的解封装实例，若enExifConfigType为OT_EXIF_CONFIG_EDITOR_S则创建的标签编辑实例。
- demuxer支持MPF格式封装的JPEG编码图片，也支持一般的JPEG编码封装格式图片。

【举例】

参考“[EXIF模块图片解封装使用实例](#)”。

【相关主题】

无。

6.2.3 SS_EXIF_GetInfo

【描述】

获取对应EXIF实例的子图信息。

【语法】

```
TD_S32 SS_EXIF_GetInfo(TD_MW_PTR exif, OT_EXIF_INFO_S* pstExifInfo);
```



【参数】

参数名称	描述	输入/输出
exif	EXIF模块实例的地址。	输入
pstExifInfo	EXIF信息结构体的地址。	输出

【返回值】

返回值	描述
0	成功。
非0	失败，参见 错误码 。

【芯片差异】

无

【需求】

- 头文件：ss_exif.h
- 库文件：libexif.a、libexif.so

【注意】

- stSubImg数组存放子图的信息，从索引0开始依次存放一张子图的信息，当遇到u32DataLen为0即终止。
- 只支持包含缩略图在内最大6张子图的图片。
- 该接口用于enExifConfigType为[OT_EXIF_CONFIG_DEMUXER_S](#)的实例。

【举例】

参考["EXIF模块图片解封装使用实例"](#)。

【相关主题】

无。

6.2.4 SS_EXIF_ReadPicData

【描述】

获取指定的子图数据。

【语法】

```
TD_S32 SS_EXIF_ReadPicData(TD_MW_PTR exif, OT\_EXIF\_DATA\_S\* pstExifData);
```

【参数】



参数名称	描述	输入/输出
exif	EXIF模块实例。	输入
pstExifData	存放子图数据的结构体。	输出

【返回值】

返回值	描述
0	成功。
非0	失败，参见 错误码 。

【需求】

- 头文件：ss_exif.h
- 库文件：libexif.a、libexif.so

【注意】

- [OT_EXIF_DATA_S](#)中的u32PicIndex为[SS_EXIF_GetInfo](#)获取到的子图索引。
- [OT_EXIF_DATA_S](#)中的pu8Data内存由外部申请，并且用户必须保证pu8Data的长度与u32DataLen保持一致，否则易发生难以预期的结果。
- 该接口用于enExifConfigType为[OT_EXIF_CONFIG_DEMUXER_S](#)的实例。当读出来的pstExifData->u32HasReadDataLen值为0时，表示子图数据已经读完。

【举例】

参考“[EXIF模块图片解封装使用实例](#)”。

【相关主题】

无。

6.2.5 SS_EXIF_GetTag

【描述】

获取图片的tag标签数据。

【语法】

TD_S32 SS_EXIF_GetTag(TD_MW_PTR exif, [OT_EXIF_TAG_LOCATION_E](#) enExifTagLocation, TD_U16 u16TagNum, [OT_EXIF_TAG_ENTRY_S*](#) pstTagEntry);

【参数】

参数名称	描述	输入/输出
exif	Exif图片标签编辑模块实例handle。	输入



参数名称	描述	输入/输出
enExifTagLocation	标签插入的位置。	输入
u16TagNum	Tag标签num，请参考Exif协议选择TagNum。	输入
pstTagEntry	标签内容地址。	输出

【返回值】

返回值	描述
0	成功。
非0	失败，参见 错误码 。

【需求】

- 头文件：ss_exif.h
- 库文件：libexif.a、libexif.so

【注意】

- 该接口用于enExifConfigType为[OT_EXIF_CONFIG_DEMUXER_S](#)的实例。
- [OT_EXIF_TAG_ENTRY_S](#)的pValue的长度和u32Cnt是一致的，其具体类型取决于enDataType。

【举例】

参考“[EXIF模块图片解封装使用实例](#)”。

【相关主题】

无。

6.2.6 SS_EXIF_SetBufferSource

【描述】

设置图片buffer数据。

【语法】

```
TD_S32 SS_EXIF_SetBufferSource(TD_MW_PTR exif, OT\_EXIF\_BUFFER\_SOURCE\_S*  
pstBufSource);
```

【参数】

参数名称	描述	输入/输出
exif	Exif图片标签编辑模块实例handle。	输入



参数名称	描述	输入/输出
pstBufSource	图片buffer的信息。	输入

【返回值】

返回值	描述
0	成功。
非0	失败，参见 错误码 。

【需求】

- 头文件：ss_exif.h
- 库文件：libexif.a、libexif.so

【注意】

- [OT_EXIF_BUFFER_SOURCE_S](#) 结构体里的u32BufSize必须与plmgBuf的数据长度一致，否则会发生难以预期的异常结果。
- 一张图片最多分成[OT_EXIF_DATA_SEG_MAX_CNT](#)个段，一张图分成多段后按顺序依次填入到plmgBuf数组与u32BufSize数组中，中间不能留空，不使用的段对应plmgBuf设置空，u32BufSize设置为0；若图片分段未按顺序设置，可能会导致不可知错误。
- 我们只支持Exif封装的图片数据。
- 该接口一个实例只支持一次调用，多次调用需要重新创建实例。
- 该接口用于enExifConfigType为[OT_EXIF_CONFIG_EDITOR_S](#)的实例。

【举例】

参考“[EXIF模块图片解封装使用实例](#)”。

【相关主题】

无

6.2.7 SS_EXIF_SetTag

【描述】

设置要增加或者修改的tag标签数据。

【语法】

```
TD_S32 SS_EXIF_SetTag(TD_MW_PTR exif, OT\_EXIF\_TAG\_ENTRY\_S* pstTagEntry, OT\_EXIF\_TAG\_LOCATION\_E enExifTagLocation);
```

【参数】



参数名称	描述	输入/输出
exif	Exif图片标签编辑模块实例handle。	输入
pstTagEntry	标签内容地址。	输入
enExifTagLocation	标签插入的位置。	输入

【返回值】

返回值	描述
0	成功。
非0	失败，参见 错误码 。

【需求】

- 头文件：ss_exif.h
- 库文件：libexif.a、libexif.so

【注意】

- [OT_EXIF_TAG_ENTRY_S](#)的u32Cnt必须与pValue对应数据长度保持一致，否则会发生难以预期的异常结果。
- [OT_EXIF_TAG_ENTRY_S](#)的enDataType必须与pValue里的数据类型保持一致，否则会发生难以预期的异常结果。
- 该接口用于enExifConfigType为[OT_EXIF_CONFIG_EDITOR_S](#)的实例。
- 请不要修改子目录偏移量，如EXIF子目录偏移量、GPS目录偏移量等。对该类tag的修改会导致图片破坏或者程序崩溃。
- 不支持对图像宽高字段的修改。

【举例】

参考“[EXIF模块图片解封装使用实例](#)”。

【相关主题】

无。

6.2.8 SS_EXIF_Close

【描述】

EXIF模块实例销毁。

【语法】

```
TD_S32 SS_EXIF_Close(TD_MW_PTR exif);
```

【参数】



参数名称	描述	输入/输出
exif	EXIF模块实例。	输入

【返回值】

返回值	描述
0	成功。
非0	失败，参见 错误码 。

【需求】

- 头文件：ss_exif.h
- 库文件：libexif.a、libexif.so

【注意】

无

【举例】

参考“[EXIF模块图片解封装使用实例](#)”。

【相关主题】

无

6.2.9 SS_EXIF_Delnit

【描述】

EXIF模块去初始化。

【语法】

TD_S32 SS_EXIF_Delnit(TD_VOID);

【参数】

无

【返回值】

返回值	描述
0	成功。
非0	失败，参见 错误码 。

【需求】



- 头文件：ss_exif.h
- 库文件：libexif.a、libexif.so

【注意】

去初始化接口内部实现中有使用引用计数，所以去初始化接口必须与初始化接口配套使用。当最后一个去初始化接口调用完后，释放内部公共资源。

【相关主题】

无。

6.3 数据类型

- **OT_EXIF_DATA_S**：定义获取的子图数据的信息。
- **OT_EXIF_INFO_S**：定义图片的信息。
- **OT_EXIF_SUBIMG_S**：定义子图信息。
- **OT_EXIF_SUBIMG_MAX_CNT**：定义子图的最大个数。
- **OT_EXIF_DATA_TYPE_E**：定义标签数据类型。
- **OT_EXIF_TAG_LOCATION_E**：定义标签数据的位置。
- **OT_EXIF_TAG_ENTRY_S**：定义标签数据。
- **OT_EXIF_CONFIG_EDITOR_S**：定义标签编辑功能的属性。
- **OT_EXIF_CONFIG_DEMUXER_S**：定义解封装功能的属性。
- **OT_EXIF_ATTR_S**：定义EXIF模块的属性。
- **OT_EXIF_CONFIG_TYPE_E**：EXIF实例类型。

6.3.1 OT_EXIF_DATA_S

【说明】

定义获取的子图数据的信息。

【定义】

```
typedef struct EXIF_DATA_S
{
    TD_U32 u32PicIndex;
    TD_U8 *pu8Data;
    TD_U32 u32DataLen;
    TD_U32 u32HasReadDataLen;
}OT_EXIF_DATA_S;
```

【成员】

成员名称	描述
u32PicIndex	输入参数，将要获取图像数据的子图索引。
pu8Data	输入参数，外部申请的装载子图数据的内存地址。
u32DataLen	输入参数，外部申请的装载子图数据的内存大小。



成员名称	描述
u32HasReadDataLen	输出参数，本次调用读到的数据个数。

【注意事项】

- pu8Data申请的内存大小必须与u32DataLen匹配。
- u32PicIndex必须为有效的子图索引。
- u32HasReadDataLen为0，表示已经读到当前图片结尾。

【相关数据类型及接口】

无。

6.3.2 OT_EXIF_INFO_S

【说明】

定义图片信息。

【定义】

```
typedef struct EXIF_INFO_S
{
    TD_S64 s64FileSize;
    OT_EXIF_SUBIMG_S stSubImg[OT_EXIF_SUBIMG_MAX_CNT];
}OT_EXIF_INFO_S;
```

【成员】

成员名称	描述
s64FileSize	图片文件大小。
stSubImg	子图信息结构体。

【注意事项】

无。

【相关数据结构及接口】

请参考OT_EXIF_SUBIMG_S。

6.3.3 OT_EXIF_SUBIMG_S

【说明】

定义子图信息。

【定义】

```
typedef struct EXIF_SUBIMG_S
{
    TD_U32 u32Width;
```




```
TD_U32 u32Height;  
TD_U32 u32DataLen;  
TD_BOOL bThumbnail;  
}OT_EXIF_SUBIMG_S;
```

【成员】

成员名称	描述
u32Width	子图的图像宽度。
u32Height	子图的图像高度。
u32DataLen	子图的数据大小。
bThumbnail	是否为缩略图。

【注意事项】

无。

【相关数据结构及接口】

无。

6.3.4 OT_EXIF_SUBIMG_MAX_CNT

【说明】

定义子图的最大个数。

【定义】

```
#define OT_EXIF_SUBIMG_MAX_CNT (6)
```

【成员】

无。

【注意事项】

无。

【相关数据结构及接口】

无。

6.3.5 OT_EXIF_DATA_SEG_MAX_CNT

【说明】

定义输入图片数据的最大分段数。

【定义】

```
#define OT_EXIF_DATA_SEG_MAX_CNT (8)
```

【成员】

无。



【注意事项】

无。

【相关数据结构及接口】

无。

6.3.6 OT_EXIF_DATA_TYPE_E

【说明】

定义标签数据类型。

【定义】

```
typedef enum EXIF_DATA_TYPE_E{
    OT_EXIF_DATA_TYPE_BYTE = 1,
    OT_EXIF_DATA_TYPE_ASCII = 2,
    OT_EXIF_DATA_TYPE_SHORT = 3,
    OT_EXIF_DATA_TYPE_LONG = 4,
    OT_EXIF_DATA_TYPE_RATIONAL = 5,
    OT_EXIF_DATA_TYPE_UNDEFINED = 7,
    OT_EXIF_DATA_TYPE_SLONG = 9,
    OT_EXIF_DATA_TYPE_SRATIONAL = 10,
} OT_EXIF_DATA_TYPE_E;
```

【成员】

成员名称	描述
OT_EXIF_DATA_TYPE_BYTE	Unsigned Byte类型，数据占8位。
OT_EXIF_DATA_TYPE_ASCII	字符类型，数据占8位。
OT_EXIF_DATA_TYPE_SHORT	Unsigned Short类型，数据占8位。
OT_EXIF_DATA_TYPE_LONG	Unsigned Long类型，数据占32位。
OT_EXIF_DATA_TYPE_RATIONAL	Unsigned Rational类型，数据占64位。
OT_EXIF_DATA_TYPE_UNDEFINED	未知类型，数据占8位。
OT_EXIF_DATA_TYPE_SLONG	Signed Long类型，数据占32位。
OT_EXIF_DATA_TYPE_SRATIONAL	Signed Rational类型，数据占64位。

【注意事项】

无。

【相关数据类型及接口】

无。

6.3.7 OT_EXIF_TAG_LOCATION_E

【说明】

定义标签数据的位置。



【定义】

```
typedef enum EXIF_TAG_LOCATION_E
{
    OT_EXIF_APP1_IFD0 = 0,
    OT_EXIF_APP1_EXIFIFD,
    OT_EXIF_APP1_EXIFINTEROP_IFD,
    OT_EXIF_APP1_IFD0_GPSINFO_IFD,
    OT_EXIF_APP1_IFD1,
} OT_EXIF_TAG_LOCATION_E;
```

【成员】

成员名称	描述
OT_EXIF_APP1_IFD0	IFD0位置。
OT_EXIF_APP1_EXIFIFD	EXIF IFD位置。
OT_EXIF_APP1_EXIFINTEROP_IFD	EXIF Interop IFD 位置。
OT_EXIF_APP1_IFD0_GPSINFO_IFD	GPSINFO IFD位置。
OT_EXIF_APP1_IFD1	IFD1位置。

【注意事项】

无。

【相关数据结构及接口】

无。

6.3.8 OT_EXIF_TAG_ENTRY_S

【说明】

定义标签数据。

【定义】

```
typedef struct EXIF_TAG_ENTRY_S
{
    TD_U16 u16TagNum;
    OT_EXIF_DATA_TYPE_E enDataType;
    TD_U32 u32Cnt;
    const void* pValue;
}OT_EXIF_TAG_ENTRY_S;
```

【成员】

成员名称	描述
u16TagNum	Tag标签num，请参考Exif协议选择TagNum。
enDataType	Tag标签的数据类型，请参考Exif协议设置。
u32Cnt	Tag标签的数据个数，注意以数据类型为单位，取值范围：(0, 102400]



成员名称	描述
pValue	Tag数据，类似于一维数组。

【注意事项】

请参考Exif协议设置tag内容。

【相关数据结构及接口】

[OT_EXIF_DATA_TYPE_E](#)

6.3.9 OT_EXIF_CONFIG_EDITOR_S

【说明】

定义标签编辑功能的属性。

【定义】

```
typedef struct EXIF_CONFIG_EDITOR_S
{
    const TD_CHAR* pszFileName;
    TD_BOOL blsOverWrite;
    TD_U32 u32FixedFileSize;
} OT_EXIF_CONFIG_EDITOR_S;
```

【成员】

成员名称	描述
pszFileName	输出图像文件的全路径名，路径长度小于PATH_MAX。
blsOverWrite	TD_TRUE：如果文件存在就覆盖； TD_FALSE：如果文件存在就报错。
u32FixedFileSize	0：不限制文件大小； >0：输出的文件大小不能大于该值，若超过该值，则返回出错。该参数主要用于预分配文件。

【注意事项】

无。

【相关数据类型及接口】

无。

6.3.10 OT_EXIF_CONFIG_DEMUXER_S

【说明】

定义解封装功能的属性。

【定义】



```
typedef struct EXIF_CONFIG_DEMUXER_S
{const TD_CHAR* pszFileName;
} OT_EXIF_CONFIG_DEMUXER_S;
```

【成员】

成员名称	描述
pszFileName	输出图像文件的全路径名，路径长度小于PATH_MAX。

【注意事项】

无。

【相关数据类型及接口】

无。

6.3.11 OT_EXIF_ATTR_S

【说明】

定义EXIF模块的属性。

【定义】

```
typedef struct EXIF_ATTR_S
{
    OT_EXIF_CONFIG_TYPE_E enExifConfigType;
    union
    {
        OT_EXIF_CONFIG_EDITOR_S stEditorConfig;
        OT_EXIF_CONFIG_DEMUXER_S stDemuxerConfig;
    };
}OT_EXIF_ATTR_S;
```

【成员】

成员名称	描述
enExifConfigType	EXIF实例的使用类型。
stEditorConfig	EXIF编辑属性。
stDemuxerConfig	EXIF解封装属性。

【注意事项】

无。

【相关数据类型及接口】

无。



6.3.12 OT_EXIF_CONFIG_TYPE_E

【说明】

EXIF实例类型。

【定义】

```
typedef enum EXIF_CONFIG_TYPE_E
{
    OT_EXIF_CONFIG_DEMUXER = 0,
    OT_EXIF_CONFIG_EDITOR,
    OT_EXIF_CONFIG_BUTT
} OT_EXIF_CONFIG_TYPE_E;
```

【成员】

成员名称	描述
OT_EXIF_CONFIG_DEMUXER	EXIF用于解封装。
OT_EXIF_CONFIG_EDITOR	EXIF用于tag编辑。
OT_EXIF_CONFIG_BUTT	数据类型枚举值结束标识。

【注意事项】

无。

【相关数据类型及接口】

无。

6.3.13 OT_EXIF_BUFFER_SOURCE_S

【说明】

EXIF的输入buffer信息。

【定义】

```
typedef struct EXIF_BUFFER_SOURCE_S
{
    TD_VOID* plmgBuf[OT_EXIF_DATA_SEG_MAX_CNT];
    TD_U32 u32BufSize[OT_EXIF_DATA_SEG_MAX_CNT];
} OT_EXIF_BUFFER_SOURCE_S;
```

【成员】

成员名称	描述
plmgBuf	图片buffer段地址。
u32BufSize	图片buffer段长度。

【注意事项】

无。



【相关数据类型及接口】

无。

6.4 错误码

EXIF错误码如表6-1所示。

表 6-1 EIXF 错误码

错误代码	宏定义	描述
0	TD_SUCCESS	成功
-1	TD_FAILURE	失败
0xA1368002	OT_ERR_EXIF_NULL_PTR	空指针
0xA1368001	OT_ERR_EXIF_ILLEGAL_PARAM	非法参数
0xA1368040	OT_ERR_EXIF_NO_AVAILABLE_HANDLE	没有可用的handle
0xA1368041	OT_ERR_EXIF_HANDLE_INVALID	无效handle
0xA1368045	OT_ERR_EXIF_STATUS	状态不对，接口调用顺序有问题
0xA1368046	OT_ERR_EXIF_INNER_FAILED	内部错误，调用内部模块接口出错
0xA1368047	OT_ERR_EXIF_MALLOC_FAILED	分配内存失败
0xA1368048	OT_ERR_EXIF_RES_OCCUPY	资源已被占用
0xA1368049	OT_ERR_EXIF_UNSUPPORT_DATA	不支持的图片数据
0xA1368050	OT_ERR_EXIF_INSUFFICIENT_FIXED_SPACE	固定文件空间不够
0xA1368051	OT_ERR_EXIF_FILE_OPERATE	非法的文件操作
0xA1368052	OT_ERR_EXIF_READ_DATA	读取数据失败
0xA1368053	OT_ERR_EXIF_OPEN_FILE	打开文件失败
0xA1368054	OT_ERR_EXIF_UNSUPPORT_FILE	不支持的文件



错误代码	宏定义	描述
0xA1368055	OT_ERR_EXIF_RES_NOT_RELEASE	资源未释放
0xA1368056	OT_ERR_EXIF_SAFE_FUNCTION	安全函数执行错误

6.5 典型场景使用指南

6.5.1 EXIF 模块图片解封装使用实例

对一个Exif图片设置，用户接口调用说明如下：

步骤1 获得一个tagdemuxer实例

```

/*****
EXIF模块进行初始化并创建一个实例。
*****/
TD_S32 s32Ret = TD_SUCCESS;
s32Ret = SS_EXIF_Init();
if (TD_SUCCESS != s32Ret)
{
    printf("SS_EXIF_Init exec failed \n");
    return -1;
}
TD_MW_PTR pPicDemuer;OT_EXIF_ATTR_S stExifAttr;
memset(&stExifAttr, 0, sizeof(OT_EXIF_ATTR_S));
stExifAttr.enExifConfigType = OT_EXIF_CONFIG_DEMUXER;
stExifAttr.stDemuxerConfig.pszFileName = inFilename;
s32Ret = SS_EXIF_Open(&pPicDemuer, &stExifAttr);
if (TD_SUCCESS != s32Ret)
{
    printf("SS_EXIF_Open exec failed \n");
    return -1;
}
}

```

步骤2 获取图片信息或者图片tag标签数据

```

OT_EXIF_INFO_S stEXIFInfo;
s32Ret = SS_EXIF_GetInfo(pPicDemuer, &stEXIFInfo);
if (TD_SUCCESS != s32Ret)
{
    printf("SS_EXIF_GetInfo exec failed \n");
    return -1;
}
OT_EXIF_TAG_ENTRY_S tagEntry = { 0 };
OT_EXIF_TAG_LOCATION_E enExifTagLocation = OT_EXIF_APP1_IFD0;
s32Ret = SS_EXIF_GetTag(pPicDemuer, enExifTagLocation, 0x0132, &tagEntry);
if (TD_SUCCESS != s32Ret)
{
    printf("get image modification time tag failed \n");
    return -1;
}
TD_CHAR *modificationTime = (TD_CHAR *)tagEntry.pValue;
printf("image modification time = %s\n", modificationTime);
enExifTagLocation = OT_EXIF_APP1_EXIFID;
s32Ret = SS_EXIF_GetTag(pPicDemuer, enExifTagLocation, 0xA002, &tagEntry);
if (TD_SUCCESS != s32Ret)

```



```
{
    printf("get image width tag failed \n");
    return -1;
}
printf("image width = %d\n", *((TD_S32 *) tagEntry.pValue));
```

步骤3 获取并保存各子图数据

```
TD_U32 u32Index = 0;
for(; u32Index < OT_EXIF_RESOLUTION_CNT; u32Index++)
{
    if (stEXIFInfo.stSubImg[u32Index].u32DataLen != 0)
    {
        char fileName[20] = {0};
        snprintf(fileName, 20, "./pic_%d.jpeg", u32Index);
        printf("filename:%s \n", fileName);
        FILE* out_pic = fopen(fileName, "w+");
        if(out_pic == NULL)
        {
            printf("create file failed, error:%d \n", errno);
            return -1;
        }

        unsigned char* pData = (unsigned char*)
        malloc(stEXIFInfo.stSubImg[u32Index].u32DataLen);
        memset(pData, 0, stEXIFInfo.stSubImg[u32Index].u32DataLen); OT_EXIF_DATA_S
        stEXIFData;
        stEXIFData.u32PicIndex = u32Index;
        stEXIFData.u32DataLen = stEXIFInfo.stSubImg[u32Index].u32DataLen;
        stEXIFData.pu8Data = pData;
        s32Ret = SS_EXIF_ReadPicData(pPicDemuer, &stEXIFData);
        if (TD_SUCCESS != s32Ret)
        {
            printf("SS_EXIF_ReadPicData exec failed \n");
            return -1;
        }
        fwrite(pData, 1, stEXIFInfo.stSubImg[u32Index].u32DataLen, out_pic);
        fclose(out_pic);
        free(pData);
    }
}
```

步骤4 销毁实例并去初始化

```
s32Ret = SS_EXIF_Close(pPicDemuer);
if (TD_SUCCESS != s32Ret)
{
    printf("SS_EXIF_Close exec failed \n");
    return -1;
}

s32Ret = SS_EXIF_DeInit();
if (TD_SUCCESS != s32Ret)
{
    printf("SS_EXIF_DeInit exec failed \n");
    return -1;
}
```

----结束

6.5.2 EXIF 模块图片编辑使用实例

对一个Exif图片设置，用户接口调用说明如下：



步骤1 获得一个tageditor实例

```
/*
*****
图片标签模块进行初始化并创建一个实例。
*****
*/
TD_S32 s32Ret = TD_SUCCESS;
s32Ret = SS_EXIF_Init();

if (TD_SUCCESS != s32Ret)
{
    printf("SS_EXIF_Init exec failed,ret:%x \n", s32Ret);
    return s32Ret;
}

TD_MW_PTR hExifHandle = NULL;
OT_EXIF_ATTR_S stExifAttr;
memset(&stExifAttr, 0, sizeof(OT_EXIF_ATTR_S));
stExifAttr.enExifConfigType = OT_EXIF_CONFIG_EDITOR;
stExifAttr.stEditorConfig.pszFileName = outFileNames;
stExifAttr.stEditorConfig.bIsOverWrite = TD_TRUE;
stExifAttr.stEditorConfig.u32FixedFileSize = 0;
s32Ret = SS_EXIF_OpenFile(&hExifHandle, &stExifAttr);

if (TD_SUCCESS != s32Ret)
{
    printf("SS_EXIF_OpenFile exec failed,ret:%x \n", s32Ret);
    return s32Ret;
}
```

步骤2 设置图片数据

```
/*
*****
fp_pic是对应的图片文件句柄，u32TotalDataLen是图片总长度
*****
*/
OT_EXIF_BUFFER_SOURCE_S stBufSource;
memset(&stBufSource, 0x00, sizeof(stBufSource));
for(i<OT_EXIF_DATA_SEG_MAX_CNT; i++)
{
    if(i == OT_EXIF_DATA_SEG_MAX_CNT -1)
    {
        stBufSource.u32BufSize[i] = (u32TotalDataLen/OT_EXIF_DATA_SEG_MAX_CNT) +
(u32TotalDataLen%OT_EXIF_DATA_SEG_MAX_CNT);
    }
    else
    {
        stBufSource.u32BufSize[i] = (u32TotalDataLen/OT_EXIF_DATA_SEG_MAX_CNT);
    }
    stBufSource.plmgBuf[i] = malloc(stBufSource.u32BufSize[i]);
    if(!stBufSource.plmgBuf[i])
    {
        printf("malloc failed\n");
        break;
    }
    memset(stBufSource.plmgBuf[i], 0, stBufSource.u32BufSize[i]);
    fread(stBufSource.plmgBuf[i], 1, stBufSource.u32BufSize[i], fp_pic);
    printf("buf idx: %d len: %d\n", i, stBufSource.u32BufSize[i]);
}
S32Ret = SS_EXIF_SetBufferSource(hExifHandle, &stBufSource);
if (TD_SUCCESS != s32Ret)
{
    printf("SS_EXIF_SetBufferSource exec failed,ret:%x \n", s32Ret);
    return s32Ret;
}
```

**步骤3 添加或者修改tag**

```
OT_EXIF_TAG_ENTRY_S stExifModelTagEntry;
stExifModelTagEntry.u16TagNum = 0xC614;
stExifModelTagEntry.enDataType = OT_EXIF_DATA_TYPE_ASCII;
char* pModelValue = "ActionCam Demo";
stExifModelTagEntry.u32Cnt = strlen(pModelValue) + 1;
stExifModelTagEntry.pValue = pModelValue; OT_EXIF_TAG_LOCATION_E enTagLocation =
OT_EXIF_APP1_IFD0;
s32Ret = SS_EXIF_SetTag(hExifHandle, &stExifModelTagEntry, enTagLocation);
if (TD_SUCCESS != s32Ret)
{
    printf("SS_EXIF_SetTag for unique camera model exec failed,ret:%x \n", s32Ret);
    return s32Ret;
}
```

步骤4 销毁实例并去初始化

```
s32Ret = SS_EXIF_Close(hExifHandle);
if (TD_SUCCESS != s32Ret)
{
    printf("SS_EXIF_Close exec failed,ret:%x \n", s32Ret);
    return s32Ret;
}
s32Ret = SS_EXIF_DeInit();
if (TD_SUCCESS != s32Ret)
{
    printf("SS_EXIF_DeInit exec failed,ret:%x \n", s32Ret);
    return s32Ret;
}
```

----结束



7 播放器模块

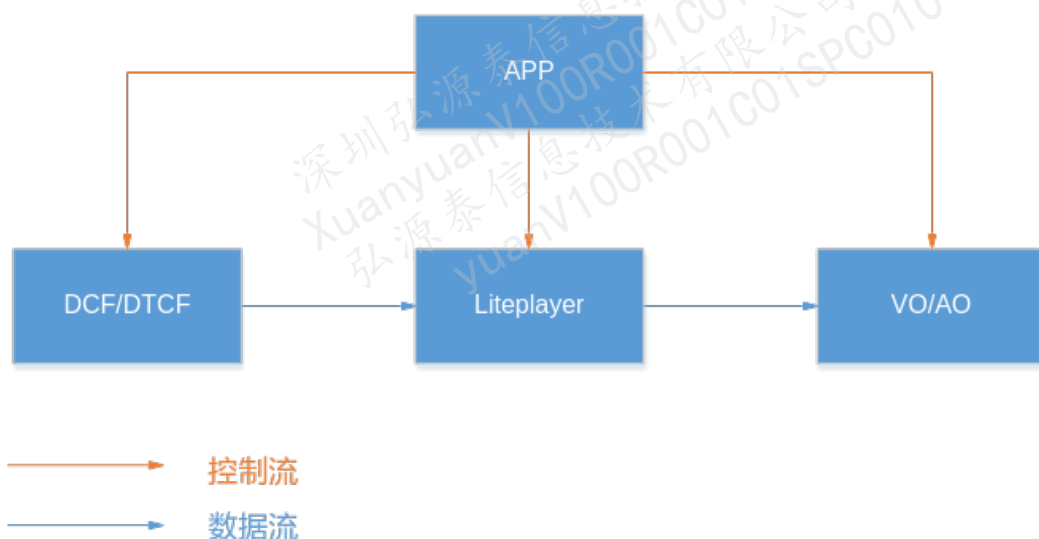
7.1 概述

播放器模块为用户提供本地媒体文件回放功能。它可以播放MP4文件（AVC/HEVC+AAC）、单音频M4A(AAC-LC)文件、单音频WAV文件、MPF文件与普通JPEG文件。它包含解封装功能与播控功能，解封装功能使用对媒体文件的解析，如把MP4解析成ES流；播控功能包括播放控制、内部数据管理等。用户可以根据demux头文件实现自己的解封装库。

此外，本模块提供了回调注册接口，让用户必须注册回调接口以接收文件播完的消息。可以接受播放进度回调消息以及播放错误消息。

播放器模块在整个系统中的位置，如图7-1所示。

图 7-1 播放器模块上下文



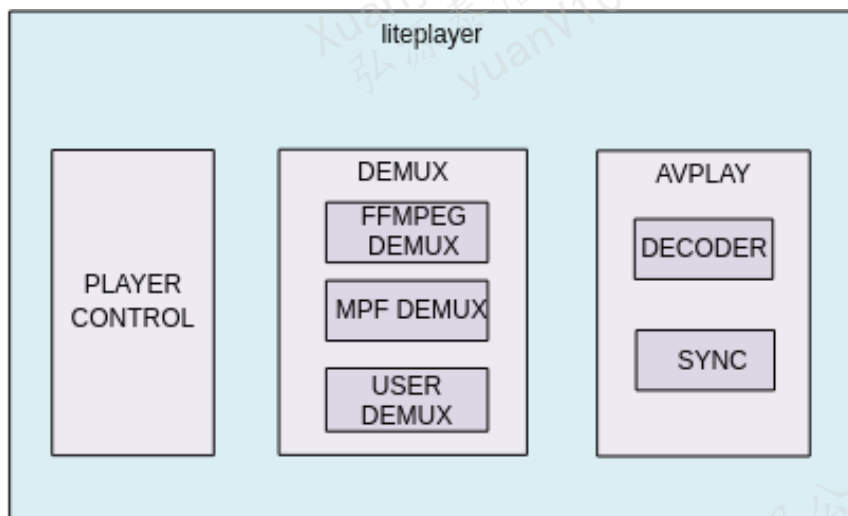
须知

本播放器模块仅支持播放本平台录制的MP4文件，且文件内视频格式限制为H.264/H.265，音频格式限制为AAC；播放视频支持的最大解码能力为请参见《MPP 媒体处理软件 Vx.0 开发参考》。

7.1.1 播放器模块层次结构

播放器模块的层次结构，如图7-2所示。

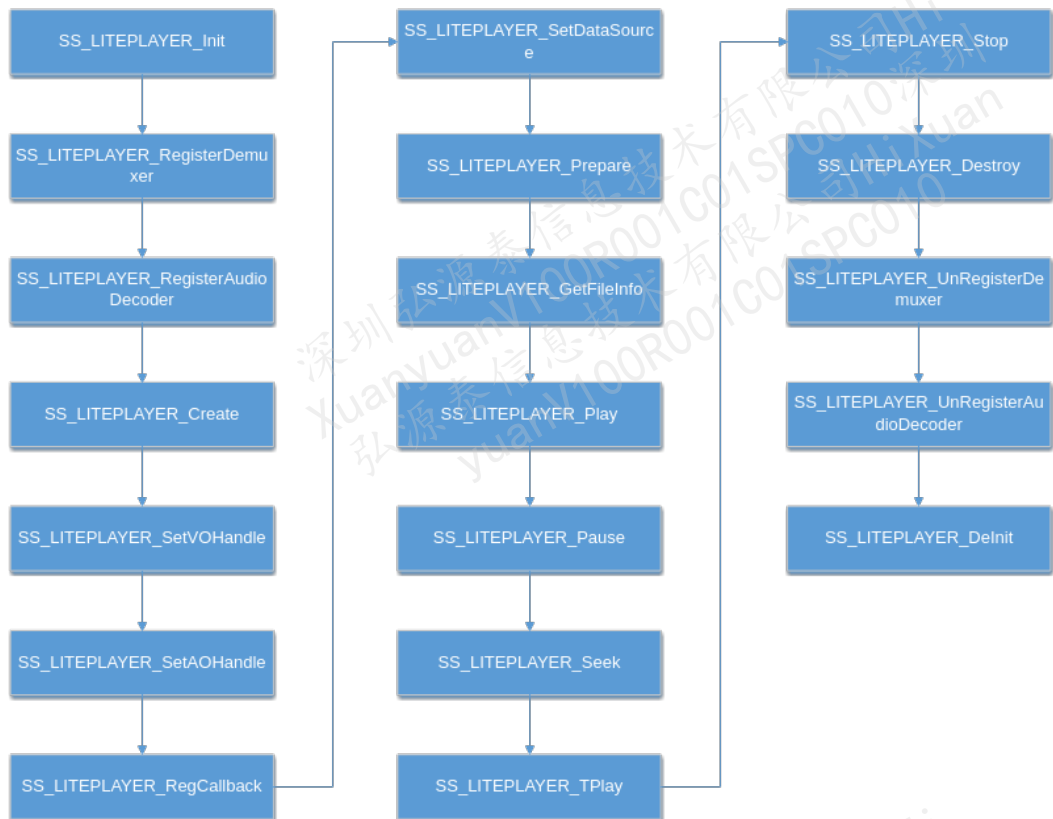
图 7-2 播放器模块层次结构图



7.1.2 播放器模块处理流程

播放器模块的处理流程如图7-3所示。

图 7-3 播放器模块处理流程



从图7-3可以看出整个播放器的处理流程：

- 步骤1 初始化播放器；
- 步骤2 注册解封装库；
- 步骤3 注册音频解码库；
- 步骤4 创建播放器实例；
- 步骤5 设置绑定的音频与视频输出；
- 步骤6 设置事件回调函数；
- 步骤7 设置要播放的文件路径；
- 步骤8 执行播放直到文件结束；
- 步骤9 播放过程中执行seek；
- 步骤10 播放过程中执行倍速播放；
- 步骤11 停止播放器；
- 步骤12 销毁播放器实例；
- 步骤13 去注册解封装库；
- 步骤14 去注册音频解码库；

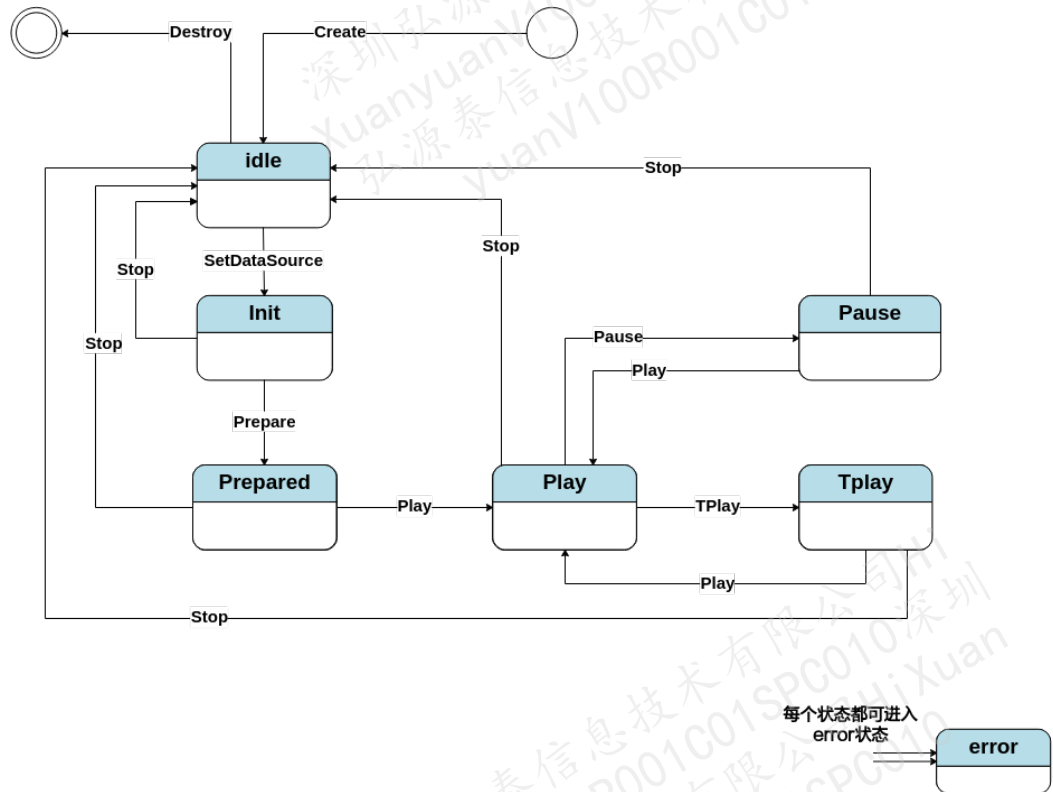
步骤15 播放器去初始化。

----结束

7.1.3 播放器状态图

播放器状态如图7-4所示。

图 7-4 播放器模块状态图



7.2 功能描述

7.2.1 解封装模块

将指定格式的媒体文件解析成ES流。主要实现接口：

- fmt_open：打开媒体文件。
- fmt_close：关闭媒体文件。
- fmt_read：读取数据包。
- fmt_free：释放数据包。
- fmt_seek：跳转到媒体文件的指定时间位置。
- fmt_probe：解析媒体媒体文件。
- fmt_getinfo：获取媒体文件的相关信息。



- `fmt_setattr`: 设置媒体文件的相关信息。

播放器模块提供注册解封装库接口，并提供MP4文件解封装库，用户可以根据`ss_demuxer.h`头文件实现自己的解封装库。

7.2.2 播控模块

播控模块提供媒体播放、暂停、跳转、停止功能。用户在回调函数里不能有阻塞操作，也不能调用播放器接口。

播放器模块的播放状态对应接口如下：

- `SS_LITEPLAYER_Create`对应`OT_LITEPLAYER_STATE_IDLE`状态。
- `SS_LITEPLAYER_SetDataSource`对应`OT_LITEPLAYER_STATE_INIT`状态。
- `SS_LITEPLAYER_Prepare`对应`OT_LITEPLAYER_STATE_PREPARED`状态。
- `SS_LITEPLAYER_Play`对应`OT_LITEPLAYER_STATE_PLAY`状态。
- `SS_LITEPLAYER_Pause`对应`OT_LITEPLAYER_STATE_PAUSE`状态。
- `SS_LITEPLAYER_TPlay`对应`OT_LITEPLAYER_STATE_TPLAY`状态。
- `SS_LITEPLAYER_Stop`对应`OT_LITEPLAYER_STATE_IDLE`状态。

7.3 API 参考

该功能模块提供以下API。

- `SS_LITEPLAYER_Init`: 初始化播放器模块。
- `SS_LITEPLAYER_DeInit`: 去初始化播放器模块。
- `SS_LITEPLAYER_RegisterDemuxer`: 注册解封装库。
- `SS_LITEPLAYER_UnRegisterDemuxer`: 去注册解封装库。
- `SS_LITEPLAYER_RegisterAudioDecoder`: 注册音频解码库。
- `SS_LITEPLAYER_UnRegisterAudioDecoder`: 去注册音频解码库。
- `SS_LITEPLAYER_Create`: 创建播放器。
- `SS_LITEPLAYER_SetVOHandle`: 设置player要绑定的Vout信息。
- `SS_LITEPLAYER_SetAOHandle`: 设置player要绑定的Aout信息。
- `SS_LITEPLAYER_Destroy`: 销毁player实例。
- `SS_LITEPLAYER_SetDataSource`: 设置player要播放的数据源。
- `SS_LITEPLAYER_Prepare`: 对player里面的数据执行预处理。
- `SS_LITEPLAYER_Play`: 开始播放媒体文件。
- `SS_LITEPLAYER_Stop`: 停止播放媒体文件并且销毁内部资源。
- `SS_LITEPLAYER_Pause`: 暂停播放媒体文件。
- `SS_LITEPLAYER_Seek`: 移动到指定播放位置。
- `SS_LITEPLAYER_RegCallback`: 注册回调函数。
- `SS_LITEPLAYER_GetFileInfo`: 获取媒体文件相关信息。
- `SS_LITEPLAYER_SetMedia`: 设置媒体文件的相关属性。
- `SS_LITEPLAYER_GetPlayStatus`: 获取当前的播放状态。



- [SS_LITEPLAYER_TPlay](#)：使能倍速播放。

7.3.1 SS_LITEPLAYER_Init

【描述】

初始化播放器模块。

【语法】

```
TD_S32 SS_LITEPLAYER_Init(TD_VOID);
```

【参数】

无

【返回值】

返回值	描述
0	成功。
非0	失败，参见 错误码 。

【需求】

- 头文件：ss_demuxer.h、ss_liteplayer.h
- 库文件：libliteplayer.a、libavplay_linux.a、libliteplayer.so

【注意】

播放器模块的使用前，必需先调用初始化接口。

【举例】

参考“[播放器使用示例](#)”。

【相关主题】

无。

7.3.2 SS_LITEPLAYER_DeInit

【描述】

去初始化播放器模块。

【语法】

```
TD_S32 SS_LITEPLAYER_DeInit(TD_VOID);
```

【参数】

无

【返回值】



返回值	描述
0	成功。
非0	失败，参见 错误码 。

【需求】

- 头文件：ss_demuxer.h、ss_liteplayer.h
- 库文件：libliteplayer.a、libavplay_linux.a、libliteplayer.so

【注意】

需要跟播放器初始化接口配套使用。

【举例】

参考“[播放器使用示例](#)”。

【相关主题】

无。

7.3.3 SS_LITEPLAYER_RegisterDemuxer

【描述】

注册解封装库。

【语法】

```
TD_S32 SS_LITEPLAYER_RegisterDemuxer(const TD_CHAR* dllName, const  
OT\_LITEPLAYER\_EXTRA\_FILESURFIX\_S\* pstExtraFileSurfix);
```

【参数】

参数名称	描述	输入/输出
dllName	解封装库文件名 (全路径)	输入
pstExtraFileSurfix	解封装库可额外支持的文件后缀	输入

【返回值】

返回值	描述
0	成功。
非0	失败，参见 错误码 。

【芯片差异】

无



【需求】

- 头文件：ss_demuxer.h、ss_liteplayer.h
- 库文件：libliteplayer.a、libavplay_linux.a、libliteplayer.so、libffmpeg_demuxer.so

【注意】

- 用户可以根据ss_demuxer.h头文件实现自己的解封装库。
- 第二个参数可为空，此时demuxer仅支持内置的文件后缀。
- 该接口要在创建liteplayer实例前调用。

【举例】

参考“[播放器使用示例](#)”。

【相关主题】

无。

7.3.4 SS_LITEPLAYER_UnRegisterDemuxer

【描述】

去注册解封装库。

【语法】

TD_S32 SS_LITEPLAYER_UnRegisterDemuxer(const TD_CHAR* pszDllName)

【参数】

参数名称	描述	输入/输出
pszDllName	解封装库文件名(全路径)	输入

【返回值】

返回值	描述
0	成功。
非0	失败，参见 错误码 。

【需求】

- 头文件：ss_demuxer.h、ss_liteplayer.h
- 库文件：libliteplayer.a、libavplay_linux.a、libliteplayer.so、libffmpeg_demuxer.so

【注意】

该接口在liteplayer实例创建后至销毁前调用会出错，此时解封装库正在使用中。请在liteplayer实例不存在时调用。



【举例】

无。

参考“[播放器使用示例](#)”。

【相关主题】

无。

7.3.5 SS_LITEPLAYER_RegisterAudioDecoder

【描述】

注册音频解码库。

【语法】

```
TD_S32 SS_LITEPLAYER_RegisterAudioDecoder(const TD_CHAR* pszAudioDecLib);
```

【参数】

参数名称	描述	输入/输出
pszAudioDecLib	音频解码库库文件名（全路径）	输入

【返回值】

返回值	描述
0	成功。
非0	失败，参见 错误码 。

【芯片差异】

无

【需求】

- 头文件：ss_demuxer.h、ss_liteplayer.h
- 库文件：libliteplayer.a、libavplay_linux.a、libliteplayer.so、libffmpeg_demuxer.so

【注意】

- 当前仅支持AAC解码库注册。
- 解码库全路径最大长度为256。
- 该接口要在所有liteplayer实例调用[SS_LITEPLAYER_Play](#)前或者[SS_LITEPLAYER_Stop](#)之后调用。

【举例】

参考“[播放器使用示例](#)”。



【相关主题】

无。

7.3.6 SS_LITEPLAYER_UnRegisterAudioDecoder

【描述】

去注册音频解码库。

【语法】

```
TD_S32 SS_LITEPLAYER_UnRegisterAudioDecoder(const TD_CHAR* pszAudioDecLib);
```

【参数】

参数名称	描述	输入/输出
pszAudioDecLib	音频解码库文件名(全路径)	输入

【返回值】

返回值	描述
0	成功。
非0	失败，参见 错误码 。

【需求】

- 头文件：ss_demuxer.h、ss_liteplayer.h
- 库文件：libliteplayer.a、libavplay_linux.a、libliteplayer.so、libffmpeg_demuxer.so

【注意】

- 解码库全路径最大长度为256。
- 该接口在所有liteplayer实例调用[SS_LITEPLAYER_Play](#)前或者[SS_LITEPLAYER_Stop](#)之后调用。

【举例】

参考“[播放器使用示例](#)”。

【相关主题】

无。

7.3.7 SS_LITEPLAYER_Create

【描述】

创建播放器。

【语法】



```
TD_S32 SS_LITEPLAYER_Create(TD_MW_PTR* ppPlayer, const OT_LITEPLAYER_PARAM_S*  
pstCreateParam);
```

【参数】

参数名称	描述	输入/输出
ppPlayer	播放器实例句柄的地址。	输出
pstCreateParam	播放器实例创建所需参数。	输入

【返回值】

返回值	描述
0	成功。
非0	失败，参见 错误码 。

【需求】

- 头文件：ss_demuxer.h、ss_liteplayer.h
- 库文件：libliteplayer.a、libavplay_linux.a、libliteplayer.so

【注意】

第二个参数不可为空，空指针会返回失败。

【举例】

参考“[播放器使用示例](#)”。

【相关主题】

无。

7.3.8 SS_LITEPLAYER_SetVOHandle

【描述】

设置player要绑定的Vout信息。

【语法】

```
TD_S32 SS_LITEPLAYER_SetVOHandle(TD_MW_PTR pPlayer, const  
OT_LITEPLAYER_VOUT_OPT_S* pstVoutOpt);
```

【参数】

参数名称	描述	输入/输出
pPlayer	播放器实例句柄。	输入
pstVoutOpt	Vout属性。	输入



【返回值】

返回值	描述
0	成功。
非0	失败，参见 错误码 。

【需求】

- 头文件：ss_demuxer.h、ss_liteplayer.h
- 库文件：libliteplayer.a、libavplay_linux.a、libliteplayer.so

【注意】

无。

【相关主题】

无。

7.3.9 SS_LITEPLAYER_SetAOHandle

【描述】

设置player要绑定的Aout信息。

【语法】

```
TD_S32 SS_LITEPLAYER_SetAOHandle(TD_MW_PTR pPlayer, const  
OT\_LITEPLAYER\_AOUT\_OPT\_S* pstAoutOpt);
```

【参数】

参数名称	描述	输入/输出
pPlayer	播放器实例句柄。	输入
pstAoutOpt	AO句柄。	输入

【返回值】

返回值	描述
0	成功。
非0	失败，参见 错误码 。

【需求】

- 头文件：ss_demuxer.h、ss_liteplayer.h



- 库文件：libliteplayer.a、libavplay_linux.a、libliteplayer.so

【注意】

无。

【举例】

参考“[播放器使用示例](#)”。

【相关主题】

无。

7.3.10 SS_LITEPLAYER_Destroy

【描述】

销毁player实例。

【语法】

TD_S32 SS_LITEPLAYER_Destroy(TD_MW_PTR pPlayer);

【参数】

参数名称	描述	输入/输出
pPlayer	播放器实例句柄。	输入

【返回值】

返回值	描述
0	成功。
非0	失败，参见 错误码 。

【需求】

- 头文件：ss_demuxer.h、ss_liteplayer.h
- 库文件：libliteplayer.a、libavplay_linux.a、libliteplayer.so

【注意】

无。

【举例】

参考“[播放器使用示例](#)”。

【相关主题】

无。



7.3.11 SS_LITEPLAYER_SetDataSource

【描述】

设置player要播放的数据源。

【语法】

TD_S32 SS_LITEPLAYER_SetDataSource (TD_MW_PTR pPlayer, const TD_CHAR* pszfilePath);

【参数】

参数名称	描述	输入/输出
pPlayer	播放器实例句柄。	输入
pszfilePath	播放数据源的路径。	输入

【返回值】

返回值	描述
0	成功。
非0	失败，参见 错误码 。

【需求】

- 头文件：ss_demuxer.h、ss_liteplayer.h
- 库文件：libliteplayer.a、libavplay_linux.a、libliteplayer.so

【注意】

filePath的路径长度不能大于1024。

【举例】

参考“[播放器使用示例](#)”。

【相关主题】

无。

7.3.12 SS_LITEPLAYER_Prepare

【描述】

对player里面的数据执行预处理。

【语法】

TD_S32 SS_LITEPLAYER_Prepare (TD_MW_PTR pPlayer)TD_MW_PTR pPlayer;

【参数】



参数名称	描述	输入/输出
pPlayer	播放器实例句柄。	输入

【返回值】

返回值	描述
0	成功。
非0	失败，参见 错误码 。

【需求】

- 头文件：ss_demuxer.h、ss_liteplayer.h
- 库文件：libliteplayer.a、libavplay_linux.a、libliteplayer.so

【注意】

无。

【举例】

参考“[播放器使用示例](#)”。

【相关主题】

无。

7.3.13 SS_LITEPLAYER_Play

【描述】

开始播放媒体文件。

【语法】

TD_S32 SS_LITEPLAYER_Play (TD_MW_PTR pPlayer)TD_MW_PTR pPlayer;

【参数】

参数名称	描述	输入/输出
pPlayer	播放器实例句柄。	输入

【返回值】

返回值	描述
0	成功。



返回值	描述
非0	失败，参见 错误码 。

【需求】

- 头文件：ss_demuxer.h、ss_liteplayer.h
- 库文件：libliteplayer.a、libavplay_linux.a、libliteplayer.so

【注意】

在[SS_LITEPLAYER_Prepare](#)接口之后调用。

【举例】

参考“[播放器使用示例](#)”。

【相关主题】

无。

7.3.14 SS_LITEPLAYER_Stop

【描述】

停止播放媒体文件并且销毁内部资源。

【语法】

```
TD_S32 SS_LITEPLAYER_Stop(TD_MW_PTR pPlayer);
```

【参数】

参数名称	描述	输入/输出
pPlayer	播放器句柄	输入

【返回值】

返回值	描述
0	成功。
非0	失败，参见 错误码 。

【需求】

- 头文件：ss_demuxer.h、ss_liteplayer.h
- 库文件：libliteplayer.a、libavplay_linux.a、libliteplayer.so

【注意】



OT_LITEPLAYER_STATE_ERR状态之后必须调用该接口才能恢复到OT_LITEPLAYER_STATE_IDLE状态。

【举例】

参考“[播放器使用示例](#)”。

【相关主题】

无。

7.3.15 SS_LITEPLAYER_Pause

【描述】

暂停播放媒体文件。

【语法】

TD_S32 SS_LITEPLAYER_Pause(TD_MW_PTR pPlayer);

【参数】

参数名称	描述	输入/输出
pPlayer	播放器实例句柄	输入

【返回值】

返回值	描述
0	成功。
非0	失败，参见 错误码 。

【需求】

- 头文件：ss_demuxer.h、ss_liteplayer.h
- 库文件：libliteplayer.a、libavplay_linux.a、libliteplayer.so

【注意】

无。

【举例】

无。

【相关主题】

无。

7.3.16 SS_LITEPLAYER_Seek

【描述】



移动到指定播放位置。

【语法】

TD_S32 SS_LITEPLAYER_Seek(TD_MW_PTR pPlayer, TD_S64 s64TimeInMs);

【参数】

参数名称	描述	输入/输出
pPlayer	播放器实例句柄	输入
s64TimeInMs	Seek的时间点，单位：ms	输入

【返回值】

返回值	描述
0	成功。
非0	失败，参见 错误码 。

【需求】

- 头文件：ss_demuxer.h、ss_liteplayer.h
- 库文件：libliteplayer.a、libavplay_linux.a、libliteplayer.so

【注意】

- 该接口是异步接口，seek完成后会在注册的回调函数中收到OT_LITEPLAYER_SEEK_END消息。
- 多seek操作需要在前次seek操作收到OT_LITEPLAYER_SEEK_END回调消息后执行，否则可能出现中间的seek操作失效。
- 该接口只能在OT_LITEPLAYER_STATE_PLAY、OT_LITEPLAYER_STATE_PAUSE两种播放状态下调用。
- 在调用seek接口与接收到seek end回调消息之间：
 - 调用[SS_LITEPLAYER_Play](#)或[SS_LITEPLAYER_Pause](#)会执行失败。
 - 调用[SS_LITEPLAYER_TPlay](#) 则seek操作可能会执行失败。

【举例】

参考“[播放器使用示例](#)”。

【相关主题】

无。

7.3.17 SS_LITEPLAYER_RegCallback

【描述】

注册回调函数。



【语法】

TD_S32 SS_LITEPLAYER_RegCallback(TD_MW_PTR pPlayer, OT_LITEPLAYER_EVENT_FN pfnCallback);

【参数】

参数名称	描述	输入/输出
pPlayer	播放器句柄	输入
pfnCallback	回调事件函数指针	输入

【返回值】

返回值	描述
0	成功。
非0	失败，参见 错误码 。

【需求】

- 头文件：ss_demuxer.h、ss_liteplayer.h
- 库文件：libliteplayer.a、libavplay_linux.a、libliteplayer.so

【注意】

- 必须在获得播放器句柄后设置回调函数句柄，以获取文件播放结束消息与播放状态改变消息等。
- 回调函数里不能有阻塞动作。
- 回调函数里不能调用liteplayer接口，否则可能会出现异常情况。
- 回调函数在用户调用完[SS_LITEPLAYER_Destroy](#)后失效。

【举例】

参考“[播放器使用示例](#)”。

【相关主题】

无。

7.3.18 SS_LITEPLAYER_GetFileInfo

【描述】

获取媒体文件相关信息。

【语法】

TD_S32 SS_LITEPLAYER_GetFileInfo(TD_MW_PTR pPlayer, OT_FORMAT_FILE_INFO_S* pstFormatInfo)

【参数】



参数名称	描述	输入/输出
pPlayer	播放器实例句柄	输入
pstFormatInfo	媒体文件信息结构体指针	输出

【返回值】

返回值	描述
0	成功。
非0	失败，参见 错误码 。

【需求】

- 头文件：ss_demuxer.h、ss_liteplayer.h
- 库文件：libliteplayer.a、libavplay_linux.a、libliteplayer.so

【注意】

只能在OT_LITEPLAYER_STATE_PREPARED、OT_LITEPLAYER_STATE_PLAY、OT_LITEPLAYER_STATE_PAUSE三种播放状态下调用。

【举例】

参考“[播放器使用示例](#)”。

【相关主题】

无。

7.3.19 SS_LITEPLAYER_SetMedia

【描述】

设置媒体文件的相关属性。

【语法】

```
TD_S32 SS_LITEPLAYER_SetMedia(TD_MW_PTR pPlayer, const OT\_LITEPLAYER\_ATTR\_S\* pstMediaAttr);
```

【参数】

参数名称	描述	输入/输出
pPlayer	播放器实例句柄	输入
pstMediaAttr	媒体文件相关属性	输入

【返回值】



返回值	描述
0	成功。
非0	失败，参见 错误码 。

【需求】

- 头文件：ss_demuxer.h、ss_liteplayer.h
- 库文件：libliteplayer.a、libavplay_linux.a、libliteplayer.so

【注意】

该接口需要在收到TD_LITEPLAYER_STATE_PREPARED消息之后，执行[SS_LITEPLAYER_Play](#)之前调用。MP4文件当前不支持修改播放的StreamId。。

【举例】

无。

【相关主题】

无。

7.3.20 SS_LITEPLAYER_GetPlayStatus

【描述】

获取播放器的播放状态。

【语法】

```
TD_S32 SS_LITEPLAYER_GetPlayStatus(TD_MW_PTR pPlayer, OT\_LITEPLAYER\_STATE\_E * penState);
```

【参数】

参数名称	描述	输入/输出
pPlayer	播放器实例句柄	输入
penState	播放器播放状态指针	输出

【返回值】

返回值	描述
0	成功。
非0	失败，参见 错误码 。

【需求】



- 头文件：ss_demuxer.h、ss_liteplayer.h
- 库文件：libliteplayer.a、libavplay_linux.a、libliteplayer.so

【注意】

该接口主要用获取播放器当前的播放状态。

【举例】

参考“[播放器使用示例](#)”。

【相关主题】

无。

7.3.21 SS_LITEPLAYER_TPlay

【描述】

使能播放器倍速播放。

【语法】

```
TD_S32 SS_LITEPLAYER_TPlay(TD_MW_PTR pPlayer, const OT\_LITEPLAYER\_TPLAY\_ATTR\_S\*
pstTrickPlayAttr);
```

【参数】

参数名称	描述	输入/输出
pPlayer	播放器实例句柄	输入
pstTrickPlayAttr	播放器倍速属性	输入

【返回值】

返回值	描述
0	成功。
非0	失败，参见 错误码 。

【需求】

- 头文件：ss_demuxer.h、ss_liteplayer.h
- 库文件：libliteplayer.a、libavplay_linux.a、libliteplayer.so

【注意】

- 只允许Play状态切换到TPlay状态。
- 单音频文件（M4A/WAV）不支持进行TPlay。

【举例】

参考“[播放器使用示例](#)”。



【相关主题】

无。

7.4 数据类型

- **OT_LITEPLAYER_ERROR_E**: 定义播放错误。
- **OT_LITEPLAYER_STATE_E**: 定义播放状态。
- **OT_LITEPLAYER_EVENT_E**: 定义播放回调事件。
- **OT_LITEPLAYER_ATTR_S**: 定义播放器属性。
- **OT_LITEPLAYER_EVENT_FN**: 定义播放回调函数指针类型。
- **OT_LITEPLAYER_PARAM_S**: 定义创建播放器实例所需参数。
- **OT_LITEPLAYER_EXTRA_FILE_SUFFIX_S**: 定义demuxer可额外支持的文件后缀信息。
- **OT_LITEPLAYER_VOUT_OPT_S**: 定义播放器绑定的VOUT属性。
- **OT_LITEPLAYER_AOUT_OPT_S**: 定义播放器绑定的AOUT属性。
- **OT_LITEPLAYER_TPLAY_ATTR_S**: 定义播放器倍速播放属性。
- **OT_LITEPLAYER_VOUT_TYPE_E**: 定义VOUT输出类型。
- **OT_LITEPLAYER_TPLAY_DIRECTION_E**: 定义倍速播放方向。
- **OT_LITEPLAYER_PLAY_SPEED_E**: 定义倍速播放速度。
- **OT_LITEPLAYER_FILE_PATH_MAX_LEN**: 定义媒体文件全路径最大长度。
- **OT_LITEPLAYER_AUD_DECODER_PATH_LEN**: 定义音频解码库最大长度。
- **OT_LITEPLAYER_MAX_FILE_SUFFIX_CNT**: 定义解封库额外支持的文件后缀最大个数。
- **OT_LITEPLAYER_MAX_FILE_SUFFIX_LEN**: 定义解封库额外支持的文件后缀最大长度。
- **OT_FORMAT_PACKET_S**: 定义解封装后的包结构体。
- **OT_FORMAT_STREAM_RESOLUTION_S**: 定义媒体分辨率。
- **OT_FORMAT_VIDEO_TYPE_E**: 定义视频类型。
- **OT_FORMAT_AUDIO_TYPE_E**: 定义音频类型。
- **OT_FORMAT_FILE_INFO_S**: 定义文件信息。
- **OT_FORMAT_FUN_S**: 定义解封装的函数。
- **OT_DEMUX_S**: 定义解封装库所包含信息的结构体。
- **OT_FORMAT_MEDIA_ATTR_S**: 定义解封装的媒体文件属性。
- **OT_DEMUXER_RESOLUTION_CNT**: 定义媒体文件对应的最大视频流个数。
- **OT_DEMUXER_NAME_LEN**: 定义解封库名字的长度。
- **OT_DEMUXER_SUPPORT_NAMES_LEN**: 定义解封库名支持的文件类型总长度。

7.4.1 OT_LITEPLAYER_ERROR_E

【说明】



定义播放器错误。

【定义】

```
typedef enum LITEPLAYER_ERROR_E
{
    OT_LITEPLAYER_ERROR_VID_PLAY_FAIL = 0x0,
    OT_LITEPLAYER_ERROR_AUD_PLAY_FAIL,
    OT_LITEPLAYER_ERROR_DEMUX_FAIL,
    OT_LITEPLAYER_ERROR_TIMEOUT,
    OT_LITEPLAYER_ERROR_NOT_SUPPORT,
    OT_LITEPLAYER_ERROR_UNKNOW,
    OT_LITEPLAYER_ERROR_ILLEGAL_STATEACTION,
    OT_LITEPLAYER_ERROR_BUTT
} OT_LITEPLAYER_ERROR_E
```

【成员】

成员名称	描述
OT_LITEPLAYER_ERROR_VID_PLAY_FAIL	视频播放失败。
OT_LITEPLAYER_ERROR_AUD_PLAY_FAIL	音频播放失败。
OT_LITEPLAYER_ERROR_DEMUX_FAIL	媒体数据解封装失败。
OT_LITEPLAYER_ERROR_TIMEOUT	媒体文件播放超时。
OT_LITEPLAYER_ERROR_NOT_SUPPORT	媒体格式不支持。
OT_LITEPLAYER_ERROR_UNKNOW	未知错误。
OT_LITEPLAYER_ERROR_ILLEGAL_STATEACTION	该状态下非法操作。

【注意事项】

无。

【相关数据类型及接口】

无。

7.4.2 OT_LITEPLAYER_STATE_E

【说明】

定义播放状态。

【定义】

```
typedef enum LITEPLAYER_STATE_E
{
    OT_LITEPLAYER_STATE_IDLE = 0,
    OT_LITEPLAYER_STATE_INIT,
    OT_LITEPLAYER_STATE_PREPARED,
    OT_LITEPLAYER_STATE_PLAY,
    OT_LITEPLAYER_STATE_TPLAY,
    OT_LITEPLAYER_STATE_PAUSE,
    OT_LITEPLAYER_STATE_ERR,
    OT_LITEPLAYER_STATE_BUTT
} OT_LITEPLAYER_STATE_E;
```




【成员】

成员名称	描述
OT_LITEPLAYER_STATE_IDLE	播放初始化之前的状态。
OT_LITEPLAYER_STATE_INIT	播放初始化状态。
OT_LITEPLAYER_STATE_PREPARED	播放准备状态。
OT_LITEPLAYER_STATE_PLAY	播放状态。
OT_LITEPLAYER_STATE_TPLAY	倍速播放状态。
OT_LITEPLAYER_STATE_PAUSE	播放暂停状态。
OT_LITEPLAYER_STATE_ERR	播放错误状态。
OT_LITEPLAYER_STATE_IDLE	播放初始化之前的状态。

【注意事项】

无。

【相关数据类型及接口】

无。

7.4.3 OT_LITEPLAYER_EVENT_E

【说明】

定义播放回调事件。

【定义】

```
typedef enum LITEPLAYER_EVENT_E
{
    OT_LITEPLAYER_EVENT_STATE_CHANGED = 0x0,
    OT_LITEPLAYER_EVENT_EOF,
    OT_LITEPLAYER_EVENT_SOF,
    OT_LITEPLAYER_EVENT_PROGRESS,
    OT_LITEPLAYER_EVENT_SEEK_END,
    OT_LITEPLAYER_EVENT_ERROR,
    OT_LITEPLAYER_EVENT_BUTTON
} OT_LITEPLAYER_EVENT_E;
```

【成员】

成员名称	描述
OT_LITEPLAYER_EVENT_STATE_CHANGED	播放状态改变事件。
OT_LITEPLAYER_EVENT_EOF	播放到文件尾事件。
OT_LITEPLAYER_EVENT_SOF	播放到文件头事件。
OT_LITEPLAYER_EVENT_PROGRESS	播放进度事件。



成员名称	描述
OT_LITEPLAYER_SEEK_END	Seek完成时间
OT_LITEPLAYER_EVENT_ERROR	播放错误事件。

【注意事项】

OT_LITEPLAYER_EVENT_SOF事件是指倍速播放快退场景下播放到文件头部。

【相关数据结构及接口】

无。

7.4.4 OT_LITEPLAYER_ATTR_S

【说明】

定义播放器属性。

【定义】

```
typedef struct LITEPLAYER_ATTR_S
{
    TD_S32 s32VidStreamId;
    TD_S32 s32AudStreamId;
} OT_LITEPLAYER_ATTR_S;
```

【成员】

成员名称	描述
s32VidStreamId	设置将要播放的视频流索引。
s32AudStreamId	设置将要播放的音频流索引。

【注意事项】

无。

【相关数据结构及接口】

无。

7.4.5 OT_LITEPLAYER_EVENT_FN

【说明】

定义播放回调函数指针类型。

【定义】

```
typedef TD_VOID (*OT_LITEPLAYER_EVENT_FN)(TD_MW_PTR pPlayer,
OT_LITEPLAYER_EVENT_E enEvent, TD_VOID * pData)
```

【成员】



成员名称	描述
pPlayer	播放器实例句柄。
enEvent	播放器事件类型。
pData	播放器对应事件携带的数据地址。 - 当事件为OT_LITEPLAYER_EVENT_STATE_CHANGED, pszData对应OT_LITEPLAYER_STATE_E类型变量的地址, 指示当前的播放状态; - 当事件为OT_LITEPLAYER_EVENT_EOF, pszData对应为NULL; - 当事件为OT_LITEPLAYER_EVENT_PROGRESS, pszData对应为unsigned int 类型数据地址, 指示当前播放的时间, 单位是毫秒; - 当事件为OT_LITEPLAYER_SEEK_END, pszData对应为int64 类型数据地址, 指示本次seek的跳转时间; - 当事件为OT_LITEPLAYER_EVENT_ERROR, pszData对应为OT_LITEPLAYER_ERROR_E类型数据地址;

【 注意事项 】

无。

【 相关数据类型及接口 】

无。

7.4.6 OT_LITEPLAYER_PARAM_S

【 说明 】

定义创建播放器实例所需参数。

【 定义 】

```
typedef struct LITEPLAYER_PARAM_S
{
    TD_U32 u32PlayPosNotifyIntervalMs; /**< Current playing Time notify interval in Ms. */
    TD_U32 u32VideoEsBufSize; /**< video es stream cache buf size.[1k,200M];if 0,use the inner default value */
    TD_U32 u32AudioEsBufSize; /**< audio es stream cache buf size.[1k,100M];if 0,use the inner default value */
    TD_U32 u32VdecFrameBufCnt; /**< Number of VB blocks for storing decoded image frames;if 0, use the inner default value */
    TD_BOOL bPauseMode; /**< pause after first frame render mode flag,TD_FALSE:do not pause, TD_TRUE:pause after first frame render. */
} OT_LITEPLAYER_PARAM_S;
```

【 成员 】

成员名称	描述
u32PlayPosNotifyIntervalMs	播放器当前播放位置回调通知时间间隔。



成员名称	描述
u32VideoEsBufSize	播放器视频流解码缓存buf最大size。取值为[1k, 200M]，单位：字节
u32AudioEsBufSize	播放器音频流解码缓存buf最大size。取值为[1k, 100M]，单位：字节
u32VdecFrameBufCnt	播放器视频解码缓存VB数量，取值为0时，使用系统默认值作为VB缓存数量。
bPauseMode	是否只播放一帧就暂停。

【注意事项】

- 当u32PlayPosNotifyIntervalMs传入值为0时，播放器内部会使用300ms作为默认回调间隔；使用非零值时，最小值硬设置为50ms；如果该值大于文件总时长时，将不会有回调通知信息。
- 当u32VideoEsBufSize和u32AudioEsBufSize传入值为0时，默认会根据播放文件分辨率进行解码缓存vdecBuffer配置: vdecBuffer为宽*高*1.5（内部还有用于核间共享的mbuffer 用来存放解封装后的码流，mbuffersize=vdecBuffer*1/2）；用户可配置这两个值，但如果配置过小，可能会导致无法播放。
- 当播放图片时，u32VideoEsBufSize在sdk解码库中有限制，因此当配置值小于该限制（当前sdk文档中为32K byte），会导致图片播放失败。
- 当u32VdecFrameBufCnt传入值为0时，播放器内部使用3作为解码VB缓存个数；当前版本中u32VdecFrameBufCnt不能小于3，小于3会创建失败。播放视频时，在一定情况下解码VB缓存个数会影响播放流畅度，如果帧率比较高出现卡顿，可酌情调高u32VdecFrameBufCnt个数；u32VdecFrameBufCnt数量大时，会占用较高内存，甚至会导致内存分配失败。

【相关数据结构及接口】

无。

7.4.7 OT_LITEPLAYER_EXTRA_FILESURFIX_S

【说明】

定义demuxer可额外支持的文件后缀信息。

【定义】

```
typedef struct LITEPLAYER_EXTRA_FILESURFIX_S
{
    TD_U32 u32SurfixCnt;
    TD_CHAR aszFileSurfix[OT_LITEPLAYER_MAX_FILE_SURFIX_CNT]
[OT_LITEPLAYER_MAX_FILE_SURFIX_LEN];
} OT_LITEPLAYER_EXTRA_FILESURFIX_S;
```

【成员】

成员名称	描述
u32SurfixCnt	可注册的额外文件后缀个数。



成员名称	描述
aszFileSurfix	可注册的额外文件后缀具体内容。

【注意事项】

无。

【相关数据结构及接口】

无。

7.4.8 OT_LITEPLAYER_VOUT_OPT_S

【说明】

定义VOUT属性信息。

【定义】

```
typedef struct LITEPLAYER_VOUT_OPT_S
{
    OT_LITEPLAYER_VOUT_TYPE_E enVoutType;
    TD_HANDLE hModHdl;
    TD_HANDLE hChnHdl;
}OT_LITEPLAYER_VOUT_OPT_S;
```

【成员】

成员名称	描述
enVoutType	VOUT输出类型。
hModHdl	根据VOUT类型不同，含义不同； 当enVoutType为VO时，hModHdl为VO设备句柄； 当enVoutType为VPSS时，hModHdl为VPSS组句柄。
hChnHdl	根据VOUT类型不同，含义不同； 当enVoutType为VO时，hChnHdl为VO窗口通道句柄； 当enVoutType为VPSS时，hChnHdl为VPSS通道句柄。

【注意事项】

无。

【相关数据结构及接口】

无。

7.4.9 OT_LITEPLAYER_AOUT_OPT_S

【说明】

定义AOUT属性信息。



【定义】

```
typedef struct LITEPLAYER_AOUT_OPT_S
{
    TD_HANDLE hAudDevHdl;//device id
    TD_HANDLE hAudTrackHdl;//chn id
}OT_LITEPLAYER_AOUT_OPT_S;
```

【成员】

成员名称	描述
hAudDevHdl	AO音频输出设备号。
hAudTrackHdl	AO音频输出通道号。

【注意事项】

本模块支持多实例，在多实例场景下请确认每个实例配置的媒体资源之间不冲突。例如：同时创建两个实例播放音频文件时，AO设备/通道配置支持该业务场景。

【相关数据结构及接口】

无。

7.4.10 OT_LITEPLAYER_TPLAY_ATTR_S

【说明】

定义倍速播放属性。

【定义】

```
typedef struct LITEPLAYER_TPLAY_ATTR_S
{
    OT_LITEPLAYER_TPLAY_DIRECTION_E enDirection;
    OT_LITEPLAYER_PLAY_SPEED_E enSpeed;
}OT_LITEPLAYER_TPLAY_ATTR_S;
```

【成员】

成员名称	描述
enDirection	倍速播放方向：向前或者向后。
enSpeed	倍速播放速度，为2的倍数。

【注意事项】

无。

【相关数据结构及接口】

无。



7.4.11 OT_LITEPLAYER_VOUT_TYPE_E

【说明】

定义VOUT输出类型。

【定义】

```
typedef enum LITEPLAYER_VOUT_TYPE_E
{
    OT_LITEPLAYER_VOUT_TYPE_VPSS,
    OT_LITEPLAYER_VOUT_TYPE_VO,
    OT_LITEPLAYER_VOUT_TYPE_BUTT
} OT_LITEPLAYER_VOUT_TYPE_E;
```

【成员】

成员名称	描述
OT_LITEPLAYER_VOUT_TYPE_VPSS	VPSS+VO输出模式。
OT_LITEPLAYER_VOUT_TYPE_VO	VO输出模式。

【注意事项】

【相关数据类型及接口】

无。

7.4.12 OT_LITEPLAYER_TPLAY_DIRECTION_E

【说明】

定义倍速播放方向类型。

【定义】

```
typedef enum LITEPLAYER_TPLAY_DIRECTION_E
{
    OT_LITEPLAYER_TPLAY_FORWARD,
    OT_LITEPLAYER_TPLAY_BACKWARD,
    OT_LITEPLAYER_TPLAY_BUTT
} OT_LITEPLAYER_TPLAY_DIRECTION_E;
```

【成员】

成员名称	描述
OT_LITEPLAYER_TPLAY_FORWARD	文件倍速正向播放。
OT_LITEPLAYER_TPLAY_BACKWARD	文件倍速反向播放。

【注意事项】

无。

【相关数据类型及接口】



无。

7.4.13 OT_LITEPLAYER_PLAY_SPEED_E

【说明】

定义倍速播放方向类型。

【定义】

```
typedef enum LITEPLAYER_PLAY_SPEED_E
{
    OT_LITEPLAYER_PLAY_SPEED_BASE = 100,
    OT_LITEPLAYER_PLAY_SPEED_2X_FAST = 2*OT_LITEPLAYER_PLAY_SPEED_BASE,
    OT_LITEPLAYER_PLAY_SPEED_4X_FAST = 4*OT_LITEPLAYER_PLAY_SPEED_BASE,
    OT_LITEPLAYER_PLAY_SPEED_8X_FAST = 8*OT_LITEPLAYER_PLAY_SPEED_BASE,
    OT_LITEPLAYER_PLAY_SPEED_16X_FAST = 16*OT_LITEPLAYER_PLAY_SPEED_BASE,
    OT_LITEPLAYER_PLAY_SPEED_32X_FAST = 32*OT_LITEPLAYER_PLAY_SPEED_BASE,
    OT_LITEPLAYER_PLAY_SPEED_64X_FAST = 64*OT_LITEPLAYER_PLAY_SPEED_BASE,
    OT_LITEPLAYER_PLAY_SPEED_128X_FAST = 128*OT_LITEPLAYER_PLAY_SPEED_BASE,
    OT_LITEPLAYER_PLAY_SPEED_BUTT
} OT_LITEPLAYER_PLAY_SPEED_E;
```

【成员】

成员名称	描述
OT_LITEPLAYER_PLAY_SPEED_2X_FAST	2倍速。
OT_LITEPLAYER_PLAY_SPEED_4X_FAST	4倍速。
OT_LITEPLAYER_PLAY_SPEED_8X_FAST	8倍速。
OT_LITEPLAYER_PLAY_SPEED_16X_FAST	16倍速。
OT_LITEPLAYER_PLAY_SPEED_32X_FAST	32倍速。
OT_LITEPLAYER_PLAY_SPEED_64X_FAST	64速。
OT_LITEPLAYER_PLAY_SPEED_128X_FAST	128倍速。

【注意事项】

无。

【相关数据类型及接口】

无。

7.4.14 OT_LITEPLAYER_FILE_PATH_MAX_LEN

【说明】

定义输入播放文件URL的最大长度。

【定义】

```
#define OT_LITEPLAYER_FILE_PATH_MAX_LEN (1024)
```

【注意事项】



无。

【相关数据类型及接口】

无。

7.4.15 OT_LITEPLAYER_AUD_DECODER_PATH_LEN

【说明】

定义音频解码库路径最大长度。

【定义】

```
#define OT_LITEPLAYER_AUD_DECODER_PATH_LEN (256)
```

【注意事项】

无。

【相关数据类型及接口】

无。

7.4.16 OT_LITEPLAYER_MAX_FILE_SURFIX_CNT

【说明】

定义解封库额外支持的文件后缀最大个数。

【定义】

```
#define OT_LITEPLAYER_MAX_FILE_SURFIX_CNT (10)
```

【注意事项】

无。

【相关数据类型及接口】

无。

7.4.17 OT_LITEPLAYER_MAX_FILE_SURFIX_LEN

【说明】

定义解封库额外支持的文件后缀最大长度。

【定义】

```
#define OT_LITEPLAYER_MAX_FILE_SURFIX_LEN (32)
```

【注意事项】

无。

【相关数据类型及接口】

无。

7.4.18 OT_FORMAT_PACKET_S

【说明】



定义解封装后的包结构体。

【定义】

```
typedef struct FORMAT_PACKET_S {  
    TD_S64 s64Pts;  
    TD_S64 s64Dts;  
    TD_U8* pu8Data;  
    TD_U32 u32Size;  
    TD_U8* pu8Header;  
    TD_U32 u32HeaderLen;  
    TD_U32 u32StreamIndex;  
    TD_BOOL bKeyFrame;  
    TD_S64 s64Duration;  
    TD_S64 s64Pos;  
} OT_FORMAT_PACKET_S;
```

【成员】

成员名称	描述
s64Pts	显示时间戳。
s64Dts	解码时间戳。
pu8Data	包的地址。
u32Size	包的长度。
pu8Header	一些文件流的头部信息地址。
u32HeaderLen	一些文件流的头部信息长度。
u32StreamIndex	该包属于的流索引。
bKeyFrame	是否是关键帧。 TD_TRUE: 是; TD_FALSE: 否。
s64Duration	两包之间的时间差。
s64Pos	该数据包在视频流中的位置。

【注意事项】

无。

【相关数据类型及接口】

无。

7.4.19 OT_FORMAT_STREAM_RESOLUTION_S

【说明】

定义媒体分辨率。

【定义】



```
typedef struct FORMAT_STREAM_RESOLUTION_S
{
    TD_S32 s32VideoStreamIndex;
    TD_U32 u32Width;
    TD_U32 u32Height;
    TD_U32 codecTag;
    OT_FORMAT_VIDEO_TYPE_E enVideoType;
}OT_FORMAT_STREAM_RESOLUTION_S;
```

【成员】

成员名称	描述
s32VideoStreamIndex	视频流索引。
u32Width	视频流分辨率的宽度。
u32Height	视频流分辨率的高度。
codecTag	视频流标签。
enVideoType	视频流类型。

【注意事项】

无。

【相关数据类型及接口】

无。

7.4.20 OT_FORMAT_VIDEO_TYPE_E

【说明】

定义视频类型。

【定义】

```
typedef enum FORMAT_VIDEO_TYPE_E
{
    OT_FORMAT_VIDEO_TYPE_H264,
    OT_FORMAT_VIDEO_TYPE_H265,
    OT_FORMAT_VIDEO_TYPE_JPEG,
    OT_FORMAT_VIDEO_TYPE_MJPEG,
    OT_FORMAT_VIDEO_TYPE_WRAPPED_AVFRAME,
    OT_FORMAT_VIDEO_TYPE_BUTT
}OT_FORMAT_VIDEO_TYPE_E;
```

【成员】

成员名称	描述
OT_FORMAT_VIDEO_TYPE_H264	H.264格式。
OT_FORMAT_VIDEO_TYPE_H265	H.265格式。
OT_FORMAT_VIDEO_TYPE_JPEG	JPEG格式。
OT_FORMAT_VIDEO_TYPE_MJPEG	MJPEG格式。



成员名称	描述
OT_FORMAT_VIDEO_TYPE_WRAPPED_AVFRAME	数据流格式。

【注意事项】

OT_FORMAT_VIDEO_TYPE_WRAPPED_AVFRAME用于获取datastream，如ts的缩略图。

【相关数据类型及接口】

无。

7.4.21 OT_FORMAT_AUDIO_TYPE_E

【说明】

定义音频类型。

【定义】

```
typedef enum FORMAT_AUDIO_TYPE_E {  
    OT_FORMAT_AUDIO_TYPE_AAC,  
    OT_FORMAT_AUDIO_TYPE_PCM,  
    OT_FORMAT_AUDIO_TYPE_G711A,  
    OT_FORMAT_AUDIO_TYPE_G711U,  
    OT_FORMAT_AUDIO_TYPE_MP3,  
    OT_FORMAT_AUDIO_TYPE_BUTT  
} OT_FORMAT_AUDIO_TYPE_E;
```

【成员】

成员名称	描述
OT_FORMAT_AUDIO_TYPE_AAC	AAC格式。
OT_FORMAT_AUDIO_TYPE_PCM	PCM格式。
OT_FORMAT_AUDIO_TYPE_G711A	G.711 A-law格式。
OT_FORMAT_AUDIO_TYPE_G711U	G.711 μ -law格式。
OT_FORMAT_AUDIO_TYPE_MP3	MP3格式。

【注意事项】

无。

【相关数据类型及接口】

无。

7.4.22 OT_FORMAT_FILE_INFO_S

【说明】



定义码流报文信息结构体。

【定义】

```
typedef struct FORMAT_FILE_INFO_S
{
    TD_S64 s64FileSize;
    TD_S64 s64StartTime;
    TD_S64 s64Duration;
    TD_S64 audioDuration;
    TD_S64 videoDuration;
    OT_FORMAT_STREAM_RESOLUTION_S
    stStreamResolution[OT_DEMUXER_RESOLUTION_CNT];
    TD_S32 s32UsedVideoStreamIndex;
    TD_FLOAT fFrameRate;
    TD_U32 u32Bitrate;
    TD_U32 u32AudioChannelNum;
    TD_U32 u32SampleRate;
    TD_S32 s32UsedAudioStreamIndex;
    OT_FORMAT_VIDEO_TYPE_E enVideoType;
    OT_FORMAT_AUDIO_TYPE_E enAudioType;
} OT_FORMAT_FILE_INFO_S;
```

【成员】

成员名称	描述
s64FileSize	媒体文件的总大小，单位：byte。
s64StartTime	媒体文件起始播放时间，单位：ms。
s64Duration	媒体文件总时长，单位：ms。
audioDuration	当前播放文件的音频流长度，单位：ms。
videoDuration	当前播放文件的视频流长度，单位：ms。
stStreamResolution	视频流的分辨率。
s32UsedVideoStreamIndex	播放使用的媒体视频流。
fFrameRate	帧率。
u32Bitrate	码率。
u32AudioChannelNum	音频流的声道数。
u32SampleRate	音频流的采样率。
s32UsedAudioStreamIndex	播放使用媒体音频流。
enVideoType	媒体文件视频类型。
enAudioType	媒体文件音频类型。

【注意事项】

无

【相关数据类型及接口】

无



7.4.23 OT_FORMAT_FUN_S

【说明】

定义解封装的函数。

【定义】

```
typedef struct FORMAT_FUN_S
{
    TD_S32 (*fmt_open)(TD_VOID* * ppFmt, const TD_CHAR* pszFileName);
    TD_S32 (*fmt_close)(const TD_VOID* pFmt);
    TD_S32 (*fmt_read)(const TD_VOID* pFmt, OT_FORMAT_PACKET_S* pstFmtFrame);
    TD_S32 (*fmt_free)(const TD_VOID* pFmt, const OT_FORMAT_PACKET_S* pstFmtFrame);
    TD_S32 (*fmt_seek)(const TD_VOID* pFmt, TD_S32 s32StreamIndex, TD_S64 s64MSec,
    TD_S32 s32Flag);
    TD_S32 (*fmt_probe)(const TD_VOID* pFmt, OT_DEMUXER_PROBE_MODE_TYPE_E
    demuxProbeMode);
    TD_S32 (*fmt_getinfo)(const TD_VOID* pFmt, OT_FORMAT_FILE_INFO_S* pstFmtInfo);
    TD_S32 (*fmt_setattr)(const TD_VOID* pFmt, OT_FORMAT_MEDIA_ATTR_S* pstFmtInfo);
}OT_DEMUX_FUN_S;
```

【成员】

成员名称	描述
fmt_open	打开媒体文件的函数指针。
fmt_close	关闭媒体文件的函数指针。
fmt_read	读取媒体包的函数指针。
fmt_free	释放媒体包的函数指针。
fmt_seek	移动媒体文件播放位置的函数指针。
fmt_probe	探测媒体文件信息的函数指针。
fmt_getinfo	获取媒体文件信息函数指针。
fmt_setattr	设置媒体文件属性函数指针。

【注意事项】

无。

【相关数据类型及接口】

无。

7.4.24 OT_DEMUX_S

【说明】

定义解封装库所包含信息的结构体。

【定义】

```
typedef struct DEMUX_S
{
    TD_CHAR aszDemuxerName[OT_DEMUXER_NAME_LEN];TD_CHAR
```




```
aszSupportFormat[OT_DEMUXER_SUPPORT_NAMES_LEN];
    OT_FORMAT_FUN_S stFmtFun;
    TD_U32 u32Priority;
}OT_DEMUX_S;
```

【成员】

成员名称	描述
aszDemuxerName	该封装库的名字。
aszSupportFormat	该封装库支持的文件格式，多种格式之间用逗号分隔。
stFmtFun	解封装库对应的解封装函数指针。
u32Priority	封装库的优先级，两个解封装库都支持都mp4文件的解析，优先选级别高的解封装库。值的范围是[0, 10]，值越小优先级越高。

【注意事项】

无。

【相关数据类型及接口】

无。

7.4.25 OT_FORMAT_MEDIA_ATTR_S

【说明】

定义指定媒体文件属性。

【定义】

```
typedef struct FORMAT_MEDIA_ATTR_S
{
    TD_S32 s32VidStreamId; /**< Video stream ID. */
    TD_S32 s32AudStreamId; /**< Audio stream ID. */
} OT_FORMAT_MEDIA_ATTR_S;
```

【成员】

成员名称	描述
s32VidStreamId	指定将要解析的视频流。
s32AudStreamId	指定将要解析的音频流。

【注意事项】

无。

【相关数据类型及接口】

无。



7.4.26 OT_DEMUXER_RESOLUTION_CNT

【说明】

定义媒体文件对应的最大视频流个数。

【定义】

```
#define OT_DEMUXER_RESOLUTION_CNT (5)
```

【注意事项】

无。

【相关数据类型及接口】

无。

7.4.27 OT_DEMUXER_NAME_LEN

【说明】

定义解封库名字的长度。

【定义】

```
#define OT_DEMUXER_NAME_LEN (20)
```

【注意事项】

无。

【相关数据类型及接口】

无。

7.4.28 OT_DEMUXER_SUPPORT_NAMES_LEN

【说明】

定义解封库名支持的文件类型总长度。

【定义】

```
#define OT_DEMUXER_SUPPORT_NAMES_LEN (56)
```

【注意事项】

无。

【相关数据类型及接口】

无。

7.4.29 OT_DEMUXER_PROBE_MODE_TYPE_E

【说明】

PROBE模式。

【定义】



```
typedef enum {  
    OT_DEMUXER_PROBE_MODE_NORMAL = 0x0000,  
    OT_DEMUXER_PROBE_MODE_GET_DATASTREAM = 0x0001}  
OT_DEMUXER_PROBE_MODE_TYPE_E;
```

【成员】

成员名称	描述
OT_DEMUXER_PROBE_MODE_NORMAL	普通模式
OT_DEMUXER_PROBE_MODE_GET_DATASTREAM	获取缩略图模式

【注意事项】

无。

【相关数据类型及接口】

无。

7.5 错误码

播放器和解封装错误码如[表7-1](#)和[表7-2](#)所示。

表 7-1 播放器错误码

错误代码	宏定义	描述
0	TD_SUCCESS	成功
-1	TD_FAILURE	失败
0xA1208001	OT_ERR_LITEPLAYER_ILLEGAL_PARAM	参数非法
0xA1208002	OT_ERR_LITEPLAYER_NULL_PTR	指针为空
0xA1208042	OT_ERR_LITEPLAYER_ILLEGAL_HANDLE	非法的handle值
0xA1208043	OT_ERR_LITEPLAYER_MEM_MALLOC	内存分配失败
0xA1208044	OT_ERR_LITEPLAYER_THREAD_CREATE_FAIL	线程创建失败
0xA1208045	OT_ERR_LITEPLAYER_MUTEX_INIT_FAIL	信号量初始化失败
0xA1208046	OT_ERR_LITEPLAYER_ADEC_IN_USE	当前音频解码器在使用中



错误代码	宏定义	描述
0xA1208047	OT_ERR_LITEPLAYER_CALLBACK_ERROR	回调函数执行失败
0xA1208048	OT_ERR_LITEPLAYER_ILLEGAL_STATE_ACTION	当前播放状态下不支持该操作
0xA1208049	OT_ERR_LITEPLAYER_DEMUX_ERROR	解封装失败
0xA1208050	OT_ERR_LITEPLAYER_DEMUX_HAS_REG	解封装库已经注册过了
0xA1208051	OT_ERR_LITEPLAYER_NOT_SUPPORT	当前播放不支持该功能
0xA1208052	OT_ERR_LITEPLAYER_REINITED	重复初始化
0xA1208053	OT_ERR_LITEPLAYER_NOT_INIT	未初始化
0xA1208054	OT_ERR_LITEPLAYER_AVPLAYER	Avplayer执行出错
0xA1208055	OT_ERR_LITEPLAYER_DLOPEN	Dlopen出错
0xA1208056	OT_ERR_LITEPLAYER_DLSYM	Dlsym出错
0xA1208057	OT_ERR_LITEPLAYER_NOT_REG_DEMUXER	未注册解封装库
0xA1208058	OT_ERR_LITEPLAYER_HAS_CREATE	已经创建了播放器实例
0xA1208059	OT_ERR_LITEPLAYER_OTHER	其它错误
0xA1208060	OT_ERR_LITEPLAYER_NDK	NDK执行出错
0xA1208061	OT_ERR_LITEPLAYER_DEMUX_IN_USE	当前demux库正被使用
0xA1208062	OT_ERR_LITEPLAYER_ADEC_NOT_REG	音频解码库未注册
0xA1208063	OT_ERR_LITEPLAYER_ADEC_HAS_REG	音频解码库已经注册过
0xA1208064	OT_ERR_LITEPLAYER_ADEC_ERROR	音频解码库使用错误
0xA1208065	OT_ERR_LITEPLAYER_REACH_MAX	实例个数到达上限



表 7-2 解封装错误码

错误代码	宏定义	描述
0	TD_SUCCESS	成功
-1	TD_FAILURE	失败
0xA1288001	OT_ERR_DEMUXER_NULL_PTR	空指针
0xA1288002	OT_ERR_DEMUXER_ILLEGAL_PARAM	非法入参
0xA1288040	OT_ERR_DEMUXER_OPEN_FILE	打开文件失败
0xA1288041	OT_ERR_DEMUXER_READ_PACKET	读取报文失败
0xA1288042	OT_ERR_DEMUXER_SEEK	Seek失败
0xA1288043	OT_ERR_DEMUXER_MEM_MALLOC	内存分配失败
0xA1288044	OT_ERR_DEMUXER_PROBE	探测媒体文件失败
0xA1288045	OT_ERR_DEMUXER_ACTION	不支持的操作
0xA1288046	OT_ERR_DEMUXER_INVALID_HANDLE	无效的handle
0xA1288047	OT_ERR_DEMUXER_FREE_PACKET	释放报文失败
0xA1288048	OT_ERR_DEMUXER_SET_ATTR	设置属性失败
0xA1288049	OT_ERR_DEMUXER_NOT_SUPPORT	不支持的功能

7.6 典型场景使用指南

7.6.1 播放器使用示例

播放一个媒体文件到结尾，用户接口调用说明如下：

步骤1 初始化设置播放器模块

```
/*  
播放器模块进行初始化并创建一个播放实例。  
*/
```



```
/******  
TD_S32 s32Ret = TD_SUCCESS;  
s32Ret = SS_LITEPLAYER_Init();  
CU_ASSERT_EQUAL_FATAL(s32Ret, TD_SUCCESS);  
  
s32Ret = SS_LITEPLAYER_RegisterAudioDecoder ("/sharefs/libss_aacdec.so");  
CU_ASSERT_EQUAL_FATAL(s32Ret, TD_SUCCESS);  
  
OT_LITEPLAYER_PARAM_S stPlayerParam = {0};  
s32Ret = SS_LITEPLAYER_Create(&pPlayer, &stPlayerParam);  
CU_ASSERT_EQUAL_FATAL(s32Ret, TD_SUCCESS);  
  
OT_LITEPLAYER_VOUT_OPT_S stVoutOpt;  
memset(&stVoutOpt, 0x00, sizeof(stVoutOpt));  
stVoutOpt.enVoutType = OT_LITEPLAYER_VOUT_TYPE_VPSS;  
stVoutOpt.hDispHdl = g_hVpssGrpHandle;  
stVoutOpt.hWndHdl = g_hVpssGrpChnHandle;  
  
ret = SS_LITEPLAYER_SetVOHandle(pPlayer, &stVoutOpt);  
CU_ASSERT_EQUAL_FATAL(s32Ret, TD_SUCCESS);  
OT_LITEPLAYER_AOUT_OPT_S stAoutOpt;  
memset(&stAoutOpt, 0x00, sizeof(stAoutOpt));  
stAoutOpt.hAudTrackHdl = g_hAoTrackHandle;  
  
ret = SS_LITEPLAYER_SetAOHandle(pPlayer, &stAoutOpt);CU_ASSERT_EQUAL_FATAL(s32Ret,  
TD_SUCCESS);
```

步骤2 定义并设置回调函数

```
/******  
//定义回调函数, 这里用全局变量g_enState/isFileEnd做线程间消息传递  
/******  
TD_VOID sample_evect_cb(TD_MW_PTR pPlayer, OT_LITEPLAYER_EVENT_E enEvent, void*  
pData)  
{  
    if (hPlayer == 0)  
    {  
        printf("the handle is error\n");  
        return TD_SUCCESS;  
    }  
  
    switch (enEvent)  
    {  
        case OT_LITEPLAYER_EVENT_STATE_CHANGED:  
            g_enState = *(OT_LITEPLAYER_STATE_E*)pData;  
            printf("@@get the new play state:%d\n", g_enState);  
            break;  
        case OT_LITEPLAYER_EVENT_EOF:  
            isFileEnd = 1;  
            printf("@@play file to the end\n");  
            break;  
        case OT_LITEPLAYER_EVENT_ERROR:  
            printf("@@play file error\n");  
            assert(0);  
            break;  
        case OT_LITEPLAYER_EVENT_PROGRESS:  
            printf("@@play file progress, data is %d\n", *(TD_U32*)pData);  
            break;  
        case OT_LITEPLAYER_SEEK_END:  
            printf("@@seek action end, time is %lld\n", *(TD_S64*)pData);  
            break;  
        default:
```




```

        break;
    }
    return TD_SUCCESS;
}

s32Ret = SS_LITEPLAYER_RegCallback(pPlayer, sample_evect_cb);
CU_ASSERT_EQUAL_FATAL(s32Ret, TD_SUCCESS);

```

步骤3 设置并准备播放数据源

```

/*****
设置播放器的数据源，等接受到OT_LITEPLAYER_STATE_PREPARED状态消息后，执行播放动作
AO_OPEN是打开ao设备动作的函数
AO资源打开之后结束时必须关闭，用全局变量g_isHasAudio表示是否打开了ao资源
*****/TD_S32 s32Ret = TD_SUCCESS;
s32Ret = SS_LITEPLAYER_SetDataSource(pPlayer, pszFileName);
CU_ASSERT_EQUAL_FATAL(s32Ret, TD_SUCCESS);

s32Ret = SS_LITEPLAYER_Prepare(pPlayer);
CU_ASSERT_EQUAL_FATAL(s32Ret, TD_SUCCESS);
OT_FORMAT_FILE_INFO_S stFormatFileInfo = {0};
s32Ret = SS_LITEPLAYER_GetFileInfo(pPlayer, &stFormatFileInfo);
CU_ASSERT_EQUAL_FATAL(s32Ret, TD_SUCCESS);

if (stFormatFileInfo.s32UsedAudioStreamIndex != TD_INVALID_HANDLE)
{
    g_isHasAudio = 1;
    AO_OPEN(stFormatFileInfo.u32SampleRate, stFormatFileInfo.u32AudioChannelNum);
}

```

步骤4 播放直到收到播放到文件尾的消息

```

/*****
isFileEnd是由回调函数收到播放到结尾消息后设置的变量值
*****/
TD_S32 s32Ret = TD_SUCCESS;
s32Ret = SS_LITEPLAYER_Play(pPlayer);
CU_ASSERT_EQUAL_FATAL(s32Ret, TD_SUCCESS);
OT_LITEPLAYER_STATE_E enState = OT_LITEPLAYER_STATE_BUTT;
s32Ret = SS_LITEPLAYER_GetPlayStatus(pPlayer, &enState);
CU_ASSERT_EQUAL_FATAL(s32Ret, TD_SUCCESS);
OT_LITEPLAYER_TPLAY_ATTR_S stTPlayAttr;
stTPlayAttr.enDirection = OT_LITEPLAYER_TPLAY_FORWARD;
stTPlayAttr.enSpeed = OT_LITEPLAYER_PLAY_SPEED_2X_FAST;
s32Ret = SS_LITEPLAYER_TPlay(pPlayer, &stTPlayAttr);
CU_ASSERT_EQUAL_FATAL(s32Ret, TD_SUCCESS);

s32Ret = SS_LITEPLAYER_Play(pPlayer);
CU_ASSERT_EQUAL_FATAL(s32Ret, TD_SUCCESS);

s32Ret = SS_LITEPLAYER_Pause(pPlayer);
CU_ASSERT_EQUAL_FATAL(s32Ret, TD_SUCCESS);

s32Ret = SS_LITEPLAYER_Play(pPlayer);
CU_ASSERT_EQUAL_FATAL(s32Ret, TD_SUCCESS);

pthread_mutex_lock(&playerMutex);
if (sample_thread_flag == 0 && isFileEnd == 0)
{
    printf("wait for liteplayer play end\n");
    pthread_cond_wait(&playerCond, &playerMutex);
}
pthread_mutex_unlock(&playerMutex);

```


**步骤5 停止播放器并销毁播放器**

```
TD_S32 s32Ret = TD_SUCCESS;
s32Ret = SS_LITEPLAYER_Stop(pPlayer);
CU_ASSERT_EQUAL_FATAL(s32Ret, TD_SUCCESS);

s32Ret = SS_LITEPLAYER_Destroy(pPlayer);
CU_ASSERT_EQUAL_FATAL(s32Ret, TD_SUCCESS);

s32Ret = SS_LITEPLAYER_UnRegisterDemuxer("./so/libffmpeg_demuxer.so");
CU_ASSERT_EQUAL_FATAL(s32Ret, TD_SUCCESS);

s32Ret = SS_LITEPLAYER_UnRegisterAudioDecoder("/sharefs/libss_aacdec.so");
CU_ASSERT_EQUAL_FATAL(s32Ret, TD_SUCCESS);
s32Ret = SS_LITEPLAYER_DeInit();
CU_ASSERT_EQUAL_FATAL(s32Ret, TD_SUCCESS);
if (g_isHasAudio != 0)
{
    AO_CLOSE();
    g_isHasAudio = 0;
}
```

----结束

7.6.2 播放器使用注意事项

由于aacdec的License问题，需要动态加载libss_aacdec.so库使用。



8 定时器模块

8.1 概述

定时器模块作为基础公共模块，提供定时任务功能。

8.2 功能描述

定时器模块主要完成的任务类型包括：

- 单次定时任务，创建定时任务之后开始计时，定时时间到达后执行一次定时动作，然后自动销毁定时器。
- 周期性定时任务，创建定时任务之后开始计时，定时动作以指定间隔周期性执行，直至定时任务销毁。

8.3 API 参考

定时器模块主要提供定时功能供外部模块使用，该功能模块提供以下API：

- **SS_Timer_Init**：初始化定时任务组。
- **SS_Timer_DeInit**：去初始化定时任务组。
- **SS_Timer_Create**：创建定时任务。
- **SS_Timer_Destroy**：销毁定时任务。
- **SS_Timer_CleanUp**：清除定时任务组中的所有定时任务。
- **SS_Timer_Reset**：重置定时任务。
- **SS_Timer_SetTickValue**：设置定时任务组中所有任务的计时精度。
- **SS_Timer_SetPeriodicAttr**：设置定时任务属性。
- **SS_Timer_GetPastTime**：获取定时任务当前计时。

8.3.1 SS_Timer_Init

【描述】

初始化定时任务组。



【语法】

```
TD_S32 SS_Timer_Init(TD_BOOL bBlock);
```

【参数】

参数名称	描述	输入/输出
bBlock	定时任务组类型标识符，阻塞或非阻塞。	输入

【返回值】

返回值	描述
非0	定时任务组句柄。
0	失败。

【需求】

- 头文件：ss_timer_ext.h
- 库文件：libottimer.a、libottimer.so

【芯片差异】

无。

【注意】

最多支持5个定时任务组。

【举例】

无。

【相关主题】

无。

8.3.2 SS_Timer_Delnit

【描述】

去初始化定时任务组。

【语法】

```
TD_S32 SS_Timer_Delnit(TD_S32 grpHdl);
```

【参数】

参数名称	描述	输入/输出
grpHdl	定时任务组句柄。	输入



【返回值】

返回值	描述
0	成功。
非0	失败，返回错误码。

【需求】

- 头文件：ss_timer_ext.h
- 库文件：libottimer.a、libottimer.so

【注意】

需要在销毁定时任务组中的所有任务之后调用。

【举例】

无。

【相关主题】

无。

8.3.3 SS_Timer_Create

【描述】

创建定时任务。

【语法】

```
TD_MW_PTR SS_Timer_Create(TD_S32 grpHdl, const OT_TIMER_S *timerConf);
```

【参数】

参数名称	描述	输入/输出
grpHdl	定时任务组句柄。	输入
timerConf	定时任务属性。	输入

【返回值】

返回值	描述
非0	创建成功，定时任务句柄。
0	创建失败。

【需求】



- 头文件：ss_timer_ext.h
- 库文件：libottimer.a、libottimer.so

【注意】

在初始化定时任务组之后调用。

【举例】

无。

【相关主题】

无。

8.3.4 SS_Timer_Destroy

【描述】

销毁定时任务。

【语法】

```
TD_S32 SS_Timer_Destroy( TD_S32 grpHdl, TD_MW_PTR tmrHdle);
```

【参数】

参数名称	描述	输入/输出
grpHdl	定时任务组句柄。	输入
tmrHdle	定时任务句柄。	输入

【返回值】

返回值	描述
0	成功。
非0	失败，返回 错误码 。

【需求】

- 头文件：ss_timer_ext.h
- 库文件：libottimer.a、libottimer.so

【注意】

创建定时任务之后调用。

【举例】

无。

【相关主题】



无。

8.3.5 SS_Timer_CleanUp

【描述】

清空定时任务组中的所有定时任务。

【语法】

```
TD_S32 SS_Timer_CleanUp(TD_S32 grpHdl);
```

【参数】

参数名称	描述	输入/输出
grpHdl	定时任务组句柄。	输入

【返回值】

返回值	描述
0	成功。
非0	失败，返回 错误码 。

【需求】

- 头文件：ss_timer_ext.h
- 库文件：libottimer.a、libottimer.so

【注意】

无。

【举例】

无。

【相关主题】

无。

8.3.6 SS_Timer_Reset

【描述】

重置定时任务。

【语法】

```
TD_S32 SS_Timer_Reset (TD_S32 grpHdl, TD_MW_PTR tmrHdle, const struct timeval* timeVal, TD_U32 timeLen);
```

【参数】



参数名称	描述	输入/输出
grpHdl	定时任务组句柄。	输入
tmrHdle	定时任务句柄。	输入
timeVal	定时起始时间。	输入
timeLen	定时时长，单位毫秒。	输入

【返回值】

返回值	描述
0	成功。
非0	失败，返回错误码。

【需求】

- 头文件：ss_timer_ext.h
- 库文件：libottimer.a、libottimer.so

【注意】

重置任务后，定时任务按照新的定时时间执行动作。

【举例】

无。

【相关主题】

无。

8.3.7 SS_Timer_SetTickValue

【描述】

设置定时任务组计时精度。

【语法】

```
TD_S32 SS_Timer_SetTickValue(TD_S32 grpHdl, TD_U32 u32TickVal_us);
```

【参数】

参数名称	描述	输入/输出
grpHdl	定时任务组句柄。	输入
u32TickVal_us	计时精度，单位微秒。	输入



【返回值】

返回值	描述
0	成功。
非0	失败，返回 错误码 。

【需求】

- 头文件：ss_timer_ext.h
- 库文件：libottimer.a、libottimer.so

【注意】

默认计时精度是100毫秒。

【举例】

无。

【相关主题】

无。

8.3.8 SS_Timer_SetPeriodicAttr

【描述】

设置定时任务属性。

【语法】

```
TD_S32 SS_Timer_SetPeriodicAttr(TD_MW_PTR tmrHdle, TD_BOOL periodic);
```

【参数】

参数名称	描述	输入/输出
tmrHdle	定时任务句柄。	输入
periodic	定时任务周期性标识。	输入

【返回值】

返回值	描述
0	成功。
非0	失败，返回 错误码 。

【需求】



- 头文件：ss_timer_ext.h
- 库文件：libottimer.a、libottimer.so

【注意】

当periodic 为TD_TRUE 时，定时任务为周期性定时任务；当periodic 为TD_FALSE 时，定时任务为单次定时任务。

【举例】

无。

【相关主题】

无。

8.3.9 SS_Timer_GetPastTime

【描述】

获取定时任务计时时间。

【语法】

```
TD_S32 TD_Timer_GetPastTime(TD_MW_PTR tmrHdle, TD_U32* pu32Time);
```

【参数】

参数名称	描述	输入/输出
tmrHdle	定时任务句柄。	输入
pu32Time	定时任务已计时时间，单位毫秒(ms)。	输出

【返回值】

返回值	描述
0	成功。
非0	失败，返回 错误码 。

【需求】

- 头文件：ss_timer_ext.h
- 库文件：libottimer.a、libottimer.so

【注意】

无。

【举例】

无。

【相关主题】



无。

8.4 数据类型

定时器模块主要提供以下数据类型：

- **TD_TIMER_HANDLE**：定义定时任务句柄。
- **OT_TIMER_S**：定义定时任务属性。
- **SS_TIMER_PROC**：定义定时任务回调函数。

8.4.1 TD_TIMER_HANDLE

【说明】

定义定时任务句柄。

【定义】

```
typedef void* TD_TIMER_HANDLE;
```

【注意事项】

无。

【相关数据类型及接口】

无。

8.4.2 OT_TIMER_S

【说明】

定义定时任务属性。

【定义】

```
typedef struct OtTimerConf
{
    struct timeval* now;
    long interval_ms;
    TD_BOOL periodic;
    SS_TIMER_PROC* timer_proc;
    TD_VOID* clientData;
} OT_TIMER_S;
```

【成员】

成员名称	描述
now	定时任务计时起始时间（以系统启动时间为基准）。
interval_ms	定时时长，单位为ms。
periodic	定时任务周期性标识。
timer_proc	定时任务计时到达事件回调。
clientData	定时任务私有数据。



【注意事项】

- now 设置为TD_NULL时，计时任务从创建之后开始计时。
- now 设置是以系统启动时间为基准，通常使用 clock_gettime(CLOCK_MONOTONIC, &time)，然后将struct timespec time 换算成struct timeval now。
- 定时时长最大值为15天（15 * 24 * 60 * 60 * 1000毫秒）。
- 当periodic为TD_TRUE时，定时任务是周期定时任务；当periodic 为TD_FALSE 时，定时任务为单次定时任务。

【相关数据类型及接口】

无。

8.4.3 SS_TIMER_PROC

【说明】

定时任务计时到达事件回调。

【定义】

```
typedef void SS_TIMER_PROC(TD_VOID* client_data, struct timeval* nowP );
```

【成员】

成员名称	描述
client_data	定时任务私有数据。
nowP	计时到达时间。

【注意事项】

无。

【相关数据类型及接口】

无。

8.5 错误码

定时器API错误码如表8-1所示。

表 8-1 定时器模块通用错误码

错误代码	宏定义	描述
0	TD_SUCCESS	成功
-1	TD_FAILURE	失败



错误代码	宏定义	描述
0xA12C8002	OT_ERR_TIMER_NULL_PTR	空指针异常
0xA12C80d4	OT_ERR_TIMER_CANCEL	取消定时器异常
0xA12C80d5	OT_ERR_TIMER_CLEANUP	清除定时器异常
0xA12C80d6	OT_ERR_TIMER_RESET	重置定时器异常



9 层次状态机

9.1 概述

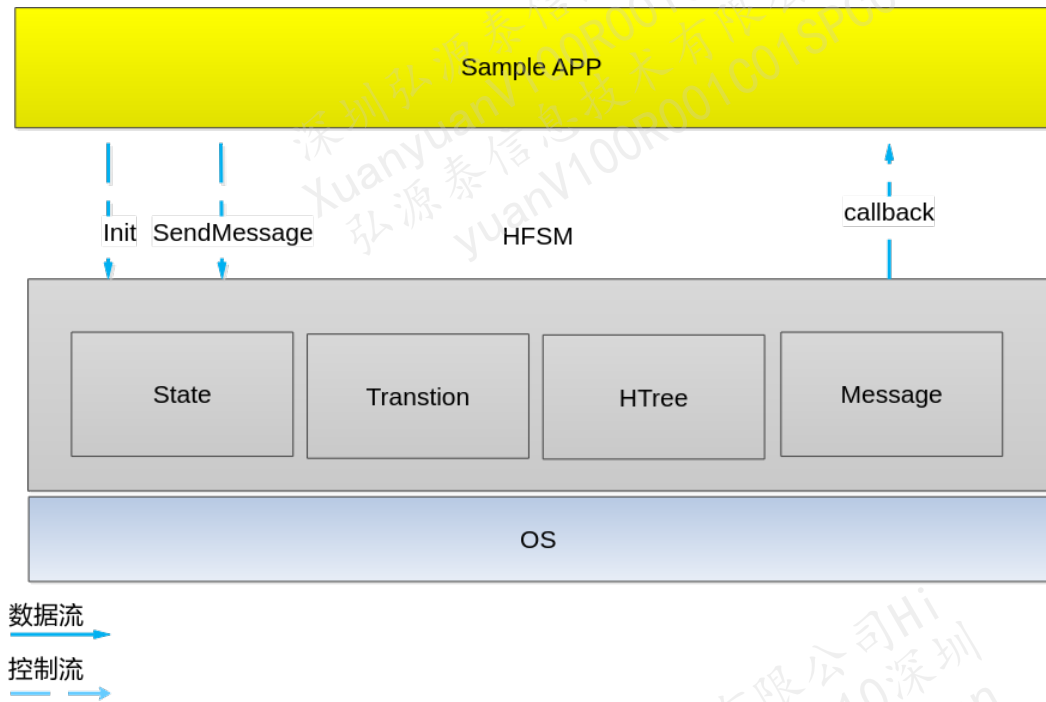
业务软件设计开发过程中会依赖一些基础组件，状态机就是常见的基础组件。复杂业务中状态机的使用可以帮助我们更好梳理业务，降低业务的复杂度。而业务是多变的，仅实现一种状态机是不够的，可扩展的易用的状态机框架应运而生。

本文主要介绍了层次状态机框架模块以及使用方法，用于指导用户开发。

9.2 架构原理

状态机模块，提供了一种层次状态机框架。用户可以自己定义状态以及状态的转移。层次状态机是基于消息驱动的。具体系统框图组成及对外接口关系如图9-1所示。

图 9-1 状态机模块系统框图



概括来说，状态机模块主要由以下部分构成：

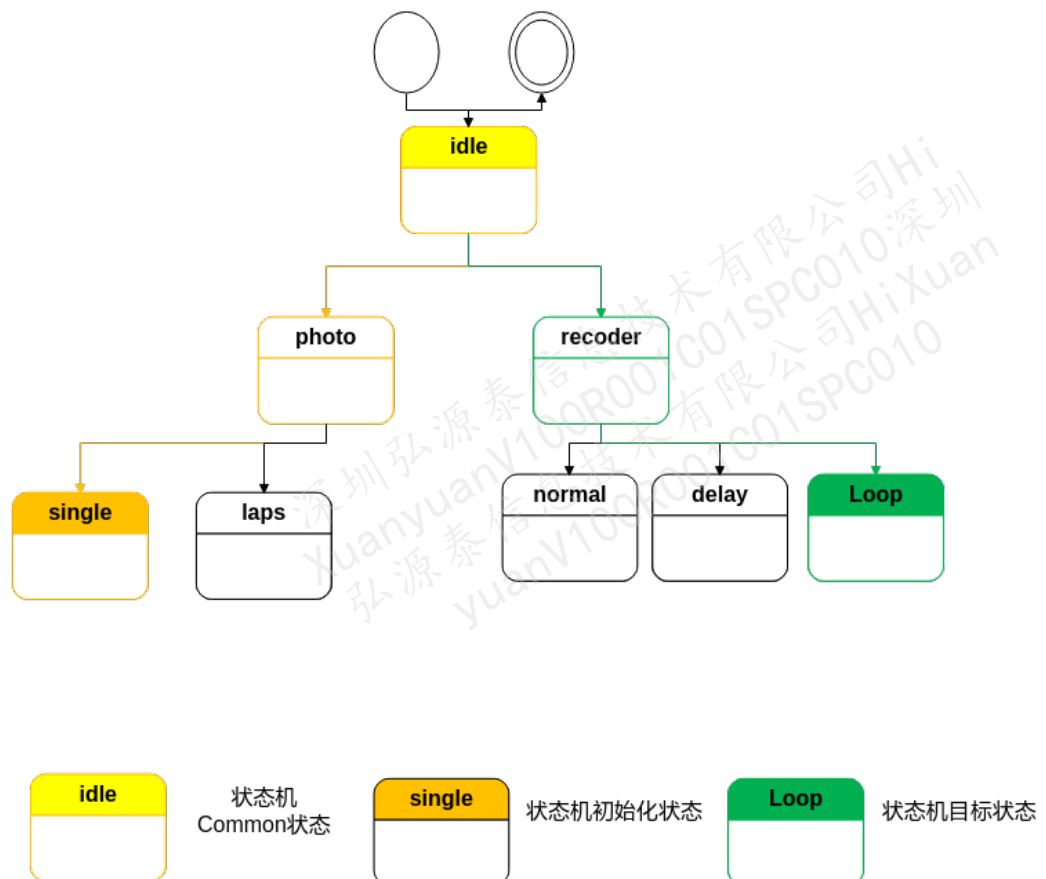
- 状态定义
用户自定义状态：状态名称，状态进入回调，状态退出回调，消息处理回调。
- 消息
消息自定义：消息，参数1，参数2，处理结果，创建时间，负载。
- 状态转移
支持状态从某一状态转移到任一其他状态，具体转移策略由用户自定义。
- 层次树。

状态机内部维护一个状态树，只有一个父节点，由用户通过添加接口构建该树。

9.2.1 状态机模型

状态机的状态是以树形状态维护的，只有一个根节点，每个节点有一个或者多个叶子节点，如图9-2所示。

图 9-2 状态机内部状态组织形式框图



9.2.2 状态切换规则

状态的切换主要有以下规则：



- 由某一状态进入其子状态时，会调用子状态的进入函数。
- 由某一状态回退到其父状态时，会调用子状态的退出函数。

举例：在图9-2中，从single状态切换到loop状态，会依次调用一下回调：

single退出->photo退出->recorder进入->Loop进入。

9.3 API 参考

该功能模块为用户提供以下API：

- **SS_HFSM_Create**：创建HFSM实例。
- **SS_HFSM_Destroy**：销毁HFSM实例。
- **SS_HFSM_AddState**：添加状态。
- **SS_HFSM_SetInitialState**：设置初始化状态。
- **SS_HFSM_GetCurrentState**：获取当前状态。
- **SS_HFSM_Start**：启动状态机。
- **SS_HFSM_Stop**：停止状态机。
- **SS_HFSM_SendAsyncMessage**：发送消息。

9.3.1 SS_HFSM_Create

【描述】

创建HFSM实例。

【定义】

```
TD_S32 SS_HFSM_Create(const OT_HFSM_ATTR_S *fsmAttr, TD_MW_PTR *hfsm);
```

【参数】

参数名称	描述	输入/输出
fsmAttr	层次状态参数指针。	输入
hfsm	状态机句柄指针。	输出

【返回值】

返回值	描述
0	成功。
非0	失败，参见 错误码 。

【需求】

- 头文件：ss_hfsm.h



- 库文件：libh fsm.a/ libh fsm.so

【注意】

无。

【举例】

无。

9.3.2 SS_HFSM_Destroy

【描述】

销毁HFSM实例。

【定义】

TD_S32 SS_HFSM_Destroy(TD_MW_PTR h fsm);

【参数】

参数名称	描述	输入/输出
h fsm	状态机实例句柄。	输入

【返回值】

返回值	描述
0	成功。
非0	失败，参见 错误码 。

【需求】

- 头文件：ss_h fsm.h
- 库文件：libh fsm.a/libh fsm.so

【注意】

HFSM创建后才能调用该接口。

【举例】

无。

9.3.3 SS_HFSM_AddState

【描述】

添加状态。

【定义】

TD_S32 SS_HFSM_AddState(TD_MW_PTR h fsm, OT_STATE_S *state, OT_STATE_S *parent);



【参数】

参数名称	描述	输入/输出
h fsm	状态机实例句柄。	输入
state	状态指针。	输入
parent	父状态指针。	输入

【返回值】

返回值	描述
0	成功。
非0	失败，参见 错误码 。

【需求】

- 头文件：ss_h fsm.h
- 库文件：libh fsm.a/ libh fsm.so

【注意】

根状态的父状态指针为空。

【举例】

无。

9.3.4 SS_HFSM_SetInitialState

【描述】

设置初始化状态。

【定义】

```
TD_S32 SS_HFSM_SetInitialState(TD_MW_PTR h fsm, const TD_CHAR *stateName);
```

【参数】

参数名称	描述	输入/输出
h fsm	状态机实例句柄。	输入
stateName	初始化状态的名称。	输入

【返回值】



返回值	描述
0	成功。
非0	失败，参见 错误码 。

【需求】

- 头文件：ss_hfsm.h
- 库文件：libhfsm.a/ libhfsm.so

【注意】

无。

【举例】

无。

9.3.5 SS_HFSM_GetCurrentState

【描述】

获取当前状态。

【定义】

```
TD_S32 SS_HFSM_GetCurrentState(TD_MW_PTR hfsm, OT\_STATE\_S *state);
```

【参数】

参数名称	描述	输入/输出
hfsm	状态机实例句柄。	输入
state	当前状态指针。	输出

【返回值】

返回值	描述
0	成功。
非0	失败，参见 错误码 。

【需求】

- 头文件：ss_hfsm.h
- 库文件：libhfsm.a/ libhfsm.so

【注意】



无。

【 举例 】

无。

9.3.6 SS_HFSM_Start

【 描述 】

启动状态机。

【 定义 】

```
TD_S32 SS_HFSM_Start(TD_MW_PTR hfsm);
```

【 参数 】

参数名称	描述	输入/输出
hfsm	状态机实例句柄。	输入

【 返回值 】

返回值	描述
0	成功。
非0	失败，参见 错误码 。

【 需求 】

- 头文件：ss_hfsm.h
- 库文件：libhfsm.a/ libhfsm.so

【 注意 】

无。

【 举例 】

无。

9.3.7 SS_HFSM_Stop

【 描述 】

停止状态机。

【 定义 】

```
TD_S32 SS_HFSM_Stop(TD_MW_PTR hfsm);
```

【 参数 】



参数名称	描述	输入/输出
hfsm	状态机实例句柄。	输入

【返回值】

返回值	描述
0	成功。
非0	失败，参见 错误码 。

【需求】

- 头文件：ss_hfsm.h
- 库文件：libhfsm.a/ libhfsm.so

【注意】

无。

【举例】

无。

9.3.8 SS_HFSM_SendAsyncMessage

【描述】

发送异步消息。

【定义】

TD_S32 SS_HFSM_SendAsyncMessage(TD_MW_PTR hfsm, const [OT_MESSAGE_S](#) *msg);

【参数】

参数名称	描述	输入/输出
hfsm	状态机实例句柄。	输入
msg	消息结构体指针。	输入

【返回值】

返回值	描述
0	成功。
非0	失败，参见 错误码 。

**【需求】**

- 头文件：ss_hfsm.h
- 库文件：libhfsm.a/ libhfsm.so

【注意】

无。

【举例】

无。

9.4 数据类型

状态机模块相关数据类型定义如下：

- **OT_STATE_NAME_LEN**：状态机名称最大长度。
- **OT_STATE_MAX_AMOUNT**：定义支持状态的最大个数。
- **OT_PROCESS_MSG_RESULTE_OK**：定义消息处理OK返回值。
- **OT_PROCESS_MSG_UNHANDLER**：消息未处理返回值。
- **OT_STATE_S**：定义状态结构体。
- **OT_HFSM_EVENT_E**：定义状态机事件回调枚举。
- **OT_HFSM_UNHANDLED_MSG_INFO_S**：定义未处理消息信息结构体。
- **OT_HFSM_TRANSITION_INFO_S**：定义状态转移信息结构体。
- **OT_HFSM_EVENT_INFO_S**：定义事件信息结构体。
- **OT_HFSM_EVENT_CALLBACK_FN**：定义状态机事件回调函数。
- **OT_HFSM_ATTR_S**：定义状态机参数结构体。
- **OT_MESSAGE_S**：定义消息结构体。

9.4.1 OT_STATE_NAME_LEN

【说明】

状态机名称的最大长度。

【定义】

```
#define OT_STATE_NAME_LEN 64
```

【注意事项】

无。

【相关数据结构及接口】

无。

9.4.2 OT_STATE_MAX_AMOUNT

【说明】

支持状态的最大个数。



【定义】

```
#define OT_STATE_MAX_AMOUNT      32
```

【注意事项】

无。

【相关数据类型及接口】

无。

9.4.3 OT_PROCESS_MSG_RESULTE_OK

【说明】

消息处理OK返回值。

【定义】

```
#define OT_PROCESS_MSG_RESULTE_OK  TD_SUCCESS
```

【注意事项】

状态的消息处理函数中，对某个消息处理完成后返回OT_PROCESS_MSG_RESULTE_OK。

【相关数据类型及接口】

无。

9.4.4 OT_PROCESS_MSG_UNHANDLER

【说明】

未处理消息返回值。

【定义】

```
#define OT_PROCESS_MSG_UNHANDLER  (-1)
```

【注意事项】

状态的消息处理函数中，没有对某个消息进行处理时返回OT_PROCESS_MSG_UNHANDLER。

【相关数据类型及接口】

无。

9.4.5 OT_STATE_S

【说明】

定义状态结构体。

【定义】

```
typedef struct StateInfo {
    OT_STATE_ID stateID;           /* state id */
    TD_CHAR name[OT_STATE_NAME_LEN]; /* state name */
    TD_S32 (*enter)(TD_MW_PTR argv); /* call when state enter */
    TD_S32 (*exit)(TD_MW_PTR argv); /* call when state exit */
}
```



```
/* call when process a message */
TD_S32 (*processMessage)(const OT_MESSAGE_S *msg, const TD_MW_PTR argv,
OT_STATE_ID *stateID);
TD_MW_PTR argv; /* User private, used by processMessage */
}OT_STATE_S;
```

【成员】

成员名称	描述
stateID	状态标识。
name	状态名称。
enter	状态进入回调函数。
exit	状态退出回调函数。
processMessage	消息处理回调函数。
argv	用户自定义指针参数。

【注意事项】

无。

【相关数据类型及接口】

无。

9.4.6 OT_HFSM_EVENT_E

【说明】

定义状态机事件回调枚举。

【定义】

```
typedef enum HfsmEventE
{
    OT_HFSM_EVENT_UNHANDLE_MSG = 0, /* unhandler message */
    OT_HFSM_EVENT_TRANSTION_ERROR, /* transtion error */
    OT_HFSM_EVENT_BUTT
} OT_HFSM_EVENT_E;
```

【成员】

成员名称	描述
OT_HFSM_EVENT_UNHANDLE_MSG	未处理的消息事件类型。
OT_HFSM_EVENT_TRANSTION_ERRO R	状态转移错误事件类型。
OT_HFSM_EVENT_BUTT	无效事件类型。

【注意事项】



无。

【相关数据类型及接口】

无。

9.4.7 OT_HFSM_UNHANDLED_MSG_INFO_S

【说明】

定义未处理消息信息结构体类型。

【定义】

```
typedef struct HfsmMsgEventInfo
{
    OT_MESSAGE_S *pstunHandlerMsg;
} OT_HFSM_UNHANDLED_MSG_INFO_S;
```

【成员】

成员名称	描述
pstunHandlerMsg	未处理的消息结构体指针。

【注意事项】

无。

【相关数据类型及接口】

无。

9.4.8 OT_HFSM_TRANSTION_INFO_S

【说明】

定义事件信息结构体。

【定义】

```
typedef struct HfsmTransitionInfo
{
    OT_STATE_S *pstOrgState;
    OT_STATE_S *pstCurrentState;
    OT_STATE_S *pstDestState;
    TD_S32 s32ErrorNo;
} OT_HFSM_TRANSTION_INFO_S;
```

【成员】

成员名称	描述
pstOrgState	原始状态结构体指针。
pstCurrentState	当前状态结构体指针。
pstDestState	目标状态结构体指针。



成员名称	描述
s32ErrorNo	错误码。

【注意事项】

无。

【相关数据类型及接口】

无。

9.4.9 OT_HFSM_EVENT_INFO_S

【说明】

定义事件信息结构体。

【定义】

```
typedef struct HfsmEventInfo
{
    OT_HFSM_EVENT_E enEventCode;
    union
    {
        OT_HFSM_UNHANDLED_MSG_INFO_S stunHandlerMsgInfo; /* unhandled message
event info */
        OT_HFSM_TRANSTION_INFO_S stTranstionInfo; /* file event info */
    } unEventInfo;
} OT_HFSM_EVENT_INFO_S;
```

【成员】

成员名称	描述
enEventCode	事件类型。
stunHandlerMsgInfo	未处理消息信息。
stTranstionInfo	状态转移信息。

【注意事项】

无。

【相关数据类型及接口】

无。

9.4.10 OT_HFSM_EVENT_CALLBACK_FN

【说明】

定义状态机事件回调函数。

【定义】



```
typedef TD_S32 (*OT_HFSM_EVENT_CALLBACK_FN)(TD_MW_PTR hfsmHandle, const
OT_HFSM_EVENT_INFO_S* eventInfo);
```

【成员】

成员名称	描述
hfsmHandle	状态机句柄。
eventInfo	事件信息结构体指针。

【注意事项】

无。

【相关数据类型及接口】

无。

9.4.11 OT_HFSM_ATTR_S

【说明】

定义状态机参数结构体。

【定义】

```
typedef struct HfsmAttr
{
    OT_HFSM_EVENT_CALLBACK_FN fnHfsmEventCallback;
    TD_U32 u32StateMaxAmount;
    TD_U32 u32MessageQueueSize;
}OT_HFSM_ATTR_S;
```

【成员】

成员名称	描述
fnHfsmEventCallback	事件回调函数指针。
u32StateMaxAmount	支持的状态数量最大数量。
u32MessageQueueSize	消息队列长度。

【注意事项】

u32MessageQueueSize约束了状态机内部一个时刻存在的未处理消息的最大数量，如果当前未处理消息数量达到该值，状态机将无法接收新的消息。用户根据实际业务场景配置该值。

【相关数据类型及接口】

无。



9.4.12 OT_MESSAGE_S

【说明】

定义消息结构体。

【定义】

```
typedef struct ot_MESSAGE_S{
    TD_S32 what;
    TD_S32 arg1;
    TD_S32 arg2;
    TD_S32 s32Result;
    TD_U64 u64CreateTime;
    TD_CHAR aszPayload[MESSAGE_PAYLOAD_LEN];
}OT_MESSAGE_S;
```

【成员】

成员名称	描述
TD_S32 what	消息内容。
TD_S32 arg1	参数一。
TD_S32 arg2	参数二。
TD_S32 s32Result	执行结果
TD_U64 u64CreateTime	创建时间。
aszPayload[MESSAGE_PAYLOAD_LEN]	负载内容。 MESSAGE_PAYLOAD_LEN负载内容长度值的定义详见common/msghandler/include/ot_message.h

【注意事项】

该结构体定义在common/msghandler/include/ot_message.h头文件中。

【相关数据类型及接口】

无。

9.5 错误码

层次状态机模块API错误码如表9-1所示。

表 9-1 层次状态机 API 错误码

错误代码	宏定义	描述
0xA1318002	OT_ERR_HFSM_NULL_PTR	空指针错误
0xA1318040	OT_ERR_HFSM_HANDLE_INVALID	非法句柄错误



错误代码	宏定义	描述
0xA1318041	OT_ERR_HFSM_INVALIDARG	非法参数
0xA1318042	OT_ERR_HFSM_MALLOC	内存申请错误
0xA1318043	OT_ERR_HFSM_CREATE	创建错误
0xA1318044	OT_ERR_HFSM_DESTROY	销毁错误
0xA1318045	OT_ERR_HFSM_NOT_CREATE	对象没有创建
0xA1318046	OT_ERR_HFSM_STATE_NO_EXIST	状态不存在
0xA1318047	OT_ERR_HFSM_ROOT_STATE_EXIST	根状态已存在
0xA1318048	OT_ERR_HFSM_STATE_EXIST	状态已经存在
0xA1318049	OT_ERR_HFSM_STATE_ERROR	状态错误
0xA1318050	OT_ERR_HFSM_STATE_EXCEED_MAX_CNT	状态添加超过最大数
0xA1318051	OT_ERR_HFSM_NOT_START	状态机未启动
0xA1318052	OT_ERR_HFSM_MSGHDL_SEND	状态机消息发送错误
0xA1318053	OT_ERR_HFSM_MSGHDL_START	状态机消息处理启动错误
0xA1318054	OT_ERR_HFSM_MSGHDL_STOP	状态机消息处理停止错误



10 事件路由

10.1 概述

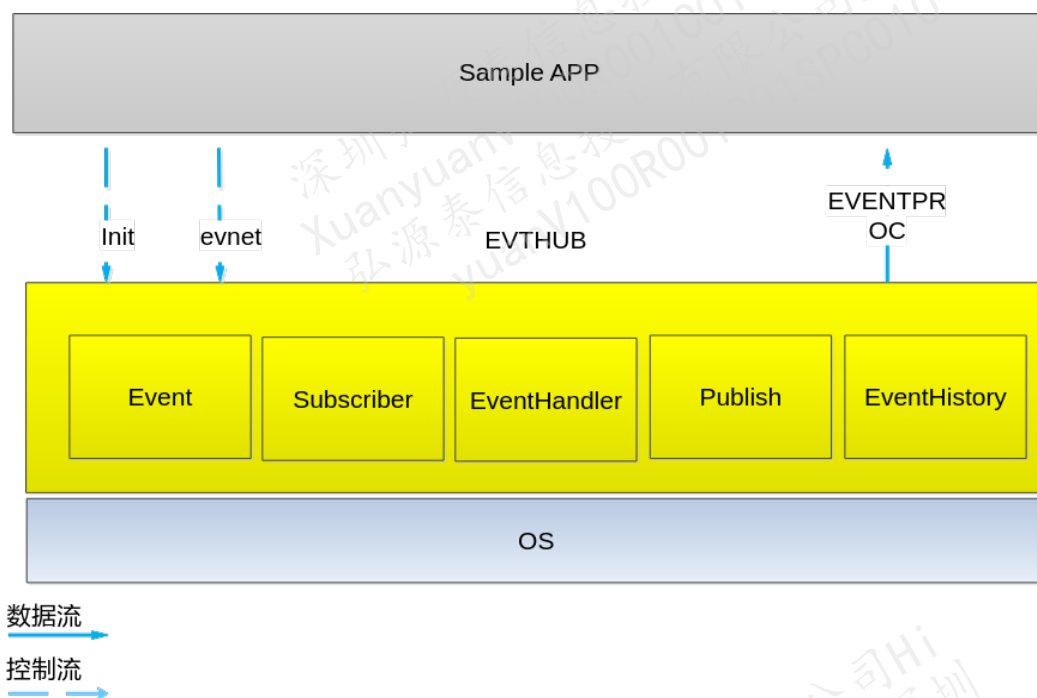
业务软件设计开发过程中会依赖一些基础组件，事件路由就是常见的基础组件。本模块结合业务场景，定制了一个基本的事件路由模块。

本文主要介绍了事件路由模块及其使用方法，用于指导用户开发。

10.2 架构原理

事件路由模块，提供了一种事件分发的框架。用户可以自己定义事件以及事件的处理策略。具体系统框图组成及对外接口关系如图10-1所示。

图 10-1 事件路由模块系统框图





概括来说，事件路由模块主要由以下部分构成：

- 订阅者
用户自定义订阅者：订阅者名称，事件处理函数，用户扩展参数，事件是否同步处理。
- 事件定义
用户自定义事件：事件ID，参数1，参数2，处理结果，创建时间，负载。
- 事件发布
支持事件统一发布。
- 事件处理器
订阅者支持异步处理事件，内部启动线程独立处理。
- 历史事件
支持上一次事件的查询。

10.3 API 参考

该功能模块为用户提供以下API：

- **SS_EVTHUB_Init**：初始化事件路由模块。
- **SS_EVTHUB_Deinit**：去初始化事件路由模块。
- **SS_EVTHUB_Register**：注册事件。
- **SS_EVTHUB_UnRegister**：解注册事件。
- **SS_EVTHUB_Publish**：发布事件。
- **SS_EVTHUB_CreateSubscriber**：创建订阅者。
- **SS_EVTHUB_DestroySubscriber**：销毁订阅者。
- **SS_EVTHUB_Subscribe**：订阅事件。
- **SS_EVTHUB_UnSubscribe**：解订阅事件。
- **SS_EVTHUB_GetEventHistory**：获取历史事件。
- **SS_EVTHUB_SetEnabled**：设置使能标记。
- **SS_EVTHUB_GetEnabled**：获取使能标记。

10.3.1 SS_EVTHUB_Init

【描述】

初始化事件路由模块。

【定义】

```
TD_S32 SS_EVTHUB_Init(TD_VOID);
```

【参数】

无。

【返回值】



返回值	描述
0	成功。
非0	失败，参见 错误码 。

【需求】

- 头文件：ss_eventhub.h
- 库文件：libeventhub.a、libeventhub.so

【注意】

该模块是单实例，仅需一次初始化。

【举例】

无。

10.3.2 SS_EVTHUB_Deinit

【描述】

去初始化事件路由模块。

【定义】

```
TD_S32 SS_EVTHUB_Deinit(TD_VOID);
```

【参数】

无。

【返回值】

返回值	描述
0	成功。
非0	失败，参见 错误码 。

【需求】

- 头文件：ss_eventhub.h
- 库文件：libeventhub.a、libeventhub.so

【注意】

无。

【举例】

无。



10.3.3 SS_EVTHUB_Register

【描述】

注册事件。

【定义】

```
TD_S32 SS_EVTHUB_Register(OT_EVENT_ID EventID);
```

【参数】

参数名称	描述	输入/输出
EventID	事件ID。	输入

【返回值】

返回值	描述
0	成功。
非0	失败，参见 错误码 。

【需求】

- 头文件：ss_eventhub.h
- 库文件：libeventhub.a、libeventhub.so

【注意】

无。

【举例】

无。

10.3.4 SS_EVTHUB_UnRegister

【描述】

解注册事件。

【定义】

```
TD_S32 SS_EVTHUB_UnRegister(OT_EVENT_ID EventID);
```

【参数】

参数名称	描述	输入/输出
EventID	事件ID。	输入



【返回值】

返回值	描述
0	成功。
非0	失败，参见 错误码 。

【需求】

- 头文件：ss_eventhub.h
- 库文件：libeventhub.a、libeventhub.so

【注意】

无。

【举例】

无。

10.3.5 SS_EVTHUB_Publish

【描述】

发布事件。

【定义】

```
TD_S32 SS_EVTHUB_Publish(const OT\_EVENT\_S *pEvent);
```

【参数】

参数名称	描述	输入/输出
pEvent	事件指针。	输入

【返回值】

返回值	描述
0	成功。
非0	失败，参见 错误码 。

【需求】

- 头文件：ss_eventhub.h
- 库文件：libeventhub.a、libeventhub.so

【注意】



无。

【举例】

无。

10.3.6 SS_EVTHUB_CreateSubscriber

【描述】

创建订阅者。

【定义】

```
TD_S32 SS_EVTHUB_CreateSubscriber(const OT_SUBSCRIBER_S *pstSubscriber,TD_MW_PTR *ppSubscriber);
```

【参数】

参数名称	描述	输入/输出
pstSubscriber	订阅者参数。	输入
ppSubscriber	订阅者句柄指针。	输出

【返回值】

返回值	描述
0	成功。
非0	失败，参见 错误码 。

【需求】

- 头文件：ss_eventhub.h
- 库文件：libeventhub.a、libeventhub.so

【注意】

无。

【举例】

无。

10.3.7 SS_EVTHUB_DestroySubscriber

【描述】

销毁订阅者。

【定义】

```
TD_S32 SS_EVTHUB_DestroySubscriber(TD_MW_PTR pSubscriber);
```



【参数】

参数名称	描述	输入/输出
pSubscriber	订阅者句柄。	输入

【返回值】

返回值	描述
0	成功。
非0	失败，参见 错误码 。

【需求】

- 头文件：ss_eventhub.h
- 库文件：libeventhub.a、libeventhub.so

【注意】

无。

【举例】

无。

10.3.8 SS_EVTHUB_Subscribe

【描述】

订阅事件。

【定义】

```
TD_S32 SS_EVTHUB_Subscribe(SS_MW_PTR pSubscriber, OT\_EVENT\_ID EventID);
```

【参数】

参数名称	描述	输入/输出
pSubscriber	订阅者句柄。	输入
EventID	事件ID。	输入

【返回值】

返回值	描述
0	成功。



返回值	描述
非0	失败，参见 错误码 。

【需求】

- 头文件：ss_eventhub.h
- 库文件：libeventhub.a、libeventhub.so

【注意】

无。

【举例】

无。

10.3.9 SS_EVTHUB_UnSubscribe

【描述】

解订阅事件。

【定义】

TD_S32 SS_EVTHUB_UnSubscribe(TD_MW_PTR pSubscriber, [OT_EVENT_ID](#) EventID);

【参数】

参数名称	描述	输入/输出
pSubscriber	订阅者句柄。	输入
EventID	事件ID。	输入

【返回值】

返回值	描述
0	成功。
非0	失败，参见 错误码 。

【需求】

- 头文件：ss_eventhub.h
- 库文件：libeventhub.a、libeventhub.so

【注意】

无。



【举例】

无。

10.3.10 SS_EVTHUB_GetEventHistory

【描述】

获取历史事件。

【定义】

```
TD_S32 SS_EVTHUB_GetEventHistory(TD_S32 EventID, OT_EVENT_S *pEvent);
```

【参数】

参数名称	描述	输入/输出
EventID	事件ID。	输入
pEvent	事件指针。	输出

【返回值】

返回值	描述
0	成功。
非0	失败，参见 错误码 。

【需求】

- 头文件：ss_eventhub.h
- 库文件：libeventhub.a、libeventhub.so

【注意】

无。

【举例】

无。

10.3.11 SS_EVTHUB_SetEnabled

【描述】

设置使能标记。

【定义】

```
TD_S32 SS_EVTHUB_SetEnabled(TD_BOOL bFlag);
```

【参数】



参数名称	描述	输入/输出
bFlag	使能标记。	输入

【返回值】

返回值	描述
0	成功。
非0	失败，参见 错误码 。

【需求】

- 头文件：ss_eventhub.h
- 库文件：libeventhub.a、libeventhub.so

【注意】

无。

【举例】

无。

10.3.12 SS_EVTHUB_GetEnabled

【描述】

获取使能标记。

【定义】

TD_S32 SS_EVTHUB_GetEnabled(TD_BOOL *pFlag);

【参数】

参数名称	描述	输入/输出
pFlag	使能标记指针。	输出

【返回值】

返回值	描述
0	成功。
非0	失败，参见 错误码 。



【需求】

- 头文件：ss_eventhub.h
- 库文件：libeventhub.a、libeventhub.so

【注意】

无。

【举例】

无。

10.4 数据类型

事件路由模块相关数据类型定义如下：

- **EVENT_PAYLOAD_LEN**：事件负载数据最大长度。
- **OT_EVTHUB_SUBSCRIBE_NAME_LEN**：订阅者名称最大长度。
- **OT_EVTHUB_MESSAGEQURUR_MAX_SIZE**：消息队列最大值。
- **OT_EVENT_ID**：事件ID。
- **OT_EVENT_S**：事件结构体。
- **OT_SUBSCRIBER_S**：订阅者结构体。

10.4.1 EVENT_PAYLOAD_LEN

【说明】

事件负载数据最大长度。

【定义】

```
#define EVENT_PAYLOAD_LEN (512)
```

【注意事项】

无。

【相关数据结构及接口】

无。

10.4.2 OT_EVTHUB_SUBSCRIBE_NAME_LEN

【说明】

订阅者名称的最大长度。

【定义】

```
#define OT_EVTHUB_SUBSCRIBE_NAME_LEN (16)
```

【注意事项】

无。

【相关数据类型及接口】



无。

10.4.3 OT_EVTHUB_MESSAGEQURUR_MAX_SIZE

【说明】

消息队列最大值。

【定义】

```
#define OT_EVTHUB_MESSAGEQURUR_MAX_SIZE (32)
```

【注意事项】

无。

【相关数据类型及接口】

无。

10.4.4 OT_EVENT_ID

【说明】

事件ID。

【定义】

```
typedef TD_U32 OT_EVENT_ID
```

【注意事项】

无。

【相关数据类型及接口】

无。

10.4.5 OT_EVENT_S

【说明】

定义事件结构体。

【定义】

```
typedef struct otEVENT_S
{
    OT_EVENT_ID EventID;
    TD_S32 arg1;
    TD_S32 arg2;
    TD_S32 s32Result;
    TD_U64 u64CreateTime;
    TD_CHAR aszPayload[EVENT_PAYLOAD_LEN];
} OT_EVENT_S;
```

【成员】

成员名称	描述
EventID	事件ID



成员名称	描述
arg1	参数一
arg2	参数二
s32Result	处理结果
u64CreateTime	创建时间
aszPayload	负载数据

【注意事项】

创建时间在事件发布时，由模块内部获取系统clock时间填充，单位秒(s)。

【相关数据类型及接口】

无。

10.4.6 OT_SUBSCRIBER_S

【说明】

定义订阅者结构体。

【定义】

```
typedef struct otSUBSCRIBER_S
{
    TD_CHAR azName[OT_EVTHUB_SUBSCRIBE_NAME_LEN];
    TD_S32 (*SS_EVTHUB_EVENTPROC_FN_PTR)(const OT_EVENT_S *pEvent, const TD_VOID
*argv);
    TD_VOID *argv;
    TD_BOOL bSync;
}OT_SUBSCRIBER_S;
```

【成员】

成员名称	描述
azName	订阅者名称。
SS_EVTHUB_EVENTPROC_FN_PTR	订阅者事件处理函数。
argv	用户定义私有参数指针。
bSync	事件处理是否同步。

【注意事项】

无。

【相关数据类型及接口】

无。



10.5 错误码

事件路由模块API错误码如表10-1所示。

表 10-1 事件路由 API 错误码

错误代码	宏定义	描述
0xA1328002	OT_ERR_EVTHUB_NULL_PTR	空指针错误
0xA1328008	OT_ERR_EVTHUB_NOT_INIT	未初始化错误
0xA1328040	OT_ERR_EVTHUB_HANDLE_INVALID	非法句柄错误。
0xA1328041	OT_ERR_EVTHUB_INVALIDARG	非法参数
0xA1328042	OT_ERR_EVTHUB_MALLOC	内存申请错误
0xA1328043	OT_ERR_EVTHUB_CREATE	创建错误
0xA1328044	OT_ERR_EVTHUB_DESTROY	销毁错误
0xA1328045	OT_ERR_EVTHUB_NOT_CREATE	对象没有创建
0xA1328046	OT_ERR_EVTHUB_EVENT_NO_RIGEST	事件未注册
0xA1328047	OT_ERR_EVTHUB_NO_EVENT_HISTORY	历史事件不存在
0xA1328048	OT_ERR_EVTHUB_MSGHDL_SEND	事件路由消息发送错误

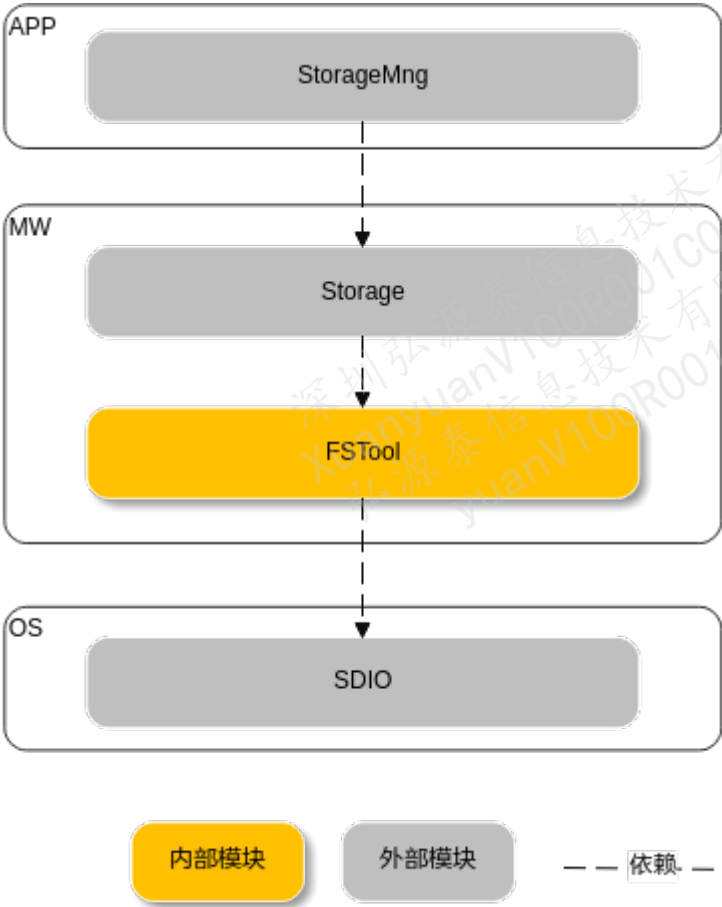


11 FSTool

11.1 概述

FSTool (File System Tool) 模块为用户提供检查和格式化文件系统的功能。模块上下文如图11-1所示。

图 11-1 FSTool 模块上下文





须知

- FSTool模块只支持FAT32文件系统。
- FSTool模块不支持多线程，请调用者保证，以免出现未知异常。

11.2 功能描述

11.2.1 检查文件系统

在录像或拍照业务中，可能会出现热插拔、异常掉电现象，导致文件系统异常。FSTool模块负责检查和修复文件系统异常，保证文件系统能正常工作。

- 检查文件系统异常
基于FAT32文件系统协议，检查文件系统的DBR、FSInfo、FAT和DirEntry。对于DBR和FSInfo中存在异常，将会上报异常错误；对于FAT和DirEntry出现异常将会进行修复。
- 检查文件系统碎片化程度
文件系统碎片化，是指文件系统中存在不连续存储的文件。这种碎片化的文件系统会导致存储设备的访问和读写性能下降。FSTool工具将统计出文件系统的碎片化程度。文件系统碎片化越严重，表示碎片化程度的值越大，反之则越小。

11.2.2 格式化文件系统

格式化方式有两种：

- 高级格式化；
- 高级格式化+擦除数据。

11.3 API 参考

该功能模块提供以下API：

- [SS_FSTOOL_Check](#)：检查文件系统。
- [SS_FSTOOL_Format](#)：格式化文件系统。
- [SS_FSTOOL_SetLogLevel](#)：设置日志级别。
- [SS_FSTOOL_RegisterBufOps](#)：注册内存操作集。
- [SS_FSTOOL_UnregisterBufOps](#)：注销内存操作集。

11.3.1 SS_FSTOOL_Check

【描述】

检查文件系统。

【语法】

```
int SS_FSTOOL_Check(const char* pszPath, const OT\_FSTOOL\_CHECK\_CFG\_S *pstCheckCfg);
```



【参数】

参数名称	描述	输入/输出
pszPath	分区路径。	输入
pstCheckCfg	文件系统检查配置参数。	输入

【返回值】

返回值	描述
0	成功。
非0	失败，参见 错误码 。

【需求】

- 头文件：ss_fstool.h
- 库文件：libfstool.a、libfstool.so

【注意】

- 如果文件系统碎片程度值大于pstCheckCfg的成员u32FragmentThr，会上报文件系统碎片化严重错误，但此时文件系统仍是可用的；
- 无论enMode为哪种方式，都会检查文件系统碎片化程度；
- 文件系统检查需要消耗一定的内存，内存消耗高于FAT表的大小，且目录项越多，碎片化越严重消耗的内存越大。以簇大小64KB、容量64GB、无碎片的文件系统为例，文件系统检查将消耗8MB的内存。如果内存不足将会上报OT_ERR_FSTOOL_NOT_ENOUGH_MEMORY的错误。

【举例】

参见Storage模块sample。

【相关主题】

无。

11.3.2 SS_FSTOOL_Format

【描述】

格式化文件系统。

【语法】

```
int SS_FSTOOL_Format(const char* pszPath, const OT_FSTOOL_FORMAT_CFG_S
*pstFormatCfg);
```

【参数】



参数名称	描述	输入/输出
pszPath	分区路径。	输入
pstFormatCfg	文件系统格式化配置参数。	输入

【返回值】

返回值	描述
0	成功。
非0	失败，参见 错误码 。

【需求】

- 头文件：ss_fstool.h
- 库文件：libfstool.a、libfstool.so

【注意】

- 格式化配置参数中的u32ClusterSize只支持65536B一种簇大小，其它数值报错；
- 格式化需要一定的内存，文件系统簇越小、容量越大，消耗的内存越大。以簇大小64KB，容量64GB为例，格式化文件系统将消耗4MB内存。如果使用过程中内存不足将会报OT_ERR_FSTOOL_NOT_ENOUGH_MEMORY错误。

【举例】

参见Storage模块sample。

【相关主题】

无。

11.3.3 SS_FSTOOL_SetLogLevel

【描述】

设置日志级别，只有等于或高于此级别的日志才会打印出来。

【语法】

```
int SS_FSTOOL_SetLogLevel(OT\_FSTOOL\_LOG\_LEVEL\_E enLogLevel);
```

【参数】

参数名称	描述	输入/输出
enLogLevel	日志级别。	输入

【返回值】



返回值	描述
0	成功。
非0	失败，参见 错误码 。

【需求】

- 头文件：ss_fstool.h
- 库文件：libfstool.a、libfst.so

【注意】

无。

【举例】

参见Storage模块sample。

【相关主题】

无。

11.3.4 SS_FSTOOL_RegisterBufOps

【描述】

注册内存操作集。

【语法】

```
int SS_FSTOOL_RegisterBufOps(const OT\_FSTOOL\_BUF\_OPS\_S* pstOPS);
```

【参数】

参数名称	描述	输入/输出
pstOPS	内存操作集。	输入

【返回值】

返回值	描述
0	成功。
非0	失败，参见 错误码 。

【需求】

- 头文件：ss_fstool.h
- 库文件：libfstool.a、libfstool.so

**【注意】**

- 模块默认申请系统内存，在系统内存紧缺的情况下，用户可通过该接口注册自定义内存操作集，从内存充裕的地方申请内存，防止因内存不足，导致模块功能不可用。
- 系统内存对于本模块是否紧缺，可参照[SS_FSTOOL_Check](#)和[SS_FSTOOL_Format](#)接口对于内存要求的介绍评判。
- 请用户保证不要在[SS_FSTOOL_Check](#)和[SS_FSTOOL_Format](#)过程中调用该接口，以免出现未知异常。

【举例】

参见Storage模块sample。

【相关主题】

[OT_FSTOOL_BUF_OPS_S](#)

11.3.5 SS_FSTOOL_UnregisterBufOps

【描述】

注销内存操作集。

【语法】

```
void SS_FSTOOL_UnregisterBufOps(void);
```

【参数】

无。

【返回值】

无。

【需求】

- 头文件：ss_fstool.h
- 库文件：libfstool.a、libfstool.so

【注意】

请用户保证不要在[SS_FSTOOL_Check](#)和[SS_FSTOOL_Format](#)过程中调用该接口，以免出现未知异常。

【举例】

参见Storage模块sample。

【相关主题】

无。

11.4 数据结构

相关数据类型、数据结构定义如下：



- **OT_FSTOOL_CHECK_MODE_E**: 文件系统检查模式。
- **OT_DISK_LABEL_LEN**: 磁盘卷标名数组长度。
- **OT_FSTOOL_FORMAT_MODE_E**: 文件系统格式化模式。
- **OT_FSTOOL_LOG_LEVEL_E**: 日志级别。
- **OT_FSTOOL_BUF_ALLOC_FN**: 内存申请函数。
- **OT_FSTOOL_BUF_FREE_FN**: 内存释放函数。
- **OT_FSTOOL_BUF_OPS_S**: 内存操作集。
- **OT_FSTOOL_CHECK_CFG_S**: 文件系统检查配置。
- **OT_FSTOOL_FORMAT_CFG_S**: 文件系统格式化配置。

11.4.1 OT_FSTOOL_CHECK_MODE_E

【说明】

文件系统检查模式。

【定义】

```
typedef enum otFSTOOL_CHECK_MODE_E
{
    OT_FSTOOL_CHECK_MODE_ALWAYS = 0x00,
    OT_FSTOOL_CHECK_MODE_DIRTY,
    OT_FSTOOL_CHECK_MODE_ALWAYS_MAIN_FAT,
    OT_FSTOOL_CHECK_MODE_DIRTY_MAIN_FAT,
    OT_FSTOOL_CHECK_MODE_BUTT
} OT_FSTOOL_CHECK_MODE_E;
```

【成员】

成员名称	描述
OT_FSTOOL_CHECK_MODE_ALWAYS	不管文件系统是否为脏，都会检查文件系统异常。
OT_FSTOOL_CHECK_MODE_DIRTY	只有在文件系统为脏的情况下，才会检查文件系统异常。
OT_FSTOOL_CHECK_MODE_ALWAYS_MAIN_FAT	不管文件系统是否为脏，都会检查文件系统，且以主FAT表信息为准进行检查。
OT_FSTOOL_CHECK_MODE_DIRTY_MAIN_FAT	只有在文件系统为脏的情况下，才会检查文件系统，且以主FAT表信息为准进行检查。

【注意事项】

无。

【相关数据类型及接口】

无。



11.4.2 OT_DISK_LABEL_LEN

【说明】

格式化磁盘卷标名数组长度。

【定义】

```
#define OT_DISK_LABEL_LEN 11
```

【成员】

无。

【注意事项】

无。

【相关数据类型及接口】

OT_FSTOOL_FORMAT_CFG_S

11.4.3 OT_FSTOOL_FORMAT_MODE_E

【说明】

文件系统格式化模式。

【定义】

```
typedef enum otFSTOOL_FORMAT_MODE_E
{
    OT_FSTOOL_FORMAT_MODE_ADVANCED = 0x00,
    OT_FSTOOL_FORMAT_MODE_DISCARD,
    OT_FSTOOL_FORMAT_MODE_BUTT
} OT_FSTOOL_FORMAT_MODE_E;
```

【成员】

成员名称	描述
OT_FSTOOL_FORMAT_MODE_ADVANCED	高级格式化，不会擦除数据区的数据。
OT_FSTOOL_FORMAT_MODE_DISCARD	高级格式化的同时，会清除掉数据区的数据。

【注意事项】

OT_FSTOOL_FORMAT_MODE_DISCARD耗时会比较久，从一秒到几十秒不等，一般在十秒以内，但经过此种格式化的存储设备，读写性能会有所提升。

【相关数据类型及接口】

无。

11.4.4 OT_FSTOOL_LOG_LEVEL_E

【说明】



日志级别。

【定义】

```
typedef enum otFSTOOL_LOG_LEVEL_E
{
    OT_FSTOOL_LOG_LEVEL_DEBUG = 0,
    OT_FSTOOL_LOG_LEVEL_INFO,
    OT_FSTOOL_LOG_LEVEL_WARN,
    OT_FSTOOL_LOG_LEVEL_ERROR,
    OT_FSTOOL_LOG_LEVEL_FATAL,
    OT_FSTOOL_LOG_LEVEL_BUTT
} OT_FSTOOL_LOG_LEVEL_E;
```

【成员】

成员名称	描述
OT_FSTOOL_LOG_LEVEL_DEBUG	DEBUG级别。
OT_FSTOOL_LOG_LEVEL_INFO	INFO级别。
OT_FSTOOL_LOG_LEVEL_WARN	WARN级别。
OT_FSTOOL_LOG_LEVEL_ERROR	ERROR级别。
OT_FSTOOL_LOG_LEVEL_FATAL	FATAL级别。

【注意事项】

无。

【相关数据类型及接口】

无。

11.4.5 OT_FSTOOL_BUF_ALLOC_FN

【说明】

内存申请函数。

【定义】

```
typedef int (*OT_FSTOOL_BUF_ALLOC_FN)(unsigned long long * phyAddr, size_t bufSize, void** virAddr);
```

【成员】

成员名称	描述
phyAddr	内存物理地址。
bufSize	内存大小。
virAddr	内存虚拟地址。



【注意事项】

无。

【相关数据类型及接口】

无。

11.4.6 OT_FSTOOL_BUF_FREE_FN

【说明】

内存释放函数。

【定义】

```
typedef int (*OT_FSTOOL_BUF_FREE_FN)(unsigned long long * phyAddr, void* virAddr);
```

【成员】

成员名称	描述
phyAddr	内存物理地址。
virAddr	内存虚拟地址。

【注意事项】

无。

【相关数据类型及接口】

无。

11.4.7 OT_FSTOOL_BUF_OPS_S

【说明】

内存操作集。

【定义】

```
typedef struct otFSTOOL_BUF_OPS_S
{
    OT_FSTOOL_BUF_ALLOC_FN pfnBufAlloc;
    OT_FSTOOL_BUF_FREE_FN pfnBufFree;
} OT_FSTOOL_BUF_OPS_S;
```

【成员】

成员名称	描述
pfnBufAlloc	内存申请函数。
pfnBufFree	内存释放函数。

【注意事项】



无。

【相关数据类型及接口】

- [OT_FSTOOL_BUF_ALLOC_FN](#)
- [OT_FSTOOL_BUF_FREE_FN](#)

11.4.8 OT_FSTOOL_CHECK_CFG_S

【说明】

文件系统检查配置。

【定义】

```
typedef struct otFSTOOL_CHECK_CFG_S
{
    OT\_FSTOOL\_CHECK\_MODE\_E enMode;
    unsigned int u32FragmentThr;
    unsigned int parts;
} OT_FSTOOL_CHECK_CFG_S;
```

【成员】

成员名称	描述
enMode	文件系统检查模式。
u32FragmentThr	文件系统碎片化程度阈值。
parts	文件系统分阶段检查阶段数目。 有效值范围为[1, 64]，其中parts越大，文件系统检查消耗内存越小，检查消耗时间会越长。

【注意事项】

无。

【相关数据类型及接口】

- [OT_FSTOOL_CHECK_MODE_E](#)
- [SS_FSTOOL_Check](#)

11.4.9 OT_FSTOOL_FORMAT_CFG_S

【说明】

文件系统格式化配置。

【定义】

```
typedef struct otFSTOOL_FORMAT_CFG_S
{
    OT\_FSTOOL\_FORMAT\_MODE\_E enMode;
    unsigned int u32ClusterSize;
    bool enable4KAlignCheck;
```



```
char diskLabel[OT_DISK_LABEL_LEN];
unsigned int parts;
} OT_FSTOOL_FORMAT_CFG_S;
```

【成员】

成员名称	描述
enMode	文件系统格式化模式。
u32ClusterSize	文件系统簇大小，单位Byte。
enable4KAlignCheck	使能4K对齐检查。 - 当为1时，格式化时检查分区是否4KB对齐，如果非对齐，会清除分区并重建4K对齐的分区，然后执行格式化。 - 当为0时，格式化时不检查分区是否4KB字节对齐。
diskLabel	格式化后的磁盘卷标名数组。 - 字符串最大长度不超过11。 - 当为空串时，磁盘不带名字。
parts	文件系统分阶段格式化数目。 有效值范围为[1, 64]，其中parts越大，文件系统格式化消耗内存越小，格式化消耗时间会越长。

【注意事项】

无

【相关数据类型及接口】

- OT_FSTOOL_CHECK_MODE_E
- SS_FSTOOL_Check

11.5 错误码

FSTool API错误码如表11-1所示。

表 11-1 FSTool API 错误码

错误代码	宏定义	描述
0x201	OT_ERR_FSTOOL_ILLEGAL_PARAM	非法入参
0x202	OT_ERR_FSTOOL_OPEN_FAILURE	打开文件失败
0x203	OT_ERR_FSTOOL_READ_FAILURE	读文件失败



错误代码	宏定义	描述
0x204	OT_ERR_FSTOOL_WRITE_FAILURE	写文件失败
0x205	OT_ERR_FSTOOL_NOT_ENOUGH_MEMORY	内存不足
0x206	OT_ERR_FSTOOL_FSTYPE_UNSupport	文件系统类型不支持
0x207	OT_ERR_FSTOOL_FS_EXCEPTION	文件系统异常，且无法修复
0x208	OT_ERR_FSTOOL_FRAGMENT_SEVERELY	文件系统碎片化严重
0x209	OT_ERR_FSTOOL_DEVICE_EXCEPTION	设备异常
0x210	OT_ERR_FSTOOL_SYSTEM_CALL_FAILURE	系统调用失败
0x211	OT_ERR_FSTOOL_NOT_64K_CLUSTER	文件系统簇大小非64KB

11.6 典型场景使用指南

参见本模块sample。

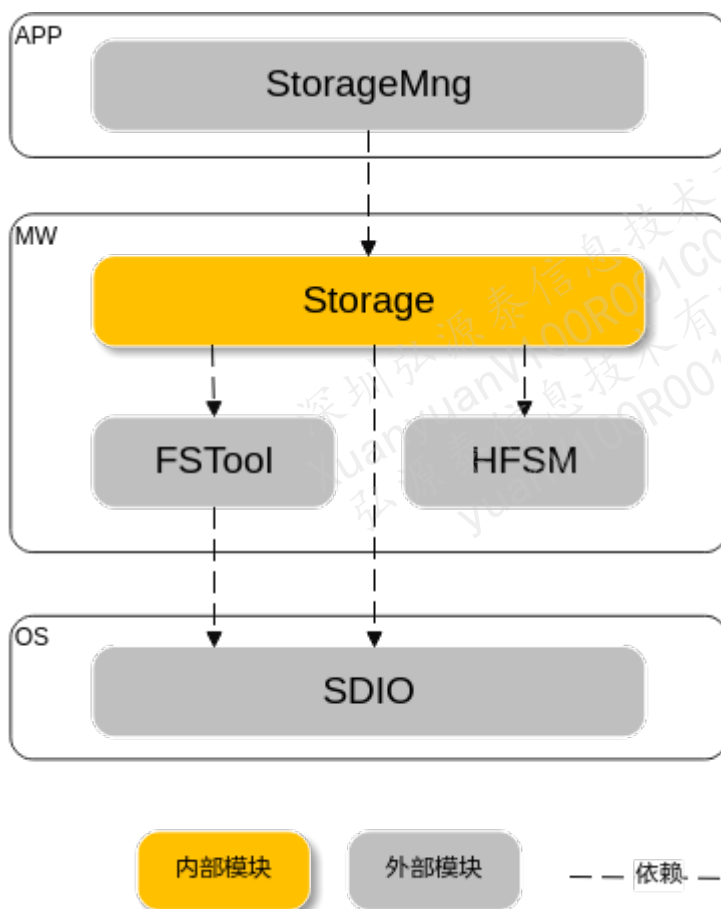


12 Storage

12.1 概述

Storage模块负责管理存储设备，为上层提供存储设备状态监听和信息获取的功能。Storage模块的上下文如图12-1所示。

图 12-1 Storage 模块上下文





12.2 功能描述

12.2.1 监听存储设备

- 当存储设备状态变化时，以事件通知的形式上报给用户；
- 当检测到存储设备上线时，便会去检查存储设备上的文件系统，并把文件系统相关的状态以事件通知的形式上报给用户。

12.2.2 获取存储设备信息

获取存储设备的信息包括硬件设备的信息和设备中的文件系统的信息。

- 设备信息包括设备CID、工作模式、工作时钟；
- 文件系统信息包括簇大小、总空间、可用空间和已用空间。

12.3 API 参考

该功能模块提供以下API：

- **SS_STORAGE_RegisterFSTool**：注册文件系统工具。
- **SS_STORAGE_UnregisterFSTool**：去注册文件系统工具。
- **SS_STORAGE_Create**：创建Storage实例。
- **SS_STORAGE_Destroy**：销毁Storage实例。
- **SS_STORAGE_GetState**：获取Storage实例状态。
- **SS_STORAGE_GetFSInfo**：获取文件系统信息。
- **SS_STORAGE_GetDevInfo**：获取存储设备信息。
- **SS_STORAGE_Format**：格式化。
- **SS_STORAGE_Pause**：暂停Storage实例。
- **SS_STORAGE_Resume**：恢复Storage实例。

12.3.1 SS_STORAGE_RegisterFSTool

【描述】

注册文件系统工具。

【语法】

```
TD_S32 SS_STORAGE_RegisterFSTool(const OT_STORAGE_FS_TOOL_S* fsTool);
```

【参数】

参数名称	描述	输入/输出
fsTool	文件系统工具（FSTool）。	输入



【返回值】

返回值	描述
0	成功。
非0	失败，参见 错误码 。

【需求】

- 头文件：ss_storage.h
- 库文件：libstorage.a、libstorage.so

【注意】

无。

【举例】

参见本模块sample。

【相关主题】

无。

12.3.2 SS_STORAGE_UnregisterFSTool

【描述】

去注册文件系统工具。

【语法】

```
TD_S32 SS_STORAGE_UnregisterFSTool(const TD_CHAR* fsType);
```

【参数】

参数名称	描述	输入/输出
fsType	文件系统类型。	输入

【返回值】

返回值	描述
0	成功。
非0	失败，参见 错误码 。

【需求】

- 头文件：ss_storage.h



- 库文件：libstorage.a、libstorage.so
- 【注意】
- 模块中必须已经注册了pszFSType类型文件系统的工具，才能去初始化成功，反之报错；
 - 如果不需要使用Storage模块了，用户需要将注册进来的文件系统工具都去注册。
- 【举例】
- 参见本模块sample。
- 【相关主题】
- 无。

12.3.3 SS_STORAGE_Create

【描述】

创建Storage实例。

【语法】

```
TD_S32 SS_STORAGE_Create(const OT_STORAGE_CFG_S* stgCfg, const OT_STORAGE_ON_STATE_CHANGE_PFN onStateChange, TD_MW_PTR* storage);
```

【参数】

参数名称	描述	输入/输出
stgCfg	Storage实例属性。	输入
onStateChange	Storage实例状态切换回调函数。	输入
storage	Storage实例句柄。	输出

【返回值】

返回值	描述
0	成功。
非0	失败，返回错误码。

【需求】

- 头文件：ss_storage.h
- 库文件：libstorage.a、libstorage.so

【注意】

最多可创建OT_STORAGE_INST_CNT_MAX个实例。



【举例】

参见本模块sample。

【相关主题】

无。

12.3.4 SS_STORAGE_Destroy

【描述】

销毁Storage实例。

【语法】

```
TD_S32 SS_STORAGE_Destroy(TD_MW_PTR storage);
```

【参数】

参数名称	描述	输入/输出
storage	Storage实例句柄。	输入

【返回值】

返回值	描述
0	成功。
非0	失败，返回 错误码 。

【需求】

- 头文件：ss_storage.h
- 库文件：libstorage.a、libstorage.so

【注意】

无。

【举例】

参见本模块sample。

【相关主题】

无。

12.3.5 SS_STORAGE_GetState

【描述】

获取Storage实例状态。

【语法】



TD_S32 SS_STORAGE_GetState(TD_MW_PTR storage, OT_STORAGE_STATE_E* state);

【参数】

参数名称	描述	输入/输出
storage	Storage实例句柄。	输入
state	Storage实例状态。	输出

【返回值】

返回值	描述
0	成功。
非0	失败，返回错误码。

【需求】

- 头文件：ss_storage.h
- 库文件：libstorage.a、libstorage.so

【注意】

无。

【举例】

参见本模块sample。

【相关主题】

无。

12.3.6 SS_STORAGE_GetFSInfo

【描述】

获取文件系统信息。

【语法】

TD_S32 SS_STORAGE_GetFSInfo(TD_MW_PTR storage, OT_STORAGE_FS_INFO_S* fsInfo);

【参数】

参数名称	描述	输入/输出
storage	Storage实例句柄。	输入
fsInfo	文件系统信息	输出



【返回值】

返回值	描述
0	成功。
非0	失败，返回 错误码 。

【需求】

- 头文件：ss_storage.h
- 库文件：libstorage.a、libstorage.so

【注意】

在Storage实例的状态处于OT_STORAGE_STATE_MOUNTED时，才能获取到文件系统的信息，否则报错。

【举例】

参见本模块sample。

【相关主题】

无。

12.3.7 SS_STORAGE_GetDevInfo

【描述】

获取存储设备信息。

【语法】

TD_S32 SS_STORAGE_GetDevInfo(TD_MW_PTR storage, [OT_STORAGE_DEV_INFO_S*](#) devInfo);

【参数】

参数名称	描述	输入/输出
storage	Storage实例句柄。	输入
devInfo	存储设备信息。	输出

【返回值】

返回值	描述
0	成功。
非0	失败，返回 错误码 。

【需求】



- 头文件：ot_storage.h
- 库文件：libstorage.a、libstorage.so

【注意】

当Storage实例的状态为OT_STORAGE_STATE_DEV_UNPLUGGED、OT_STORAGE_STATE_DEV_CONNECTING和OT_STORAGE_STATE_DEV_ERROR时，无法获取存储设备的信息，反之，则可以。

【举例】

参见本模块sample。

【相关主题】

无。

12.3.8 SS_STORAGE_Format

【描述】

格式化，按指定文件系统类型和簇大小格式化存储设备。

【语法】

```
TD_S32 SS_STORAGE_Format(TD_MW_PTR storage, const TD_CHAR* fsType, TD_U64 clusterSize);
```

【参数】

参数名称	描述	输入/输出
storage	Storage实例句柄。	输入
fsType	文件系统类型。	输入
clusterSize	簇大小(单位Byte)	输入

【返回值】

返回值	描述
0	成功。
非0	失败，返回 错误码 。

【需求】

- 头文件：ss_storage.h
- 库文件：libstorage.a、libstorage.so

【注意】

- 该功能实质上是注册进来的文件系统工具中的格式化功能，如果注册进来的文件系统工具不支持pszFSType和u64ClusterSize，则会报错；



- 该接口调用发生错误时，可能会返回OT_STORAGE_FSTOOL_ERR_E中定义的错误码。

【举例】

参见本模块sample。

【相关主题】

无。

12.3.9 SS_STORAGE_Pause

【描述】

暂停Storage实例，暂停后将不再监听Storage实例所代表的存储设备的状态。

【语法】

TD_S32 SS_STORAGE_Pause(TD_MW_PTR storage);

【参数】

参数名称	描述	输入/输出
storage	Storage实例句柄。	输入

【返回值】

返回值	描述
0	成功。
非0	失败，返回错误码。

【需求】

- 头文件：ss_storage.h
- 库文件：libstorage.a、libstorage.so

【注意】

- Storage实例暂停后，该Storage实例的SS_STORAGE_GetState、SS_STORAGE_GetFSInfo、SS_STORAGE_GetDevInfo和SS_STORAGE_Format接口将不可用，直到恢复，其他接口可用。
- 如果pStorage所表示的Storage实例已经暂停，调用该接口则会报错。

【举例】

参见本模块sample。

【相关主题】

无。



12.3.10 SS_STORAGE_Resume

【描述】

恢复Storage实例，恢复后的Storage实例将会继续监听其所代表的存储设备状态。

【语法】

```
TD_S32 SS_STORAGE_Resume(TD_MW_PTR storage);
```

【参数】

参数名称	描述	输入/输出
storage	Storage实例句柄。	输入

【返回值】

返回值	描述
0	成功。
非0	失败，返回 错误码 。

【需求】

- 头文件：ss_storage.h
- 库文件：libstorage.a、libstorage.so

【注意】

如果pStorage所表示的Storage实例并未暂停，调用该接口则报错；

【举例】

参见本模块sample。

【相关主题】

无。

12.4 数据结构

相关数据类型、数据结构定义如下：

- [OT_STORAGE_PATH_LEN_MAX](#)：路径最大长度。
- [OT_STORAGE_INST_CNT_MAX](#)：Storage实例最大个数。
- [OT_STORAGE_CFG_S](#)：Storage实例属性。
- [OT_STORAGE_STATE_E](#)：Storage实例状态。
- [OT_STORAGE_TRANSMISSION_SPEED_E](#)：存储设备传输速率。



- [OT_STORAGE_DEV_INFO_S](#): 存储设备信息。
- [OT_STORAGE_FS_INFO_S](#): 文件系统信息。
- [OT_STORAGE_FSTOOL_ERR_E](#): 文件系统工具错误码。
- [OT_STORAGE_FS_TOOL_S](#): 文件系统工具。
- [OT_STORAGE_ON_STATE_CHANGE_PFN](#): Storage实例状态变化回调函数。

12.4.1 OT_STORAGE_PATH_LEN_MAX

【说明】

路径最大长度。

【定义】

```
#define OT_STORAGE_PATH_LEN_MAX 256
```

【成员】

无。

【注意事项】

无。

【相关数据类型及接口】

无。

12.4.2 OT_STORAGE_INST_CNT_MAX

【说明】

Storage实例最大数量，该数据结构约束了Storage模块可创建的最大Storage实例数。

【定义】

```
#define OT_STORAGE_INST_CNT_MAX 2
```

【成员】

无。

【注意事项】

无。

【相关数据类型及接口】

无。

12.4.3 OT_STORAGE_CFG_S

【说明】

Storage实例属性。

【定义】

```
typedef struct otSTORAGE_CFG_S  
{
```



```
TD_U8 u8DevPortNo;
TD_U8 u8DevErrCountThr;
TD_CHAR aszPartitionPath[OT_STORAGE_PATH_LEN_MAX];
TD_CHAR aszMountPath[OT_STORAGE_PATH_LEN_MAX];
} OT_STORAGE_CFG_S;
```

【成员】

成员名称	描述
u8DevPortNo	存储设备端口号，即存储设备对应MCI卡槽号。
u8DevErrCountThr	存储设备错误次数阈值。 典型值：3。
aszPartitionPath	存储设备分区路径。
aszMountPath	分区挂载路径。

【注意事项】

- 存储设备在运行过程可能会出现错误，如读写异常，如果出现错误的次数超过u8DevErrCountThr，设备将会进入OT_STORAGE_STATE_DEV_ERROR状态；
- 保证各Storage实例之间的aszPartitionPath和aszMountPath互斥。

【相关数据类型及接口】

OT_STORAGE_PATH_LEN_MAX

12.4.4 OT_STORAGE_STATE_E

【说明】

Storage实例状态。

【定义】

```
typedef enum otSTORAGE_STATE_E
{
    OT_STORAGE_STATE_DEV_UNPLUGGED = 0x00,
    OT_STORAGE_STATE_DEV_CONNECTING,
    OT_STORAGE_STATE_DEV_CONNECT_FAILED,
    OT_STORAGE_STATE_DEV_ERROR,
    OT_STORAGE_STATE_FS_CHECKING,
    OT_STORAGE_STATE_FS_CHECK_FAILED,
    OT_STORAGE_STATE_FS_EXCEPTION,
    OT_STORAGE_STATE_MOUNTED,
    OT_STORAGE_STATE_MOUNT_FAILED,
    OT_STORAGE_STATE_IDEL
} OT_STORAGE_STATE_E;
```

【成员】

成员名称	描述
OT_STORAGE_STATE_DEV_UNPLUGGED	存储设备已拔出。



成员名称	描述
OT_STORAGE_STATE_DEV_CONNECTING	存储设备连接中。
OT_STORAGE_STATE_DEV_CONNECT_FAILED	存储设备连接失败。
OT_STORAGE_STATE_DEV_ERROR	存储设备损坏。
OT_STORAGE_STATE_FS_CHECKING	文件系统检查中。
OT_STORAGE_STATE_FS_CHECK_FAILED	文件系统检查失败
OT_STORAGE_STATE_FS_EXCEPTION	文件系统异常。
OT_STORAGE_STATE_MOUNTED	文件系统已挂载
OT_STORAGE_STATE_MOUNT_FAILED	文件系统挂载失败
OT_STORAGE_STATE_IDEL	空闲状态，Storage时创建后的状态。

【注意事项】

- 使用过程中，如果Storage实例多次处于OT_STORAGE_STATE_DEV_ERROR状态，说明该存储设备可能存在损坏，请考虑更换存储设备；如果只是偶现，可考虑插拔存储设备或者上电下电来恢复。
- 如果Storage实例出现OT_STORAGE_STATE_DEV_CONNECT_FAILED，说明存储设备损坏，请考虑更换存储设备。
- 如果Storage实例处于OT_STORAGE_STATE_FS_EXCEPTION状态，说明存储设备上的文件系统出现异常，且文件系统工具无法修复这样的异常，请考虑格式化进行恢复。

【相关数据类型及接口】

无。

12.4.5 OT_STORAGE_TRANSMISSION_SPEED_E

【说明】

存储设备传输速率。

【定义】

```
typedef enum otSTORAGE_TRANSMISSION_SPEED_E
{
    OT_STORAGE_TRANSMISSION_SPEED_1_4M = 0x00,
    OT_STORAGE_TRANSMISSION_SPEED_4_10M,
    OT_STORAGE_TRANSMISSION_SPEED_10_30M,
    OT_STORAGE_TRANSMISSION_SPEED_30_50M,
    OT_STORAGE_TRANSMISSION_SPEED_50_100M,
    OT_STORAGE_TRANSMISSION_SPEED_EXCEED_100M,
    OT_STORAGE_TRANSMISSION_SPEED_BUTT
} OT_STORAGE_TRANSMISSION_SPEED_E;
```

【成员】



成员名称	描述
OT_STORAGE_TRANSMISSION_SPEED_1_4M	存储设备存储速率在1~4MB/S。
OT_STORAGE_TRANSMISSION_SPEED_4_10M	存储设备存储速率在4~10MB/S。
OT_STORAGE_TRANSMISSION_SPEED_10_30M	存储设备存储速率在10~30MB/S。
OT_STORAGE_TRANSMISSION_SPEED_30_50M	存储设备存储速率在30~50MB/S。
OT_STORAGE_TRANSMISSION_SPEED_50_100M	存储设备存储速率在50~100MB/S
OT_STORAGE_TRANSMISSION_SPEED_EXCEED_100M	存储设备存储速率在100MB/S以上

【注意事项】

存储设备的传输速率受很多因素影响，此处描述的速率等级是根据存储设备的硬件属性和工作模式评估的理论传输速率，可能与业务运行时的实际速率有差别。

【相关数据类型及接口】

无。

12.4.6 OT_STORAGE_DEV_INFO_S

【说明】

存储设备信息。

【定义】

```
typedef struct otSTORAGE_DEV_INFO_S
{
    TD_CHAR aszDevType[OT_STORAGE_PATH_LEN_MAX];
    TD_CHAR aszCID[OT_STORAGE_PATH_LEN_MAX];
    TD_CHAR aszWorkMode[OT_STORAGE_PATH_LEN_MAX];
    TD_CHAR aszWorkClock[OT_STORAGE_PATH_LEN_MAX];
    OT_STORAGE_TRANSMISSION_SPEED_E enTranSpeed;
} OT_STORAGE_DEV_INFO_S;
```

【成员】

成员名称	描述
aszDevType	存储设备类型，如：SD、MMC。
aszCID	存储设备CID号。
aszWorkMode	工作模式。
aszWorkClock	工作时钟。



成员名称	描述
enTranSpeed	传输速率。

【注意事项】

无。

【相关数据类型及接口】

- [OT_STORAGE_PATH_LEN_MAX](#)
- [OT_STORAGE_TRANSMISSION_SPEED_E](#)

12.4.7 OT_STORAGE_FS_INFO_S

【说明】

文件系统信息。

【定义】

```
typedef struct otSTORAGE_FS_INFO_S
{
    TD_U64 u64ClusterSize;
    TD_U64 u64TotalSize;
    TD_U64 u64AvailableSize;
    TD_U64 u64UsedSize;
} OT_STORAGE_FS_INFO_S;
```

【成员】

成员名称	描述
u64ClusterSize	簇大小，单位Byte。
u64TotalSize	总空间，单位Byte。
u64AvailableSize	可用空间，单位Byte。
u64UsedSize	已用空间，单位Byte。

【注意事项】

无。

【相关数据类型及接口】

无。

12.4.8 OT_STORAGE_FSTOOL_ERR_E

【说明】

文件系统工具错误码，定义了文件系统的错误码，用户根据这一套错误码实现自己的文件系统工具。



【定义】

```
typedef enum otSTORAGE_FSTOOL_ERR_E
{
    OT_STORAGE_FSTOOL_SUCCESS=0x00,
    OT_STORAGE_FSTOOL_ERR_ILLEGAL_PARAM=0x201,
    OT_STORAGE_FSTOOL_ERR_OPEN_FAIL=0x202,
    OT_STORAGE_FSTOOL_ERR_READ_FAIL=0x203,
    OT_STORAGE_FSTOOL_ERR_WRITE_FAIL=0x204,
    OT_STORAGE_FSTOOL_ERR_NOT_ENOUGH_MEMORY=0x205,
    OT_STORAGE_FSTOOL_ERR_FSTYPE_UNSUPPORT=0x206,
    OT_STORAGE_FSTOOL_ERR_FS_EXCEPTION=0x207,
    OT_STORAGE_FSTOOL_ERR_FRAGMENT_SEVERELY=0x208,
    OT_STORAGE_FSTOOL_ERR_DEVICE_EXCEPTION=0x209,
    OT_STORAGE_FSTOOL_ERR_SYSTEM_CALL_FAILURE=0x210,
    OT_STORAGE_FSTOOL_ERR_FS_NOT_64K_CLUSTER=0x211,
    OT_STORAGE_FSTOOL_ERR_ERR_BUTT
}OT_STORAGE_FSTOOL_ERR_E;
```

【成员】

成员名称	描述
OT_STORAGE_FSTOOL_SUCCESS	接口调用成功。
OT_STORAGE_FSTOOL_ERR_ILLEGAL_PARAM	入参非法。
OT_STORAGE_FSTOOL_ERR_OPEN_FAIL	打开文件失败。
OT_STORAGE_FSTOOL_ERR_READ_FAIL	读文件失败。
OT_STORAGE_FSTOOL_ERR_WRITE_FAIL	写文件失败。
OT_STORAGE_FSTOOL_ERR_NOT_ENOUGH_MEMORY	内存不足。
OT_STORAGE_FSTOOL_ERR_FSTYPE_UNSUPPORT	文件系统类型不支持。
OT_STORAGE_FSTOOL_ERR_FS_EXCEPTION	文件系统异常。
OT_STORAGE_FSTOOL_ERR_FRAGMENT_SEVERELY	文件系统碎片化严重。
OT_STORAGE_FSTOOL_ERR_DEVICE_EXCEPTION	设备异常。
OT_STORAGE_FSTOOL_ERR_SYSTEM_CALL_FAILURE	系统调用失败。
OT_STORAGE_FSTOOL_ERR_FS_NOT_64K_CLUSTER	簇大小非64KB
OT_STORAGE_FSTOOL_ERR_ERR_BUTT	未知错误

【注意事项】

无。

【相关数据类型及接口】

无。

12.4.9 OT_STORAGE_FS_TOOL_S

【说明】



文件系统工具，定义了文件系统工具的函数原型，用户可根据需要定制自己的文件系统工具。

【定义】

```
typedef struct ot_STORAGE_FS_TOOL_S
{
    OT_STORAGE_FSTOOL_ERR_E (*pfnCheck)(const TD_CHAR *partitionPath);
    OT_STORAGE_FSTOOL_ERR_E (*pfnFormat)(const TD_CHAR *partitionPath, TD_U64
clusterSize);
    TD_CHAR aszFSType[OT_STORAGE_PATH_LEN_MAX];
}OT_STORAGE_FS_TOOL_S;
```

【成员】

成员名称	描述
pfnCheck	文件系统检查工具。
pfnFormat	文件系统格式化工具。
aszFSType	文件系统类型。

【注意事项】

- 用户在实现定制化文件系统工具时，前提要保证操作系统支持该类型文件系统；
- 当前只支持用户定制化FAT32和EXFAT的文件系统工具。

【相关数据类型及接口】

- OT_STORAGE_FSTOOL_ERR_E
- OT_STORAGE_PATH_LEN_MAX

12.4.10 OT_STORAGE_ON_STATE_CHANGE_PFN

【说明】

Storage实例状态变化回调函数。

【定义】

```
typedef TD_VOID (*OT_STORAGE_ON_STATE_CHANGE_PFN)(TD_MW_PTR storage, const
TD_CHAR* partitionPath, OT_STORAGE_STATE_E state, TD_S32 errCode);
```

【成员】

成员名称	描述
storage	Storage实例句柄。
partitionPath	存储设备分区路径。
state	Storage实例状态。
errCode	错误码。



【注意事项】

无。

【相关数据类型及接口】

无。

12.5 错误码

Storage API错误码如表12-1所示。

表 12-1 Storage API 错误码

错误代码	宏定义	描述
0xa1188040	OT_ERR_STORAGE_SYSTEM_CALL_FAILURE	系统调用失败
0xa1188041	OT_ERR_STORAGE_NULL_PTR	空指针
0xa1188050	OT_ERR_STORAGE_ILLEGAL_PARAM	入参非法
0xa1188051	OT_ERR_STORAGE_INVALID_HANDLE	实例句柄无效
0xa1188052	OT_ERR_STORAGE_INST_CNT_UPPER_LIMIT	实例数量超过上限
0xa1188060	OT_ERR_STORAGE_DEV_DISCONNECT	存储设备未连接
0xa1188061	OT_ERR_STORAGE_DEV_ERROR	存储设备错误
0xa1188070	OT_ERR_STORAGE_FS_CHECKING	文件系统检查中
0xa1188071	OT_ERR_STORAGE_FS_EXCEPTION	文件系统异常
0xa1188072	OT_ERR_STORAGE_FS_NOT_MOUNTED	文件系统未挂载
0xa1188080	OT_ERR_STORAGE_RUNNING	正在运行中
0xa1188081	OT_ERR_STORAGE_PAUSED	已暂停

12.6 典型场景使用指南

参见本模块sample。



12.7 Proc 调试

在Linux系统上，Storage模块支持Proc调试，程序运行后，通过串口在板端执行“cat /proc/ot/middleware/storage”便可以查看Storage模块的Proc信息。Storage模块提供Proc信息如表12-2所示。

表 12-2 Storage Proc 信息

定义	说明
aszPartition	分区路径。
bRunning	Storage实例是否在运行中。 0：暂停； 1：运行中。
enCurState	Storage实例当前状态。
u64FSCheckDuration(us)	文件系统检查耗时，单位us。
u64FSMountDuration(us)	文件系统挂载耗时，单位us。
u64FormatDuration(us)	格式化耗时，单位us。
bFragmentSeverely	文件系统是否碎片化严重。 0：碎片化不严重； 1：碎片化严重。

Storage模块Proc信息样例如图12-2所示。

图 12-2 Storage 模块 Proc 信息

```
~ # cat /proc/ot/middleware/storage
Storage
  partition : /dev/mmcblk1p1
  isRunning : 1
  curState  : 7
  fsCheckTime(us): 122358
  mountTime(us)  : 2701
  formatTime(us)  : 0
  bFragmentSeverely: 0
```



13 XMP 封装模块

13.1 概述

XMP封装模块用于将XMP协议数据封装到媒体文件中，本模块主要是用于将XMP数据封装到MP4文件中。

13.2 功能描述

13.2.1 基本概念

Adobe XMP（可扩展元数据平台，Adobe's Extensible Metadata Platform）是一种标签技术，可以帮助用户将文件信息(元数据)添加到以PNG、GIF、JPEG、Photoshop、TIFF、PDF和MP4等格式保存的文件中。XMP 为 Adobe 应用程序间的元数据交换提供了便利。

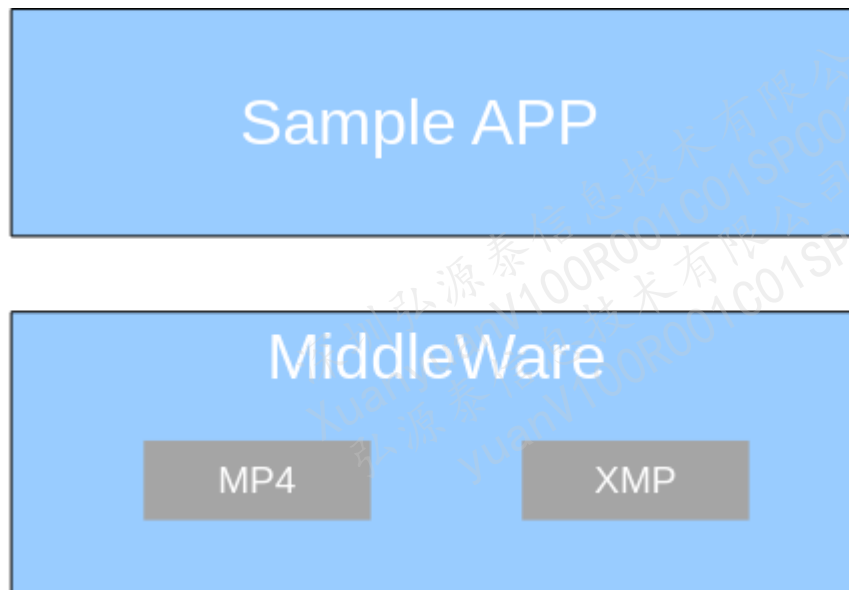
13.2.2 功能描述

XMP模块，主要将XMP格式数据拼装成符合Adobe协议的XML数据、对符合Adobe的XML数据进行解析。

13.2.3 上下文描述

XMP封装组件软件上下文依赖关系如[图13-1](#)所示。

图 13-1 XMP 封装组件软件上下文关系图



- XMP模块提供对外接口用于对XMP数据的拼装和解析；主要是以XML格式拼装XMP数据；以及把XML格式数据解析成XMP数据。
- MP4模块提供对外接口用来将拼装好的XMP数据进行私有BOX封装，封装到MP4文件中。也提供对外接口用来解析MP4文件里私有BOX，以获取封装在BOX里面的XMP数据。
- Sample App模块调用MP4模块、XMP模块的对外接口完成MP4的封装与解封装。

13.3 API 参考

该模块支持：

- XMP封装初始化、去初始化；
- XMP对象创建、销毁；
- XMP命名空间和属性的扩展；
- XMP协议数据属性值的设置；
- XMP协议数据生成、解析和遍历。

该功能模块提供以下API：

- **SS_XMP_Init**：初始化XMP模块。
- **SS_XMP_DeInit**：去初始化XMP模块。
- **SS_XMP_Create**：创建XMP对象并初始化，并且添加命名空间。
- **SS_XMP_Destroy**：销毁XMP实例。
- **SS_XMP_GetProperty**：获取XMP属性值。
- **SS_XMP_AddProperty**：添加XMP属性和属性值。
- **SS_XMP_ToString**：获取XMP数据拼装后的XML数据。
- **SS_XMP_ForeachKeyValue**：遍历XMP属性和属性值。



13.3.1 SS_XMP_Init

【描述】

初始化XMP模块。

【语法】

```
TD_S32 SS_XMP_Init();
```

【参数】

无

【返回值】

返回值	描述
0	成功。
非0	失败，参见 错误码 。

【需求】

- 头文件：ss_xmp.h
- 库文件：libxmp.a、libxmp.so

【注意】

此接口添加了引用计数，初始化仅需一次，也支持重复初始化，与去初始化配对使用。

【举例】

本例中演示了XMP模块的初始化过程，具体可以参见本模块的sample代码。

```
TD_S32 s32ret = 0;
s32ret = SS_XMP_Init();
if (TD_SUCCESS != s32ret)
{
    printf("SS_XMP_Init fail s32ret=[%d]\n", s32ret);
}
```

【相关主题】

无

13.3.2 SS_XMP_DeInit

【描述】

去初始化XMP模块。

【语法】

```
TD_S32 SS_XMP_DeInit();
```

【参数】

无



【返回值】

返回值	描述
0	成功。
非0	失败，参见 错误码 。

【需求】

- 头文件：ss_xmp.h
- 库文件：libxmp.a、libxmp.so

【注意】

与初始化配对使用。

【举例】

本例中演示了XMP模块的去初始化过程，具体可以参见本模块的sample代码。

```
s32ret = SS_XMP_DeInit();
if (TD_SUCCESS!= s32ret)
{
    printf("SS_XMP_DeInit fail s32ret=[%d]\n", s32ret);
}
```

【相关主题】

无

13.3.3 SS_XMP_Create

【描述】

创建XMP对象并初始化，并且添加命名空间。

【语法】

```
TD_S32 SS_XMP_Create(OT\_XMP\_ATTR\_S *pstAttr, TD\_MW\_PTR *ppXMP);
```

【参数】

参数名称	描述	输入/输出
pstAttr	Namespace 参数	输入
ppXMP	XMP模块句柄	输出

【返回值】

返回值	描述
0	成功。



返回值	描述
非0	失败，参见 错误码 。

【需求】

- 头文件：ss_xmp.h
- 库文件：libxmp.a、libxmp.so

【注意】

- 命名空间限制大小128；该模块支持多实例，最多支持10个XMP实例。
- 当创建多个实例时，多个实例之间的Key值与NameSpaces值二者不能有一个相同，不然会创建失败；二者全部相同时，创建时则会进行覆盖。
- 属性aszPrefix 不能含有特殊符号，例如*、@、#、¥、%等特殊符号，且不能以数字开头否则会创建失败。属性aszURI 则没有要求。
- stAttr.enXMPAttrType = OT_XMP_ATTR_WRITER，此对象为封装XMP协议数据；
- stAttr.enXMPAttrType = OT_XMP_ATTR_READER，此对象为解析XMP协议数据。

【举例】

本例中演示了XMP模块的创建XMP对象过程，具体可以参见本模块的sample代码。

```
OT_XMP_HANDLE hXMPHandle = NULL;
stAttr.enXMPAttrType = OT_XMP_ATTR_WRITER;
stAttr.stWriterAttr.s32NSCnt = 1;
stAttr.stWriterAttr.pstNameSpaces[0] = (OT_XMP_NS_S*)malloc(sizeof(OT_XMP_NS_S));
if (NULL == stAttr.stWriterAttr.pstNameSpaces)
{
    return TD_FAILURE;
}
(TD_VOID)strncpy(stAttr.stWriterAttr.pstNameSpaces[0]->aszPrefix, "GSpherical",
strlen( "GSpherical"));
(TD_VOID)strncpy(stAttr.stWriterAttr.pstNameSpaces[0]->aszURI, "http://ns.google.com/
videos/1.0/spherical/", strlen( "http://ns.google.com/videos/1.0/spherical/"));
if (TD_SUCCESS!= SS_XMP_Create(&stAttr, &hXMPHandle))
{
    FREE_ARRAY(i, stAttr.stWriterAttr.pstNameSpaces, stAttr.stWriterAttr.s32NSCnt);
    return TD_FAILURE;
}
```

【相关主题】

无

13.3.4 SS_XMP_Destroy

【描述】

销毁XMP实例。

【语法】

```
TD_S32 SS_XMP_Destroy(TD_MW_PTR pXMP);
```

【参数】



参数名称	描述	输入/输出
pXMP	XMP模块句柄	输入

【返回值】

返回值	描述
0	成功。
非0	失败，参见 错误码 。

【需求】

- 头文件：ss_xmp.h
- 库文件：libxmp.a、libxmp.so

【注意】

无

【举例】

本例中演示了XMP模块的销毁XMP对象过程，具体可以参见本模块的sample代码。

```
if ( TD_SUCCESS!= SS_XMP_Destroy(hXMPHandle))
{
    FREE_ARRAY(i, stAttr.stWriterAttr.pstNameSpaces, stAttr.stWriterAttr.s32NSCnt);
    free(pBuf);
    pBuf = NULL;
    return TD_FAILURE;
}
```

【相关主题】

无

13.3.5 SS_XMP_GetProperty

【描述】

获取XMP属性值。

【语法】

```
TD_S32 SS_XMP_GetProperty(TD\_MW\_PTR pXMP, const TD_CHAR* pszNameSpace,const  
TD_CHAR* pszkey, TD_CHAR* pszValue, TD_S32 *ps32ValueBuflen);
```

【参数】

参数名称	描述	输入/输出
pXMP	Xmp模块句柄	输入



参数名称	描述	输入/输出
pszNameSpace	命名空间	输入
pszkey	属性	输入
pszValue	获取的属性值	输出
ps32ValueBuflen	获取属性值输入为需要分配的空间/输出为分配的空间	输入/输出

【返回值】

返回值	描述
0	成功。
非0	失败，参见 错误码 。

【需求】

- 头文件：ss_xmp.h
- 库文件：libxmp.a、libxmp.so

【注意】

当分配的Buff的长度过小（即小于Value的长度），无法存放对应的数据时，但是ValueBuffLen长度可以正确输出，但是Buff不会存放数据。

【举例】

无

【相关主题】

无

13.3.6 SS_XMP_AddProperty

【描述】

添加XMP属性和属性值。

【语法】

```
TD_S32 SS_XMP_AddProperty(TD\_MW\_PTR pXMP, const TD_CHAR* pszNameSpace,const TD_CHAR* pszkey,const TD_CHAR* pszValue);
```

【参数】

参数名称	描述	输入/输出
pXMP	Xmp模块句柄	输入



参数名称	描述	输入/输出
pszNameSpace	命名空间	输入
pszkey	属性	输入
pszValue	属性值	输入

【返回值】

返回值	描述
0	成功。
非0	失败，参见 错误码 。

【需求】

- 头文件：ss_xmp.h
- 库文件：libxmp.a、libxmp.so

【注意】

当添加属性值的时候，所添加的pszkey不能包含特殊符号，例如*、@、#、¥、%等特殊符号，也不支持以数字开头，否则会添加失败。

【举例】

本例中演示了XMP模块的添加属性和属性值过程，具体可以参见本模块的sample代码。

```
if (TD_SUCCESS!= SS_XMP_AddProperty(hXMPHandle, stAttr.stWriterAttr.pstNameSpaces[0]->aszURI, "ProjectionType", "equirectangular"))
{
    FREE_ARRAY(i, stAttr.stWriterAttr.pstNameSpaces, stAttr.stWriterAttr.s32NSCnt);
    return TD_FAILURE;
}
```

【相关主题】

无

13.3.7 SS_XMP_ToString

【描述】

获取XMP数据拼装后的XML数据。

【语法】

```
TD_S32 SS_XMP_ToString(TD\_MW\_PTR pXMP, TD_CHAR* pszBuf, TD_S32 *ps32Buflen);
```

【参数】



参数名称	描述	输入/输出
pXMP	Xmp模块句柄	输入
pszBuf	获取拼装后的数据	输出
ps32Buflen	获取属性值输入为需要分配的空间/输出为分配的空间	输入/输出

【返回值】

返回值	描述
0	成功。
非0	失败，参见 错误码 。

【需求】

- 头文件：ss_xmp.h
- 库文件：libxmp.a、libxmp.so

【注意】

当分配的Buff的长度过小（即小于Value的长度），无法存放对应的数据时，ps32Buflen长度可以正确输出，但是Buff不会存放数据。

【举例】

本例中演示了XMP模块的获取XMP数据拼装后的XML数据过程，具体可以参见本模块的sample代码。

```
if (TD_SUCCESS!= SS_XMP_ToString(hXMPhandle, &cTmp, &len))
{
    FREE_ARRAY(i, stAttr.stWriterAttr.pstNameSpaces, stAttr.stWriterAttr.s32NSCnt);
    return TD_FAILURE;
}
```

【相关主题】

无

13.3.8 SS_XMP_ForeachKeyValue

【描述】

遍历XMP属性和属性值。

【语法】

```
TD_S32 SS_XMP_ForeachKeyValue(TD\_MW\_PTR pXMP, const TD_CHAR* pszNameSpace,
OT\_XMP\_HANDLE\_KEYVALUE\_FN FnKeyValue);
```

【参数】



参数名称	描述	输入/输出
pXMP	Xmp模块句柄	输入
pszNameSpace	命名空间	输出
FnKeyValue	遍历属性和属性值的回调函数	输入/输出

【返回值】

返回值	描述
0	成功。
非0	失败，参见 错误码 。

【需求】

- 头文件：ss_xmp.h
- 库文件：libxmp.a、libxmp.so

【注意】

无

【举例】

本例中演示了遍历XMP属性和属性值的过程，具体可以参见本模块的sample代码。

```
if (TD_SUCCESS!= SS_XMP_ForeachKeyValue(hXMPhandle, kSpherical, Proc_KeyValue))
{
    printf("SS_XMP_Create fail \n");
    free(stAttr.stReaderAttr.pszInputBuf);
    stAttr.stReaderAttr.pszInputBuf= NULL;
    return TD_FAILURE;
}
```

【相关主题】

无

13.4 数据结构

相关数据类型、数据结构定义如下：

- [TD_MW_PTR](#)：XMP模块句柄。
- [OT_XMP_ATTR_S](#)：XMP配置参数结构体。
- [OT_XMP_READER_ATTRS_S](#)：解析XMP协议数据结构体。
- [OT_XMP_WRITER_ATTRS_S](#)：封装XMP协议数据结构体。
- [OT_XMP_NS_S](#)：命名空间注册结构体。
- [OT_XMP_HANDLE_KEYVALUE_FN](#)：遍历属性和属性值的回调函数。



- [OT_XMP_ATTR_TYPE_E](#): XMP对象功能类型枚举。
- [OT_XMP_MAX_KEY_PATH](#): KEY最大长度。
- [OT_XMP_MAX_NAMESPACE_PATH](#): 命名空间最大长度。
- [OT_XMP_MAX_NAMESPACE_NUM](#): 命名空间可封装最大个数。

13.4.1 TD_MW_PTR

【说明】

XMP模块句柄。

【定义】

```
typedef TD_VOID* TD_MW_PTR;
```

【注意事项】

无

【相关数据类型及接口】

无

13.4.2 OT_XMP_ATTR_S

【说明】

XMP配置参数结构体。

【定义】

```
typedef struct XMP_ATTR_S
{
    OT\_XMP\_ATTR\_TYPE\_E enXMPAttrType;
    union {
        OT\_XMP\_READER\_ATTRS\_S stReaderAttr;
        OT\_XMP\_WRITER\_ATTRS\_S stWriterAttr;
    };
}OT_XMP_ATTR_S;
```

【成员】

成员名称	描述
enXMPAttrType	XMP实例类型
stReaderAttr	解析XMP结构体
stWriterAttr	封装XMP结构体

【注意事项】

无

【相关数据类型及接口】

无



13.4.3 OT_XMP_READER_ATTRS_S

【说明】

解析XMP协议数据结构体。

【定义】

```
typedef struct XMP_READER_ATTRS_S
{
    TD_S32 s32InputBufLen;
    TD_CHAR *pszInputBuf;
} OT_XMP_READER_ATTRS_S;
```

【成员】

成员名称	描述
s32InputBufLen	需解析数据长度
pszInputBuf	需解析数据

【注意事项】

无

【相关数据类型及接口】

无。

13.4.4 OT_XMP_WRITER_ATTRS_S

【说明】

封装XMP协议数据结构体。

【定义】

```
typedef struct XMP_WRITER_ATTRS_S
{
    TD_S32 s32NSCnt;
    OT_XMP_NS_S *pstNameSpaces[OT_XMP_MAX_NAMESPACE_NUM];
} OT_XMP_WRITER_ATTRS_S;
```

【成员】

成员名称	描述
s32NSCnt	注册namespace个数[0, 10]
pstNameSpaces	注册namespace结构体

【注意事项】

无

【相关数据类型及接口】

无



13.4.5 OT_XMP_NS_S

【说明】

命名空间注册结构体。

【定义】

```
typedef struct XMP_NS_S
{TD_CHAR aszPrefix[OT_XMP_MAX_KEY_PATH]; /*key (0,64)*/TD_CHAR
aszURI[OT_XMP_MAX_NAMESPACE_PATH];/*namespace (0,128)*/
} OT_XMP_NS_S;
```

【成员】

成员名称	描述
aszPrefix	注册的key值
aszURI	注册的namespace

【注意事项】

无

【相关数据类型及接口】

无

13.4.6 OT_XMP_HANDLE_KEYVALUE_FN

【说明】

遍历属性和属性值的回调函数。

【定义】

```
typedef TD_S32 (*OT_XMP_HANDLE_KEYVALUE_FN)(const TD_CHAR *pszkey, const TD_CHAR
*pszValue);
```

【成员】

成员名称	描述
pszkey	遍历属性
pszValue	属性值

【注意事项】

无

【相关数据类型及接口】

无



13.4.7 OT_XMP_ATTR_TYPE_E

【说明】

XMP对象功能类型枚举。

【定义】

```
typedef enum XMP_ATTR_TYPE_E
{OT_XMP_ATTR_READER = 0,
  OT_XMP_ATTR_WRITER,
  OT_XMP_ATTR_BUTT
} OT_XMP_ATTR_TYPE_E;
```

【成员】

成员名称	描述
OT_XMP_ATTR_READER	XMP对象为解析类型。
OT_XMP_ATTR_WRITER	XMP对象为封装类型。
OT_XMP_ATTR_BUTT	无效值。

【注意事项】

无

【相关数据类型及接口】

无

13.4.8 OT_XMP_MAX_KEY_PATH

【说明】

KEY最大长度。

【定义】

```
#define OT_XMP_MAX_KEY_PATH (64)
```

【注意事项】

无

【相关数据类型及接口】

无

13.4.9 OT_XMP_MAX_NAMESPACE_PATH

【说明】

命名空间最大长度。

【定义】

```
#define OT_XMP_MAX_NAMESPACE_PATH (128)
```



【注意事项】

无

【相关数据类型及接口】

无

13.4.10 OT_XMP_MAX_NAMESPACE_NUM

【说明】

命名空间可封装最大个数。

【定义】

```
#define OT_XMP_MAX_NAMESPACE_NUM (10)
```

【注意事项】

无

【相关数据类型及接口】

无

13.5 错误码

XMP封装API错误码如表13-1所示。

表 13-1 XMP 封装 API 错误码

错误代码	宏定义	描述
0	TD_SUCCESS	成功
-1	TD_FAILURE	失败
0xA12B8061	OT_ERR_XMP_EINVAL_PAEAMETER	非法参数
0xA12B8062	OT_ERR_XMP_NULL_PTR	指针为空
0xA12B8063	OT_ERR_XMP_NOMEM	内存不够，或者申请内存失败。
0xA12B8064	OT_ERR_XMP_GET_PAEAMETER	获取参数失败
0xA12B8065	OT_ERR_XMP_INIT	XMP初始化失败
0xA12B8066	OT_ERR_XMP_INSTANTE	XMP实例化失败
0xA12B8067	OT_ERR_XMP_DEINIT	XMP去初始化失败
0xA12B8068	OT_ERR_XMP_TYPE	XMP实例功能类型错误



13.6 典型场景使用指南

参见版本sample。

13.7 第三方库依赖说明

XMP封装模块依赖Adobe提供的开源第三方代码，库名为xmp-toolkit-sdk；本章节将详细介绍如何移植xmp-toolkit-sdk。

- xmp-toolkit-sdk 源码下载
下载地址：<https://www.adobe.com/devnet/xmp/sdk/eula-cc20168.html>
获取到文件名为XMP-Toolkit-SDK-CC201607.zip，然后移到linux虚拟机里解压
解压后有的文件目录如下：

```
1535 Aug  3 2016 BSD-License.txt
4096 Jul 26 17:29 build
4096 Jun 13 18:54 docs
4096 Jun 14 10:55 public
4096 Jun 13 18:54 samples
4096 Jun 19 11:46 source
4096 Jun 14 10:32 third-party
4096 Jun 13 18:54 tools
4096 Jun 13 18:54 XMPCommon
4096 Jun 13 18:54 XMPCore
4096 Jun 13 18:54 XMPFiles
4096 Jun 13 18:55 XMPFilesPlugins
163352 Aug  1 2016 XMP-Toolkit-SDK-Overview.pdf
```

- expat源码下载
下载地址：<http://sourceforge.net/projects/expat/files/expat/2.1.0/>
获取到文件名为expat-2.1.0.tar.gz，然后解压到third-party/expat下
- xmp-toolkit-sdk 源码编译
 - 删除source/Host_IO-POSIX.cpp source/Host_IO-Win.cpp
 - 编辑Makefile
 - 执行make进行编译
- 生成的第三方静态库与动态库在public/lib下libxmpcore.a libxmpcore.so。



14 TS 封装模块

14.1 概述

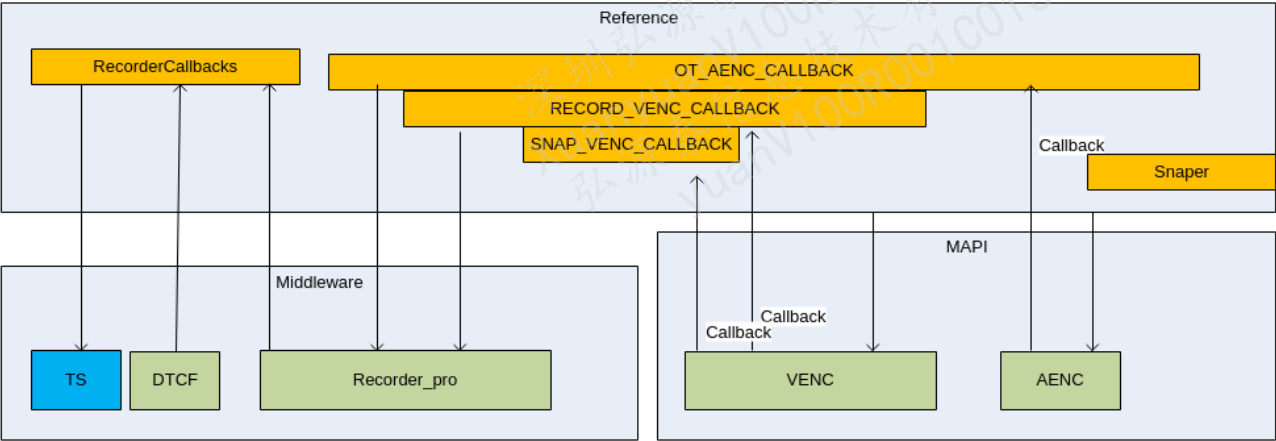
TS封装用于录像时音视频以及私有数据以标准的TS格式封装。

14.2 功能描述

当前TS封装支持标准H.264、H.265视频编码格式，AAC、MP3音频编码格式TS文件封装，同时也支持JPEG私有数据封装。

TS封装组件软件上下文依赖关系如图14-1所示。对下依赖具体封装实现，TS封装根据上层sample app对录像（RECORD）功能调用，依赖文件管理模块提供的文件路径文件名，对接收到的媒体流数据进行标准格式TS文件封装。

图 14-1 TS 封装组件软件上下文依赖关系



14.3 API 参考

该功能模块为用户提供以下API：



- [SS_TS_Create](#): 创建一个TS实例。
- [SS_TS_Destroy](#): 销毁一个TS实例。
- [SS_TS_CreateStream](#): 创建一个TS流。
- [SS_TS_DestroyAllStreams](#): 销毁所有流。
- [SS_TS_WriteFrame](#): 写入一帧数据。
- [SS_TS_GetFileSize](#): 获取文件大小。

14.3.1 SS_TS_Create

【描述】

创建一个TS实例。

【定义】

```
TD_S32 SS_TS_Create(const OT_TS_CONFIG_S* pstTsCfg, TD_MW_PTR* ppTsMuxer);
```

【参数】

参数名称	描述	输入/输出
pstTsCfg	配置参数。	输入
ppTsMuxer	TS实例句柄指针。	输出

【返回值】

返回值	描述
0	成功。
非0	失败，参见 错误码 。

【需求】

- 头文件：ss_ts_format.h
- 库文件：libts.a、libts.so

【注意】

- 支持多实例，最多可以创建[OT_TS_MAX_AMOUNT](#)个实例。
- 当前暂时不支持解封装相关功能。
- 创建文件时，全路径文件名存在时会进行写覆盖。

【举例】

无。

14.3.2 SS_TS_Destroy

【描述】



销毁一个TS实例。

【定义】

```
TD_S32 SS_TS_Destroy(TD_MW_PTR pTsMuxer);
```

【参数】

参数名称	描述	输入/输出
pTsMuxer	TS封装实例句柄。	输入

【返回值】

返回值	描述
0	成功。
非0	失败，参见 错误码 。

【需求】

- 头文件：ss_ts_format.h
- 库文件：libts.a、libts.so

【注意】

无。

【举例】

无。

14.3.3 SS_TS_CreateStream

【描述】

创建一个TS流。

【定义】

```
TD_S32 SS_TS_CreateStream(TD_MW_PTR pTsMuxer, TD_MW_PTR* ppStream, const  
OT\_TS\_STREAM\_INFO\_S* pstStreamInfo);
```

【参数】

参数名称	描述	输入/输出
pTsMuxer	TS封装实例句柄。	输入
ppStream	TS流句柄指针。	输出
pstStreamInfo	创建的流的信息指针。	输入



【返回值】

返回值	描述
0	成功。
非0	失败，参见 错误码 。

【需求】

- 头文件：ss_ts_format.h
- 库文件：libts.a、libts.so

【注意】

无。

【举例】

无。

14.3.4 SS_TS_DestroyAllStreams

【描述】

销毁所有流。

【定义】

```
TD_S32 SS_TS_DestroyAllStreams(TD_MW_PTR pTsMuxer);
```

【参数】

参数名称	描述	输入/输出
pTsMuxer	TS封装实例句柄。	输入

【返回值】

返回值	描述
0	成功。
非0	失败，参见 错误码 。

【需求】

- 头文件：ss_ts_format.h
- 库文件：libts.a、libts.so

【注意】



该接口功能为销毁TS文件中PMT表中所有的流，请在调用SS_TS_Destroy前调用该接口，确认所有的流被销毁。

【举例】

无。

14.3.5 SS_TS_WriteFrame

【描述】

写入一帧数据。

【定义】

```
TD_S32 SS_TS_WriteFrame(TD_MW_PTR pTsMuxer, TD_MW_PTR pStream, const
OT_TS_FRAME_DATA_S* pstFrameData);
```

【参数】

参数名称	描述	输入/输出
pTsMuxer	TS封装实例句柄。	输入
pStream	TS流句柄。	输入
pstFrameData	帧数据指针。	输入

【返回值】

返回值	描述
0	成功。
非0	失败，参见 错误码 。

【需求】

- 头文件：ss_ts_format.h
- 库文件：libts.a、libts.so

【注意】

无。

【举例】

无。

14.3.6 SS_TS_GetFileSize

【描述】

获取文件大小。

【定义】



```
TD_U64 SS_TS_GetFileSize(TD_MW_PTR pTsMuxer);
```

【参数】

参数名称	描述	输入/输出
pTsMuxer	TS封装实例句柄。	输入

【返回值】

返回值	描述
非0	文件大小。

【需求】

- 头文件：ss_ts_format.h
- 库文件：libts.a、libts.so

【注意】

无。

【举例】

无。

14.4 数据类型

TS封装模块相关数据类型定义如下：

- **OT_TS_MAX_AMOUNT**：定义TS封装实例的最大数量。
- **OT_TS_MAX_STREAM_AMOUNT**：定义每个TS封装实例的TS流的最大数量。
- **OT_TS_MAX_FILE_NAME**：定义TS文件名称的最大长度。
- **OT_TS_VBUF_MAX_SIZE**：定义TS文件设置Vbuffer的最大值，单位字节。
- **OT_TS_PREALLOC_MAX_SIZE**：定义TS文件预分配的最大值，单位字节。
- **OT_TS_STREAM_TYPE_E**：定义TS流的类型集。
- **OT_TS_CODECID_E**：定义TS支持的编码类型集。
- **OT_TS_STREAM_INFO_S**：定义TS流信息结构体。
- **OT_TS_FRAME_DATA_S**：定义TS帧数据信息结构体。
- **OT_TS_MUXER_CONFIG_S**：定义TS封装的配置参数结构体。
- **OT_TS_DEMUXER_CONFIG_S**：定义TS解封装的配置参数结构体。
- **OT_TS_CONFIG_S**：定义TS模块的配置参数结构体。
- **OT_TS_CONFIG_TYPE_E**：定义TS配置类型。



14.4.1 OT_TS_MAX_AMOUNT

【说明】

定义TS封装实例的最大数量。

【定义】

```
#define OT_TS_MAX_AMOUNT (4)
```

【注意事项】

无。

【相关数据结构及接口】

无。

14.4.2 OT_TS_MAX_STREAM_AMOUNT

【说明】

定义每个TS封装实例支持的TS流的最大数量。

【定义】

```
#define OT_TS_MAX_STREAM_AMOUNT (6)
```

【注意事项】

无。

【相关数据类型及接口】

无。

14.4.3 OT_TS_MAX_FILE_NAME

【说明】

定义TS文件名称的最大长度。

【定义】

```
#define OT_TS_MAX_FILE_NAME (256)
```

【注意事项】

无。

【相关数据类型及接口】

无。

14.4.4 OT_TS_VBUF_MAX_SIZE

【说明】

定义TS文件设置Vbuffer的最大值，单位字节。

【定义】

```
#define OT_TS_VBUF_MAX_SIZE (5*1024*1024)
```



【注意事项】

无。

【相关数据类型及接口】

无。

14.4.5 OT_TS_PREALLOC_MAX_SIZE

【说明】

定义TS文件预分配的最大值，单位字节。

【定义】

```
#define OT_TS_PREALLOC_MAX_SIZE (100*1024*1024)
```

【注意事项】

无。

【相关数据类型及接口】

无。

14.4.6 OT_TS_STREAM_TYPE_E

【说明】

定义TS流的类型集。

【定义】

```
typedef enum TS_STREAM_TYPE_E {  
    OT_TS_STREAM_VIDEO = 1,  
    OT_TS_STREAM_AUDIO,  
    OT_TS_STREAM_DATA,  
    OT_TS_STREAM_BUTT  
} OT_TS_STREAM_TYPE_E;
```

【成员】

成员名称	描述
OT_TS_STREAM_VIDEO	视频流类型。
OT_TS_STREAM_AUDIO	音频流类型。
OT_TS_STREAM_DATA	自定义数据流类型。
OT_TS_STREAM_BUTT	无效TS流类型。

【注意事项】

无。

【相关数据类型及接口】

无。



14.4.7 OT_TS_CODECID_E

【说明】

定义TS支持的编码类型集。

【定义】

```
typedef enum TS_CODECID_E {  
    OT_TS_CODECID_ID_H264 = 1,  
    OT_TS_CODECID_ID_H265,  
    OT_TS_CODECID_ID_AAC,  
    OT_TS_CODECID_ID_MP3,  
    OT_TS_CODECID_ID_JPG,  
    OT_TS_CODECID_ID_GPS,  
    OT_TS_CODECID_ID_BUTT  
} OT_TS_CODECID_E;
```

【成员】

成员名称	描述
OT_TS_CODECID_ID_H264	H.264编码类型。
OT_TS_CODECID_ID_H265	H.265编码类型。
OT_TS_CODECID_ID_AAC	AAC编码类型。
OT_TS_CODECID_ID_MP3	MP3编码类型。
OT_TS_CODECID_ID_JPG	JPEG编码类型。
OT_TS_CODECID_ID_GPS	GPS编码类型。
OT_TS_CODECID_ID_BUTT	无效编码类型。

【注意事项】

当前支持视频类型：H.264和H.265，当前支持音频类型AAC、MP3，当前支持私有数据类型JPEG、GPS。其他类型暂时不支持。

【相关数据类型及接口】

无。

14.4.8 OT_TS_STREAM_INFO_S

【说明】

定义TS流信息结构体。

【定义】

```
typedef struct TS_STREAM_INFO_S {  
    OT_TS_CODECID_E enCodecID; /*codec type*/  
    TD_FLOAT fSpeed; /* play speed, (0,1]for slow,(1,~) for fast */  
} OT_TS_STREAM_INFO_S;
```

【成员】



成员名称	描述
enCodecID	编码类型。
fSpeed	设置播放速度

【注意事项】

无。

【相关数据类型及接口】

无。

14.4.9 OT_TS_FRAME_DATA_S

【说明】

定义TS帧数据信息结构体。

【定义】

```
typedef struct TS_FRAME_DATA_S {  
    TD_U64 u64TimeStamp; /*frame timestamp*/  
    TD_U8* pu8DataBuffer; /*frame data buffer*/  
    TD_U32 u32DataLength; /*frame data len*/  
    TD_BOOL bKeyFrameFlag; /*key frame flag, used for video*/  
} OT_TS_FRAME_DATA_S;
```

【成员】

成员名称	描述
u64TimeStamp	帧数据的时间戳。
pu8DataBuffer	帧数据的数据缓冲区指针。
u32DataLength	帧数据的数据长度。
bKeyFrameFlag	是否关键帧。 TD_TRUE：是； TD_FALSE：否。

【注意事项】

无。

【相关数据类型及接口】

无。

14.4.10 OT_TS_MUXER_CONFIG_S

【说明】



定义TS封装的配置参数结构体。

```
【定义】
typedef struct TS_MUXER_CONFIG_S {
    TD_U32 u32PreAllocUnit;
    TD_U32 u32VBufSize;
} OT_TS_MUXER_CONFIG_S;
```

【成员】

成员名称	描述
u32PreAllocUnit	预分配单元大小。为0时表示不开启预分配功能，取值范围：[0,100M]，单位：字节。 注意：由于ts是流式格式，预分配模式的启用将对其解析末尾数据造成影响，请客户酌情开启。
u32VBufSize	写数据缓冲区大小，取值范围：(0, 5M]。单位：字节。

【注意事项】

无。

【相关数据类型及接口】

无。

14.4.11 OT_TS_DEMUXER_CONFIG_S

【说明】

定义TS解封装的配置参数结构体。

【定义】

```
typedef struct TS_DEMUXER_CONFIG_S {
    TD_U32 u32VBufSize;
} OT_TS_DEMUXER_CONFIG_S;
```

【成员】

成员名称	描述
u32VBufSize	读数据缓冲区大小，取值范围：(0, 5M]。单位：字节

【注意事项】

当前暂时不支持demuxer相关功能。

【相关数据类型及接口】

无。



14.4.12 OT_TS_CONFIG_S

【说明】

定义TS模块的配置参数结构体。

【定义】

```
typedef struct TS_CONFIG_S {
    TD_CHAR aszFileName[OT_TS_MAX_FILE_NAME];/*file path and file name*/
    OT_TS_CONFIG_TYPE_E enConfigType;
    union
    {
        OT_TS_MUXER_CONFIG_S stMuxerConfig;
        OT_TS_DEMUXER_CONFIG_S stDemuxerConfig;
    };
} OT_TS_CONFIG_S;
```

【成员】

成员名称	描述
aszFileName	文件名称。
enConfigType	文件配置类型。
stMuxerConfig	文件封装配置。
stDemuxerConfig	文件解封装配置。

【注意事项】

无。

【相关数据类型及接口】

无。

14.4.13 OT_TS_CONFIG_TYPE_E

【说明】

定义TS配置类型。

【定义】

```
typedef enum TS_CONFIG_TYPE_E {
    OT_TS_CONFIG_MUXER = 1,
    OT_TS_CONFIG_DEMUXER,
    OT_TS_CONFIG_BUTT,
}OT_TS_CONFIG_TYPE_E;
```

【成员】

成员名称	描述
OT_TS_CONFIG_MUXER	TS封装类型。



成员名称	描述
OT_TS_CONFIG_DEMUXER	TS解封装类型。
OT_TS_CONFIG_BUTT	TS配置非法类型。

【注意事项】

无。

【相关数据类型及接口】

无。

14.5 错误码

TS封装的API错误码如表14-1所示。

表 14-1 TS 封装的 API 错误码

错误代码	宏定义	描述
0XA12F8002	OT_ERR_TS_NULL_PTR	空指针错误
0XA12F8040	OT_ERR_TS_HANDLE_INVALID	无效句柄错误
0XA12F8041	OT_ERR_TS_INVALIDARG	无效参数错误
0XA12F8042	OT_ERR_TS_MALLOC	内存申请错误
0XA12F8043	OT_ERR_TS_CREATE_MUXER	创建错误
0XA12F8044	OT_ERR_TS_DESTROY_MUXER	销毁错误
0XA12F8045	OT_ERR_TS_CREATE_STREAM	创建流错误
0XA12F8046	OT_ERR_TS_DESTROY_STREAM	销毁流错误
0XA12F8047	OT_ERR_TS_NOT_CREATE	对象没有创建错误
0XA12F8048	OT_ERR_TS_READ_FRAME	读帧错误
0XA12F8049	OT_ERR_TS_WRITE_FRAME	写帧错误
0XA12F8061	OT_ERR_TS_UNSUPPORT_CODEC	不支持的编码类型
0XA12F8062	OT_ERR_TS_OPEN_FILE	打开文件错误
0XA12F8063	OT_ERR_TS_END_FILE	关闭文件错误
0XA12F8064	OT_ERR_TS_READ_FILE	读取文件错误
0XA12F8065	OT_ERR_TS_WRITE_FILE	写文件错误
0XA12F8066	OT_ERR_TS_FALLOCATE_FILE	文件预分配错误

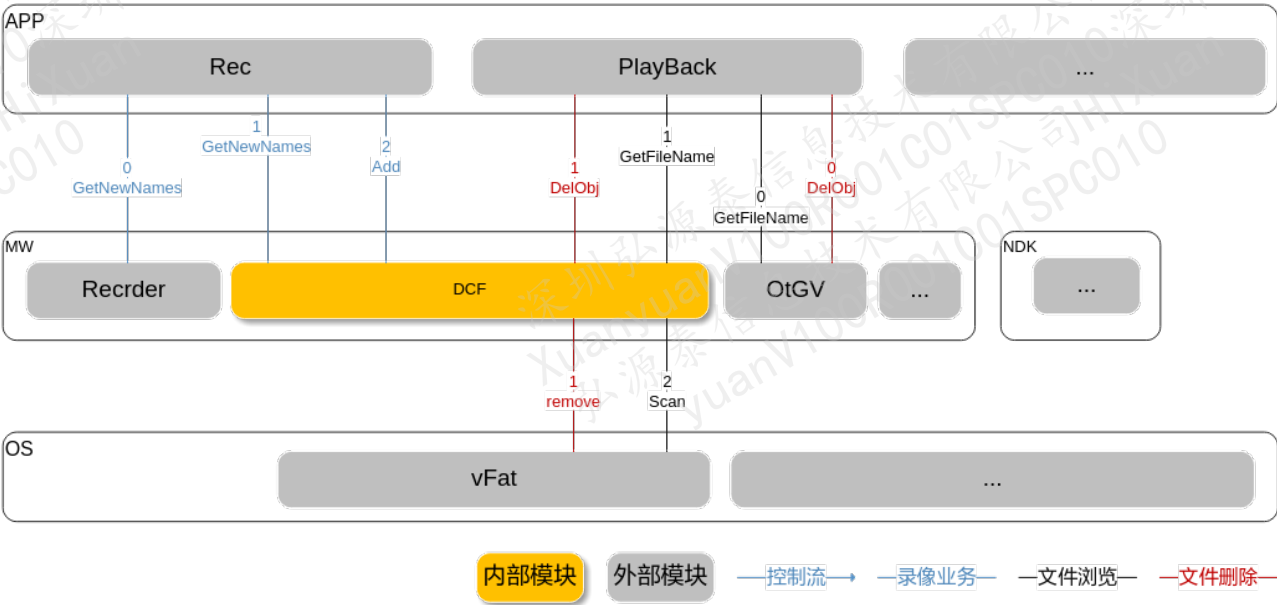


15 DCF

15.1 概述

DCF (Design Rule for Camera File System) 模块是录像、拍照、预览和回放业务的支撑模块。为录像和拍照业务提供文件名；为预览和回放提供检索和删除功能。模块上下文如图15-1所示，展示了录像、文件浏览和文件删除的典型场景的控制流图。

图 15-1 DCF 模块上下文



15.2 功能描述



15.2.1 文件命名

DCF模块约定了目录和文件的命名规则。DCF模块为录像或拍照业务提供符合约定的文件名。

15.2.1.1 DCF 根目录

DCF根目录是存放DCF目录和文件的顶级目录，目录名为“DCIM”。

15.2.1.2 DCF 目录

存储DCF对象的目录称为DCF目录，是DCF根目录的子目录。DCF目录命名规则符合以下条件：

- 目录名长度为8；
- 目录名前段是目录号，由3为数字组成，取值范围：[100, 999]；
- 目录名后段是自由字符，由5位字符组成，字符取值范围为[0,9]、[A,Z]和“_”。

DCF目录命名样例如表15-1所示。

表 15-1 DCF 目录命名样例

规范	目录号			自由字符（5位字符）				
样例	1	0	0	H	S	C	A	M

15.2.1.3 DCF 文件

DCF文件是媒体文件，命名规则符合以下条件：

- 文件名长度为12（包括后缀名，分隔符“.”占去1位）；
- 文件名前段是自由字符，由1位或者4位字符组成，字符取值范围[A,Z]和“_”。如果自由字符长度为1，则后面接着组号，由3位数字组成，组号取值范围为[001,999]；
- 文件名中段是文件号，由4位数字组成，取值范围为[0001,0999]；
- 文件名后段是后缀名，由3位字符组成，字符取值范围[0,9]、[A,Z]和“_”。

DCF文件命名样例如表15-2所示。

表 15-2 DCF 文件命名样例

规范	自由字符（1或4位）				文件号（4位）				分隔符	后缀名（3位）		
样例	L	0	0	1	0	0	0	1	.	M	P	4
样例	N	O	R	M	0	0	0	2	.	M	P	4

15.2.1.4 DCF 对象

在一个DCF目录中，具有相同文件号的文件为一个DCF对象，DCF对象符合以下条件：



- 对象内文件号相同；
- 对象内文件名自由字符长度相同；
- 一个对象最多包含6个文件；
- 一个对象有且只有一个主文件。

15.2.1.5 DCF 组

在一个DCF目录中，对于文件名自由字符长度为1的对象，组号相同的对象为一个组；对于文件名自由字符长度为4的对象，单个对象为一组。

DCF目录与文件结构如图15-2所示。

图 15-2 DCF 目录和文件结构

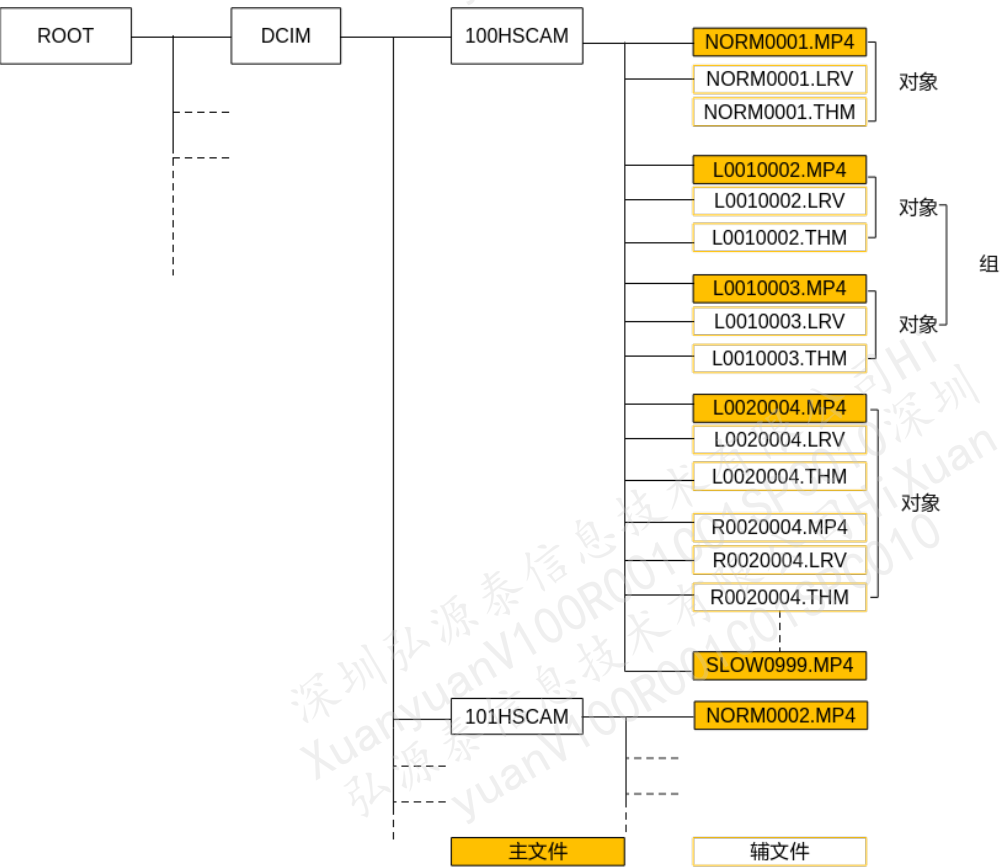


图15-2所示，文件号相同的为一个对象，每个对象中有主文件，如“NORM0001.MP4”就是“0001”这个对象的主文件，“NORM0001.LRV”和“NORM0001.THM”为辅文件。组号相同的所有对象归为一个组，如“001”这个组包含了“0002”和“0003”这两个对象。

15.2.2 文件检索、添加、删除

DCF模块为文件预览回放提供检索和删除功能；为录像或拍照提供添加功能，在业务结束后将生成的媒体文件添加到DCF模块中以供后续检索和删除。



15.2.2.1 检索

DCF模块提供了两种检索方式：

- 从组到对象的嵌套检索，首先检索组，再去检索组中的每一个对象；
- 对象直接检索，用户设置基准对象，以基准对象向前或向后进行检索对象。

15.2.2.2 添加

DCF模块只支持对象添加，不支持组的添加。

15.2.2.3 删除

DCF模块提供了两种删除方式：

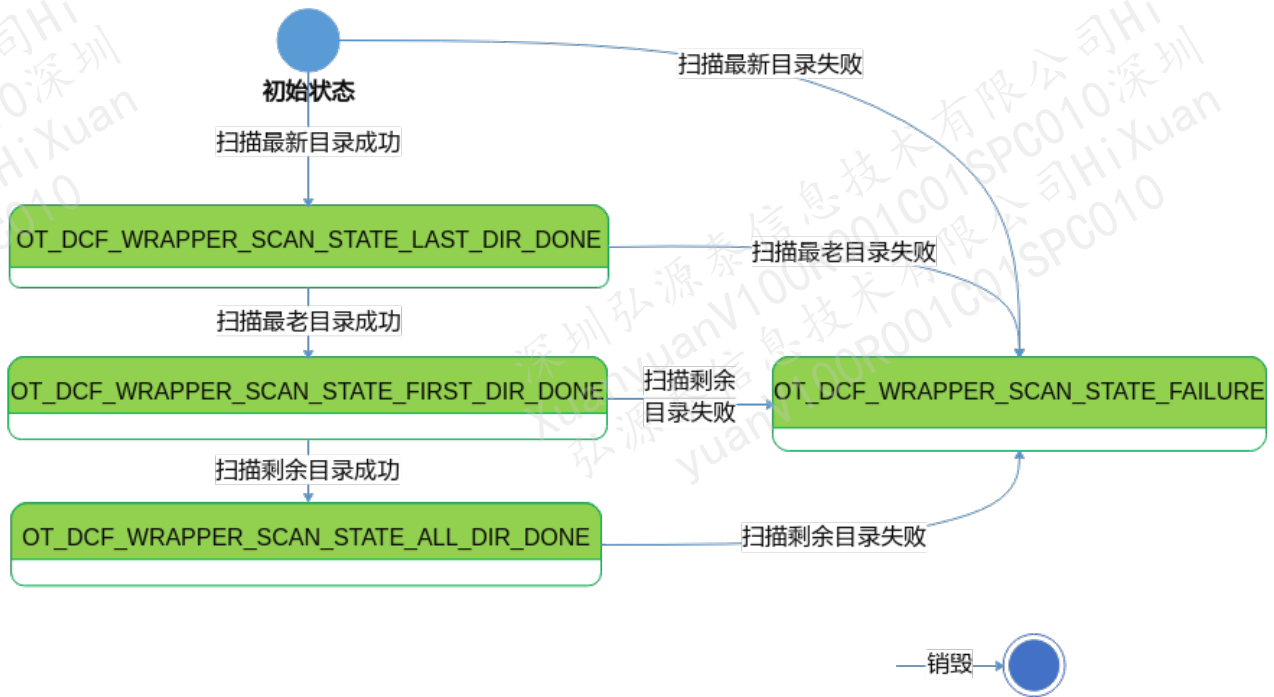
- 组删除，删除组中所有的对象；
- 对象删除，删除指定对象。

须知

DCF实例对外提供的功能，是根据扫描状态决定的，各状态的切换如图15-3所示。

- 当扫描状态为OT_DCF_WRAPPER_SCAN_STATE_LAST_DIR_DONE时，只支持新建对象名和添加对象；
- 当扫描状态为OT_DCF_WRAPPER_SCAN_STATE_FIRST_DIR_DONE时，较前一状态可支持检索和删除；
- 当扫描状态为OT_DCF_WRAPPER_SCAN_STATE_ALL_DIR_DONE时，较前一状态可支持获取组或对象的数量；
- 当扫描状态为OT_DCF_WRAPPER_SCAN_STATE_FAILURE时，代表该DCF实例出现异常，无法提供新建对象、添加、检索和删除的功能，需要销毁实例，重新创建。

图 15-3 各状态的切换图



15.3 API 参考

该功能模块提供以下API：

- **SS_DCF_WRAPPER_Init**：模块初始化。
- **SS_DCF_WRAPPER_DeInit**：模块去初始化。
- **SS_DCF_WRAPPER_Create**：创建DCF实例。
- **SS_DCF_WRAPPER_Destroy**：销毁DCF实例。
- **SS_DCF_WRAPPER_GetFirstGrp**：获取最老组。
- **SS_DCF_WRAPPER_GetLastGrp**：获取最新组。
- **SS_DCF_WRAPPER_GetNextGrp**：向后获取组。
- **SS_DCF_WRAPPER_GetPrevGrp**：向前获取组。
- **SS_DCF_WRAPPER_DelCurGrp**：删除当前组。
- **SS_DCF_WRAPPER_GetTotalGrpCnt**：获取DCF实例中的总组数。
- **SS_DCF_WRAPPER_GetGrpCnt**：获取某一类型的组数。
- **SS_DCF_WRAPPER_GetCurGrpFirstObj**：获取当前组最老对象。
- **SS_DCF_WRAPPER_GetCurGrpLastObj**：获取当前组最新对象。
- **SS_DCF_WRAPPER_GetCurGrpNextObj**：获取当前组当前对象的后一个对象。
- **SS_DCF_WRAPPER_GetCurGrpPrevObj**：获取当前组当前对象的前一个对象。
- **SS_DCF_WRAPPER_GetNextObjList**：向后获取对象列表。
- **SS_DCF_WRAPPER_GetPrevObjList**：向前获取对象列表。



- [SS_DCF_WRAPPER_AddObj](#): 添加对象。
- [SS_DCF_WRAPPER_DelObj](#): 删除对象。
- [SS_DCF_WRAPPER_GetTotalObjCnt](#): 获取DCF实例中的总对象数。
- [SS_DCF_WRAPPER_GetObjCnt](#): 获取某一类型的对象数。
- [SS_DCF_WRAPPER_GetNewFilePaths](#): 获取新对象的文件路径列表。

15.3.1 SS_DCF_WRAPPER_Init

【描述】

模块初始化。

【语法】

```
TD_S32 SS_DCF_WRAPPER_Init(TD_VOID);
```

【参数】

无。

【返回值】

返回值	描述
0	成功。
非0	失败，参见 错误码 。

【需求】

- 头文件: `ss_dcf_wrapper.h`
- 库文件: `libdcf_impl.a`、`libdcf_impl.so`、`libdcf_core.a`、`libdcf_core.so`、`libdcf_wrapper.a`、`libdcf_wrapper.so`

【注意】

无。

【举例】

参见本模块[sample](#)。

【相关主题】

无。

15.3.2 SS_DCF_WRAPPER_DeInit

【描述】

模块去初始化。

【语法】

```
TD_S32 SS_DCF_WRAPPER_DeInit(TD_VOID);
```

【参数】



无。

【返回值】

返回值	描述
0	成功。
非0	失败，参见 错误码 。

【需求】

- 头文件：ss_dcf_wrapper.h
- 库文件：libdcf_impl.a、libdcf_impl.so、libdcf_core.a、libdcf_core.so、libdcf_wrapper.a、libdcf_wrapper.so

【注意】

无。

【举例】

参见本模块sample。

【相关主题】

无。

15.3.3 SS_DCF_WRAPPER_Create

【描述】

创建DCF实例。实例是DCF模块对外提供功能的操作实体，DCF模块支持多实例，各实例之间互不干扰。

【语法】

```
TD_S32 SS_DCF_WRAPPER_Create(const OT_DCF_WRAPPER_ON_SCAN_EVENT_PFN  
pfnScanEvent, const OT_DCF_WRAPPER_NAMERULE_CFG_S* pstCfg,  
OT_DCF_WRAPPER_INDEX* piDCF);
```

【参数】

参数名称	描述	输入/输出
pfnScanEvent	扫描事件回调函数。	输入
pstCfg	命名规则配置属性。	输入
piDCF	DCF实例索引。	输出

【返回值】



返回值	描述
0	成功。
非0	失败，返回 错误码 。

【需求】

- 头文件：ss_dcf_wrapper.h
- 库文件：libdcf_impl.a、libdcf_impl.so、libdcf_core.a、libdcf_core.so、libdcf_wrapper.a、libdcf_wrapper.so

【注意】

- 最多可创建[OT_DCF_WRAPPER_MAX_INSTANCE](#)个实例；
- 各实例pstCfg的根目录路径互斥；
- pfnScanEvent可为空。

【举例】

参见本模块sample。

【相关主题】

无。

15.3.4 SS_DCF_WRAPPER_Destroy

【描述】

销毁DCF实例。

【语法】

```
TD_S32 SS_DCF_WRAPPER_Destroy(OT\_DCF\_WRAPPER\_INDEX iDCF);
```

【参数】

参数名称	描述	输入/输出
iDCF	DCF实例索引。	输入

【返回值】

返回值	描述
0	成功。
非0	失败，返回 错误码 。

【需求】



- 头文件：ss_dcf_wrapper.h
- 库文件：libdcf_impl.a、libdcf_impl.so、libdcf_core.a、libdcf_core.so、libdcf_wrapper.a、libdcf_wrapper.so

【注意】

无。

【举例】

参见本模块sample。

【相关主题】

无。

15.3.5 SS_DCF_WRAPPER_GetFirstGrp

【描述】

获取最老组。

【语法】

```
TD_S32 SS_DCF_WRAPPER_GetFirstGrp(OT_DCF_WRAPPER_INDEX iDCF,  
OT_DCF_WRAPPER_GRP_S* pstGrp);
```

【参数】

参数名称	描述	输入/输出
iDCF	DCF实例索引。	输入
pstGrp	组。	输出

【返回值】

返回值	描述
0	成功。
非0	失败，返回错误码。

【需求】

- 头文件：ss_dcf_wrapper.h
- 库文件：libdcf_impl.a、libdcf_impl.so、libdcf_core.a、libdcf_core.so、libdcf_wrapper.a、libdcf_wrapper.so

【注意】

- 此接口必须在OT_DCF_WRAPPER_SCAN_STATE_FIRST_DIR_DONE之后调用；
- 获取出来的组作为实例的当前组。

【举例】



参见本模块sample。

【相关主题】

无。

15.3.6 SS_DCF_WRAPPER_GetLastGrp

【描述】

获取最新组。

【语法】

```
TD_S32 SS_DCF_WRAPPER_GetLastGrp(OT_DCF_WRAPPER_INDEX iDCF,  
OT_DCF_WRAPPER_GRP_S* pstGrp);
```

【参数】

参数名称	描述	输入/输出
iDCF	DCF实例索引。	输入
pstGrp	组。	输出

【返回值】

返回值	描述
0	成功。
非0	失败，返回错误码。

【需求】

- 头文件：ss_dcf_wrapper.h
- 库文件：libdcf_impl.a、libdcf_impl.so、libdcf_core.a、libdcf_core.so、libdcf_wrapper.a、libdcf_wrapper.so

【注意】

- 此接口必须在OT_DCF_WRAPPER_SCAN_STATE_FIRST_DIR_DONE之后调用；
- 获取出来的组作为实例的当前组。

【举例】

参见本模块sample。

【相关主题】

无。

15.3.7 SS_DCF_WRAPPER_GetNextGrp

【描述】



获取当前组之后的第N个组。

【语法】

```
TD_S32 SS_DCF_WRAPPER_GetNextGrp(OT_DCF_WRAPPER_INDEX iDCF, TD_U32 u32Interval,  
OT_DCF_WRAPPER_GRP_S* pstGrp);
```

【参数】

参数名称	描述	输入/输出
iDCF	DCF实例索引。	输入
u32Interval	组间隔。	输入
pstGrp	组	输出

【返回值】

返回值	描述
0	成功。
非0	失败，返回错误码。

【需求】

- 头文件：ss_dcf_wrapper.h
- 库文件：libdcf_impl.a、libdcf_impl.so、libdcf_core.a、libdcf_core.so、libdcf_wrapper.a、libdcf_wrapper.so

【注意】

- 向后获取的基准是当前组，故此接口在SS_DCF_WRAPPER_GetFirstGrp或SS_DCF_WRAPPER_GetLastGrp之后调用；
- 接口调用成功后，当前组变更为获取出来的组，作为下一次检索的基准；
- 如果u32Interval大于当前组之后的组数，则报错，且当前组不变更。

【举例】

参见本模块sample。

【相关主题】

无。

15.3.8 SS_DCF_WRAPPER_GetPrevGrp

【描述】

获取当前组之前的第N个组。

【语法】



TD_S32 SS_DCF_WRAPPER_GetPrevGrp([OT_DCF_WRAPPER_INDEX](#) iDCF, TD_U32 u32Interval, [OT_DCF_WRAPPER_GRP_S*](#) pstGrp);

【参数】

参数名称	描述	输入/输出
iDCF	DCF实例索引。	输入
u32Interval	组间隔。	输入
pstGrp	组	输出

【返回值】

返回值	描述
0	成功。
非0	失败，返回 错误码 。

【需求】

- 头文件：ss_dcf_wrapper.h
- 库文件：libdcf_impl.a、libdcf_impl.so、libdcf_core.a、libdcf_core.so、libdcf_wrapper.a、libdcf_wrapper.so

【注意】

- 向前获取的基准是当前组，故此接口在[SS_DCF_WRAPPER_GetFirstGrp](#)或[SS_DCF_WRAPPER_GetLastGrp](#)之后调用；
- 接口调用成功后，当前组变更为获取出来的组，作为下一次检索的基准；
- 如果u32Interval大于当前组之前的组数，则报错，且当前组不变更。

【举例】

参见本模块sample。

【相关主题】

无。

15.3.9 SS_DCF_WRAPPER_DelCurGrp

【描述】

删除当前组，删除当前组中所有对象在数据库中的记录和存储介质中的实体文件。

【语法】

TD_S32 SS_DCF_WRAPPER_DelCurGrp([OT_DCF_WRAPPER_INDEX](#) iDCF);

【参数】



参数名称	描述	输入/输出
iDCF	DCF实例索引。	输入

【返回值】

返回值	描述
0	成功。
非0	失败，返回 错误码 。

【需求】

- 头文件：ss_dcf_wrapper.h
- 库文件：libdcf_impl.a、libdcf_impl.so、libdcf_core.a、libdcf_core.so、libdcf_wrapper.a、libdcf_wrapper.so

【注意】

当前组删除成功后，当前组不变更，其它组操作接口仍以此组为基准。

【举例】

参见本模块sample。

【相关主题】

无。

15.3.10 SS_DCF_WRAPPER_GetTotalGrpCnt

【描述】

获取DCF实例中的总组数。

【语法】

```
TD_S32 SS_DCF_WRAPPER_GetTotalGrpCnt(OT\_DCF\_WRAPPER\_INDEX iDCF, TD_U32* pu32Cnt);
```

【参数】

参数名称	描述	输入/输出
iDCF	DCF实例索引。	输入
pu32Cnt	总组数	输出

【返回值】



返回值	描述
0	成功。
非0	失败，返回 错误码 。

【需求】

- 头文件：ss_dcf_wrapper.h
- 库文件：libdcf_impl.a、libdcf_impl.so、libdcf_core.a、libdcf_core.so、libdcf_wrapper.a、libdcf_wrapper.so

【注意】

此接口在TD_DCF_WRAPPER_SCAN_STATE_ALL_DIR_DONE后调用。

【举例】

参见本模块sample。

【相关主题】

无。

15.3.11 SS_DCF_WRAPPER_GetGrpCnt

【描述】

获取某个类型的组的数量。

【语法】

```
TD_S32 SS_DCF_WRAPPER_GetGrpCnt(OT\_DCF\_WRAPPER\_INDEX iDCF, TD_U8 u8ObjTypIndex,  
TD_U32 *pu32Cnt);
```

【参数】

参数名称	描述	输入/输出
iDCF	DCF实例索引。	输入
u8ObjTypIndex	DCF对象类型索引。	输入
pu32Cnt	组数	输出

【返回值】

返回值	描述
0	成功。
非0	失败，返回 错误码 。



【需求】

- 头文件：ss_dcf_wrapper.h
- 库文件：libdcf_impl.a、libdcf_impl.so、libdcf_core.a、libdcf_core.so、libdcf_wrapper.a、libdcf_wrapper.so

【注意】

- 此接口在OT_DCF_WRAPPER_SCAN_STATE_ALL_DIR_DONE后调用；
- u8ObjTypIndex为指定对象类型在对象类型表中的下标，详见 [OT_DCF_WRAPPER_NAMERULE_CFG_S](#)中的“astObjTypTbl”。

【举例】

参见本模块sample。

【相关主题】

无。

15.3.12 SS_DCF_WRAPPER_GetCurGrpFirstObj

【描述】

获取当前组的最老对象。

【语法】

TD_S32 SS_DCF_WRAPPER_GetCurGrpFirstObj([OT_DCF_WRAPPER_INDEX](#) iDCF, [OT_DCF_WRAPPER_OBJ_S*](#) pstObj);

【参数】

参数名称	描述	输入/输出
iDCF	DCF实例索引。	输入
pstObj	DCF对象	输出

【返回值】

返回值	描述
0	成功。
非0	失败，返回 错误码 。

【需求】

- 头文件：ss_dcf_wrapper.h
- 库文件：libdcf_impl.a、libdcf_impl.so、libdcf_core.a、libdcf_core.so、libdcf_wrapper.a、libdcf_wrapper.so

【注意】



此接口在当前组存在的情况下调用，如果未进行组检索或者当前组被删除则报错。

【 举例 】

参见本模块sample。

【 相关主题 】

无。

15.3.13 SS_DCF_WRAPPER_GetCurGrpLastObj

【 描述 】

获取当前组最新对象。

【 语法 】

```
TD_S32 SS_DCF_WRAPPER_GetCurGrpLastObj(OT_DCF_WRAPPER_INDEX iDCF,  
OT_DCF_WRAPPER_OBJ_S* pstObj);
```

【 参数 】

参数名称	描述	输入/输出
iDCF	DCF实例索引。	输入
pstObj	DCF对象	输出

【 返回值 】

返回值	描述
0	成功。
非0	失败，返回 错误码 。

【 需求 】

- 头文件：ss_dcf_wrapper.h
- 库文件：libdcf_impl.a、libdcf_impl.so、libdcf_core.a、libdcf_core.so、libdcf_wrapper.a、libdcf_wrapper.so

【 注意 】

此接口在当前组存在的情况下调用，如果未进行组检索或者当前组被删除则报错。

【 举例 】

参见本模块sample。

【 相关主题 】

无。



15.3.14 SS_DCF_WRAPPER_GetCurGrpNextObj

【描述】

获取当前组当前对象的后一个对象。

【语法】

```
TD_S32 SS_DCF_WRAPPER_GetCurGrpNextObj(OT_DCF_WRAPPER_INDEX iDCF,
OT_DCF_WRAPPER_OBJ_S* pstObj);
```

【参数】

参数名称	描述	输入/输出
iDCF	DCF实例索引。	输入
pstObj	DCF对象	输出

【返回值】

返回值	描述
0	成功。
非0	失败，返回错误码。

【需求】

- 头文件：ss_dcf_wrapper.h
- 库文件：libdcf_impl.a、libdcf_impl.so、libdcf_core.a、libdcf_core.so、libdcf_wrapper.a、libdcf_wrapper.so

【注意】

- 此接口在SS_DCF_WRAPPER_GetCurGrpFirstObj或SS_DCF_WRAPPER_GetCurGrpLastObj后调用；
- 接口调用成功，当前对象变更为获取出来的对象，反之则不变更。

【举例】

参见本模块sample。

【相关主题】

无。

15.3.15 SS_DCF_WRAPPER_GetCurGrpPrevObj

【描述】

获取当前组当前对象的前一个对象。

【语法】



TD_S32 SS_DCF_WRAPPER_GetCurGrpPrevObj([OT_DCF_WRAPPER_INDEX](#) iDCF, [OT_DCF_WRAPPER_OBJ_S*](#) pstObj);

【参数】

参数名称	描述	输入/输出
iDCF	DCF实例索引。	输入
pstObj	DCF对象	输出

【返回值】

返回值	描述
0	成功。
非0	失败，返回 错误码 。

【需求】

- 头文件：ss_dcf_wrapper.h
- 库文件：libdcf_impl.a、libdcf_impl.so、libdcf_core.a、libdcf_core.so、libdcf_wrapper.a、libdcf_wrapper.so

【注意】

- 此接口在[SS_DCF_WRAPPER_GetCurGrpFirstObj](#)或[SS_DCF_WRAPPER_GetCurGrpLastObj](#)后调用；
- 接口调用成功，当前对象变更为获取出来的对象，反之则不变更。

【举例】

参见本模块sample。

【相关主题】

无。

15.3.16 SS_DCF_WRAPPER_GetNextObjList

【描述】

获取某个对象之后的N个DCF对象，并且将这个N个DCF对象保存在列表中返回。

【语法】

TD_S32 SS_DCF_WRAPPER_GetNextObjList([OT_DCF_WRAPPER_INDEX](#) iDCF, const TD_CHAR* pszMainFile, TD_U32 u32Cnt, [OT_DCF_WRAPPER_OBJ_LIST_S*](#) pstObjList);

【参数】



参数名称	描述	输入/输出
iDCF	DCF实例索引。	输入
pszMainFile	基准DCF对象的主文件。	输入
u32Cnt	预获取的对象数量	输入
pstObjList	DCF对象列表	输出

【返回值】

返回值	描述
0	成功。
非0	失败，返回 错误码 。

【需求】

- 头文件：ss_dcf_wrapper.h
- 库文件：libdcf_impl.a、libdcf_impl.so、libdcf_core.a、libdcf_core.so、libdcf_wrapper.a、libdcf_wrapper.so

【注意】

- 此接口在OT_DCF_WRAPPER_SCAN_STATE_FIRST_DIR_DONE后调用；
- pszMainFile必须为DCF对象的主文件，表征基准对象；
- 如果pszMainFile为“TD_NULL”，则以实例中最老对象为基准，检索结果包含最老对象；
- 如果pszMainFile合法且表征的基准对象不存在，仍以该对象为基准向前检索；
- 如果基准对象已是最新对象，则报错；
- 如果u32Cnt大于基准对象之后的对象总数，pstObjList则返回基准之后所有的对象；
- pstObjList必须有足够的空间保存u32Cnt个对象；
- 此接口检索功能与组方式检索互不干扰，接口调用后不会变更当前组和当前组的当前对象。

【举例】

参见本模块sample。

【相关主题】

无。

15.3.17 SS_DCF_WRAPPER_GetPrevObjList

【描述】

获取某个对象之前的N个DCF对象，并且将这个N个DCF对象保存在列表中返回。



【语法】

```
TD_S32 SS_DCF_WRAPPER_GetPrevObjList(OT_DCF_WRAPPER_INDEX iDCF, const TD_CHAR*  
pszMainFile, TD_U32 u32Cnt, OT_DCF_WRAPPER_OBJ_LIST_S* pstObjList);
```

【参数】

参数名称	描述	输入/输出
iDCF	DCF实例索引。	输入
pszMainFile	基准DCF对象的主文件。	输入
u32Cnt	预获取的对象数量	输入
pstObjList	DCF对象列表	输出

【返回值】

返回值	描述
0	成功。
非0	失败，返回错误码。

【需求】

- 头文件：ss_dcf_wrapper.h
- 库文件：libdcf_impl.a、libdcf_impl.so、libdcf_core.a、libdcf_core.so、libdcf_wrapper.a、libdcf_wrapper.so

【注意】

- 此接口在OT_DCF_WRAPPER_SCAN_STATE_FIRST_DIR_DONE后调用；
- pszMainFile必须为DCF对象的主文件，表征基准对象；
- 如果pszMainFile为“TD_NULL”，则以实例中最新对象为基准，检索结果包含最新对象；
- 如果pszMainFile合法且表征的基准对象不存在，仍以该对象为基准向后检索；
- 如果基准对象已是最老对象，则报错；
- 如果u32Cnt大于基准对象之前的对象总数，pstObjList则返回基准之前所有的对象；
- pstObjList必须有足够的空间保存u32Cnt个对象；
- 此接口检索功能与组方式检索互不干扰，接口调用后不会变更当前组和当前组的当前对象。

【举例】

参见本模块sample。

【相关主题】

无。



15.3.18 SS_DCF_WRAPPER_AddObj

【描述】

添加DCF对象，向数据库中添加一个对象记录。

【语法】

TD_S32 SS_DCF_WRAPPER_AddObj(**OT_DCF_WRAPPER_INDEX** iDCF, const TD_CHAR* pszMainFile);

【参数】

参数名称	描述	输入/输出
iDCF	DCF实例索引。	输入
pszMainFile	基准DCF对象的主文件。	输入

【返回值】

返回值	描述
0	成功。
非0	失败，返回 错误码 。

【需求】

- 头文件：ss_dcf_wrapper.h
- 库文件：libdcf_impl.a、libdcf_impl.so、libdcf_core.a、libdcf_core.so、libdcf_wrapper.a、libdcf_wrapper.so

【注意】

- 此接口在OT_DCF_WRAPPER_SCAN_STATE_LAST_DIR_DONE后调用；
- pszMainFile必须为DCF对象的主文件；
- 如果预添加的对象在数据库中已有记录，则报错。

【举例】

参见本模块sample。

【相关主题】

无。

15.3.19 SS_DCF_WRAPPER_DelObj

【描述】

删除DCF对象，向数据库中添加一个对象记录，并删除存储介质中的实体文件。

【语法】



TD_S32 SS_DCF_WRAPPER_DelObj (**OT_DCF_WRAPPER_INDEX** iDCF, const TD_CHAR* pszMainFile);

【参数】

参数名称	描述	输入/输出
iDCF	DCF实例索引。	输入
pszMainFile	基准DCF对象的主文件。	输入

【返回值】

返回值	描述
0	成功。
非0	失败，返回 错误码 。

【需求】

- 头文件：ss_dcf_wrapper.h
- 库文件：libdcf_impl.a、libdcf_impl.so、libdcf_core.a、libdcf_core.so、libdcf_wrapper.a、libdcf_wrapper.so

【注意】

- 此接口在OT_DCF_WRAPPER_SCAN_STATE_FIRST_DIR_DONE后调用；
- pszMainFile必须为DCF对象的主文件。

【举例】

参见本模块sample。

【相关主题】

无。

15.3.20 SS_DCF_WRAPPER_GetTotalObjCnt

【描述】

获取DCF实例中的总对象数。

【语法】

TD_S32 SS_DCF_WRAPPER_GetTotalObjCnt(**OT_DCF_WRAPPER_INDEX** iDCF, TD_U32* pu32Cnt);

【参数】

参数名称	描述	输入/输出
iDCF	DCF实例索引。	输入



参数名称	描述	输入/输出
pu32Cnt	总对象数	输出

【返回值】

返回值	描述
0	成功。
非0	失败，返回 错误码 。

【需求】

- 头文件：ss_dcf_wrapper.h
- 库文件：libdcf_impl.a、libdcf_impl.so、libdcf_core.a、libdcf_core.so、libdcf_wrapper.a、libdcf_wrapper.so

【注意】

此接口在OT_DCF_WRAPPER_SCAN_STATE_ALL_DIR_DONE后调用。

【举例】

参见本模块sample。

【相关主题】

无。

15.3.21 SS_DCF_WRAPPER_GetObjCnt

【描述】

获取某个类型的DCF对象的数量。

【语法】

```
TD_S32 SS_DCF_WRAPPER_GetObjCnt(OT\_DCF\_WRAPPER\_INDEX iDCF, TD_U8 u8ObjTypIndex,  
TD_U32 *pu32Cnt);
```

【参数】

参数名称	描述	输入/输出
iDCF	DCF实例索引。	输入
u8ObjTypIndex	DCF对象类型索引。	输入
pu32Cnt	对象数	输出

【返回值】



返回值	描述
0	成功。
非0	失败，返回 错误码 。

【需求】

- 头文件：ss_dcf_wrapper.h
- 库文件：libdcf_impl.a、libdcf_impl.so、libdcf_core.a、libdcf_core.so、libdcf_wrapper.a、libdcf_wrapper.so

【注意】

- 此接口在OT_DCF_WRAPPER_SCAN_STATE_ALL_DIR_DONE后调用；
- u8ObjTypIndex为指定对象类型在对象类型表中的下标，详见[OT_DCF_WRAPPER_NAMERULE_CFG_S](#)中的“astObjTypTbl”。

【举例】

参见本模块sample。

【相关主题】

无。

15.3.22 SS_DCF_WRAPPER_GetNewFilePaths

【描述】

获取新DCF对象的文件路径列表，只生成文件路径，不在存储设备中创建文件实体。

【语法】

TD_S32 SS_DCF_WRAPPER_GetNewFilePaths([OT_DCF_WRAPPER_INDEX](#) iDCF, TD_BOOL bNewGrp, TD_U8 u8ObjTypIndex, [OT_DCF_WRAPPER_OBJ_S*](#) pstObj);

【参数】

参数名称	描述	输入/输出
iDCF	DCF实例索引。	输入
bNewGrp	是否以新组创建	输入
u8ObjTypIndex	DCF对象类型索引。	输入
pstObj	DCF对象	输出

【返回值】



返回值	描述
0	成功。
非0	失败，返回 错误码 。

【需求】

- 头文件：ss_dcf_wrapper.h
- 库文件：libdcf_impl.a、libdcf_impl.so、libdcf_core.a、libdcf_core.so、libdcf_wrapper.a、libdcf_wrapper.so

【注意】

- 此接口在OT_DCF_WRAPPER_SCAN_STATE_LAST_DIR_DONE后调用；
- 对于文件类型符长度为4的对象，文件名中不包含组信息，bNewGrp无效；
- u8ObjTypIndex为指定对象类型在对象类型表中的下标，详见[OT_DCF_WRAPPER_NAMERULE_CFG_S](#)中的“astObjTypTbl”；
- 新建DCF对象只有通过[SS_DCF_WRAPPER_AddObj](#)接口添加到数据库中，才会被检索到。

【举例】

参见本模块sample。

【相关主题】

无。

15.4 数据结构

相关数据类型、数据结构定义如下：

- [OT_DCF_WRAPPER_MAX_INSTANCE](#)：DCF实例最大数量。
- [OT_DCF_WRAPPER_MAX_OBJ_TYPE_CNT](#)：DCF对象最大种类数量。
- [OT_DCF_WRAPPER_MAX_FILE_CNT_IN_OBJ](#)：单个DCF对象中最大文件数量。
- [OT_DCF_WRAPPER_MAX_FILE_PATH_LENGTH](#)：文件路径最大长度。
- [OT_DCF_WRAPPER_MAX_ROOT_PATH_LENGTH](#)：根目录路径最大长度。
- [OT_DCF_WRAPPER_DIR_FREE_CHAR_LENGTH](#)：目录自由字符长度。
- [OT_DCF_WRAPPER_FILE_FREE_CHAR_LENGTH](#)：文件名自由字符长度。
- [OT_DCF_WRAPPER_FILE_EXTENSION_LENGTH](#)：文件后缀名长度。
- [OT_DCF_WRAPPER_INDEX](#)：DCF实例索引。
- [OT_DCF_WRAPPER_OBJ_S](#)：DCF对象。
- [OT_DCF_WRAPPER_OBJ_LIST_S](#)：DCF对象列表。
- [OT_DCF_WRAPPER_GRP_S](#)：DCF组。
- [OT_DCF_WRAPPER_SCAN_STATE_E](#)：目录扫描状态集。



- [OT_DCF_WRAPPER_FILE_TYPE_S](#): DCF文件类型。
- [OT_DCF_WRAPPER_OBJ_TYPE_S](#): DCF对象类型。
- [OT_DCF_WRAPPER_NAMERULE_CFG_S](#): 命名规则配置参数。
- [OT_DCF_WRAPPER_ON_SCAN_EVENT_PFN](#): 扫描事件回调函数原型。

15.4.1 OT_DCF_WRAPPER_MAX_INSTANCE

【说明】

DCF实例最大数量。

【定义】

```
#define OT_DCF_WRAPPER_MAX_INSTANCE 4
```

【成员】

无。

【注意事项】

无。

【相关数据类型及接口】

无。

15.4.2 OT_DCF_WRAPPER_MAX_OBJ_TYPE_CNT

【说明】

DCF对象最大种类数量。

【定义】

```
#define OT_DCF_WRAPPER_MAX_OBJ_TYPE_CNT 16
```

【成员】

无。

【注意事项】

无。

【相关数据类型及接口】

无。

15.4.3 OT_DCF_WRAPPER_MAX_FILE_CNT_IN_OBJ

【说明】

单个DCF对象中最大文件数量。

【定义】

```
#define OT_DCF_WRAPPER_MAX_FILE_CNT_IN_OBJ 6
```

【成员】



无。

【注意事项】

无。

【相关数据类型及接口】

无。

15.4.4 OT_DCF_WRAPPER_MAX_FILE_PATH_LENGTH

【说明】

文件路径最大长度。

【定义】

```
#define OT_DCF_WRAPPER_MAX_FILE_PATH_LENGTH 256
```

【成员】

无。

【注意事项】

无。

【相关数据类型及接口】

无。

15.4.5 OT_DCF_WRAPPER_MAX_ROOT_PATH_LENGTH

【说明】

根目录路径最大长度。

【定义】

```
#define OT_DCF_WRAPPER_MAX_ROOT_PATH_LENGTH 229
```

【成员】

无。

【注意事项】

无。

【相关数据类型及接口】

无。

15.4.6 OT_DCF_WRAPPER_DIR_FREE_CHAR_LENGTH

【说明】

DCF目录自由字符长度。

【定义】

```
#define OT_DCF_WRAPPER_DIR_FREE_CHAR_LENGTH 6
```

**【成员】**

无。

【注意事项】

除去字符串尾部 ‘\0’ 占用的1位，目录自由字符实际长度为 (OT_DCF_WRAPPER_DIR_FREE_CHAR_LENGTH -1) 。

【相关数据类型及接口】

无。

15.4.7 OT_DCF_WRAPPER_FILE_FREE_CHAR_LENGTH

【说明】

DCF文件名自由字符长度。

【定义】

```
#define OT_DCF_WRAPPER_FILE_FREE_CHAR_LENGTH 5
```

【成员】

无。

【注意事项】

除去字符串尾部 ‘\0’ 占用的1位，文件自由字符实际长度为 (OT_DCF_WRAPPER_FILE_FREE_CHAR_LENGTH -1) 。

【相关数据类型及接口】

无。

15.4.8 OT_DCF_WRAPPER_FILE_EXTENSION_LENGTH

【说明】

DCF文件后缀名长度。

【定义】

```
#define OT_DCF_WRAPPER_FILE_EXTENSION_LENGTH 4
```

【成员】

除去字符串尾部 ‘\0’ 占用的1位，文件后缀名实际长度为 (OT_DCF_WRAPPER_FILE_EXTENSION_LENGTH -1) 。

【注意事项】

无。

【相关数据类型及接口】

无。

15.4.9 OT_DCF_WRAPPER_INDEX

【说明】



DCF实例索引。

【定义】

```
typedef TD_U8 OT_DCF_WRAPPER_INDEX;
```

【成员】

无。

【注意事项】

DCF最多支持OT_DCF_WRAPPER_MAX_INSTANCE个实例，所以OT_DCF_WRAPPER_INDEX的有效范围为[0, OT_DCF_WRAPPER_MAX_INSTANCE-1]。

【相关数据类型及接口】

无。

15.4.10 OT_DCF_WRAPPER_OBJ_S

【说明】

DCF对象，在同一个DCF目录中具有相同文件号的文件归为一个对象。

【定义】

```
typedef struct otDCF_WRAPPER_OBJ_S
{
    TD_U8 u8FileCnt;
    TD_CHAR aszFilePath[OT_DCF_WRAPPER_MAX_FILE_CNT_IN_OBJ]
[OT_DCF_WRAPPER_MAX_FILE_PATH_LENGTH];
}OT_DCF_WRAPPER_OBJ_S;
```

【成员】

成员名称	描述
u8FileCnt	DCF对象中的文件数量。
aszFilePath	媒体流帧数据buffer。

【注意事项】

一个DCF对象中最多含有OT_DCF_WRAPPER_MAX_FILE_CNT_IN_OBJ个文件。

【相关数据类型及接口】

- OT_DCF_WRAPPER_MAX_FILE_CNT_IN_OBJ
- OT_DCF_WRAPPER_MAX_FILE_PATH_LENGTH

15.4.11 OT_DCF_WRAPPER_OBJ_LIST_S

【说明】

DCF对象列表。

【定义】



```
typedef struct otDCF_WRAPPER_OBJ_LIST_S
{
    TD_U32 u32ObjCnt;
    OT_DCF_WRAPPER_OBJ_S* apstObj;
}OT_DCF_WRAPPER_OBJ_LIST_S;
```

【成员】

成员名称	描述
u32ObjCnt	列表中对象数量。
apstObj	对象列表。

【注意事项】

apstObj是存放DCF对象的结构体数组指针。

【相关数据类型及接口】

[OT_DCF_WRAPPER_OBJ_S](#)

15.4.12 OT_DCF_WRAPPER_GRP_S

【说明】

组，具有相同组号的对象为一组。

【定义】

```
typedef struct otDCF_WRAPPER_GRP_S
{
    TD_U8 u8ObjTypIdx;
    TD_U16 u16Gnum;
    TD_U16 u16ObjCnt;
}OT_DCF_WRAPPER_GRP_S;
```

【成员】

成员名称	描述
u8ObjTypIdx	DCF对象类型索引。
u16Gnum	组号。
u16ObjCnt	组中所包含的对象数量。

【注意事项】

对于文件名自由字符中没有组属性的组，u16Gnum为无效值0。

【相关数据类型及接口】

无。



15.4.13 OT_DCF_WRAPPER_SCAN_STATE_E

【说明】

DCF目录扫描状态集。

【定义】

```
typedef enum otDCF_WRAPPER_SCAN_STATE_E
{
    OT_DCF_WRAPPER_SCAN_STATE_LAST_DIR_DONE    = 0x01,
    OT_DCF_WRAPPER_SCAN_STATE_FIRST_DIR_DONE   = 0x02,
    OT_DCF_WRAPPER_SCAN_STATE_ALL_DIR_DONE     = 0x03,
    OT_DCF_WRAPPER_SCAN_STATE_FAILURE          = 0x04,
    OT_DCF_WRAPPER_SCAN_STATE_BUTTON
}OT_DCF_WRAPPER_SCAN_STATE_E;
```

【成员】

成员名称	描述
OT_DCF_WRAPPER_SCAN_STATE_LAST_DIR_DONE	最新目录扫描完成，该状态表示DCF模块可以对外提供新建和添加DCF对象的功能了。
OT_DCF_WRAPPER_SCAN_STATE_FIRST_DIR_DONE	最老目录扫描完成，该状态表示DCF模块可以对外提供检索和删除功能了。
OT_DCF_WRAPPER_SCAN_STATE_ALL_DIR_DONE	所有目录扫描完成，该状态表示DCF模块可以对外提供全局信息查询的功能了。
OT_DCF_WRAPPER_SCAN_STATE_FAILURE	扫描失败，该状态表示DCF模块无法正常对外提供功能，实例无效，需要销毁。用户可根据失败的具体原因，解决后再重新创建实例。

【注意事项】

无。

【相关数据类型及接口】

无。

15.4.14 OT_DCF_WRAPPER_FILE_TYPE_S

【说明】

DCF文件类型。

【定义】

```
typedef struct otDCF_WRAPPER_FILE_TYPE_S
{
    TD_CHAR aszTypChar[OT_DCF_WRAPPER_FILE_FREE_CHAR_LENGTH];
    TD_CHAR aszExtension[OT_DCF_WRAPPER_FILE_EXTENSION_LENGTH];
}OT_DCF_WRAPPER_FILE_TYPE_S;
```

【成员】



成员名称	描述
aszTypChar	文件类型符。
aszExtension	文件后缀名。

【注意事项】

- aszTypChar字符只允许[A, Z]和“_”；
- aszExtension字符只允许[0, 9]、[A, Z]和“_”；
- aszTypChar长度只能为1或者4：长度为1代表此类型的文件名中包含组号信息；长度为4代表此类文件名不包含组号信息。如“NORM”类型的文件名“NORM0001.MP4”，“L”类型的文件名“L0010002.MP4”；
- aszExtension长度必须为（[OT_DCF_WRAPPER_FILE_EXTENSION_LENGTH - 1](#)）。

【相关数据类型及接口】

- [OT_DCF_WRAPPER_FILE_FREE_CHAR_LENGTH](#)
- [OT_DCF_WRAPPER_FILE_EXTENSION_LENGTH](#)

15.4.15 OT_DCF_WRAPPER_OBJ_TYPE_S

【说明】

DCF对象类型，多种文件类型绑定成一种对象。

【定义】

```
typedef struct otDCF_WRAPPER_OBJ_TYPE_S
{
    TD_U8 u8FileCnt;
    OT\_DCF\_WRAPPER\_FILE\_TYPE\_S astFileTbl[OT\_DCF\_WRAPPER\_MAX\_FILE\_CNT\_IN\_OBJ];
}OT_DCF_WRAPPER_OBJ_TYPE_S;
```

【成员】

成员名称	描述
u8FileCnt	文件种数。
astFileTbl	文件类型表。

【注意事项】

- astFileTbl最多绑定[OT_DCF_WRAPPER_MAX_FILE_CNT_IN_OBJ](#)种文件类型；
- astFileTbl中的各文件类型互斥；
- astFileTbl中的各文件类型符长度一致；
- astFileTbl中各文件类型顺序存放，不允许间隔存放；
- u8FileCnt与astFileTbl中存放的文件类型数保持一致。



【相关数据类型及接口】

- [OT_DCF_WRAPPER_FILE_TYPE_S](#)
- [OT_DCF_WRAPPER_MAX_FILE_CNT_IN_OBJ](#)

15.4.16 OT_DCF_WRAPPER_NAMERULE_CFG_S

【说明】

文件命名规则配置属性，确定DCF实例的根目录、目录自由字符和文件名的自由字符和文件后缀名。

【定义】

```
typedef struct otDCF_WRAPPER_NAMERULE_CFG_S
{
    TD_CHAR aszRoot[OT_DCF_WRAPPER_MAX_ROOT_PATH_LENGTH];
    TD_CHAR aszDirFreeChar[OT_DCF_WRAPPER_DIR_FREE_CHAR_LENGTH];
    TD_U8 u8ObjTypCnt;
    OT_DCF_WRAPPER_OBJ_TYPE_S astObjTypTbl[OT_DCF_WRAPPER_MAX_OBJ_TYPE_CNT];
}OT_DCF_WRAPPER_NAMERULE_CFG_S;
```

【成员】

成员名称	描述
aszRoot	根目录路径。
aszDirFreeChar	DCF目录自由字符。
u8ObjTypCnt	DCF对象种数。
astObjTypTbl	DCF对象类型表。

【注意事项】

- aszRoot除去结束符‘\0’占去的一位，长度最大为 ([OT_DCF_WRAPPER_MAX_ROOT_PATH_LENGTH](#) - 1) ；
- aszDirFreeChar除去结束符‘\0’占去的一位，长度最大为 ([OT_DCF_WRAPPER_DIR_FREE_CHAR_LENGTH](#) - 1) ；
- aszDirFreeChar字符只允许[0,9]、[A,Z]和“_” ；
- astObjTypTbl中各对象类型互斥，对象中各文件类型互斥；
- astObjTypTbl中各对象类型顺序存放，不允许间隔存放；
- astObjTypTbl中存放的对象种数大于等于u8ObjTypCnt，否则报错。

【相关数据类型及接口】

- [OT_DCF_WRAPPER_MAX_ROOT_PATH_LENGTH](#)
- [OT_DCF_WRAPPER_DIR_FREE_CHAR_LENGTH](#)
- [OT_DCF_WRAPPER_OBJ_TYPE_S](#)
- [OT_DCF_WRAPPER_MAX_OBJ_TYPE_CNT](#)



15.4.17 OT_DCF_WRAPPER_ON_SCAN_EVENT_PFN

【说明】

扫描事件回调函数原型。

【定义】

```
typedef TD_VOID (*OT_DCF_WRAPPER_ON_SCAN_EVENT_PFN)(OT_DCF_WRAPPER_INDEX  
iDCF, OT_DCF_WRAPPER_SCAN_STATE_E enState, TD_S32 s32ScanErrCode);
```

【成员】

成员名称	描述
iDCF	DCF实例索引。
enState	扫描状态。
s32ScanErrCode	扫描失败的具体错误码。

【注意事项】

s32ScanErrCode只有在扫描失败情况下有效。

【相关数据类型及接口】

[OT_DCF_WRAPPER_SCAN_STATE_E](#)

15.5 错误码

DCF API错误码如[表15-3](#)所示。

表 15-3 DCF API 错误码

错误代码	宏定义	描述
0xa12e8040	OT_ERR_DCF_WRAPPER_NULL_PTR	指针为空
0xa12e8041	OT_ERR_DCF_WRAPPER_ILLEGAL_PA RAM	参数非法
0xa12e8042	OT_ERR_DCF_WRAPPER_REINIT	重复初始化
0xa12e8043	OT_ERR_DCF_WRAPPER_NOT_INIT	未初始化
0xa12e8050	OT_ERR_DCF_WRAPPER_INVALID_IN DEX	DCF实例索引无效
0xa12e8051	OT_ERR_DCF_WRAPPER_NOT_ENOU GH_INDEX	DCF实例索引不足



错误代码	宏定义	描述
0xa12e8052	OT_ERR_DCF_WRAPPER_INSTANCE_NOT_DESTROY	DCF实例未销毁
0xa12e8053	OT_ERR_DCF_WRAPPER_SCAN_THREAD_CREATE_FAILURE	扫描线程创建失败
0xa12e8054	OT_ERR_DCF_WRAPPER_LAST_DIR_NOT_SCANDONE	最新目录未扫描完成
0xa12e8055	OT_ERR_DCF_WRAPPER_FIRST_DIR_NOT_SCANDONE	最老目录未扫描完成
0xa12e8056	OT_ERR_DCF_WRAPPER_ALL_DIR_NOT_SCANDONE	所有目录未扫描完成
0xa12e8057	OT_ERR_DCF_WRAPPER_SCAN_FAILURE	扫描失败
0xa12d8060	OT_ERR_DCF_WRAPPER_ROOT_NOT_DIR	存储设备中该根目录不是一个目录文件
0xa12d8061	OT_ERR_DCF_WRAPPER_ROOT_NOT_EXIST	存储设备中不存在该根目录
0xa12d8062	OT_ERR_DCF_WRAPPER_ROOT_ALREADY_OCCUPIED	根目录已经被占用，与其它实例使用的根目录重复
0xa12e8063	OT_ERR_DCF_WRAPPER_IMAGE_ROOT_NOT_DIR	存储设备中DCF图像根目录不是一个目录文件
0xa12e8064	OT_ERR_DCF_WRAPPER_IMAGE_ROOT_NOT_EXIST	存储设备中不存在该DCF图像根目录
0xa12e8065	OT_ERR_DCF_WRAPPER_DIR_NOT_DIR	存储设备中DCF目录不是一个目录文件
0xa12e8066	OT_ERR_DCF_WRAPPER_DIR_NOT_EXIST	存储设备中不存在该DCF目录
0xa12e8067	OT_ERR_DCF_WRAPPER_DIR_NOT_EXIST_IN_DB	数据库中不存在该目录
0xa12e8071	OT_ERR_DCF_WRAPPER_MAKE_DIR_FAILURE	创建目录失败
0xa12e8072	OT_ERR_DCF_WRAPPER_NOT_ENOUGH_DIR	没有足够DCF目录，目录号已达上限
0xa12e8080	OT_ERR_DCF_WRAPPER_NOT_ENOUGH_SIZE	大小不够
0xa12e8081	OT_ERR_DCF_WRAPPER_NOT_ENOUGH_MEMORY	没有足够内存
0xa12e8082	OT_ERR_DCF_WRAPPER_SYSTEM_CALL_FAILURE	调用系统函数失败



错误代码	宏定义	描述
0xa12e8090	OT_ERR_DCF_WRAPPER_EMPTY_DB	数据库为空
0xa12e8091	OT_ERR_DCF_WRAPPER_NO_NEXT	没有后一个组或对象
0xa12e8092	OT_ERR_DCF_WRAPPER_NO_PREV	没有前一个组或对象
0xa12e80a0	OT_ERR_DCF_WRAPPER_EMPTY_GRP	组为空，组内没有对象
0xa12e80a1	OT_ERR_DCF_WRAPPER_UNKNOW_CUR_GRP	当前组未知
0xa12e80b0	OT_ERR_DCF_WRAPPER_UNKNOW_CUR_OBJ	当前对象未知
0xa12e80b1	OT_ERR_DCF_WRAPPER_INVALID_OBJ_TYPE_INDEX	对象类型索引无效
0xa12e80b2	OT_ERR_DCF_WRAPPER_BUSY_OBJ	对象处于使用状态
0xa12e80b3	OT_ERR_DCF_WRAPPER_OBJ_NOT_EXIST	对象不存在
0xa12e80b4	OT_ERR_DCF_WRAPPER_OBJ_ALREADY_EXIST	对象已经存在
0xa12e80c1	OT_ERR_DCF_WRAPPER_INVALID_FILEPATH	文件路径无效
0xa12e80c2	OT_ERR_DCF_WRAPPER_NOT_MAIN_FILE	非对象的主文件

15.6 典型场景使用指南

参见本模块sample。

15.7 Proc 调试

在Linux系统上，DCF模块支持Proc调试，程序运行后，通过串口在板端执行“cat /proc/ot/middleware/dcf”便可以查看DCF模块的Proc信息。DCF模块提供Proc信息如表15-4所示。

表 15-4 DCF Proc 信息

定义	说明
InstIndex	DCF实例索引



定义	说明
CurDnum	文件检索时的当前目录号
CurFnum	文件检索时的当前文件号
LastTbl(Dnum/bComplete)	最新目录数据库表（目录号/是否组建完成）
SubLastTbl(Dnum/bComplete)	次新目录数据库表（目录号/是否组建完成）
TmpTbl(Dnum/bComplete)	临时目录数据库表（目录号/是否组建完成）
FirstTbl(Dnum/bComplete)	最老目录数据库表（目录号/是否组建完成）
CurScanJob(Dnum/CPUUR)	当前扫描任务（目录号/扫描CPU占用率）
LastDirScanDuration(us)	最新目录扫描完成耗时（单位：微秒）
FirstDirScanDuration(us)	最老目录扫描完成耗时（单位：微秒）
AllDirScanDuration(us)	所有目录扫描完成耗时（单位：微秒）

DCF模块Proc信息样例如图15-4所示。

图 15-4 DCF 模块 Proc 信息

```
~ # cat /proc/ot/middleware/dcf
DCF
    InstIndex : 0
    CurDnum   : 0
    CurFnum   : 0
    LastTbl(Dnum/isComplete) : 0/0
    SubLastTbl(Dnum/isComplete) : 0/0
    TmpTbl(Dnum/isComplete) : 0/0
    FirstTbl(Dnum/isComplete) : 0/0
    CurScanJob(Dnum/cpuRate) : 0/0
    LastDirScanDuration(us) : 18
    FirstDirScanDuration(us) : 59
    AllDirScanDuration(us) : 81
```




16 DNG 封装解封装模块

16.1 概述

DNG封装解封装模块用于拍照时的标准DNG格式文件封装，本地回放时的DNG文件解封装预览缩略图等功能，支持DNG标准协议。

16.2 功能描述

16.2.1 基本概念

DNG(Digital Negative)是Adobe开发的一种开放的raw image file format,里面使用的tag基本都定义在TIFF或者TIFF/EP中，在DNG Sepcification中只是定义或者建议了数据的组织方式，颜色空间的转换等等。在DNG出现以前，各个数码相机制造商都有自己的格式，比如Canon(cr2/crw), Nikon(nef), Olympus(orf), Pentex(pef)等等，一方面的原因是在这之前没有统一的raw格式，更重要的是，各个厂商希望用这个只对自己公开的数据格式来保护自己的私密信息。而DNG的出现主要是为了统一不同厂商raw格式。DNG规范基本和TIFF,TIFF/EP兼容，一个DNG文件由IFH和IFD组成，IFD由DE组成。



图 16-1 DNG 文件基本组织结构图

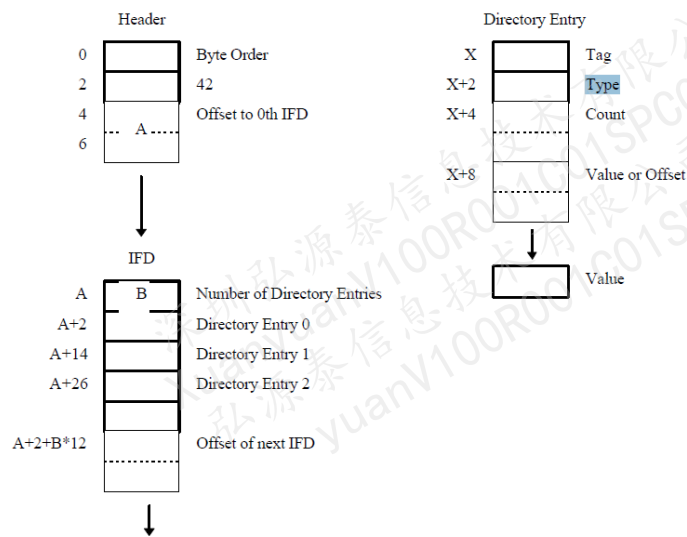


Figure 1 — TIFF File structure

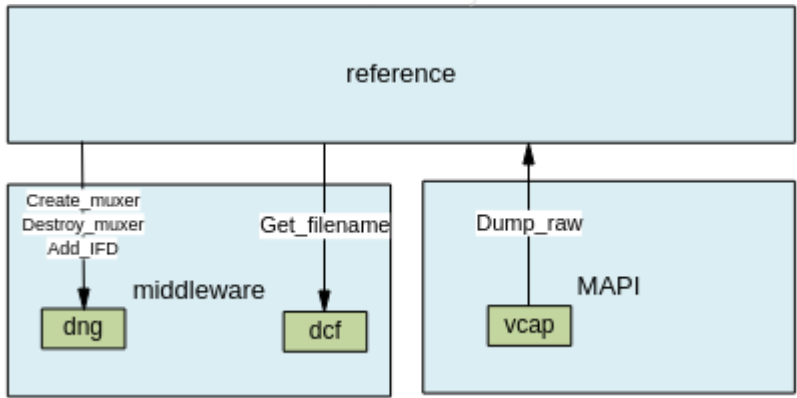
16.2.2 功能描述

当前文件封装DNG支持标准bayer raw数据封装，jpeg缩略图封装，同时支持GPS、XMP、EXIF等信息封装。

16.2.3 上下文描述

文件封装DNG组件软件上下文依赖关系如图16-2所示。对下依赖SDK的raw数据获取及相关媒体信息获取，DNG根据上层sample app对数据的组织格式，依赖文件管理(DCF)模块提供的文件路径文件名，对接收到的媒体流数据进行标准格式DNG文件封装。由于DNG文件内部存储结构具有较强的灵活性，本模块按照海思默认方式组织数据存储格式，如果客户需要定义差异化存储结构，需要使用更底层API自行实现。

图 16-2 DNG 组件上下文依赖关系





16.3 API 参考

该模块支持：

- DNG文件封装解封装实例创建、销毁；
- IFD添加；
- IFD中图片信息获取；
- IFD中图片数据获取；

该功能模块提供以下API：

- **SS_DNG_Create**：DNG封装实例创建。
- **SS_DNG_Destroy**：DNG封装实例销毁。
- **SS_DNG_AddIFD**：为DNG文件添加IFD。
- **SS_DNG_GetImageInfo**：DNG解封装获取图片信息。
- **SS_DNG_GetImageData**：DNG解封装获取图片数据。

16.3.1 SS_DNG_Create

【描述】

DNG封装实例创建。

【语法】

```
TD_S32 SS_DNG_Create(TD_MW_PTR* ppDng, OT_DNG_CONFIG_S* pstDngCfg);
```

【参数】

参数名称	描述	输入/输出
ppDng	DNG模块句柄。	输出
pstDngCfg	参数配置信息。	输入

【返回值】

返回值	描述
0	成功。
非0	失败，参见 错误码 。

【需求】

- 头文件：ss_dng.h
- 库文件：libdng.a、libdng.so

【注意】



该模块支持多实例，支持多线程写数据。不支持多线程。根据配置类型选择创建封装或者解封装实例，其中最大支持32个实例。

【举例】

本例中演示了DNG封装模块的初始化过程，包括必要的参数设置，具体可以参见本模块的sample代码。

```
//create muxer
OT_DNG_CONFIG_S stCfg;
memset(&stCfg, 0x00, sizeof(OT_DNG_CONFIG_S));
snprintf(stCfg.aszFileName, DNG_MAX_FILE_NAME, "%d%s", u32PhotoSeq, "sample.dng");
s32Ret = SS_DNG_Create(&pDNG, &stCfg);
if (s32Ret != TD_SUCCESS)
{
    printf("SS_DNG_Create failed %d \n", s32Ret);
    return TD_FAILURE;
}
```

【相关主题】

无。

16.3.2 SS_DNG_Destroy

【描述】

DNG封装实例销毁。

【语法】

```
TD_S32 SS_DNG_Destroy(TD_MW_PTR pDng);
```

【参数】

参数名称	描述	输入/输出
pDng	DNG模块句柄。	输入

【返回值】

返回值	描述
0	成功。
非0	失败，返回 错误码 。

【需求】

- 头文件：ss_dng.h
- 库文件：libdng.a、libdng.so

【注意】



不支持重复销毁，重复调用会返回错误。用户若需要验证该组件封装的DNG图片格式是否正确，请用ExifTool检查该文件各字段是否可被正确解析，并且使用主流DNG图片播放器（Lightroom Photoshop等工具）查看图片内容是否正确。

【相关主题】

无。

16.3.3 SS_DNG_AddIFD

【描述】

为DNG添加IFD。

【语法】

```
TD_S32 SS_DNG_AddIFD(TD_MW_PTR pDng, OT_DNG_MUXER_IFD_INFO_S* pstIFDInfo);
```

【参数】

参数名称	描述	输入/输出
pDng	DNG模块句柄。	输入
pstIFDInfo	IFD配置信息。	输入

【返回值】

返回值	描述
0	成功。
非0	失败，返回错误码。

【需求】

- 头文件：ss_dng.h
- 库文件：libdng.a、libdng.so

【注意】

- 请在创建DNG实例后调用此接口，仅支持在OT_DNG_CONFIG_TYPE_MUXER模式下调用该接口。
- DNG实例不支持链式存储，只支持树式存储，需要保证至少有一个IFD，其余IFD可以设置为IFD的子IFD。
- DNG文件中至少有一张raw图，可以根据需要添加缩略图和中图；如果DNG文件中需要封装用于预览的缩略图，一般封装在IFD0中，然后将raw图封装在IFD0的subIFD中。
- 请先添加IFD0后再添加相应的subIFD；并且个数与IFD0中的子IFD个数匹配。

【举例】

无。



【相关主题】

无。

16.3.4 SS_DNG_GetImageInfo

【描述】

DNG解封装获取图片信息。

【语法】

```
TD_S32 SS_DNG_GetImageInfo(TD_MW_PTR pDng, TD_U32 u32IfdIndex,  
OT\_DNG\_DEMUXER\_IMAGE\_INFO\_S\* pstImageInfo);
```

【参数】

参数名称	描述	输入/输出
pDng	DNG解封装实例句柄。	输入
u32IfdIndex	图片IFD index。	输入
pstImageInfo	图片宽高，数据长度等信息。	输出

【返回值】

返回值	描述
0	成功。
非0	失败，返回 错误码 。

【需求】

- 头文件：ss_dng.h
- 库文件：libdng.a、libdng.so

【注意】

- 请在创建DNG解封装实例后，每次获取图片数据前调用该接口，仅支持在OT_DNG_CONFIG_TYPE_DEMUXER模式下调用该接口。
- 当前DNG解封装功能支持解析本模块封装功能封装出来的DNG图片，暂时不兼容其他平台下的DNG文件。
- Libtiff开源库限制，不支持连续获取同一个IFD中图片信息，仅支持顺序获取。例如：先获取第0个，再获取第1个。不支持两次获取第0个。

【举例】

无。

【相关主题】

无。



16.3.5 SS_DNG_GetImageData

【描述】

DNG解封装获取图片数据。

【语法】

```
TD_S32 SS_DNG_GetImageData(TD_MW_PTR pDng, TD_U8* pu8Buffer, TD_U32 u32DataLen);
```

【参数】

参数名称	描述	输入/输出
pDng	DNG解封装指针	输入
pu8Buffer	数据指针	输出
u32DataLen	数据长度，单位：字节	输入

【返回值】

返回值	描述
0	修复成功。
非0	修复失败，返回 错误码 。

【需求】

- 头文件：ss_dng.h
- 库文件：libdng.a、libdng.so

【注意】

- 请数据调用者保证pu8Buffer指针长度足够，且与u32DataLen参数相符合，pu8Buffer内存外部创建外部销毁。
- 在调用该接口前请确保DNG解封装实例创建，仅支持在OT_DNG_CONFIG_TYPE_DEMUXER模式下调用该接口，且已经调用[SS_DNG_GetImageInfo](#)接口获取了DNG文件中缩略图的信息，u32DataLen参数是从该接口中获取的数据长度。

【举例】

无。

【相关主题】

无。

16.4 数据结构

相关数据类型、数据结构定义如下：



- **OT_DNG_CONFIG_S**: DNG实例配置参数结构体。
- **OT_DNG_MUXER_IFD_INFO_S**: 定义DNG实例IFD信息参数结构体。
- **OT_DNG_MUXER_IFD0_INFO_S**: 定义DNG实例中IFD0配置参数结构体。
- **OT_DNG_MUXER_RAWIFD_INFO_S**: raw IFD信息结构体。
- **OT_DNG_MUXER_SCREENIFD_INFO_S**: 中图IFD信息结构体。
- **OT_DNG_MUXER_IMAGE_DATA_S**: 图像数据结构体。
- **OT_DNG_MUXER_EXIF_INFO_S**: EXIF信息结构体。
- **OT_DNG_MUXER_GPS_INFO_S**: GPS信息结构体。
- **OT_DNG_MUXER_IMAGE_DYNAMIC_INFO_S**: 图像动态信息结构体。
- **OT_DNG_MUXER_IMAGE_STATIC_INFO_S**: 图像静态信息结构体。
- **OT_DNG_MUXER_RAW_FORMAT_S**: raw数据格式信息结构体。
- **OT_DNG_MUXER_CFALAYOUT_TYPE_E**: DNG图片raw数据CFA排布类型枚举。
- **OT_DNG_MUXER_IMAGE_TYPE_E**: 定义DNG封装IFD中图片类型枚举。
- **OT_DNG_DEMUXER_IMAGE_INFO_S**: 图像信息结构体。
- **OT_DNG_CONFIG_TYPE_E**: DNG实例配置类型。
- **DNG_MAX_FILE_NAME**: DNG文件名长度最大值。
- **DNG_MUXER_DCF_DESCRIPTION_LENGTH**: DCF描述长度。
- **DNG_MUXER_DATETIME_LENGTH**: 日期时间长度。
- **DNG_MUXER_PROFILEHUESATMAPDIM_LENGTH**: profilehusatmapdim描述长度。
- **DNG_MUXER_LENINFO_LEN**: 镜头描述长度。
- **DNG_MUXER_MATRIX_SIZE**: 矩阵大小。
- **DNG_MUXER_GPSVER_LEN**: GPS版本信息描述长度。
- **DNG_MUXER_TUDE_LEN**: 经纬度信息长度。
- **DNG_MUXER_ISP_BAYER_CHN**: 贝尔通道数目。
- **DNG_MUXER_CFACOLORPLANE**: 色彩平面个数。
- **DNG_MUXER_REPEAT_LEN**: 电平重复度描述长度。
- **DNG_MUXER_SCALE_LEN**: 比例描述长度。
- **DNG_MUXER_NP_SIZE**: 噪声描述长度。
- **DNG_MUXER_REF_BW**: ReferenceBlackWhite描述长度。
- **DNG_MUXER_CROP_LEN**: 裁剪描述长度。

16.4.1 OT_DNG_CONFIG_S

【说明】

定义DNG实例配置参数结构体。

【定义】

```
typedef struct DNG_CONFIG_S
{
    OT_DNG_CONFIG_TYPE_E enConfigType;
```



```
TD_CHAR aszFileName[DNG_MAX_FILE_NAME];/*file path and file name*/
} OT_DNG_CONFIG_S;
```

【成员】

成员名称	描述
enConfigType	配置类型，以OT_DNG_CONFIG_MUXER或者OT_DNG_CONFIG_DEMUXER模式打开文件。
aszFileName	文件名，包含文件路径。

【注意事项】

aszFileName保证路径存在，路径不存在，接口返回错误。

【相关数据类型及接口】

OT_DNG_CONFIG_TYPE_E

16.4.2 OT_DNG_MUXER_IFD_INFO_S

【说明】

定义DNG实例IFD信息参数结构体。

【定义】

```
typedef struct DNG_MUXER_IFD_INFO_S
{
    TD_U16 u16Photometric;//photometric interpretation
    TD_U32 u32Width;
    TD_U32 u32Height;
    TD_U16 u16BitPerSample;
    TD_U16 u16SamplePerPixel;
    TD_U16 u16Compression;//such as jpegTD_U32 u32SubfileType;
    TD_U16 u16PlanarConfig;//storage organization
    OT_DNG_MUXER_IMAGE_TYPE_E enImageType;//thm or screannail or raw
    union
    {
        OT_DNG_MUXER_IFD0_INFO_S stIFD0Info;/*IFD0 info*/
        OT_DNG_MUXER_RAWIFD_INFO_S stRawIFDInfo;/*IFD0 subifd: raw IFD info*/
        OT_DNG_MUXER_SCREENIFD_INFO_S stScreenIFDInfo;/*IFD0 subifd: scrennail IFD info*/
    };
    OT_DNG_MUXER_IMAGE_DATA_S stImageData;
} OT_DNG_MUXER_IFD_INFO_S;
```

【成员】

成员名称	描述
u16Photometric	颜色空间
u32Width	图像宽
u32Height	图像高
u16BitPerSample	采样位宽



成员名称	描述
u16SamplePerPixel	每像素采样数
u16Compression	压缩模式，当前只支持COMPRESSION_NONE与COMPRESSION_JPEG模式
u16PlanarConfig	像素平面排布格式
u32SubfileType	子文件类型标志。 0：源文件； 1：缩略图。
enImageType	图像类型
stIFD0Info	IFD信息结构体
stImageData	图像数据结构体

【 注意事项 】

请注意u16Photometric u16Compression enImageType等参数的匹配，即raw图和缩略图等配置差异，参见模块sample。

【 相关数据类型及接口 】

- OT_DNG_MUXER_IMAGE_TYPE_E
- OT_DNG_MUXER_IFD0_INFO_S
- OT_DNG_MUXER_RAWIFD_INFO_S
- OT_DNG_MUXER_SCREENIFD_INFO_S
- OT_DNG_MUXER_IMAGE_DATA_S

16.4.3 OT_DNG_MUXER_IFD0_INFO_S

【 说明 】

定义DNG实例中IFD0配置参数结构体。

【 定义 】

```
typedef struct DNG_MUXER_IFD0_INFO_S
{
    TD_U32 u32SubIFDs;//count of subifd
    TD_CHAR aszUniqueCameraModel[DNG_MUXER_DCF_DESCRIPTION_LENGTH];
    TD_CHAR aszCameraSerialNumber[DNG_MUXER_DCF_DESCRIPTION_LENGTH];
    TD_CHAR aszMake[DNG_MUXER_DCF_DESCRIPTION_LENGTH];
    TD_CHAR aszModel[DNG_MUXER_DCF_DESCRIPTION_LENGTH];
    TD_CHAR aszSoftware[DNG_MUXER_DCF_DESCRIPTION_LENGTH];
    TD_CHAR aszDateTime[DNG_MUXER_DATETIME_LENGTH];TD_U8 u8Orientation;
    TD_U32 au32ProfileHueSatMapDim[DNG_MUXER_PROFILEHUESATMAPDIM_LEN];
    TD_FLOATTD_FLOAT* pfProfileHueSatMapData1;
    TD_U32 u32Map1Len;
    TD_FLOATTD_FLOAT* pfProfileHueSatMapData2;
    TD_U32 u32Map2Len;
    TD_FLOATTD_FLOAT afLensInfo[DNG_MUXER_LENINFO_LEN];TD_U8* pu8XmpBuf;//
    optional ApplicationNotes TIFFTAG_XMLPACKET
}
```



```

TD_U32 u32XmpDataLen;
TD_U8* pu8XPComment;//optional
TD_U32 u32XPCommentLen;
TD_U8* pu8XPKeywords;//optional
TD_U32 u32XPKeywordsLen;
TD_U8* pu8DNGPrivData;//optional
TD_U32 u32DNGPrivDataLen;
TD_BOOL bHasExifIFD;
TD_BOOL bHasGpsIFD;
OT_DNG_MUXER_EXIF_INFO_S stExifInfo;
OT_DNG_MUXER_GPS_INFO_S stGpsInfo;
OT_DNG_MUXER_IMAGE_DYNAMIC_INFO_S stDynamicInfo;
OT_DNG_MUXER_IMAGE_STATIC_INFO_S stStaticInfo;
OT_DNG_MUXER_RAW_FORMAT_S stRawFormat;//if have no raw subifd,should muxer
rawformat param in ifd0
} OT_DNG_MUXER_IFD0_INFO_S;

```

【成员】

成员名称	描述
u32SubIFDs	子IFD数目
aszUniqueCameraModel	唯一相机型号
aszCameraSerialNumber	相机序列号
aszMake	厂商
aszModel	设备型号
aszSoftware	软件名
aszDateTime	拍摄时间
u8Orientation	排布方向
au32ProfileHueSatMapDim	色彩、饱和度映射维度配置
pfProfileHueSatMapData1	色彩、饱和度映射维度配置数据1（可选）
u32Map1Len	色彩、饱和度映射维度配置数据1长度
pfProfileHueSatMapData2	色彩、饱和度映射维度配置数据2（可选）
u32Map2Len	色彩、饱和度映射维度配置数据2长度
afLensInfo	镜头信息
pu8XmpBuf	XMP元数据指针（可选）
u32XmpDataLen	XMP元数据长度，单位：字节
pu8XPComment	Windows备注数据（可选）
u32XPCommentLen	Windows备注数据长度，单位：字节
pu8XPKeywords	Windows关键词数据（可选）
u32XPKeywordsLen	Windows关键词数据长度，单位：字节
pu8DNGPrivData	DNG隐私数据（可选）



成员名称	描述
u32DNGPrivDataLen	DNG隐私数据长度，单位：字节
bHasExifIFD	是否有EXIF信息 IFD标志，为true时表示有。
bHasGpsIFD	是否有GPS 信息IFD标志，为true时表示有。
stExifInfo	EXIF信息 IFD（当bHasExifIFD为false时，该项可不配置）
stGpsInfo	GPS 信息IFD（当bHasGpsIFD为false时，该项可不配置）
stDynamicInfo	图像动态信息
stStaticInfo	图像静态信息
stRawFormat	Raw图格式信息

【注意事项】

- 可选配置项根据用户需要进行配置，例如当不需要添加pu8XmpBuf信息时，pu8XmpBuf和u32XmpDataLen不需要设置；同理GPS EIXF等也是。
- 当u32SubIFDs为0时，表明该IFD0不包含子IFD, 该IFD0为raw IFD ,本身只包含一个raw图，请注意此时的压缩类型(非压缩的原始数据类型)和subFileType（参见dng规范）配置；此时需要配置该结构体中的stRawFormat参数。
- 当u32SubIFDs不为0时，表明该IFD0包含子IFD, 该IFD0为本身包含一个缩略图，同时包含封装有raw图的子IFD，请注意此时的压缩类型（缩略图的jpeg压缩类型）和subFileType（参见DNG协议规范）配置；此时不需要配置该结构体中的stRawFormat参数。

【相关数据类型及接口】

- [OT_DNG_MUXER_EXIF_INFO_S](#)
- [OT_DNG_MUXER_GPS_INFO_S](#)
- [OT_DNG_MUXER_IMAGE_DYNAMIC_INFO_S](#)
- [OT_DNG_MUXER_IMAGE_STATIC_INFO_S](#)
- [OT_DNG_MUXER_RAW_FORMAT_S](#)

16.4.4 OT_DNG_MUXER_RAWIFD_INFO_S

【说明】

定义raw IFD信息结构体。

【定义】

```
typedef struct DNG_MUXER_RAWIFD_INFO_S
{
    TD_FLOAT afDefaultCropOrigin[DNG_MUXER_CROP_LEN];
    TD_FLOAT afDefaultCropSize[DNG_MUXER_CROP_LEN];TD_VOID* pu8OpcodeList1;//
    optional
    TD_U32 u32OpcodeList1DataLen;
```



```
OT_DNG_MUXER_RAW_FORMAT_S stRawFormat;  
} OT_DNG_MUXER_RAWIFD_INFO_S;
```

【成员】

成员名称	描述
afDefaultCropOrigin	缺省原始图像边界。
afDefaultCropSize	缺省原始图像大小。
pu8OpcodeList1	操作码1数据（可选）
u32OpcodeList1DataLen	操作码1数据长度，单位：字节
stRawFormat	Raw图像信息。

【注意事项】

无。

【相关数据类型及接口】

OT_DNG_MUXER_RAW_FORMAT_S

16.4.5 OT_DNG_MUXER_SCREENIFD_INFO_S

【说明】

中图IFD信息结构体。

【定义】

```
typedef struct DNG_MUXER_SCREENIFD_INFO_S  
{  
    TD_FLOAT fReferenceBlackWhite[DNG_MUXER_REF_BW];  
} OT_DNG_MUXER_SCREENIFD_INFO_S;
```

【成员】

成员名称	描述
fReferenceBlackWhite	参考黑白点。

【注意事项】

无。

【相关数据类型及接口】

无。

16.4.6 OT_DNG_MUXER_IMAGE_DATA_S

【说明】

定义图像数据信息。



【定义】

```
typedef struct DNG_MUXER_IMAGE_DATA_S
{
    TD_U32 u32DataLen;
    TD_U8* pu8DataBuf;
    TD_U32 u32Stride;//for row data
} OT_DNG_MUXER_IMAGE_DATA_S;
```

【成员】

成员名称	描述
u32DataLen	图像数据长度。
pu8DataBuf	图像数据指针。
u32Stride	步幅（raw数据相关描述，缩略图可不配置）。

【注意事项】

无。

【相关数据类型及接口】

无。

16.4.7 OT_DNG_MUXER_EXIF_INFO_S

【说明】

EXIF数据信息结构体。

【定义】

```
typedef struct DNG_MUXER_EXIF_INFO_S
{
    TD_CHAR    aszDateTimeOriginal[DNG_MUXER_DATETIME_LENGTH];
    TD_CHAR    aszCaptureTime[DNG_MUXER_DATETIME_LENGTH];
    TD_FLOAT    fExposureTime;
    TD_U16    u16ExposureMode; /
    TD_U16    u16ExposureProgram;
    TD_U16    u16LightSource;
    TD_U16    u16ISOSpeedRatings;
    TD_FLOAT    fExposureBiasValue;
    TD_U16    u16MeteringMode;
    TD_FLOAT    fFocalLength;
    TD_U16    u16WhiteBalance;
    TD_U16    u16Saturation;
    TD_U16    u16Sharpness;
    TD_FLOAT    fMaxApertureValue;
    TD_U8    u8SceneType;
    TD_U16    u16CustomRendered;
    TD_U16    u16FocalLengthIn35mmFilm;
    TD_U16    u16SceneCaptureType;
    TD_U16    u16GainControl;
    TD_U16    u16Contrast;
    TD_U16    u16Flash;
    TD_FLOAT    fDigitalZoomRatio;
} OT_DNG_MUXER_EXIF_INFO_S;
```




【成员】

成员名称	描述
aszDateTimeOriginal	图像原始时间
aszCaptureTime	抓拍时间
fExposureTime	曝光时间
u16ExposureMode	曝光模式
u16ExposureProgram	曝光程序
u16LightSource	光源
fFNumber	F系数
u16ISOSpeedRatings	ISO速度等级
fExposureBiasValue	曝光偏差值
u16MeteringMode	测光模式
fFocalLength	焦距
u16WhiteBalance	白平衡
u16Saturation	饱和度
u16Sharpness	锐度
fMaxApertureValue	最大光圈
u8SceneType	场景
u16CustomRendered	自定义获取
u16FocalLengthIn35mmFilm	35mm焦距
u16SceneCaptureType	场景捕捉类型
u16GainControl	收益控制
u16Contrast	对比度
u16Flash	Flash
fDigitalZoomRatio	数字压缩率

【注意事项】

无。

【相关数据类型及接口】

无。



16.4.8 OT_DNG_MUXER_GPS_INFO_S

【说明】

GPS信息结构体。

【定义】

```
typedef struct DNG_MUXER_IMAGE_GPS_INFO_S
{
    TD_U8    au8GpsVersion[DNG_MUXER_GPSVER_LEN];
    TD_CHAR  chGPSLatitudeRef; /* GPS LatitudeRef Indicates whether the latitude is north
    or south latitude, 'N'/'S', default 'N' */
    TD_FLOAT afGPSLatitude[DNG_MUXER_TUDE_LEN]; /* GPS Latitude is expressed as
    degrees, minutes and seconds,a typical format like "dd/1,mm/1,ss/1", default 0/1, 0/1, 0/1 */
    TD_CHAR  chGPSLongitudeRef; /* GPS LongitudeRef Indicates whether the longitude is
    east or west longitude, 'E'/'W', default 'E' */
    TD_FLOAT afGPSLongitude[DNG_MUXER_TUDE_LEN]; /* GPS Longitude is expressed as
    degrees, minutes and seconds,a typical format like "dd/1,mm/1,ss/1", default 0/1, 0/1, 0/1 */
    TD_U8    u8GPSAltitudeRef; /* GPS AltitudeRef Indicates the reference altitude used, 0 -
    above sea level, 1 - below sea level * default 0 */
    TD_FLOAT fGPSAltitude; /* GPS AltitudeRef Indicates the altitude based on the reference
    u8GPSAltitudeRef, the reference unit is meters, default 0/1 */
} OT_DNG_MUXER_GPS_INFO_S;
```

【成员】

成员名称	描述
au8GpsVersion	GPS版本。
chGPSLatitudeRef	纬度方位（南北方向）。
afGPSLatitude	纬度。
chGPSLongitudeRef	经度方位（东西方向）。
afGPSLongitude	经度。
u8GPSAltitudeRef	水平线方位（高于，低于）
fGPSAltitude	水平高度，单位：米

【注意事项】

无。

【相关数据类型及接口】

无。

16.4.9 OT_DNG_MUXER_IMAGE_DYNAMIC_INFO_S

【说明】

DNG图片动态信息结构体。

【定义】

```
typedef struct DNG_MUXER_IMAGE_DYNAMIC_INFO_S
{
```



```
TD_FLOAT afAsShotNeutral[DNG_MUXER_CFACOLORPLANE];
TD_DOUBLE adNoiseProfile[DNG_MUXER_NP_SIZE];
} OT_DNG_MUXER_IMAGE_DYNAMIC_INFO_S; //for every image
```

【成员】

成员名称	描述
afAsShotNeutral	中性白平衡。
adNoiseProfile	噪音配置。

【注意事项】

无。

【相关数据类型及接口】

无。

16.4.10 OT_DNG_MUXER_IMAGE_STATIC_INFO_S

【说明】

DNGraw图片静态文件信息结构体。

【定义】

```
typedef struct DNG_MUXER_IMAGE_STATIC_INFO_S
{
    TD_FLOAT afColorMatrix1[DNG_MUXER_MATRIX_SIZE];
    TD_FLOAT afColorMatrix2[DNG_MUXER_MATRIX_SIZE];
    TD_FLOAT afCameraCalibration1[DNG_MUXER_MATRIX_SIZE];
    TD_FLOAT afCameraCalibration2[DNG_MUXER_MATRIX_SIZE];
    TD_U8 u8CalibrationIlluminant1;
    TD_U8 u8CalibrationIlluminant2;
    TD_FLOAT afForwadMatrix1[DNG_MUXER_MATRIX_SIZE];
    TD_FLOAT afForwadMatrix2[DNG_MUXER_MATRIX_SIZE];
} OT_DNG_MUXER_IMAGE_STATIC_INFO_S; //For raw
```

【成员】

成员名称	描述
afColorMatrix1	颜色矩阵1。
afColorMatrix2	颜色矩阵2。
afCameraCalibration1	相机校准1。
afCameraCalibration2	相机校准2。
u8CalibrationIlluminant1	校准光源1。
u8CalibrationIlluminant2	校准光源2。
afForwadMatrix1	转换矩阵1。



成员名称	描述
afForwadMatrix2	转换矩阵2。

【注意事项】

无。

【相关数据类型及接口】

无。

16.4.11 OT_DNG_MUXER_RAW_FORMAT_S

【说明】

DNG图片raw数据格式信息结构体。

【定义】

```
typedef struct DNG_MUXER_RAW_FORMAT_S
{
    TD_FLOATTD_FLOAT af32BlackLevel[DNG_MUXER_ISP_BAYER_CHN];
    TD_U8 au8CfaPattern[DNG_MUXER_ISP_BAYER_CHN]; /* 0:red 1:green 2: blue*/
    TD_U8 au8CfaPlaneColor[DNG_MUXER_CFACOLORPLANE]; /* 0:red 1:green 2: blue*/
    OT_DNG_MUXER_CFALAYOUT_TYPE_E enCfaLayout;
    TD_U16 au16BlcRepeatDim[DNG_MUXER_REPEAT_LEN];
    TD_U32 u32WhiteLevel;
    TD_FLOATTD_FLOAT afDefaultScale[DNG_MUXER_SCALE_LEN];
    TD_U16 au16CfaRepeatPatternDim[DNG_MUXER_REPEAT_LEN];
} OT_DNG_MUXER_RAW_FORMAT_S;
```

【成员】

成员名称	描述
af32BlackLevel	黑平衡。
au8CfaPattern	CFA矩阵。
au8CfaPlaneColor	CFA色彩空间。
enCfaLayout	CFA排布方式。
au16BlcRepeatDim	黑电平重复度。
u32WhiteLevel	白电平。
afDefaultScale	缺省尺度。
au16CfaRepeatPatternDim	CFA重复度。

【注意事项】

无。

【相关数据类型及接口】



无。

16.4.12 OT_DNG_MUXER_CFALAYOUT_TYPE_E

【说明】

DNG图片raw数据CFA排布类型枚举。

【定义】

```
typedef enum DNG_MUXER_CFALAYOUT_TYPE_E
{
    DNG_MUXER_CFALAYOUT_TYPE_RECTANGULAR = 1,
    DNG_MUXER_CFALAYOUT_TYPE_A,
    DNG_MUXER_CFALAYOUT_TYPE_B,
    DNG_MUXER_CFALAYOUT_TYPE_C,
    DNG_MUXER_CFALAYOUT_TYPE_D,
    DNG_MUXER_CFALAYOUT_TYPE_E,
    DNG_MUXER_CFALAYOUT_TYPE_F,
    DNG_MUXER_CFALAYOUT_TYPE_G,
    DNG_MUXER_CFALAYOUT_TYPE_H,
    DNG_MUXER_CFALAYOUT_TYPE_BUTT
} OT_DNG_MUXER_CFALAYOUT_TYPE_E;
```

【成员】

成员名称	描述
DNG_MUXER_CFALAYOUT_TYPE_RECTANGULAR	矩形排布
DNG_MUXER_CFALAYOUT_TYPE_A	A型排布
DNG_MUXER_CFALAYOUT_TYPE_B	B型排布
DNG_MUXER_CFALAYOUT_TYPE_C	C型排布
DNG_MUXER_CFALAYOUT_TYPE_D	D型排布
DNG_MUXER_CFALAYOUT_TYPE_E	E型排布
DNG_MUXER_CFALAYOUT_TYPE_F	F型排布
DNG_MUXER_CFALAYOUT_TYPE_G	G型排布
DNG_MUXER_CFALAYOUT_TYPE_H	H型排布

【注意事项】

A型排布、B型排布和C型排布不支持。

【相关数据类型及接口】

无。

16.4.13 OT_DNG_MUXER_IMAGE_TYPE_E

【说明】

定义DNG封装IFD中图片类型枚举。



【定义】

```
typedef enum DNG_MUXER_IMAGE_TYPE_E
{
    OT_DNG_MUXER_IMAGE_IFD0 = 1, // it may have subifds when it's thm data; if no subifds, it's raw data
    OT_DNG_MUXER_IMAGE_SCREEN, // sub ifd of ifd0, must have ifd0 already
    OT_DNG_MUXER_IMAGE_RAW, // subifd of ifd0, must have ifd0 already
} OT_DNG_MUXER_IMAGE_TYPE_E;
```

【成员】

成员名称	描述
OT_DNG_MUXER_IMAGE_IFD0	当IFD0有子IFD表明该IFD中存储的图片为缩略图，当没有子IFD时，表明该IFD内图片为原始RAW图。
OT_DNG_MUXER_IMAGE_SCREEN	IFD中存储的图片类型为缩图。
OT_DNG_MUXER_IMAGE_RAW	IFD中存储的图片类型为原始RAW图。

【注意事项】

DNG图片可封装缩略图和中图，当有缩略图时通常存储在IFD0中，然后在IFD0的子IFD中存储原始RAW图和中图等数据；当前组件支持DNG中图片中不存储缩略图和中图，只存储原始RAW图，即只有IFD0且该IFD中存储原始RAW图。

【相关数据类型及接口】

无。

16.4.14 OT_DNG_DEMUXER_IMAGE_INFO_S

【说明】

定义DNG解封装图像缩略图数据信息。

【定义】

```
typedef struct DNG_DEMUXER_IMAGE_INFO_S
{
    TD_U32 u32Width;
    TD_U32 u32Height;
    TD_U32 u32DataLen;
} OT_DNG_DEMUXER_IMAGE_INFO_S;
```

【成员】

成员名称	描述
u32DataLen	图像数据长度。
u32Height	图像高。
u32Width	图像宽。



【注意事项】

无。

【相关数据类型及接口】

无。

16.4.15 OT_DNG_CONFIG_TYPE_E

【说明】

定义DNG实例配置类型。

【定义】

```
typedef enum DNG_CONFIG_TYPE_E
{
    OT_DNG_CONFIG_TYPE_MUXER = 1, // usually thm or raw
    OT_DNG_CONFIG_TYPE_DEMUXER,
    OT_DNG_CONFIG_TYPE_BUTT,
} OT_DNG_CONFIG_TYPE_E
```

【成员】

成员名称	描述
OT_DNG_CONFIG_TYPE_MUXER	Muxer模式，对输入文件进行封装。
OT_DNG_CONFIG_TYPE_DEMUXER	Demuxer模式，对输入文件名进行解封装。
OT_DNG_CONFIG_TYPE_BUTT	无效值。

【注意事项】

无。

【相关数据类型及接口】

无。

16.4.16 DNG_MAX_FILE_NAME

【说明】

定义DNG图片文件名最大长度。

【定义】

```
#define DNG_MAX_FILE_NAME (128) /*file name path max*/
```

【注意事项】

无。

【相关数据类型及接口】

无。



16.4.17 DNG_MUXER_DCF_DESCRIPTION_LENGTH

【说明】

定义DCF信息描述长度。

【定义】

```
#define DNG_MUXER_DCF_DESCRIPTION_LENGTH (32)
```

【注意事项】

无。

【相关数据类型及接口】

无。

16.4.18 DNG_MUXER_DATETIME_LENGTH

【说明】

定义时间日期信息描述长度。

【定义】

```
#define DNG_MUXER_DATETIME_LENGTH (20)
```

【注意事项】

无。

【相关数据类型及接口】

无。

16.4.19 DNG_MUXER_PROFILEHUESATMAPDIM_LENGTH

【说明】

定义profilehuesatmap描述信息长度。

【定义】

```
#define DNG_MUXER_PROFILEHUESATMAPDIM_LENGTH (3)
```

【注意事项】

无。

【相关数据类型及接口】

无。

16.4.20 DNG_MUXER_LENINFO_LEN

【说明】

定义镜头信息描述长度。

【定义】

```
#define DNG_MUXER_LENINFO_LEN (4)
```

**【注意事项】**

无。

【相关数据类型及接口】

无。

16.4.21 DNG_MUXER_MATRIX_SIZE

【说明】

定义矩阵长度。

【定义】

```
#define DNG_MUXER_MATRIX_SIZE (9)
```

【注意事项】

无。

【相关数据类型及接口】

无。

16.4.22 DNG_MUXER_GPSVER_LEN

【说明】

定义GPS版本描述信息长度。

【定义】

```
#define DNG_MUXER_GPSVER_LEN (4)
```

【注意事项】

无。

【相关数据类型及接口】

无。

16.4.23 DNG_MUXER_TUDE_LEN

【说明】

定义经纬度信息描述长度。

【定义】

```
#define DNG_MUXER_TUDE_LEN (3)
```

【注意事项】

无。

【相关数据类型及接口】

无。



16.4.24 DNG_MUXER_ISP_BAYER_CHN

【说明】

定义贝尔通道数目。

【定义】

```
#define DNG_MUXER_ISP_BAYER_CHN (4)
```

【注意事项】

无。

【相关数据类型及接口】

无。

16.4.25 DNG_MUXER_CFACOLORPLANE

【说明】

定义颜色平面个数。

【定义】

```
#define DNG_MUXER_CFACOLORPLANE (3)
```

【注意事项】

无。

【相关数据类型及接口】

无。

16.4.26 DNG_MUXER_REPEAT_LEN

【说明】

定义电平重复度描述长度。

【定义】

```
#define DNG_MUXER_REPEAT_LEN (2)
```

【注意事项】

无。

【相关数据类型及接口】

无。

16.4.27 DNG_MUXER_SCALE_LEN

【说明】

定义比例信息描述长度。

【定义】

```
#define DNG_MUXER_SCALE_LEN (2)
```

**【注意事项】**

无。

【相关数据类型及接口】

无。

16.4.28 DNG_MUXER_NP_SIZE

【说明】

定义噪声信息描述长度。

【定义】

```
#define DNG_MUXER_NP_SIZE (6)
```

【注意事项】

无。

【相关数据类型及接口】

无。

16.4.29 DNG_MUXER_REF_BW

【说明】

定义ReferenceBlackWhite描述长度。

【定义】

```
#define DNG_MUXER_REF_BW (6)
```

【注意事项】

无。

【相关数据类型及接口】

无。

16.4.30 DNG_MUXER_CROP_LEN

【说明】

定义裁剪信息描述长度。

【定义】

```
#define DNG_MUXER_CROP_LEN (2)
```

【注意事项】

无。

【相关数据类型及接口】

无。



16.5 错误码

DNG封装API错误码如表16-1所示。

表 16-1 DNG 封装 API 错误码

错误代码	宏定义	描述
0	TD_SUCCESS	成功
-1	TD_FAILURE	失败
0xA1388002	OT_ERR_DNG_NULL_PTR	空指针
0xA1388040	OT_ERR_DNG_HANDLE_INVALID	无效句柄
0xA1388041	OT_ERR_DNG_INVALIDARG	非法参数
0xA1388042	OT_ERR_DNG_MALLOC	内存分配失败
0xA1388043	OT_ERR_DNG_CREATE_MUXER	DNG创建muxer失败
0xA1388044	OT_ERR_DNG_DESTROY_MUXER	DNG销毁muxer失败
0xA1388045	OT_ERR_DNG_WRITE	DNG写失败
0xA1388046	OT_ERR_DNG_READ	DNG读失败
0xA1388047	OT_ERR_DNG_CREATE_AGAIN	DNG重复创建
0xA1388048	OT_ERR_DNG_NOT_CREATE	DNG未创建
0xA1388049	OT_ERR_DNG_ADD_IFD	DNG添加IFD失败
0xA1388050	OT_ERR_DNG_GET_INFO	DNG获取信息失败
0xA1388051	OT_ERR_DNG_GET_IMAGE	DNG获取图片数据失败
0xA1388052	OT_ERR_DNG_CREATE_DEMUXER	DNG创建demuxer失败
0xA1388053	OT_ERR_DNG_DESTROY_DEMUXER	DNG销毁demuxer失败



错误代码	宏定义	描述
0xA1388054	OT_ERR_DNG_REACH_MAX	DNG实例超上限

16.6 典型场景使用指南

参见版本sample。



A 缩略语

A		
AAC	Advanced Audio Coding	高级音频编码，音频格式
API	Application Programming Interface	应用程序接口
APP	Application	应用程序
AVC	Advanced Video Coding	高级视频编码
ATTR	Attribute	属性
C		
CFG	Config	配置
CID	Card Identification	SD卡ID
CODEC	COder-DECoder	编解码器
D		
DBR	Dos Boot Record	分区引导扇区
DCF	Design Rule for Camera File System	相机文件系统
DIR	Directory	目录
DNG	Digital Negative	数字负片，Adobe原始图像格式
DTCF	Date Time Camera File System	日期时间文件系统
E		
EMR	Emergency	紧急
ERR	Error	错误
ES	Elementary Stream	原始流
EXIF	Exchangeable Image File Format	可交换图像文件格式



F		
FAT	File Allocation Table	文件分配表
FN	Function	函数
FSTool	File System Tool	文件系统工具
H		
HEVC	High Efficiency Video Coding	高效率视频编码
HFSM	Hierarchical Finite State Machine	分层有限状态机
I		
ISO	International Organization for Standardization	国际标准化组织
IEC	International Electrotechnical Commission	国际电工委员会
M		
Mbuf	Media Buffer	媒体缓存
MMZ	Media Memory Zone	媒体内存区
MPEG	Moving Picture Experts Group	动态图像专家组
MPF	Multi-Picture Format	多图片格式
R		
REC	Record	录像
S		
SDIO	SD Card IO	SD卡输入输出
SDK	Software Development Kit	软件开发工具包
T		
TS	Transport Stream	传输流
X		
XMP	Extensible Metadata Platform	可扩展元数据平台